Theoretical Computer Science
Logic & Puzzles

TypeScript Edition

Prof. Dr. Karl Stroetmann

October 7, 2026

Copyright © 2018–2026 Karl Stroetmann

The text of these lecture notes is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy, distribute, and adapt the text for non-commercial purposes, provided that you give appropriate credit and distribute your contributions under the same license.

The programs discussed in these lecture notes are licensed under the MIT License. They are provided "as is", without warranty of any kind.

The TypeScript programs were translated from the original Python programs by Tobias Neithöfer and Leo Vatter; some of them were later revised by the author.

The sources of these lecture notes and the programs are available at

https://github.com/karlstroetmann/Logic.

ISBN 979-8161584897 (paperback)
ISBN 979-8161596531 (hardcover)

Contents

Acronyms 6

1 Introduction 7
- 1.1 Motivation . . . 7
- 1.2 Overview . . . 8
- 1.3 Check Your Understanding . . . 10

2 Limits of Computability 11
- 2.1 The Halting Problem . . . 11
- 2.2 The Equivalence Problem . . . 17
- 2.3 Concluding Remarks . . . 19
- 2.4 Chapter Review . . . 20
- 2.5 Further Reading . . . 20

3 Correctness Proofs 21
- 3.1 Computational Induction . . . 21
- 3.2 Symbolic Execution . . . 28
  - 3.2.1 Iterative Squaring . . . 28
  - 3.2.2 Integer Square Root . . . 30
- 3.3 Check Your Understanding . . . 34

4 Propositional Calculus 35
- 4.1 Introduction . . . 35
- 4.2 Applications of Propositional Logic . . . 37
- 4.3 The Formal Definition of Propositional Formulas . . . 38
  - 4.3.1 The Syntax of Propositional Formulas . . . 38
  - 4.3.2 Semantics of Propositional Formulas . . . 40
  - 4.3.3 Implementation . . . 41
  - 4.3.4 An Application . . . 45
- 4.4 Tautologies . . . 48
  - 4.4.1 *TypeScript* Implementation . . . 49

4.5 Conjunctive Normal Form . . . 51
4.5.1 Transforming a Formula into CNF . . . 54
4.5.2 Computing the Conjunctive Normal Form in *TypeScript* . . . 55
4.6 The Concept of a Derivation . . . 62
4.6.1 Properties of the Concept of a Formal Derivation . . . 66
4.6.2 Satisfiability . . . 67
4.6.3 Refutational Completeness . . . 68
4.6.4 Constructive Interpretation of the Proof of Refutation Completeness . . . 70
4.7 The Algorithm of Davis and Putnam . . . 74
4.7.1 Simplification with the Cut Rule . . . 76
4.7.2 Simplification via Subsumption . . . 76
4.7.3 Simplification through Case Distinction . . . 77
4.7.4 The Algorithm . . . 78
4.7.5 An Example . . . 78
4.7.6 Implementation . . . 79
4.7.7 The Jeroslow-Wang Heuristic . . . 84
4.8 Solving Logical Puzzles . . . 87
4.8.1 The 8-Queens Problem . . . 87
4.8.2 The Zebra Puzzle . . . 95
4.9 Sudoku . . . 102
4.10 Sudoku and Logic-Solver . . . 107
4.10.1 Implementation of the Sudoku Solver . . . 107
4.11 Check Your Comprehension . . . 112

5 First-Order Logic **113**
5.1 Syntax of First-Order Logic . . . 114
5.1.1 Bound Variables and Free Variables . . . 116
5.1.2 Σ-Formulas . . . 117
5.2 Semantics of First-Order Logic . . . 120
5.3 Implementing Σ-Structures in *TypeScript* . . . 124
5.4 Constraint Programming . . . 126
5.4.1 Constraint Satisfaction Problems . . . 126
5.4.2 Example: Map Colouring . . . 127
5.4.3 Example: The Eight Queens Puzzle . . . 129
5.4.4 A Backtracking Constraint Solver . . . 131
5.5 Solving Search Problems by Constraint Programming . . . 138
5.6 The Z3 Solver . . . 142
5.6.1 A Simple Text Problem . . . 142
5.6.2 The Knight's Tour . . . 145

5.6.3 Solving Search Problems with Z3 . . . 151
5.7 Check Your Comprehension . . . 156

6 Automatic Theorem Proving **157**
6.1 FOL Conjunctive Normal Form . . . 157
6.1.1 Significance of Clausal Form in Automated Reasoning . . . 158
6.2 The Transformation Process: From FOL to Clauses . . . 158
6.2.1 Step 1: Elimination of Implications and Biconditionals . . . 158
6.2.2 Step 2: Conversion to Negation Normal Form - NNF . . . 158
6.2.3 Step 3: Standardization of Variables (Renaming Variables) . . . 159
6.2.4 Step 4: Conversion to Prenex Normal Form . . . 159
6.2.5 Step 5: Skolemization: Eliminating Existential Quantifiers . . . 160
6.2.6 Step 6: Dropping Universal Quantifiers . . . 161
6.2.7 Step 7: Conversion of the Matrix to Conjunctive Normal Form (CNF) . . . 161
6.2.8 Step 8: Using Set Notation . . . 161
6.2.9 Comprehensive Example Illustrating All Steps . . . 161
6.2.10 Refutational Theorem Proving . . . 163
6.3 Unification . . . 165
6.3.1 Substitutions . . . 165
6.3.2 The Algorithm of Martelli and Montanari . . . 166
6.4 A Deductive System for First-Order Logic without Equality . . . 169
6.5 Vampire* . . . 175
6.5.1 Proving Theorems in Group Theory . . . 175
6.5.2 Who killed Agatha? . . . 177
6.6 *Prover9* and *Mace4** . . . 179
6.6.1 The Automatic Prover *Prover9* . . . 180
6.6.2 *Mace4* . . . 181
6.7 Check Your Comprehension . . . 183

Acronyms

CI computational induction. 21

CSP constraint satisfaction problem. 9, 126, 127, 130, 131, 133, 135, 136, 140, 141, 149, 151, 153, 155, 187

FOL first order logic. 9

IT information technology. 7

SE symbolic execution. 21

STS Symbolic Transition System. 138

Chapter 1

Introduction

For the uninitiated, mathematical logic is both quite abstract and pretty arcane. In this short chapter, I would like to motivate why you have to learn logic in order to become a computer scientist. After that, I will give a short overview of the topics covered in this lecture.

1.1 Motivation

In the lecture on algorithms, we have discussed three important properties that an algorithm should have: An algorithm should be

- correct,
- efficient, and
- simple.

The efficiency of algorithms has been discussed in the lecture on algorithms. This lecture will therefore focus on the correctness of algorithms. The rest of this section will further motivate the importance of the correctness of algorithms.

Modern software systems are among the most complex systems developed by mankind. You can get a sense of the complexity of these systems if you look at the amount of work that is necessary to build and maintain complex software systems. Today it is quite common that complex software projects require more than a thousand developers. Of course, the failure of a project of this size is very costly and can have catastrophic consequences. Nevertheless, history shows that these failures happen. Here is a list of software problems that have made it to the headlines in recent years.

1. A stark example of information technology (IT) failure consequences is the December 2022 Southwest Airlines meltdown. A number of delayed flights due to a severe winter storm triggered the collapse of the Southwest Airlines crew scheduling system SkySolver, leading to over 16,700 canceled flights and stranding hundreds of thousands of passengers during peak holiday travel. Southwest estimated the financial impact at over $1 billion.

2. In 2018 and 2019 two Boeing 737 MAX planes crashed because of a software problem in the Maneuvering Characteristics Augmentation System.

   This error led to the death of 346 passengers and crew.

3. Between 1999 and 2015 the British Post Office used faulty accounting software provided by Fujitsu. As a result of the buggy accounting, over 900 employees of the British Post Office were falsely convicted of embezzlement. As a consequence, some of these employees were imprisoned. Four of those that were falsely convicted committed suicide. These tragic incidents are known as the Horizon IT scandal.

4. In 1996, the very first Ariane 5 rocket self-destructed as a result of a software error.

These and numerous other examples show that the development of complex software systems requires a high level of precision and diligence. Hence, the development of software needs a solid scientific foundation. Mathematical logic is an important part of this foundation that has immediate applications in computer science.

(a) Logic can be used to specify the interfaces of complex systems.

(b) Logic is used to build interactive theorem provers that are able to establish the correctness of software. For example, *Microsoft*™ has built the Lean Prover and the Z3, which is a logic constraint solver, as part of their research in software engineering.

(c) The correctness of digital circuits can be verified using automatic theorem provers that are based on propositional logic. For example, *Cadence*™ has built the Jasper Formal Verification Platform.

Personally, I worked on hardware verification at the Siemens Research Center until 2002. Their current product is called Questa One.

It is easy to extend this enumeration. However, besides their immediate applications, there is another reason you have to study both logic and set theory: Without the proper use of abstractions, complex software systems cannot be managed. After all, nobody is able to keep millions of lines of program code in her head. The only way to construct and manage a software system of this size is to introduce the right abstractions and to develop the system in layers. Hence, the ability to work with abstract concepts is one of the main virtues of a modern computer scientist. Exposing students to mathematics in general and logic in particular trains their abilities to grasp abstract concepts.

From my past teaching experience I know that many students think that a good programmer already is a good computer scientist. However, computer science is much more than just programming. This should not be too surprising. After all, there is no reason to believe that a good bricklayer is a good architect and neither is a good architect necessarily a good bricklayer. In computer science, a good programmer need not be a scientist at all, while a computer scientist, by its very name, is a scientist. There is no denying that mathematics in general and logic in particular is an important part of science. Furthermore, these topics form the foundation of computer science. Therefore, you should master them. In addition, this part of your scientific education is much more permanent than the knowledge of any particular programming language. Nobody knows which programming language will be *en vogue* 10 years from now. In three years, when you start your professional career, most of you will have to learn new programming languages. Then your ability to quickly grasp new concepts will be much more important than your skills in any particular programming language.

1.2 Overview

This lecture deals mostly with mathematical logic and is structured as follows.

(a) We begin our lecture by investigating the limits of computability.

For certain problems there is no algorithm that can solve the problem algorithmically. For example, the question whether a given program will terminate for a given input is not decidable. This is known as the halting problem. We will prove the undecidability of the halting problem in the second chapter.

(b) The third chapter discusses two different methods that can be used to prove the correctness of a program:

- Computational induction is the method of choice for proving the correctness of recursive algorithms.
- Symbolic execution is used to verify iterative algorithms.

(c) The fourth chapter discusses propositional logic.

In logic, we distinguish between propositional logic, first order logic, and higher order logic. Propositional logic is only concerned with the logical connectives

- $\neg$ (not),
- $\wedge$ (and),
- $\vee$ (or),
- $\rightarrow$ (if $\cdots$ then),
- $\leftrightarrow$ (if and only if).

First-order logic also investigates the quantifiers

- $\forall$ (for all),
- $\exists$ (there exists).

where these quantifiers range over the objects of the domain of discourse. Finally, in higher order logic these quantifiers also range over sets, functions, and predicates.

As propositional logic is easier to grasp than first-order logic, we start our investigation of logic with propositional logic. Furthermore, propositional logic has the advantage of being decidable: We will present an algorithm that can check whether a propositional formula is satisfiable. In contrast to propositional logic, first-order logic is not decidable.

Next, we discuss applications of propositional logic: We will show how the 8 queens problem can be reduced to the question whether a formula from propositional logic is satisfiable. We present the algorithm of Davis and Putnam that can decide the satisfiability of a propositional formula. We will use it to solve various logical puzzles. In particular, we solve

(a) the 8 queens puzzle, and
(b) a Sudoku puzzle.

(d) Next, we discuss first-order logic.

We begin by defining first order logic (FOL) formulas and their meaning. Next, we introduce constraint satisfaction problems (CSPs) and show how to solve these problems via backtracking. Then we show how to formalise certain problems as CSPs and then solve them. In particular, we will discuss the following problems:

(a) the eight queens problem,

(b) the Zebra Puzzle,

(c) the missionaries and cannibals problem and similar search problems.

To this end we will introduce Z3, which is a high-performance constraint solver developed by Microsoft.

(e) In the last chapter, the most important concept will be the notion of a formal proof in first order logic. To this end, we introduce a formal proof system that is complete for first order logic. Completeness means that we will develop an algorithm that can prove the correctness of every first-order formula that is universally valid. This algorithm is the foundation of automated theorem proving.

As an application of theorem proving we discuss the systems Vampire, Prover9 and Mace4. Prover9 is an automated theorem prover, while Mace4 can be used to refute a mathematical conjecture.

1.3 Check Your Understanding

- Is it possible that a coding mistake in accounting software can kill people?
- Provide three examples of faulty software systems that have caused great harm. Do not cite the examples given in this chapter but rather search the web to find more examples.
- Provide two examples of automatic theorem provers that are used in industry.
- Is first order logic more expressive than propositional logic or is it the other way around?

Chapter 2

Limits of Computability

> The only reliable way to test if a program will run forever is to run it forever.
>
> — *Anonymous*

Every discipline of the sciences has its limits: Students of the medical sciences soon realize that it is difficult to raise the dead and even religious zealots have trouble to walk on water. Only very experienced bishops can do it. Similarly, computer science has its limits. We will discuss these limits next. First, we show that we cannot decide whether a computer program will eventually terminate or whether it will run forever. Second, we prove that it is impossible to automatically check whether two functions are equivalent.

2.1 The Halting Problem

In this subsection we prove that it is not possible for a computer program to decide whether another computer program does terminate. This problem is known as the *halting problem*. Before we give a formal proof that the halting problem is undecidable, let us discuss one example that shows why it is indeed difficult to decide whether a program always terminates. Consider the program shown in Figure 2.1 on page 13. This program contains a `while`-loop in line 27. If there is a natural number $k \geq n$ such that the expression,

$$\texttt{legendre}(k)$$

in line 28 evaluates to `false`, then the program prints a message and terminates. However, if $\texttt{legendre}(k)$ is true for all $k \geq n$, then the `while`-loop does not terminate. Given a natural number `n`, the expression $\texttt{legendre}(n)$ tests whether there is a prime number between $\texttt{n}^2$ and $(\texttt{n}+1)^2$. If, however, the set

$$\{k \in \mathbb{N} \mid n^2 \leq k \wedge k \leq (n+1)^2\}$$

does not contain a prime number, then $\texttt{legendre}(n)$ evaluates to `false` for this value of n. The function `legendre` is defined in line 14. Given a natural number n, it returns `true` if and only if the formula

$$\exists k \in \mathbb{N} : \big(n^2 < k \wedge k < (n+1)^2 \wedge \mathit{isPrime}(k)\big)$$

holds true. The French mathematician Adrien-Marie Legendre (1752 – 1833) conjectured that for any natural number $n \in \mathbb{N}$ there is a prime number p such that

$$n^2 < p \wedge p < (n+1)^2$$

holds. Although there are a number of arguments in support of Legendre's conjecture, to this day nobody has been able to prove it. The answer to the question, whether the invocation of the function `findCounterExample` will terminate for every natural number n is, therefore, unknown as it depends on the truth of Legendre's conjecture: If we had some procedure that could check whether the function call `findCounterExample`(1) does terminate, then this procedure would be able to decide whether Legendre's conjecture is true. Therefore, it should come as no surprise that such a procedure does not exist.

Let us proceed to prove formally that the halting problem is not solvable. To this end, we need the following definition.

Definition 1 (Test Function) *A string t is a test function with name f iff t has the form*

```
`function f(x) {
  body
}`
```

and, furthermore, the string t can be parsed as a TypeScript function, that is the evaluation of the expression

$$\texttt{eval}(t)$$

does not yield an error. The set of all test functions is denoted as TF. If $t \in TF$ and t has the name f, then this is written as

$$\texttt{name}(t) = f.$$ □

Examples:

1. We define the string s_1 as follows:

   ```
   const s1 = `function simple(x) {
       return 0;
   }`;
   ```

   Then s_1 is a test function with the name `simple`.

2. We define the string s_2 as

   ```
   const s2 = `function loop(x) {
       let x = 0;
       while (true) {
           x = x + 1;
       }
   }`;
   ```

   Then s_2 is a test function with the name `loop`.

3. We define the string s_3 as

```
function range(n: number): number[] {
    return Array.from({ length: n }, (_, t) => t + 1);
}

function divisors(k: number): number[] {
    return range(k).filter(t => k % t == 0);
}

function isPrime(k: number): boolean {
    const divisorList = divisors(k);
    return k != 1 && divisorList.length == 2;
}

function legendre(n: number): boolean {
    let k = n * n + 1;
    while (k < (n + 1) ** 2) {
        if (isPrime(k)) {
            console.log(`${n}**2 < ${k} < ${n + 1}**2`);
            return true;
        }
        k += 1;
    }
    return false;
}

function findCounterExample(n: number): void {
    while (true) {
        if (legendre(n)) {
            n += 1;
        } else {
            console.log(`No prime between ${n}**2 and ${n + 1}**2!`);
            return;
        }
    }
}
```

Figure 2.1: A program checking Legendre's conjecture.

```
const s3 = `function hugo(x):
    return ++x;
`;
```

Then s_3 is not a test function. The reason is that in *TypeScript* the body of a function needs to be enclosed in braces.

Therefore,

```
eval(s3)
```

yields an error message complaining about an unexpected token.

In order to be able to formalize the halting problem succinctly, we introduce three additional notations.

Notation 2 ($\rightsquigarrow, \downarrow, \uparrow$) *If n is the name of a TypeScript function that takes k arguments $a_1, \cdots, a_k$, then we write*

$$n(a_1, \cdots, a_k) \rightsquigarrow r$$

iff the evaluation of the expression $n(a_1, \cdots, a_k)$ yields the result r. If we are not concerned with the result r but only want to state that the evaluation terminates eventually, then we will write

$$n(a_1, \cdots, a_k) \downarrow$$

*and read this notation as "*evaluation of $n(a_1, \cdots, a_k)$ terminates*". If the evaluation of the expression $n(a_1, \cdots, a_k)$ does not terminate, this is written as*

$$n(a_1, \cdots, a_k) \uparrow.$$

*This notation is read as "*evaluation of $n(a_1, \cdots, a_k)$ diverges*".* □

Examples: Using the test functions defined earlier, we have:

1. `simple("emil")` $\rightsquigarrow 0$,
2. `simple("emil")` $\downarrow$,
3. `loop(2)` $\uparrow$.

The halting problem for *TypeScript* functions is the question whether there is a *TypeScript* function

```
function stops(t, a) {
    ⋮
}
```

that takes as input a test function t and a string a and that satisfies the following specification:

1. $t \notin TF \quad \Leftrightarrow \quad$ `stops`$(t, a) \rightsquigarrow 2$. If the first argument of `stops` is not a test function, then `stops(`t`, `a`)` returns the number 2.
2. $t \in TF \wedge$ `name`$(t) = n \wedge n(a) \downarrow \quad \Leftrightarrow \quad$ `stops`$(t, a) \rightsquigarrow 1$. If the first argument of `stops` is a test function with name n and, furthermore, the evaluation of $n(a)$ terminates, then `stops(`t`, `a`)` returns the number 1.
3. $t \in TF \wedge$ `name`$(t) = n \wedge n(a) \uparrow \quad \Leftrightarrow \quad$ `stops`$(t, a) \rightsquigarrow 0$. If the first argument of `stops` is a test function with name n but the evaluation of $n(a)$ diverges, then `stops(`t`, `a`)` returns the number 0.

If there was a *TypeScript* function `stops` that did satisfy the specification given above, then the halting problem for *TypeScript* would be decidable.

Theorem 3 (Alan Turing, 1937) *The halting problem is undecidable.*

Proof: In order to prove the undecidability of the halting problem we have to show that there can be no function `stops` satisfying the specification given above. This calls for an indirect proof also known as a *proof by contradiction*. We will therefore assume that a function `stops` solving the halting problem does exist and we will then show that this assumption leads to a contradiction. This contradiction will leave us with the conclusion that there can be no function `stops` that satisfies the specification given above and that, therefore, the halting problem is undecidable. In order to proceed, let us assume that a *TypeScript* function `stops` satisfying the specification given above exists and let us define the string *turing* as shown in Figure 2.2 below.

```
const turing = `
function alan(x) {
    const result = stops(x, x);
    if (result == 1) {
        while (true) {
            console.log("... looping ...");
        }
    }
    return result;
};`
```

Figure 2.2: Definition of the string *turing*.

Given this definition it is easy to check that *turing* is, indeed, a test function with the name "`alan`", that is we have

$$\mathit{turing} \in \mathit{TF} \ \wedge \ \texttt{name}(\mathit{turing}) = \texttt{alan}.$$

Therefore, we can use the string *turing* as the first argument of the function `stops`. Let us determine the value of the following expression:

$$\texttt{stops}(\mathit{turing}, \mathit{turing})$$

Since we have already noted that *turing* is a test function, according to the specification of the function `stops` there are only two cases left:

$$\texttt{stops}(\mathit{turing}, \mathit{turing}) \rightsquigarrow 0 \quad \vee \quad \texttt{stops}(\mathit{turing}, \mathit{turing}) \rightsquigarrow 1.$$

Let us consider these cases in turn.

1. $\texttt{stops}(\mathit{turing}, \mathit{turing}) \rightsquigarrow 0$. According to the specification of `stops` we should then have

   $$\texttt{alan}(\mathit{turing}) \uparrow.$$

   Let us check whether this is true. In order to do this, we have to check what happens when the expression

   $$\texttt{alan}(\mathit{turing})$$

   is evaluated:

(a) Since we have assumed for this case that the expression `stops`(*turing*, *turing*) yields 0, in line 3, the variable `result` is assigned the value 0.

(b) Line 4 now tests whether `result` is 1. Of course, this test fails. Therefore, the block of the `if`-statement is not executed.

(c) Finally, in line 9 the value of the variable `result` is returned.

All in all we see that the call of the function `alan` does terminate when given the argument *turing*. However, this is the opposite of what the function `stops` has claimed. Therefore, this case has led us to a contradiction.

2. `stops`(*turing*, *turing*) $\rightsquigarrow$ 1. According to the specification of `stops` we should then have

$$\texttt{alan}(\textit{turing})\downarrow,$$

i.e. the evaluation of `alan`(*turing*) should terminate. Again, let us check in detail whether this is true.

(a) Since we have assumed for this case that the expression `stops`(*turing*, *turing*) yields 1, in line 3, the variable `result` is assigned the value 1.

(b) Line 4 now tests whether `result` is 1. Of course, this time the test succeeds. Therefore, the block of the `if`-statement is executed.

(c) However, this block contains an infinite loop. Therefore, the evaluation of `alan`(*turing*) diverges. But this contradicts the specification of `stops`!

Therefore, the second case also leads to a contradiction.

As we have obtained contradictions in both cases, the assumption that there is a function `stops` that solves the halting problem is refuted. □

Remark: The proof of the fact that the halting problem is undecidable was published in 1937 by Alan Turing (1912 – 1954) [Tur37]. Of course, Turing did not solve the problem for *TypeScript* but rather for the so-called *Turing machines*. A Turing machine can be interpreted as a formal description of an algorithm. Therefore, Turing has shown that there is no algorithm that is able to decide whether some given algorithm will always terminate.

Remark: At this point you might wonder whether there might be another programming language that is more powerful so that it would be possible to solve the halting problem if we use this more powerful programming language. However, if you check the proof given for *TypeScript* you will easily see that this proof can be adapted to any other programming language that is at least as powerful as *TypeScript*. ◇

Of course, if a programming language is very restricted, then it might be possible to check the halting problem for this weak programming language. But for any programming language that supports at least `while`-loops, `if`-statements, and the definition of procedures the argument given above shows that the halting problem is not solvable.

Exercise 1: Show that if the halting problem were solvable, then it would be possible to write a program that checks whether there are infinitely many twin primes. A twin prime is a pair of natural

numbers $\langle p, p+2 \rangle$ such that both p and $p+2$ are prime numbers. The *twin prime conjecture* is one of the oldest unsolved mathematical problems. ◇

Exercise 2: A set X is countable iff there is a function

$$f : \mathbb{N} \to X$$

such that for all $x \in X$ there is an $n \in \mathbb{N}$ such that x is the image of n under f:

$$\forall x \in X : \exists n \in \mathbb{N} : x = f(n).$$

A function of this kind is called surjective. Prove that the set $2^{\mathbb{N}}$, which is the set of all subsets of $\mathbb{N}$, is not countable.

Hint: Your proof should be similar to the proof that the halting problem is undecidable. Proceed as follows: Assume that there is a function f enumerating the subsets of $\mathbb{N}$, that is assume that

$$\forall x \in 2^{\mathbb{N}} : \exists n \in \mathbb{N} : x = f(n)$$

holds. Next, and this is the crucial step, define a set `Cantor` as follows:

$$\texttt{Cantor} := \{n \in \mathbb{N} \mid n \notin f(n)\}.$$

Now try to derive a contradiction. ◇

Exercise 3: Prove that the set $\mathbb{Q}$ of all rational numbers is countable. The set $\mathbb{Q}$ can be defined as follows:

$$\mathbb{Q} = \left\{ \frac{p}{q} \;\middle|\; p \in \mathbb{Z} \wedge q \in \mathbb{N} \wedge q \geq 1 \right\}.$$

2.2 Undecidability of the Equivalence Problem

Unfortunately, the halting problem is not the only undecidable problem in computer science. Another important problem that is undecidable is the question whether two given functions always compute the same result. To state this more formally, we need the following definition.

Definition 4 ($\simeq$) *Assume n_1 and n_2 are the names of two TypeScript functions that take arguments $a_1, \cdots, a_k$. Let us define*

$$n_1(a_1, \cdots, a_k) \simeq n_2(a_1, \cdots, a_k)$$

if and only if either of the following cases is true:

1. $n_1(a_1, \cdots, a_k) \uparrow \quad \wedge \quad n_2(a_1, \cdots, a_k) \uparrow$,
   that is both function calls diverge.
2. $\exists r : \Big(n_1(a_1, \cdots, a_k) \rightsquigarrow r \quad \wedge \quad n_2(a_1, \cdots, a_k) \rightsquigarrow r\Big)$
   that is both function calls terminate and compute the same result.

If $n_1(a_1, \cdots, a_k) \simeq n_2(a_1, \cdots, a_k)$ holds, then the expressions $n_1(a_1, \cdots, a_k)$ and $n_2(a_1, \cdots, a_k)$ are partially equivalent. □

We are now ready to state the equivalence problem. A *TypeScript* function `equal` solves the *equivalence problem* if it is defined as

```
function equal(p1, p2, a) {
    body
}
```

and, furthermore, it satisfies the following specification:

1. $p_1 \notin TF \ \vee \ p_2 \notin TF \quad \Leftrightarrow \quad \texttt{equal}(p_1, p_2, a) \rightsquigarrow 2.$
2. If
   (a) $p_1 \in TF \ \wedge \ \texttt{name}(p_1) = n_1,$
   (b) $p_2 \in TF \ \wedge \ \texttt{name}(p_2) = n_2$ and
   (c) $n_1(a) \simeq n_2(a)$

   holds, then we must have:

   $$\texttt{equal}(p_1, p_2, a) \rightsquigarrow 1.$$
3. Otherwise we must have

   $$\texttt{equal}(p_1, p_2, a) \rightsquigarrow 0.$$

Theorem 5 *The equivalence problem is undecidable.*

Proof: The proof is by contradiction. Therefore, assume that there is a function `equal` such that `equal` solves the equivalence problem. Assuming `equal` exists, we will then proceed to define a function `stops` that solves the halting problem. Figure 2.3 shows this construction of the function `stops`.

```
1   function stops(t, a) {
2       let l = `function loop(x) {
3                   while (true) {
4                       x = 1;
5                   }
6               }`;
7       let e = equal(l, t, a);
8       if (e == 2) {
9           return 2;
10      } else {
11          return 1 - e;
12      }
13  }
```

Figure 2.3: An implementation of the function `stops`.

Notice that in line 7 the function `equal` is called with a string that is a test function with the name `loop`. This test function has the following form:

```
function loop(x) {
     while (true) {
         x = 1;
     }
}
```

Independent of the argument x, the function `loop` does not terminate. Therefore, if the first argument t of `stops` is a test function with name n, the function `equal` will return 1 if $n(a)$ diverges, and will return 0 otherwise. But this implementation of `stops` would then solve the halting problem as for a given test function t with name n and argument a the function `stops` would return 1 if and only if the evaluation of $n(a)$ terminates. As we have already proven that the halting problem is undecidable, there can be no function `equal` that solves the equivalence problem either. □

Remark: The unsolvability of the equivalence problem was proven by Henry Gordon Rice [Ric53] in 1953. ◇

2.3 Concluding Remarks

Although, in general, we cannot decide whether a program terminates for a given input, this does not mean that we should not attempt to do so. After all, we have only proven that there is no procedure that can always check whether a given program will terminate. There might well exist a procedure for termination checking that works most of the time. Indeed, there are a number of systems that try to check whether a program will terminate for every input. For example, for Prolog programs, the paper "*Automated Modular Termination Proofs for Real Prolog Programs*" [MGS96] describes a successful approach. Recent years have seen a lot of progress in this area. The article "Proving Program Termination" [CPR11] reviews these developments. However, as the recently developed systems rely on both *automatic theorem proving* and *Ramsey theory* they are quite out of the scope of this lecture.

2.4 Chapter Review

You should be able to solve the following exercises.

(a) Define the notion of a test function.

(b) Define the halting problem.

(c) Prove that the halting problem is not decidable.

(d) Show how the notion $\simeq$ is defined.

(e) Define the equivalence problem.

(f) Prove that the equivalence problem is not decidable.

(g) Define the notion of a countable set.

(h) Prove that the set $2^{\mathbb{N}}$ is not countable.

(i) Provide an example of an unsolved mathematical problem that could be easily solved if the halting problem were decidable.

2.5 Further Reading

The book "*Introduction to the Theory of Computation*" by Michael Sipser [Sip96] discusses the undecidability of the halting problem in section 4.2. It also covers many related undecidable problems.

Another good book discussing undecidability is the book "*Introduction to Automata Theory, Languages, and Computation*" written by John E. Hopcroft, Rajeev Motwani and Jeffrey D. Ullman [HMU06]. This book is the third edition of a classic text. In this book, the topic of undecidability is discussed in chapter 9.

The exposition in these books is based on Turing machines and is therefore more formal than the exposition given here. This increased formality is necessary to prove that, for example, it is undecidable whether two context free grammars are equivalent. A word of warning: The two books mentioned above are not intended to be read by undergraduates in their first year. If you want to dive deeper into the concept of undecidability, you should do so only after you have finished your second year.

Chapter 3

Correctness Proofs

There are two ways to write error-free programs; only the third one works.

— *Alan Perlis*

In this chapter we will show two different methods that can be used to prove the correctness of a *TypeScript* function.

(a) The method of computational induction (CI) can be used to verify the correctness of a *TypeScript* function that is defined recursively.

(b) In order to establish the correctness of a *TypeScript* function that is defined iteratively we use symbolic execution (SE).

3.1 Computational Induction

Figure 3.1 shows the definition of the function `power`(m, n) that computes the value m^n. We will verify the correctness of this function.

It is by no means obvious that the function shown in Figure 3.1 does compute the value m^n. We prove this claim by computational induction. Technically, CI is an induction on the number of recursive invocations of a function that is defined recursively. This method is the method of choice to prove the correctness of a function if this function is defined recursively. A proof by computational induction consists of three parts:

1. The base case.

   In the base case we have to show that the function definition is correct in all those cases where the function does not invoke itself recursively.

2. The induction step.

   In the induction step we have to prove that the function definition works in all those cases where the function does invoke itself recursively. In order to carry out this proof we may assume that the results computed by the recursive invocations are correct. This assumption is called the induction hypothesis.

```
function power(m: bigint, n: number): bigint {
    if (n == 0) {
        return 1n;
    }
    const p = power(m, Math.floor(n / 2));
    if (n % 2 == 0) {
        return p * p;
    } else {
        return p * p * m;
    }
}
```

Figure 3.1: Computation of m^n for $m, n \in \mathbb{N}$.

3. The termination proof.

   In this final step we have to show that the recursive definition of the function is well founded, i.e. we have to prove that the recursive invocations terminate.

Let us prove the claim

$$\texttt{power}(m, n) = m^n$$

by computational induction. In order to simplify the notation we use Gauß's bracket notation for the floor function and define

$$\lfloor x \rfloor := \texttt{Math.floor}(x).$$

1. **Base case**:

   The only case where `power` does not invoke itself recursively is the case $n = 0$. In this case, we have

   $$\texttt{power}(m, 0) = 1 = m^0. \quad \surd$$

2. **Induction step**:

   The recursive invocation of `power` has the form $\texttt{power}(m, \lfloor n/2 \rfloor)$. By the induction hypothesis we may assume that

   $$\texttt{power}\big(m, \lfloor n/2 \rfloor\big) = m^{\lfloor n/2 \rfloor}$$

   holds. After the recursive invocation there are two cases that have to be dealt with separately.

   (a) $n \,\%\, 2 = 0$, therefore n is even.
   Then there exists a number $k \in \mathbb{N}$ such that $n = 2 \cdot k$ and therefore $\lfloor n/2 \rfloor = k$. Hence we have:

$$\begin{aligned}
\texttt{power}(m,n) &= \texttt{power}(m,k) \cdot \texttt{power}(m,k) \\
&\overset{\text{IV}}{=} m^k \cdot m^k \\
&= m^{2\cdot k} \\
&= m^n.
\end{aligned}$$

(b) $n \,\%\, 2 = 1$, therefore n is odd.
Then there exists a number $k \in \mathbb{N}$ such that $n = 2 \cdot k + 1$ and we have $\lfloor n/2 \rfloor = k$. In this case we have:

$$\begin{aligned}
\texttt{power}(m,n) &= \texttt{power}(m,k) \cdot \texttt{power}(m,k) \cdot m \\
&\overset{\text{IV}}{=} m^k \cdot m^k \cdot m \\
&= m^{2\cdot k+1} \\
&= m^n.
\end{aligned}$$

As we have shown that $\texttt{power}(m,n) = m^n$ in both cases, the induction step is finished. √

3. **Termination proof**:
Every time the function `power` is invoked as $\texttt{power}(m,n)$ and $n > 0$, the recursive invocation has the form $\texttt{power}(m, \lfloor n/2 \rfloor)$ and, since $\lfloor n/2 \rfloor < n$ for all $n > 0$, the second argument is decreased. As this argument is a natural number, it must eventually reach 0. But if the second argument of the function `power` is 0, the function terminates immediately. √ □

```
function divMod(m: number, n: number): [number, number] {
    if (m < n) {
        return [0, m];
    }
    const [q, r] = divMod(Math.floor(m / 2), n);
    if (2 * r + (m % 2) < n) {
        return [2 * q, 2 * r + (m % 2)];
    } else {
        return [2 * q + 1, 2 * r + (m % 2) - n];
    }
}
```

Figure 3.2: The function `divMod`.

Example: The function `divMod` that is shown in Figure 3.2 satisfies the specification

$$\texttt{divMod}(m,n) = (q,r) \rightarrow m = q \cdot n + r \;\wedge\; r < n.$$ ◇

Proof: Before we start with the details of the proof, let us note that the following equation is valid for all $m \in \mathbb{N}$:

$$2 \cdot \lfloor m/2 \rfloor + m\%2 = m.$$

The easiest way to see this is to write the number m in the binary system. If the binary representation of m is written with k bits $b_0, b_1, \cdots, b_{k-1}$ where b_0 is the least significant bit, then we have

$$m = \sum_{i=0}^{k-1} b_i \cdot 2^i.$$

But then it is easy to see that the following holds:

1. $\lfloor m/2 \rfloor = \left\lfloor \sum_{i=1}^{k-1} b_i \cdot 2^{i-1} + b_0 \cdot \frac{1}{2} \right\rfloor = \sum_{i=1}^{k-1} b_i \cdot 2^{i-1}$,
2. $m \,\%\, 2 = b_0$.

From this we get

$$2 \cdot \lfloor m/2 \rfloor + m \,\%\, 2 = 2 \cdot \sum_{i=1}^{k-1} b_i \cdot 2^{i-1} + b_0 = \sum_{i=0}^{k-1} b_i \cdot 2^i = m.$$

Now we can start with the proof of the correctness of the function `divMod`. We assume that $m, n \in \mathbb{N}$, where $n > 0$. Furthermore, assume

$$\bar{q}, \bar{r} = \texttt{divMod}(m, n).$$

In order to prove the correctness of `divMod`, we have to show two formulas:

$$m = \bar{q} \cdot n + \bar{r} \tag{3.1}$$
$$\bar{r} < n \tag{3.2}$$

Since `divMod` is defined recursively, the proof of these formulas is done by computational induction.

B.C.: $m < n$

In this case we have $\bar{q} = 0$ and $\bar{r} = m$. In order to prove (3.1) we note that

$$\begin{aligned} & m = \bar{q} \cdot n + \bar{r} \\ \Leftrightarrow \quad & m = 0 \cdot n + m \quad \surd \end{aligned}$$

To prove (3.2) we note that

$$\begin{aligned} & \bar{r} < n \\ \Leftrightarrow \quad & m < n \quad \surd \end{aligned}$$

Here $m < n$ is true because this condition is the assumption of the base case.

I.S.: $\lfloor m/2 \rfloor \mapsto m$

By the induction hypothesis we know that our claim is true for the recursive invocation of `divMod` in line 5. Therefore we have the following:

$$\lfloor m/2 \rfloor = q \cdot n + r \tag{3.3}$$
$$r < n \tag{3.4}$$

In order to complete the induction step we have to perform a case distinction that is analogous to the test of the second `if`-statement in the implementation of `divMod`.

(a) $2 \cdot r + m\,\%\,2 < n$

In this case we have $\bar{q} = 2 \cdot q$ and $\bar{r} = 2 \cdot r + m\,\%\,2$. In order to prove (3.1) we note the following:

$$m = \bar{q} \cdot n + \bar{r} \tag{3.5}$$
$$\Leftrightarrow \quad m = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2 \tag{3.6}$$

We will derive equation (3.6) from equation (3.3). To this end, we multiply equation (3.3) by 2. This yields:

$$2 \cdot \lfloor m/2 \rfloor = 2 \cdot q \cdot n + 2 \cdot r.$$

If we add $m\,\%\,2$ to this equation we get

$$2 \cdot \lfloor m/2 \rfloor + m\,\%\,2 = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2.$$

As we have $2 \cdot \lfloor m/2 \rfloor + m\,\%\,2 = m$ the last equation can be simplified to

$$m = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2.$$

However, this is just equation (3.6) which we had to prove. $\surd$

Next, we show that $\bar{r} < n$. This is equivalent to

$$2 \cdot r + m\,\%\,2 < n.$$

However, this inequation is the condition of this case of the case distinction and hence it is true. $\surd$

(b) $2 \cdot r + m\,\%\,2 \geq n$

In this case we have $\bar{q} = 2 \cdot q + 1$ and $\bar{r} = 2 \cdot r + m\,\%\,2 - n$. We start with the proof of (3.1).

$$\begin{aligned} & m = \bar{q} \cdot n + \bar{r} \\ \Leftrightarrow \quad & m = (2 \cdot q + 1) \cdot n + 2 \cdot r + m\,\%\,2 - n \\ \Leftrightarrow \quad & m = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2 \end{aligned}$$

This last equation follows from equation (3.3) as follows:

$$\begin{aligned} & \lfloor m/2 \rfloor = q \cdot n + r \\ \Rightarrow \quad & 2 \cdot \lfloor m/2 \rfloor = 2 \cdot q \cdot n + 2 \cdot r \\ \Rightarrow \quad & 2 \cdot \lfloor m/2 \rfloor + m\,\%\,2 = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2 \\ \Rightarrow \quad & m = 2 \cdot q \cdot n + 2 \cdot r + m\,\%\,2 \end{aligned}$$

Next, we show that $\bar{r} < n$. This is equivalent to

$$2 \cdot r + m\,\%\,2 - n < n$$

From (3.4) we know that

$$\begin{aligned} & r < n \\ \Rightarrow \quad & r + 1 \leq n \\ \Rightarrow \quad & 2 \cdot r + 2 \leq 2 \cdot n \\ \Rightarrow \quad & 2 \cdot r + m\,\%\,2 + 1 \leq 2 \cdot n \quad \text{since } m\,\%\,2 \leq 1 \\ \Rightarrow \quad & 2 \cdot r + m\,\%\,2 < 2 \cdot n \\ \Rightarrow \quad & 2 \cdot r + m\,\%\,2 - n < n \surd \end{aligned}$$

T.: As $\lfloor m/2 \rfloor < m$ for all $m \geq n$ and $n > 0$ it is obvious that we will eventually have $m < n$. But then the function `divMod` terminates. □

Before we can tackle the next exercise, we need to prove the following lemma.

Lemma 6 (Euclid) *Assume $a, b \in \mathbb{N}$ such that $b > 0$. Then we have*

$$\texttt{gcd}(a, b) = \texttt{gcd}(b, a \,\%\, b).$$

Proof: The function $\texttt{cd}(a, b)$ computes the set of common divisors of a and b and is therefore defined as

$$\texttt{cd}(a, b) := \big\{t \in \mathbb{N} \mid a \,\%\, t = 0 \wedge b \,\%\, t = 0\big\}.$$

The function gcd is related to the function cd by the equation

$$\texttt{gcd}(a, b) = \max\big(\texttt{cd}(a, b)\big).$$

Hence it is sufficient to show that

$$\texttt{cd}(a, b) = \texttt{cd}(b, a \,\%\, b).$$

This is an equation between two sets and therefore is equivalent to showing that both

$$\texttt{cd}(a, b) \subseteq \texttt{cd}(b, a \,\%\, b) \quad \text{and} \quad \texttt{cd}(b, a \,\%\, b) \subseteq \texttt{cd}(a, b)$$

holds. We show these two statements separately.

1. We show that $\texttt{cd}(a, b) \subseteq \texttt{cd}(b, a \,\%\, b)$.

   Assume $t \in \texttt{cd}(a, b)$. Then there are $u, v \in \mathbb{N}$ such that

   $$a = t \cdot u \quad \text{and} \quad b = t \cdot v.$$

   Since $a = q \cdot b + a \,\%\, b$ where $q = \lfloor a/b \rfloor$ we have

   $$a \,\%\, b = a - q \cdot b = t \cdot u - q \cdot t \cdot v = t \cdot (u - q \cdot v).$$

   This shows that t divides $a \,\%\, b$. Since t also divides b we therefore have $t \in \texttt{cd}(b, a \,\%\, b)$. √

2. We show that $\texttt{cd}(b, a \,\%\, b) \subseteq \texttt{cd}(a, b)$.

   Assume $t \in \texttt{cd}(b, a \,\%\, b)$. Then there are $u, v \in \mathbb{N}$ such that

   $$b = t \cdot u \quad \text{and} \quad a \,\%\, b = t \cdot v.$$

   Since $a = q \cdot b + a \,\%\, b$ where $q = \lfloor a/b \rfloor$ we have

   $$a = q \cdot t \cdot u + t \cdot v = t \cdot (q \cdot u + v).$$

   This shows that t divides a. Since t also divides b we therefore have $t \in \texttt{cd}(a, b)$. √

This concludes the proof. □

Exercise 4: Prove that the function ggt that is shown in Figure 3.3 computes the greatest common divisor of its arguments, i.e. prove that $\texttt{ggt}(a, b) = \texttt{gcd}(a, b)$ holds for all $a, b \in \mathbb{N}$. ◇

Exercise 5: The integer square root of a natural number n is defined as

$$\texttt{isqrt}(n) := \max\big(\{r \in \mathbb{N} \mid r^2 \leq n\}\big).$$

```
function ggt(a: number, b: number): number {
    if (b == 0) {
        return a;
    }
    return ggt(b, a % b);
}
```

Figure 3.3: The function ggt.

```
function root(n: bigint): bigint {
    if (n == 0n) {
        return 0n;
    }
    const r = root(n / 4n);
    if ((2n * r + 1n) * (2n * r + 1n) <= n) {
        return 2n * r + 1n;
    } else {
        return 2n * r;
    }
}
```

Figure 3.4: The function isqrt.

Prove that the function root that is shown in Figure 3.4 on page 27 computes the integer square root of its argument. ◇

Exercise 6: Figure 3.5 shows a *TypeScript* program implementing the function square. Show that the equation

$$\texttt{square}(n) = n^2$$

holds for every $n \in \mathbb{N}$. You should use computational induction to verify the claim. ◇

Exercise 7: The binomial coefficient $\binom{n}{k}$ is defined as follows:

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Figure 3.6 shows a *TypeScript* program implementing the function binomial. Prove that

$$\texttt{binomial}(n,k) = \binom{n}{k}$$

holds for all $n \in \mathbb{N}$ and all $k \in \{0, \cdots, n\}$. ◇

```
1  function square(n: number): number {
2      if (n == 0) {
3          return 0;
4      }
5      return square(n - 1) + 2 * n - 1;
6  }
```

Figure 3.5: The function `square`.

```
1  function binomial(n: number, k: number): number {
2      if (k == 0 || k == n) {
3          return 1;
4      } else {
5          return binomial(n - 1, k - 1) + binomial(n - 1, k);
6      }
7  }
```

Figure 3.6: The function `binomial`.

3.2 Symbolic Execution

In the previous section we saw how to prove the correctness of a recursive function via computational induction. If a function is instead implemented via loops, then the method of computational induction is not applicable. Instead, the method of symbolic execution is used. We introduce this method via two examples.

3.2.1 Iterative Squaring

In the previous section we implemented a recursive version of iterative squaring and verified its correctness via computational induction. In this section we implement an iterative version of iterative squaring and verify its correctness via symbolic execution. Consider the program shown in Figure 3.7.

The main difference between a mathematical formula and a program is that in a formula all occurrences of a variable refer to the same value. This is different in a program because the variables change their values dynamically. In order to deal with this property of program variables, we have to be able to distinguish the different occurrences of a given variable. To this end, we index the program variables. When doing this, we have to be aware of the fact that the same occurrence of a program variable can still denote different values if the variable occurs inside a loop. In this case we have to index the variables in a way such that the index includes a counter that counts the number of loop iterations. For concreteness, consider the program shown in Figure 3.7. Here, in line 5 the variable `r` has the index n on the right side of the assignment, while it has the index $n+1$ on the left side of the

```
1  function power(x0: bigint, y0: number): bigint {
2      let r0 = 1n;
3      while (yn > 0) {
4          if (yn % 2 == 1) {
5              rn+1 = rn * xn;
6          }
7          xn+1 = xn * xn;
8          yn+1 = Math.floor(yn / 2);
9      }
10     return rN;
11 }
```

Figure 3.7: An annotated program to compute powers.

assignment in line 5. The index n denotes the number of times that the test `y > 0` of the `while` loop has been executed. After the `while`-loop finishes, the variable `r` is indexed as $\texttt{r}_N$ in line 10, where N denotes the total number of times that the test `y > 0` has been executed. We show the correctness of the given program next. Let us define

$$a := x_0, \quad b := y_0.$$

We will show that, provided $a \geq 1$, the `while` loop satisfies the invariant

$$r_n \cdot x_n^{y_n} = a^b. \tag{3.7}$$

This claim is proven by induction on the number of loop iterations.

B.C.: $n = 0$.

Since we have $r_0 = 1$, $x_0 = a$, and $y_0 = b$ we have

$$r_n \cdot x_n^{y_n} = r_0 \cdot x_0^{y_0} = 1 \cdot a^b = a^b. \quad \surd$$

I.S.: $n \mapsto n + 1$.

Our proof proceeds by a case distinction with respect to the expression $y_n \,\%\, 2$:

(a) $y_n \,\%\, 2 = 1$.
Then we have $y_n = 2 \cdot \lfloor y_n / 2 \rfloor + 1$, $r_{n+1} = r_n \cdot x_n$ and $y_{n+1} = \lfloor y_n / 2 \rfloor$. Hence

$$
\begin{aligned}
& r_{n+1} \cdot x_{n+1}^{y_{n+1}} \\
= \;& (r_n \cdot x_n) \cdot (x_n \cdot x_n)^{\lfloor y_n/2 \rfloor} \\
= \;& r_n \cdot x_n^{2 \cdot \lfloor y_n/2 \rfloor + 1} \\
= \;& r_n \cdot x_n^{2 \cdot \lfloor y_n/2 \rfloor + y_n \,\%\, 2} \\
= \;& r_n \cdot x_n^{y_n} \\
\overset{i.h.}{=} \;& a^b \quad \surd
\end{aligned}
$$

(b) $y_n \,\%\, 2 = 0$.

Then we have $y_n = 2 \cdot \lfloor y_n/2 \rfloor$ and $r_{n+1} = r_n$. Therefore

$$
\begin{aligned}
& r_{n+1} \cdot x_{n+1}^{y_{n+1}} \\
= \;& r_n \cdot (x_n \cdot x_n)^{\lfloor y_n/2 \rfloor} \\
= \;& r_n \cdot x_n^{2 \cdot \lfloor y_n/2 \rfloor + 0} \\
= \;& r_n \cdot x_n^{2 \cdot \lfloor y_n/2 \rfloor + y_n \,\%\, 2} \\
= \;& r_n \cdot x_n^{y_n} \\
\overset{i.h.}{=} \;& a^b \quad \surd
\end{aligned}
$$

This shows the validity of equation (3.7). If the `while` loop terminates, we must have $y_N = 0$. If $n = N$, then equation (3.7) yields:

$$
\begin{aligned}
& r_N \cdot x_N^{y_N} = a^b \\
\Leftrightarrow \;& r_N \cdot x_N^0 = a^b \\
\Leftrightarrow \;& r_N \cdot 1 = a^b \qquad \text{because } x_N \neq 0 \\
\Leftrightarrow \;& r_N = a^b
\end{aligned}
$$

In the step from the second line to the third line we have used the fact that, since $x_0 \neq 0$, we also have that $x_n \neq 0$ for all $n \in \{0, 1, \cdots, N\}$ and therefore $x_N^0 = 1$. Hence we have shown that $r_N = a^b$. The `while` loop terminates because we have

$$y_{n+1} = \lfloor y_n/2 \rfloor < y_n \quad \text{as long as} \quad y_n > 0$$

and therefore y_n must eventually become 0. Thus we have proven that $\texttt{power}(a, b) = a^b$ holds. □

3.2.2 Integer Square Root

We continue with another example that demonstrates the method of symbolic execution. Figure 3.8 shows a function that is able to compute the integer square root of its argument a as long as a is less than $2^{128\,000}$, i.e. if a is represented as an unsigned number, then it can be represented with 128 000

bits. In the implementation of the function `root` we have already indexed the variables. Note that there is no need to index the variable a since this variable is never updated. To prove the correctness of this program, we first have to establish invariants for the variables p_k and r_k. In order to be able to formulate the invariant for the variable `r`, we have to understand that the function `root` computes the integer square root of its argument a bit by bit starting at the most significant bit. Let us denote the mathematical function that computes the integer square root of a number a with `isqrt`. If we represent $\texttt{isqrt}(a)$ in the binary system, we have

$$\texttt{isqrt}(a) = \sum_{i=0}^{n} b_i \cdot 2^i, \quad \text{where } b_i \in \{0,1\}$$

and $n+1$ is the number of bits that are needed to represent $\texttt{isqrt}(a)$. Note that then the binary representation of the number a needs at most twice as many bits. In the program below we have $n+1 = 64\,000$. Therefore, we assume that the argument a of `isqrt` is less than or equal to $2^{2\cdot(n+1)}$, i.e. we assume that

$$a \leq 2^{128\,000},$$

since then we are sure that we can represent $\texttt{isqrt}(a)$ with $64,000$ bits. In order to compute $\texttt{isqrt}(a)$ we start with the last bit b_n. If the square of 2^n is less than or equal to a, then we must have $b_n = 1$. Otherwise we have $b_n = 0$. Assume now that we have already computed the bits $b_n, b_{n-1}, \cdots, b_{n-k}$. In order to compute $b_{n-(k+1)}$ we have to check whether

$$\left(2^{n-(k+1)} + \sum_{i=n-k}^{n} b_i \cdot 2^i\right)^2 \leq a.$$

If it is, then $b_{n-(k+1)} = 1$, else $b_{n-(k+1)} = 0$. With this in mind, we can state the invariants that hold when the `while` loop in line 4 is entered:

1. $p_k = 2^{n-k} \quad$ for $k = 0, \cdots, n$.
2. $r_k = \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i} \quad$ where the b_i are the bits of the binary representation of $\texttt{isqrt}(a)$.

We proceed to prove the first invariant $p_k = 2^{n-k}$ by induction.

B.C.: $k = 0$

$p_0 = 2^{64\,000} = 2^n = 2^{n-0}$.

I.S.: $k \mapsto k+1$

$p_{k+1} = \lfloor p_k/2 \rfloor \overset{IV}{=} \lfloor 2^{n-k}/2 \rfloor = 2^{n-(k+1)}$.

This holds as long as $k+1 \leq n$. Since we have $p_n = 2^{n-n} = 2^0 = 1$ it follows that

$$p_{n+1} = \lfloor p_n/2 \rfloor = \lfloor 1/2 \rfloor = 0.$$

This shows that the body of the `while` loop is executed $n+1$ times. This was to be expected, as each execution of this loop computes one bit of the result.

We proceed to prove the second invariant $r_k = \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i}$ by induction.

```
1  function root(a: bigint): bigint {
2      let r_0 = 0n;
3      let p_0 = 2n ** 64_000n;
4      while (p_k > 0) {
5          if (a >= (r_k + p_k) ** 2n) {
6              r_{k+1} = r_k + p_k;
7          }
8          p_{k+1} = p_k / 2n;
9      }
10     return r_K;
11 }
```

Figure 3.8: An annotated program to compute the integer square root.

B.C.: $k = 0$

We have $r_0 = 0$ and also

$$\sum_{i=0}^{0-1} b_{n-i} \cdot 2^{n-i} = 0,$$

since the sum $\sum_{i=0}^{-1} \cdots$ is empty and hence has the value 0.

I.S.: $k \mapsto k+1$

Now we need to perform a case distinction with respect to the test $a \geq (r_k + p_k)^2$.

(a) $a \geq (r_k + p_k)^2$.

In this case we have that $r_k + p_k \leq \texttt{isqrt}(a)$ and since $p_k = 2^{n-k}$ this shows that

$$r_k + 2^{n-k} \leq \texttt{isqrt}(a).$$

This implies that $b_{n-k} = 1$. Therefore we have

$$\begin{aligned} r_{k+1} &= r_k + p_k \\ &\overset{IV}{=} \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i} + 2^{n-k} \\ &= \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i} + b_{n-k} \cdot 2^{n-k} \\ &= \sum_{i=0}^{k} b_i \cdot 2^i \end{aligned}$$

(b) $a < (r_k + p_k)^2$.

In this case we have that $r_k + p_k > \texttt{isqrt}(a)$ and since $p_k = 2^{n-k}$ this shows that

$$r_k + 2^{n-1} > \texttt{isqrt}(a).$$

This implies that $b_{n-k} = 0$. Therefore, we have

$$\begin{aligned} r_{n+1} &= r_k \\ &\overset{IV}{=} \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i} \\ &= \sum_{i=0}^{k-1} b_{n-i} \cdot 2^{n-i} + 0 \cdot 2^{n-k} \\ &= \sum_{i=0}^{k-1} b_{n-1} \cdot 2^{n-i} + b_k \cdot 2^{n-k} \\ &= \sum_{i=0}^{k} b_i \cdot 2^i \end{aligned}$$

Since the program terminates when $p_k = 0$ and this is the case when $k = n + 1$ we have

$$\texttt{root}(a) = r_K = r_{n+1} = \sum_{i=0}^{n} b_{n-i} \cdot 2^{n-i} = \sum_{i=0}^{n} b_i \cdot 2^i = \texttt{isqrt}(a). \qquad \square$$

Exercise 8: Use the method of symbolic program execution to prove the correctness of the implementation of the Euclidean algorithm that is shown in Figure 3.9 on page 33. During the proof you should make use of the fact that for all positive natural numbers x and y the function `gcd` that computes the greatest common divisor of x and y satisfies the equation

$$\texttt{gcd}(x, y) = \texttt{gcd}(y, x \,\%\, y).$$

Furthermore, the invariant of the `while` loop is

$$\texttt{gcd}(x_n, y_n) = \texttt{gcd}(a, b) \quad \text{where } a := x_0 \text{ and } b := y_0.$$

Using this invariant you should be able to prove that $\texttt{ggt}(a, b) = \texttt{gcd}(a, b)$ for all $a, b \in \mathbb{N}$ such that $a > 0$. Note that in order to carry out the proof you have to distinguish between the mathematical function `gcd` that computes the greatest common divisor and the *TypeScript* function `ggt` that is implemented in Figure 3.9. $\diamond$

```
function ggt(x0: number, y0: number): number {
    while (yn != 0) {
        [xn+1, yn+1] = [yn, xn % yn];
    }
    return xN;
}
```

Figure 3.9: An iterative implementation of the Euclidean algorithm.

Exercise 9: Figure 3.10 shows a *TypeScript* program implementing the function `square`. Show that the equation

$$\texttt{square}(s) = s^2$$

holds for every $s \in \mathbb{N}$. You should use symbolic execution to verify the claim. ◇

```
function square(s₀: number): number {
    let r₀ = 0;
    let u₀ = 1;
    while (sₙ > 0) {
        rₙ₊₁ += uₙ;
        uₙ₊₁ += 2;
        sₙ₊₁ -= 1;
    }
    return r_N;
}
```

Figure 3.10: The function `square` with annotations.

3.3 Check Your Understanding

(a) Explain the method of computational induction.

(b) What are the three steps that have to be executed to prove the correctness of the implementation of a *TypeScript* function via computational induction?

(c) What are the two properties of Euclidean division of natural numbers?

(d) Are you able to prove the correctness of a recursive implementation of Euclid's algorithm?

(e) What is the difference between a variable occurring in a mathematical formula and a variable occurring in a *TypeScript* program?

(f) How do we transform a variable occurring in a *TypeScript* program into a mathematical variable?

(g) Explain the method of symbolic execution.

(h) Are you able to prove the correctness of an iterative implementation of Euclid's algorithm?

(i) When would you use computational induction and when would you choose symbolic execution instead?

Chapter 4

Propositional Calculus

4.1 Introduction

Propositional calculus (also known as propositional logic) deals with the connection of propositions (a.k.a. simple statements) through logical connectives. Here, logical connectives are words like "and", "or", "not", "if $\cdots$, then", and "exactly if". To start with, we define the notion of an atomic proposition: An atomic proposition is a sentence that

- expresses a fact that is either `true` or `false` and
- that does not contain any logical connectives.

  This last property is the reason for calling the proposition atomic.

Examples of atomic propositions are the following:

1. "*The sun is shining*"
2. "*It is raining.*"
3. "*There is a rainbow in the sky.*"

Atomic propositions can be combined by means of logical connectives into composite propositions. An example of a composite proposition would be

If the sun is shining and it is raining, then there is a rainbow in the sky. (1)

This statement is composed of the three atomic propositions

- "*The sun is shining.*",
- "*It is raining.*", and
- "*There is a rainbow in the sky.*"

using the logical connectives "*and*" and "*if $\cdots$, then*". Propositional calculus investigates how the truth value of composite statements is calculated from the truth values of the propositions. Furthermore, it investigates how new statements can be derived from given statements.

In order to analyze the structure of complex statements we introduce propositional variables. These propositional variables are just names that denote atomic propositions. Furthermore, we introduce symbols serving as mathematical operators for the logical connectives "not", "and", "or", "if $\cdots$, then", and "if and only if".

1. $\neg a$ is read as not a
2. $a \wedge b$ is read as a and b
3. $a \vee b$ is read as a or b
4. $a \rightarrow b$ is read as if a, then b
5. $a \leftrightarrow b$ is read as a if and only if b

Propositional formulas are built from propositional variables using the propositional operators shown above and can have an arbitrary complexity. Using propositional operators, the statement (1) can be written as follows:

$$\texttt{sunny} \wedge \texttt{rainy} \rightarrow \texttt{rainBow}.$$

Here, we have used `sunny`, `rainy` and `rainBow` as propositional variables.

Some propositional formulas are always `true`, no matter how the propositional variables are interpreted. For example, the propositional formula

$$p \vee \neg p$$

is always `true`, it does not matter whether the proposition denoted by p is `true` or `false`. A propositional formula that is always `true` is known as a tautology. There are also propositional formulas that are never `true`. For example, the propositional formula

$$p \wedge \neg p$$

is always `false`. A propositional formula is called satisfiable, if there is at least one way to assign truth values to the variables such that the formula is `true`. Otherwise the formula is called unsatisfiable. In this lecture we will discuss a number of different algorithms to check whether a formula is satisfiable. These algorithms are very important in a number of industrial applications. For example, a very important application is the design of digital circuits. Furthermore, a number of logic puzzles can be translated into propositional formulas and finding a solution to these puzzles amounts to checking the satisfiability of these formulas. For example, we will solve the eight queens puzzle in this way.

The rest of this chapter is structured as follows:

1. We list several applications of propositional logic.
2. We define the notion of propositional formulas, i.e. we define the set of strings that are propositional formulas.

   This is known as the syntax of propositional formulas.
3. Next, we discuss the evaluation of propositional formulas and implement the evaluation in *TypeScript*.

   This is known as the semantics of propositional formulas.
4. Then we formally define the notions tautology and satisfiability for propositional formulas.

5. We discuss algebraic manipulations of propositional formulas and introduce the conjunctive normal form.

   Some algorithms discussed later require that the propositional formulas have conjunctive normal form.

6. After that we discuss the concept of a logical derivation. The purpose of a logical derivation is to derive new formulas from a given set of formulas.

7. Finally, we discuss the Davis-Putnam algorithm for checking the satisfiability of a set of propositional formulas. As an application, we solve the eight queens puzzle using this algorithm.

4.2 Applications of Propositional Logic

Propositional logic is not only the basis of first order logic, but it also has important practical applications. As there are many different applications of propositional logic, I will only list those applications which I have seen myself during the years when I did work in industry.

1. Analysis and design of electronic circuits.

   Modern digital circuits are comprised of hundreds of billions of logical gates.[1] A logical gate is a building block, that represents a logical connective such as "and", "or", "not" as an electronic circuit.

   The complexity of modern digital circuits would be unmanageable without computer-aided verification methods. The methods used are applications of propositional logic. A very concrete application is circuit comparison. Here two digital circuits are represented as propositional formulas. Afterwards, one tries to show the equivalence of these formulas by means of propositional logic. Software tools that are used for the verification of digital circuits sometimes cost more than 100 000 $. For example, the company Magma offers the equivalence checker Quartz Formal at a price of 150 000 $ per license. Such a license is then valid for three years.

2. Controlling the signals and switches of railroad stations.

   At a large railway station, there are several hundred switches and signals that have to be reset all the time to provide routes for the trains. For safety reasons, different routes must not cross each other. The individual routes are described by so-called closure plans. The correctness of these closure plans can be analyzed via propositional formulas.

3. A number of logical puzzles can be coded as propositional formulas and can then be solved with the algorithm of Davis and Putnam. For example, we will discuss the eight queens puzzle in this lecture. This puzzle asks to place eight queens on a chess board such that no two queens can attack each other.

[1] The web page `https://en.wikipedia.org/wiki/Transistor_count` gives an overview of the complexity of modern processors.

4.3 The Formal Definition of Propositional Formulas

In this section we first cover the syntax of propositional formulas. After that, we discuss their semantics. The syntax defines the way in which we represent formulas as strings and how we can combine formulas into a proof. The semantics of propositional logic is concerned with the meaning of propositional formulas. We will first define the semantics of propositional logic with the help of set theory. Then, we implement this semantics in *TypeScript*.

4.3.1 The Syntax of Propositional Formulas

We define propositional formulas as strings. To this end we assume a set $\mathcal{P}$ of so-called propositional variables as given. Typically, $\mathcal{P}$ is the set of all lower case Latin characters, which additionally may be indexed. For example, we will use

$$p, q, r, p_1, p_2, p_3$$

as propositional variables. Then, propositional formulas are strings that are formed from the alphabet

$$\mathcal{A} := \mathcal{P} \cup \{\top, \bot, \neg, \vee, \wedge, \rightarrow, \leftrightarrow, (,)\}.$$

We define the set $\mathcal{F}$ of propositional formulas by induction:

1. $\top \in \mathcal{F}$ and $\bot \in \mathcal{F}$.

   Here $\top$ denotes the formula that is always `true`, while $\bot$ denotes the formula that is always `false`. The formula $\top$ is called "verum"[2], while $\bot$ is called "falsum"[3].

2. If $p \in \mathcal{P}$, then $p \in \mathcal{F}$.

   Every propositional variable is also a propositional formula.

3. If $f \in \mathcal{F}$, then $\neg f \in \mathcal{F}$.

   The formula $\neg f$ (read: not f) is called the negation of f.

4. If $f_1, f_2 \in \mathcal{F}$, then we also have

$(f_1 \vee f_2) \in \mathcal{F}$	(*read*:	f_1 or f_2	also: disjunction	of f_1 and f_2),
$(f_1 \wedge f_2) \in \mathcal{F}$	(*read*:	f_1 and f_2	also: conjunction	of f_1 and f_2),
$(f_1 \rightarrow f_2) \in \mathcal{F}$	(*read*:	if f_1, then f_2	also: implication	of f_1 and f_2),
$(f_1 \leftrightarrow f_2) \in \mathcal{F}$	(*read*:	f_1 if and only if f_2	also: biconditional	of f_1 and f_2).

The set $\mathcal{F}$ of propositional formulas is the smallest set of those strings formed from the characters in the alphabet $\mathcal{A}$ that has the closure properties given above.

Example: Assume that $\mathcal{P} := \{p, q, r\}$. Then we have the following:

1. $p \in \mathcal{F}$,
2. $(p \wedge q) \in \mathcal{F}$,

[2] "verum" is the Latin word for "`true`".
[3] "falsum" is the Latin word for "`false`".

3. $\Big(\big((\neg p \to q) \vee (q \to \neg p) \big) \to r \Big) \in \mathcal{F}$. □

In order to save parentheses we agree on the following rules:

1. Outermost parentheses are dropped. Therefore, we write
   $$p \wedge q \quad \text{instead of} \quad (p \wedge q).$$
2. The negation operator $\neg$ has a higher precedence than all other operators.
3. The operators $\vee$ and $\wedge$ associate to the left. Therefore, we write
   $$p \wedge q \wedge r \quad \text{instead of} \quad (p \wedge q) \wedge r.$$
4. **In this lecture** the logical operators $\wedge$ and $\vee$ have the same precedence. This is different from the programming language *TypeScript*. In the programming languages *TypeScript*, `C`, and *Java* the operator "`&&`" has a higher precedence than the operator "`||`".
5. The operator $\to$ is right associative, i.e. we write
   $$p \to q \to r \quad \text{instead of} \quad p \to (q \to r).$$
6. The operators $\vee$ and $\wedge$ have a higher precedence than the operator $\to$. Therefore, we write
   $$p \wedge q \to r \quad \text{instead of} \quad (p \wedge q) \to r.$$
7. The operator $\to$ has a higher precedence than the operator $\leftrightarrow$. Therefore, we write
   $$p \to q \leftrightarrow r \quad \text{instead of} \quad (p \to q) \leftrightarrow r.$$
8. You should note that the operator $\leftrightarrow$ neither associates to the left nor to the right. Therefore, the expression
   $$p \leftrightarrow q \leftrightarrow r$$
   is **ill-defined** and has to be parenthesised. If you encounter this type of expression in a book it is usually meant as an abbreviation for the expression
   $$(p \leftrightarrow q) \wedge (q \leftrightarrow r).$$
   We will not use this kind of abbreviation.

Remark: Later, we will conduct a series of proofs that prove mathematical statements about formulas. In these proofs we will make use of propositional connectives. In order to distinguish these connectives from the connectives of propositional logic we agree on the following:

1. Inside a propositional formula, the propositional connective "not" is written as "$\neg$".
   When we prove a statement about propositional formulas, we use the word "`not`" instead.
2. Inside a propositional formula, the propositional connective "and" is written as "$\wedge$".
   When we prove a statement about propositional formulas, we use the word "`and`" instead.
3. Inside a propositional formula, the propositional connective "or" is written as "$\vee$".
   When we prove a statement about propositional formulas, we use the word "`or`" instead.

4. Inside a propositional formula, the propositional connective "if $\cdots$, then" is written as "$\rightarrow$". When we prove a statement about propositional formulas, we use the symbol "$\Rightarrow$" instead.
5. Inside a propositional formula, the propositional connective "if and only if" is written as "$\leftrightarrow$". When we prove a statement about propositional formulas, we use the symbol "$\Leftrightarrow$" instead. $\diamond$

4.3.2 Semantics of Propositional Formulas

In this section we define the meaning a.k.a. the semantics of propositional formulas. To this end we assign truth values to propositional formulas. First, we define the set $\mathbb{B}$ of truth values:

$$\mathbb{B} := \{\texttt{true}, \texttt{false}\}.$$

Next, we define the notion of a propositional valuation.

Definition 7 (Propositional Valuation) *A propositional valuation is a function*

$$\mathcal{I} : \mathcal{P} \to \mathbb{B},$$

that maps the propositional variables $p \in \mathcal{P}$ to truth values $\mathcal{I}(p) \in \mathbb{B}$. $\diamond$

A propositional valuation $\mathcal{I}$ maps only the propositional variables to truth values. In order to map propositional formulas to truth values we need to interpret the propositional operators "$\neg$", "$\wedge$", "$\vee$", "$\rightarrow$", and "$\leftrightarrow$" as functions on the set $\mathbb{B}$. To this end we define the functions $\ominus$, $\oslash$, $\otimes$, $\oplus$, and $\ominus$. These functions have the following signatures:

1. $\ominus\colon \mathbb{B} \to \mathbb{B}$
2. $\oslash\colon \mathbb{B} \times \mathbb{B} \to \mathbb{B}$
3. $\otimes\colon \mathbb{B} \times \mathbb{B} \to \mathbb{B}$
4. $\oplus : \mathbb{B} \times \mathbb{B} \to \mathbb{B}$
5. $\ominus : \mathbb{B} \times \mathbb{B} \to \mathbb{B}$

We will use these functions as the valuations of the propositional operators. It is easiest to define the functions $\ominus$, $\oslash$, $\otimes$, $\oplus$, and $\ominus$ via the following truth table (Table 4.1):

p	q	$\ominus(p)$	$\otimes(p,q)$	$\oslash(p,q)$	$\oplus(p,q)$	$\ominus(p,q)$
true	true	false	true	true	true	true
true	false	false	true	false	false	false
false	true	true	true	false	true	false
false	false	true	false	false	true	true

Table 4.1: Interpretation of the propositional operators.

Then the truth value of a propositional formula f under a given propositional valuation $\mathcal{I}$ is defined by induction on f. We will denote the truth value as $\widehat{\mathcal{I}}(f)$. We have

1. $\widehat{\mathcal{I}}(\bot) := \texttt{false}$.

2. $\widehat{\mathcal{I}}(\top) := \texttt{true}$.
3. $\widehat{\mathcal{I}}(p) := \mathcal{I}(p)$ for all $p \in \mathcal{P}$.
4. $\widehat{\mathcal{I}}(\neg f) := \ominus\big(\widehat{\mathcal{I}}(f)\big)$ for all $f \in \mathcal{F}$.
5. $\widehat{\mathcal{I}}(f \wedge g) := \oslash\big(\widehat{\mathcal{I}}(f), \widehat{\mathcal{I}}(g)\big)$ for all $f, g \in \mathcal{F}$.
6. $\widehat{\mathcal{I}}(f \vee g) := \otimes\big(\widehat{\mathcal{I}}(f), \widehat{\mathcal{I}}(g)\big)$ for all $f, g \in \mathcal{F}$.
7. $\widehat{\mathcal{I}}(f \to g) := \ominus\big(\widehat{\mathcal{I}}(f), \widehat{\mathcal{I}}(g)\big)$ for all $f, g \in \mathcal{F}$.
8. $\widehat{\mathcal{I}}(f \leftrightarrow g) := \ominus\big(\widehat{\mathcal{I}}(f), \widehat{\mathcal{I}}(g)\big)$ for all $f, g \in \mathcal{F}$.

In order to simplify the notation we will not distinguish between the function

$$\widehat{\mathcal{I}} : \mathcal{F} \to \mathbb{B}$$

that is defined on all propositional formulas and the function

$$\mathcal{I} : \mathcal{P} \to \mathbb{B}.$$

Hence, from here on we will write $\mathcal{I}(f)$ instead of $\widehat{\mathcal{I}}(f)$.
Example: We show how to compute the truth value of the formula

$$(p \to q) \to (\neg p \to q) \to q$$

for the propositional valuation

$$\mathcal{I} := \{p \mapsto \texttt{true}, q \mapsto \texttt{false}\}.$$

$$\begin{aligned}
\mathcal{I}\Big((p \to q) \to (\neg p \to q) \to q\Big) &= \ominus\Big(\mathcal{I}\big((p \to q)\big), \mathcal{I}\big((\neg p \to q) \to q\big)\Big) \\
&= \ominus\Big(\ominus\big(\mathcal{I}(p), \mathcal{I}(q)\big), \mathcal{I}\big((\neg p \to q) \to q\big)\Big) \\
&= \ominus\Big(\ominus\big(\texttt{true}, \texttt{false}\big), \mathcal{I}\big((\neg p \to q) \to q\big)\Big) \\
&= \ominus\Big(\texttt{false}, \mathcal{I}\big((\neg p \to q) \to q\big)\Big) \\
&= \texttt{true}
\end{aligned}$$

◇

Note that we only evaluated some parts of the formula. The reason is that as soon as we know that the first argument of $\ominus$ is `false` the value of the corresponding formula is already determined. Nevertheless, this approach is too cumbersome. In practice, we do not evaluate large propositional formulas ourselves. Instead, we will next implement a *TypeScript* program that evaluates these formulas for us.

4.3.3 Implementation

In this section we develop a *TypeScript* program that can evaluate propositional formulas. Every time we develop a program to compute something useful we have to decide which data structures are most appropriate to represent the information that is to be processed by the program. In this case we want to process propositional formulas. Therefore, we have to decide how to represent propositional formulas in *TypeScript*. One obvious possibility would be to use strings. However,

this would be a bad choice as it would then be difficult to access the parts of a given formula. It is far more suitable to represent propositional formulas as nested lists. A nested list is a list that contains both strings and nested lists. For example,

```
['∧', ['¬', 'p'], 'q']
```

is a nested list that represents the propositional formula $\neg p \wedge q$.

Formally, the representation of propositional formulas is defined by a function

$$\mathit{rep} : \mathcal{F} \to \mathit{TypeScript}$$

that maps a propositional formula f to the corresponding nested list $\mathit{rep}(f)$. We define $\mathit{rep}(f)$ inductively by induction on f.

1. $\top$ is represented as the list `['⊤']`.

   This is possible because `'⊤'` is a unicode symbol and *TypeScript* supports the use of unicode symbols in strings. Therefore, we have

   $$\mathit{rep}(\top) := \texttt{['⊤']}.$$

2. $\bot$ is represented as the list `['⊥']`. Therefore, we have

   $$\mathit{rep}(\bot) := \texttt{['⊥']}.$$

3. Since propositional variables are strings we can represent these variables by themselves:

   $$\mathit{rep}(p) := p \quad \text{for all } p \in \mathcal{P}.$$

4. If f is a propositional formula, the negation $\neg f$ is represented as a pair where we put the unicode symbol `'¬'` at the first position, while the representation of f is put at the second position. We have

   $$\mathit{rep}(\neg f) := \texttt{['¬'}, \mathit{rep}(f)\texttt{]}.$$

5. If f_1 and f_2 are propositional formulas, we represent $f_1 \wedge f_2$ with the help of the unicode symbol `'∧'`. This symbol has the name "`logical and`". Hence we have

   $$\mathit{rep}(f \wedge g) := \texttt{['∧'}, \mathit{rep}(f), \mathit{rep}(g)\texttt{]}.$$

6. If f_1 and f_2 are propositional formulas, we represent $f_1 \vee f_2$ with the help of the unicode symbol `'∨'`. Hence we have

   $$\mathit{rep}(f \vee g) := \texttt{['∨'}, \mathit{rep}(f), \mathit{rep}(g)\texttt{]}.$$

7. If f_1 and f_2 are propositional formulas, we represent $f_1 \to f_2$ with the help of the unicode symbol `'→'`. Hence we have

   $$\mathit{rep}(f \to g) := \texttt{['→'}, \mathit{rep}(f), \mathit{rep}(g)\texttt{]}.$$

8. If f_1 and f_2 are propositional formulas, we represent $f_1 \leftrightarrow f_2$ with the help of the unicode symbol `'↔'`. Hence we have

   $$\mathit{rep}(f \leftrightarrow g) := \texttt{['↔'}, \mathit{rep}(f), \mathit{rep}(g)\texttt{]}.$$

When choosing the representation of a formula in *TypeScript* we have a lot of freedom. We could as well have represented formulas as objects. A good representation should have the following properties:

1. It should be intuitive, i.e. we do not want to use any obscure encoding.
2. It should be adequate.
   (a) It should be easy to recognize whether a formula is a propositional variable, a negation, a conjunction, etc.
   (b) It should be easy to access the components of a formula.
   (c) Given a formula f, it should be easy to generate the representation of f.
3. It should be memory efficient.

A propositional valuation is a function

$$\mathcal{I} : \mathcal{P} \to \mathbb{B}$$

mapping the set of propositional variables $\mathcal{P}$ into the set of truth values $\mathbb{B} = \{\texttt{true}, \texttt{false}\}$. We represent a propositional valuation $\mathcal{I}$ as the set of all propositional variables that are mapped to `true` by $\mathcal{I}$:

$$\mathit{rep}(\mathcal{I}) := \{x \in \mathcal{P} \mid \mathcal{I}(x) = \texttt{true}\}.$$

This enables us to implement a simple function that evaluates a propositional formula f with a given propositional valuation $\mathcal{I}$. The *TypeScript* function `evaluate` is shown in Figure 4.1 on page 44.

1. First we define the type that we are going to use.
   (a) A `Variable` is just a `string`.
   (b) A `Formula` is either a `Variable` or a nested list as discussed above.
2. The function `evaluate` takes two arguments.
   (a) The first argument F is a propositional formula that is represented as a nested list.
   (b) The second argument I is a propositional valuation. This valuation is represented as a set of propositional variables. Here, we use the type `RecursiveSet` from the library `recursive-set`. For conciseness, we abbreviate this type as `Set`. In order to do so, we import the type `RecursiveSet` as follows:

       ```
       import RecursiveSet as Set from 'recursive-set';
       ```

       Given a propositional variable p, the value of $\mathcal{I}(p)$ is then computed by the expression $\mathcal{I}.\texttt{has}(p)$.

Next, we discuss the implementation of the function `evaluate()`.

1. Line 8 deals with the case that the argument f is a propositional variable. We can recognize this by the fact that f is a string, which we can check with the operator `typeof`.

   In this case we have to check whether the variable p is an element of the set I, because p is interpreted as `true` if and only if $p \in I$. The method $I.\texttt{has}(x)$ takes a set I and an element x and checks whether x is an element of I.
2. If f is $\top$, then evaluating f yields `true`.
3. If f is $\bot$, then evaluating f yields `false`.

```typescript
type Variable = string;
type Formula = Variable
             | ['⊤' | '⊥']
             | ['¬', Formula]
             | ['↔' | '→' | '∧' | '∨', Formula, Formula];

function evaluate(f: Formula, I: Set<Variable>): boolean {
    if (typeof f == 'string') { return I.has(f); }
    switch (f[0]) {
        case '⊤': return true;
        case '⊥': return false;
        case '¬': { const [_, g] = f;
                    return !evaluate(g, I); }
        default:  {
            const [op, g, h] = f;
            const [gs, hs] = [g, h].map(e => evaluate(e, I));
            switch (op) {
                case '∧': return gs && hs;
                case '∨': return gs || hs;
                case '→': return gs <= hs;
                case '↔': return gs == hs;
            }
        }
    }
}
```

Figure 4.1: Evaluation of a propositional formula.

4. If f has the form $\neg g$, we recursively evaluate g and negate the result.

5. If f has the form $g \odot h$, where $\odot$ is some binary operator, i.e. $\odot \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$ we recursively evaluate g and h. The resulting Boolean values are stored in gs and hs and are then combined according to the operator $\odot$.

   (a) If $\odot = \wedge$ we can use the *TypeScript* operator "`&&`" to combine gs and hs.

   (b) If $\odot = \vee$ we can use the *TypeScript* operator "`||`" to combine gs and hs.

   (c) If $\odot = \rightarrow$ then we exploit the fact that in *TypeScript* the values `true` and `false` are ordered and we have

   `false < true`.

   Now the evaluation of the formula $g \rightarrow h$ is only `false` if the evaluation of g yields `true` but the evaluation of h yields `false`. Therefore,

   `evaluate`$(g \rightarrow h, \mathrm{I})$ has the same value as `evaluate`$(g, I) \leq$ `evaluate`(h, I).

Hence we combine *gs* and *hs* using the *less-or-equal* operator "<=".

(d) If $\odot = \leftrightarrow$ we exploit the fact that the formula $g \leftrightarrow h$ is `true` if and only if g and h have the same truth value. Hence we can use the *TypeScript* operator "==" to combine *gs* and *hs*.

4.3.4 An Application

Next, we showcase a playful application of propositional logic. Inspector Watson is called to investigate a burglary at a jewelry store. Three suspects have been detained in the vicinity of the jewelry store. Their names are Aaron, Bernard, and Caine. The evaluation of the files reveals the following facts.

1. At least one of these suspects must have been involved in the crime.

   If the propositional variable a is interpreted as claiming that Aaron is guilty, while b and c stand for the guilt of Bernard and Caine respectively, then this statement is captured by the following formula:

   $$f_1 := a \vee b \vee c.$$

2. If Aaron is guilty, then he has exactly one accomplice.

   To formalize this statement, we decompose it into two statements.

   (a) If Aaron is guilty, then he has at least one accomplice.

   $$f_2 := a \rightarrow b \vee c$$

   (b) If Aaron is guilty, then he has at most one accomplice.

   $$f_3 := a \rightarrow \neg(b \wedge c)$$

3. If Bernard is innocent, then Caine is innocent too.

   $$f_4 := \neg b \rightarrow \neg c$$

4. If exactly two of the suspects are guilty, then Caine is one of them.

   It is not straightforward to translate this statement into a propositional formula. One trick we can try is to negate the statement and then try to translate the negation. Now the statement given above is wrong if and only if Caine is innocent but both Aaron and Bernard are guilty. Therefore we can translate this statement as follows:

   $$f_5 := \neg(\neg c \wedge a \wedge b)$$

5. If Caine is innocent, then Aaron is guilty.

   This translates into the following formula:

   $$f_6 := \neg c \rightarrow a$$

We now have a set $F = \{f_1, f_2, f_3, f_4, f_5, f_6\}$ of propositional formulas. The question then is to find all propositional valuations $\mathcal{I}$ that evaluate all formulas from F as `true`. If there is exactly one such propositional valuation, then this valuation gives us the culprits. As it is too time consuming to try all possible valuations by hand we will write a program that performs the required computations. Figure 4.2 shows the program `02-Usual-Suspects.ipynb`. We discuss this program next.

```
import { RecursiveSet as Set, Value } from 'recursive-set';

function set<T extends Value>(...elements: T[]): Set<T> {
    return new Set(...elements);
}

import { parse } from './PropositionalLogicParser'

const P = set('a', 'b', 'c');

const f1 = 'a ∨ b ∨ c';
const f2 = 'a → b ∨ c';
const f3 = 'a → ¬(b ∧ c)';
const f4 = '¬b → ¬c';
const f5 = '¬(¬c ∧ a ∧ b)';
const f6 = '¬c → a';
const Fs = [f1, f2, f3, f4, f5, f6];

type Variable = string;
type Formula = Variable
             | ['⊤' | '⊥']
             | ['¬', Formula]
             | ['↔' | '→' | '∧' | '∨', Formula, Formula];

const Gs = Fs.map(f => parse(f));

function allTrue(Gs: Formula[], I: Set<Variable>): boolean {
    return Gs.every(f => evaluate(f, I));
}

const validAssignments = P.powerset().filter(I => allTrue(Gs, I));
```

Figure 4.2: A program to investigate the burglary.

1. We import the type `RecursiveSet` from the library `recursive-set` and abbreviate this type as `Set`.

2. Since it is rather tedious to write `new Set(···)` whenever we want to create a new set, we define the function `set`. We will use this function in all of the following programs.

3. We input propositional formulas as strings. However, the function `evaluate` needs nested lists as input. Therefore, we first import our parser for propositional formulas.

4. We define the set P of propositional variables. We use the propositional variable `a` to express that Aaron is guilty, `b` is short for Bernard is guilty and `c` is `true` if and only if Caine is guilty.

5. Next, we define the propositional formulas $f_1, \cdots, f_6$ as strings.
6. `Fs` is the list of all these propositional formulas.
7. These formulas are transformed into nested lists.
8. The function `allTrue`(Fs, I) takes two inputs.
   (a) `Fs` is a set of propositional formulas that are represented as nested lists.
   (b) `I` is a propositional valuation that is represented as a set of propositional variables. Hence `I` is a subset of `P`

   If all propositional formulas f from the set `Fs` evaluate as `true` given the valuation `I`, then `allTrue` returns the result `true`, otherwise `false` is returned.
9. Finally we compute the set of all propositional variables that render all formulas from `Fs` `true`. The function `powerset` takes a set M and returns the power set of M, i.e. it returns the set 2^M.

When we run this program we see that there is just a single propositional valuation I such that all formulas from `Fs` are rendered `true` under I. This propositional valuation has the form

```
{'b', 'c'}.
```

Thus, the given problem is solvable and both Bernard and Caine are guilty, while Aaron is innocent.

Exercise 10: The following puzzle appeared in the book *Logical Deduction Puzzles* [Sum06]. Solve this puzzle by editing the notebook

Chapter-4/Murder-Frame.ipynb.

We all know Alice and Bob from Cryptography. However, little did we know about their difficult family relations, which might be the reason for their secrecy.
Unfortunately, things have escalated recently: Murder occurred one evening in the home of their father and mother. Alice and Bob are the only children of their parents. One member of the family murdered another member, the third member witnessed the crime, and the fourth member was an accessory after the fact. Furthermore, we know the following:

1. The accessory and the witness were of opposite sex.
2. The oldest member and the witness were of opposite sex.
3. The youngest member and the victim were of opposite sex.
4. The accessory was older than the victim.
5. The father was the oldest member.
6. The murderer was not the youngest member.

Question: Which of the four — the father, the mother, Alice, or Bob — was the murderer? ◇

4.4 Tautologies

Using the program to evaluate a propositional formula we can see that the formula

$$(p \to q) \to (\neg p \to q) \to q$$

is `true` for every propositional valuation $\mathcal{I}$. This property gives rise to a definition.

Definition 8 (Tautology) *If f is a propositional formula and we have*

$$\mathcal{I}(f) = \texttt{true} \quad \text{for every propositional valuation } \mathcal{I},$$

then f is a tautology. This is written as

$$\models f.$$ ◇

If f is a tautology, then we say that f is universally valid .

Examples:

1. $\models p \vee \neg p$
2. $\models p \to p$
3. $\models p \wedge q \to p$
4. $\models p \to p \vee q$
5. $\models (p \to \bot) \leftrightarrow \neg p$
6. $\models p \wedge q \leftrightarrow q \wedge p$

One way to prove that a formula F is universally valid is to evaluate it for every possible valuation. However, if there are n propositional variables occurring in f, then there are 2^n different possible valuations. Hence for values of n that are greater than thirty this method is hopelessly inefficient. Therefore, our goal in the rest of this chapter is to develop a method that can often deal with hundreds of propositional variables.

Definition 9 (Equivalent) *Two formulas f and g are equivalent if and only if*

$$\models f \leftrightarrow g.$$ ◇

Examples: We have the following equivalences:

$\models \neg\bot \leftrightarrow \top$	$\models \neg\top \leftrightarrow \bot$	
$\models p \vee \neg p \leftrightarrow \top$	$\models p \wedge \neg p \leftrightarrow \bot$	tertium-non-datur
$\models p \vee \bot \leftrightarrow p$	$\models p \wedge \top \leftrightarrow p$	identity element
$\models p \vee \top \leftrightarrow \top$	$\models p \wedge \bot \leftrightarrow \bot$	
$\models p \wedge p \leftrightarrow p$	$\models p \vee p \leftrightarrow p$	idempotency
$\models p \wedge q \leftrightarrow q \wedge p$	$\models p \vee q \leftrightarrow q \vee p$	commutativity
$\models (p \wedge q) \wedge r \leftrightarrow p \wedge (q \wedge r)$	$\models (p \vee q) \vee r \leftrightarrow p \vee (q \vee r)$	associativity
$\models \neg\neg p \leftrightarrow p$		elimination of $\neg\neg$
$\models p \wedge (p \vee q) \leftrightarrow p$	$\models p \vee (p \wedge q) \leftrightarrow p$	absorption
$\models p \wedge (q \vee r) \leftrightarrow (p \wedge q) \vee (p \wedge r)$	$\models p \vee (q \wedge r) \leftrightarrow (p \vee q) \wedge (p \vee r)$	distributivity
$\models \neg(p \wedge q) \leftrightarrow \neg p \vee \neg q$	$\models \neg(p \vee q) \leftrightarrow \neg p \wedge \neg q$	DeMorgan
$\models (p \rightarrow q) \leftrightarrow \neg p \vee q$		elimination of $\rightarrow$
$\models (p \leftrightarrow q) \leftrightarrow (\neg p \vee q) \wedge (\neg q \vee p)$		elimination of $\leftrightarrow$

Notation: If the formulas f and g are equivalent, we can write this as

$$\models f \leftrightarrow g.$$

However, since this notation is rather clumsy, we will denote this fact as $f \Leftrightarrow g$ instead. Furthermore, in this context we use *chaining* for the operator "$\Leftrightarrow$", that is we write

$$f \Leftrightarrow g \Leftrightarrow h$$

to denote the fact that we have both

$$\models f \leftrightarrow g \quad \text{and} \quad \models g \leftrightarrow h.$$

◇

4.4.1 *TypeScript* Implementation

In this section we develop a *TypeScript* program that is able to decide whether a given propositional formula f is a tautology. The idea is that the program evaluates f for all possible propositional valuations. Hence we have to compute the set of all propositional valuations for a given set of propositional variables. We have already seen that the propositional valuations are in a 1-to-1 correspondence with the subsets of the set $\mathcal{P}$ of all propositional variables because we can represent a propositional valuation $\mathcal{I}$ as the set of all propositional variables that evaluate as `true`:

$$\{q \in \mathcal{P} \mid \mathcal{I}(q) = \texttt{true}\}.$$

If we have a propositional formula f and want to check whether f is a tautology, we first have to determine the set of propositional variables occurring in f. To this end we define a function

$$\texttt{collectVars} : \mathcal{F} \rightarrow 2^{\mathcal{P}}$$

such that $\texttt{collectVars}(f)$ is the set of propositional variables occurring in f. This function can be defined recursively.

1. $\texttt{collectVars}(p) = \{p\}$ for all propositional variables p.

2. $\texttt{collectVars}(\top) = \{\}$.
3. $\texttt{collectVars}(\bot) = \{\}$.
4. $\texttt{collectVars}(\neg f) := \texttt{collectVars}(f)$.
5. $\texttt{collectVars}(f \wedge g) := \texttt{collectVars}(f) \cup \texttt{collectVars}(g)$.
6. $\texttt{collectVars}(f \vee g) := \texttt{collectVars}(f) \cup \texttt{collectVars}(g)$.
7. $\texttt{collectVars}(f \rightarrow g) := \texttt{collectVars}(f) \cup \texttt{collectVars}(g)$.
8. $\texttt{collectVars}(f \leftrightarrow g) := \texttt{collectVars}(f) \cup \texttt{collectVars}(g)$.

Figure 4.3 on page 50 shows how to implement this definition. Note that we have been able to combine the last four cases.

```
1   function collectVars(f: Formula): Set<Variable> {
2       if (typeof f == 'string') { return set(f); }
3       switch (f[0]) {
4           case '⊤':
5           case '⊥': return set();
6           case '¬': { const [_, g] = f; return collectVars(g); }
7           default: {  // case '↔', '→', '∧', '∨':
8               const [_, g, h] = f;
9               const [gs, hs] = [g, h].map(collectVars);
10              return gs.union(hs);
11          }
12      }
13  }
```

Figure 4.3: Checking that a formula is a tautology.

Now we are able to implement the function

$$\texttt{tautology} : \mathcal{F} \rightarrow \mathbb{B}$$

that takes a formula f and returns `true` if and only if $\models f$ holds. Figure 4.4 on page 51 shows the implementation.

1. First, we compute the set P of all propositional variables occurring in f.
2. Then we try to find a propositional valuation $\mathcal{I}$ such that $\mathcal{I}(f) = \texttt{false}$. In this case $\mathcal{I}$ is returned.
3. Otherwise f is a tautology and we return `true`.

```
function tautology(f: Formula): true | Set<Variable> {
    const P = collectVars(f);
    for (const I of P.powerset()) {
        if (!evaluate(f, I)) {
            return I;
        }
    }
    return true;
}
```

Figure 4.4: Checking that f is a tautology.

4.5 Conjunctive Normal Form

The following section discusses algebraic manipulations of propositional formulas. Concretely, we will define the notion of a conjunctive normal form and show how a propositional formula can be turned into conjunctive normal form. The Davis-Putnam algorithm that is discussed later requires us to put the given formulas into conjunctive normal form.

Definition 10 (Literal) *A propositional formula f is a literal if and only if we have one of the following cases:*

1. $f = \top$ *or* $f = \bot$.
2. $f = p$, *where p is a propositional variable.*
   In this case f is a positive literal.
3. $f = \neg p$, *where p is a propositional variable.*
   In this case f is a negative literal.

The set of all literals is denoted as $\mathcal{L}$. ◇

If l is a literal, then the complement of l is denoted as $\overline{l}$. It is defined by a case distinction.

1. $\overline{\top} = \bot$ and $\overline{\bot} = \top$.
2. $\overline{p} := \neg p$, if $p \in \mathcal{P}$.
3. $\overline{\neg p} := p$, if $p \in \mathcal{P}$.

The complement $\overline{l}$ of a literal l is equivalent to the negation of l, we have

$$\models \overline{l} \leftrightarrow \neg l.$$

However, the complement of a literal l is also a literal, while the negation of a literal is in general not a literal. For example, if p is a propositional variable, then the complement of $\neg p$ is p, while the negation of $\neg p$ is $\neg\neg p$ and this is not a literal.

Definition 11 (Clause) *A propositional formula K is a clause when it has the form*

$$K = l_1 \vee \cdots \vee l_r$$

where l_i is a literal for all $i = 1, \cdots, r$. Hence a clause is a disjunction of literals. The set of all clauses is denoted as $\mathcal{K}$. ◇

Often, a clause is seen as a set of its literals. Interpreting a clause as a set of its literals abstracts from both the order and the number of the occurrences of its literals. This is possible because the operator "$\vee$" is associative, commutative, and idempotent. Hence the clause $l_1 \vee \cdots \vee l_r$ will be written as the set

$$\{l_1, \cdots, l_r\}.$$

This notation is called the set notation for clauses. The following example shows how the set notation is beneficial. We take the clauses

$$p \vee (q \vee \neg r) \vee p \quad \text{and} \quad \neg r \vee q \vee (\neg r \vee p).$$

Although these clauses are equivalent, they are not syntactically identical. However, if we transform these clauses into set notation, we get

$$\{p, q, \neg r\} \quad \text{and} \quad \{\neg r, q, p\}.$$

In a set every element occurs at most once. Furthermore, the order of the elements does not matter. Hence the sets given above are the same!

Let us now transfer the propositional equivalence

$$l_1 \vee \cdots \vee l_r \vee \bot \;\Leftrightarrow\; l_1 \vee \cdots \vee l_r$$

into set notation. We get

$$\{l_1, \cdots, l_r, \bot\} \;\Leftrightarrow\; \{l_1, \cdots, l_r\}.$$

This shows that the element $\bot$ can be discarded from a clause. If we write this equivalence for $r = 0$, we get

$$\{\bot\} \Leftrightarrow \{\}.$$

Hence the empty set of literals is to be interpreted as $\bot$.

Definition 12 *A clause K is trivial, if and only if one of the following cases occurs:*

1. $\top \in K$.
2. *There is a variable $p \in \mathcal{P}$, such that we have both $p \in K$ as well as $\neg p \in K$.*
   In this case p and $\neg p$ are called complementary literals. ◇

Proposition 13 *A clause K is a tautology if and only if it is trivial.*

Proof: We first assume that the clause K is trivial. If $\top \in K$, then because we have $f \vee \top \;\Leftrightarrow\; \top$ obviously $K \Leftrightarrow \top$. If p is a propositional variable, so that both $p \in K$ and $\neg p \in K$ are valid, then due to the equivalence $p \vee \neg p \Leftrightarrow \top$ we immediately have $K \Leftrightarrow \top$.

Next we assume that the clause K is a tautology. We carry out the proof indirectly and assume that K is non-trivial. This means that $\top \notin K$ and K cannot contain any complementary literals. Thus

K must have the form

$$K = \{\neg p_1, \cdots, \neg p_m, q_1, \cdots, q_n\} \quad \text{with } p_i \neq q_j \text{ for all } i \in \{1, \cdots, m\} \text{ and } j \in \{1, \cdots, n\}.$$

Then we can define a propositional valuation $\mathcal{I}$ as follows:

1. $\mathcal{I}(p_i) = \texttt{true}$ for all $i = 1, \cdots, m$ and
2. $\mathcal{I}(q_j) = \texttt{false}$ for all $j = 1, \cdots, n$,

We have $\mathcal{I}(K) = \texttt{false}$ with this valuation and therefore K isn't a tautology. So the assumption that K is not trivial is `false`. □

Definition 14 (Conjunctive Normal Form) *A formula F is in conjunctive normal form (short CNF) if and only if F is a conjunction of clauses, i.e. if*

$$F = K_1 \wedge \cdots \wedge K_n,$$

where the K_i are clauses for all $i = 1, \cdots, n$. ◇

The following is an immediate corollary of the definition of a CNF.

Corollary 15 *If $F = K_1 \wedge \cdots \wedge K_n$ is in conjunctive normal form, then*

$$\models F \quad \text{if and only if} \quad \models K_i \quad \text{for all } i = 1, \cdots, n.$$ □

Thus, for a formula $F = K_1 \wedge \cdots \wedge K_n$ in conjunctive normal form, we can easily decide whether F is a tautology, because F is a tautology iff all clauses K_i are trivial.

Since the associative, commutative and idempotent laws apply to a conjunction in the same way as to a disjunction it is beneficial to also use a set notation: If the formula

$$F = K_1 \wedge \cdots \wedge K_n$$

is in conjunctive normal form, we represent this formula by the set of its clauses and write

$$F = \{K_1, \cdots, K_n\}.$$

The clauses themselves are also specified in set notation. We provide an example: If p, q and r are propositional variables, the formula

$$(p \vee q \vee \neg r) \wedge (q \vee \neg r \vee p \vee q) \wedge (\neg r \vee p \vee \neg q)$$

is in conjunctive normal form. In set notation, this becomes

$$\big\{\{p, q, \neg r\}, \{p, \neg q, \neg r\}\big\}.$$

Since we have $K \wedge \top \Leftrightarrow K$ we have that

$$\{\} \Leftrightarrow \top,$$

when the empty set is interpreted as an empty set of clauses. Furthermore, we have

$$\big\{\{\}\big\} \Leftrightarrow \bot.$$

4.5.1 Transforming a Formula into CNF

Next, we present a method that can be used to transform any propositional formula F into CNF. As we have seen above, we can then easily decide whether F is a tautology.

1. Eliminate all occurrences of the operator "$\leftrightarrow$" using the equivalence
$$F \leftrightarrow G \Leftrightarrow (F \rightarrow G) \wedge (G \rightarrow F).$$
2. Eliminate all occurrences of the operator "$\rightarrow$" using the equivalence
$$F \rightarrow G \Leftrightarrow \neg F \vee G.$$
3. Move the negation signs inwards as far as possible. Use the following equivalences:
   (a) $\neg\bot \Leftrightarrow \top$
   (b) $\neg\top \Leftrightarrow \bot$
   (c) $\neg\neg F \Leftrightarrow F$
   (d) $\neg(F \wedge G) \Leftrightarrow \neg F \vee \neg G$
   (e) $\neg(F \vee G) \Leftrightarrow \neg F \wedge \neg G$

   In the result that we obtain after this step, the negation signs occur only immediately before propositional variables. Formulas with this property are also referred to as formulas in negation-normal-form.
4. If the formula contains "$\vee$" operators on top of "$\wedge$" operators, we use the distributive law
$$\begin{aligned} & (F_1 \wedge \cdots \wedge F_m) \vee (G_1 \wedge \cdots \wedge G_n) \\ \Leftrightarrow \quad & (F_1 \vee G_1) \wedge \cdots \wedge (F_1 \vee G_n) \ \wedge \ \cdots \ \wedge \ (F_m \vee G_1) \wedge \cdots \wedge (F_m \vee G_n) \end{aligned}$$
   to move the disjunction "$\vee$" inwards.
5. In the final step we convert the formula into set notation.

We should note that the formula can grow considerably when using the distributive law. This is because the formula F occurs twice on the right-hand side of the equivalence $F \vee (G \wedge H) \leftrightarrow (F \vee G) \wedge (F \vee H)$, while it occurs only once on the left.

We demonstrate the procedure using the example of the formula

$$(p \rightarrow q) \rightarrow (\neg p \rightarrow \neg q).$$

1. Since the formula does not contain the operator "$\leftrightarrow$" there is nothing to do in the first step.
2. In order to eliminate the operator "$\rightarrow$", we eliminate the inner occurrences of "$\rightarrow$" first. This yields
$$(\neg p \vee q) \rightarrow (\neg\neg p \vee \neg q).$$
   When we eliminate the remaining occurrence of "$\rightarrow$" we get
$$\neg(\neg p \vee q) \vee (\neg\neg p \vee \neg q).$$
3. The conversion to negation normal form results in
$$(p \wedge \neg q) \vee (p \vee \neg q).$$

4. By using the distributive law we get

$$\big(p \vee (p \vee \neg q)\big) \wedge \big(\neg q \vee (p \vee \neg q)\big).$$

5. The conversion to set notation yields the following clauses

$$\{p, p, \neg q\} \quad \text{and} \quad \{\neg q, p, \neg q\}.$$

However, since the order of the elements of a set is unimportant and, moreover, a set contains each element only once, we realize that these two clauses are the same. Therefore, if we combine these clauses into a set, we get

$$\big\{\{p, \neg q\}\big\}.$$

Note that the formula is significantly simplified by converting it into set notation.

The formula is now converted to CNF.

4.5.2 Computing the Conjunctive Normal Form in *TypeScript*

Next, we specify a number of functions that can be used to convert a given formula formula f into conjunctive normal form. These functions are part of the Jupyter notebook

TypeScript/Chapter-4/04-CNF.ipynb.

We start with the function

$$\texttt{elimBiconditional} : \mathcal{F} \to \mathcal{F}$$

which has the task of transforming a given propositional formula f into an equivalent formula which no longer contains the operator "$\leftrightarrow$". The function $\texttt{elimBiconditional}(f)$ is defined by induction with respect to the propositional formula f. In order to present this inductive definition, we set up recursive equations that describe the behavior of the function `elimBiconditional`:

1. If f is a propositional variable p, there is nothing to do:

$$\texttt{elimBiconditional}(p) = p \quad \text{for all } p \in \mathcal{P}.$$

2. The cases in which f is equal to verum or falsum are also trivial:

$$\texttt{elimBiconditional}(\top) = \top \quad \text{and} \quad \texttt{elimBiconditional}(\bot) = \bot.$$

3. If f has the form $f = \neg g$, we eliminate the operator "$\leftrightarrow$" recursively from the formula g and negate the resulting formula:

$$\texttt{elimBiconditional}(\neg g) = \neg\texttt{elimBiconditional}(g).$$

4. In the cases $f = g_1 \wedge g_2$, $f = g_1 \vee g_2$ and $f = g_1 \to g_2$ we recursively eliminate the operator "$\leftrightarrow$" from the formulas g_1 and g_2 and then reassemble the formula together again:

   (a) $\texttt{elimBiconditional}(g_1 \wedge g_2) = \texttt{elimBiconditional}(g_1) \wedge \texttt{elimBiconditional}(g_2)$.

   (b) $\texttt{elimBiconditional}(g_1 \vee g_2) = \texttt{elimBiconditional}(g_1) \vee \texttt{elimBiconditional}(g_2)$.

   (c) $\texttt{elimBiconditional}(g_1 \to g_2) = \texttt{elimBiconditional}(g_1) \to \texttt{elimBiconditional}(g_2)$.

5. If f has the form $f = g_1 \leftrightarrow g_2$, we use the equivalence

   $$(g_1 \leftrightarrow g_2) \Leftrightarrow \big((g_1 \rightarrow g_2) \wedge (g_2 \rightarrow g_1)\big).$$

   This leads to the equation:

   $$\texttt{elimBiconditional}(g_1 \leftrightarrow g_2) = \texttt{elimBiconditional}\big((g_1 \rightarrow g_2) \wedge (g_2 \rightarrow g_1)\big).$$

   It is necessary to call the function `elimBiconditional` on the right-hand side of the equation, because the operator "$\leftrightarrow$" can still occur in g_1 and g_2.

```
1   type Formula1 = Variable
2                 | ['⊤' | '⊥']
3                 | ['¬', Formula1]
4                 | ['→' | '∧' | '∨', Formula1, Formula1];
5   function eliminateBiconditional(f: Formula): Formula1 {
6       if (typeof f == 'string') { return f; }
7       switch (f[0]) {
8           case '⊤':
9           case '⊥': return f;
10          case '¬': const [_, g] = f; return ['¬', eliminateBiconditional(g)];
11          default:
12              const [op, h, k] = f;
13              const [hs, ks] = [h, k].map(eliminateBiconditional);
14              switch (op) {
15                  case '↔': return ['∧', ['→', hs, ks], ['→', ks, hs]];
16                  default: return [op, hs, ks];
17  }}}
```

Figure 4.5: Elimination of $\leftrightarrow$

Figure 4.5 on page 56 shows the implementation of the function `elimBiconditional`.

1. In line 6, we check whether f is a string. In this case f must be a propositional variable, because all other propositional formulas are represented as nested lists. Therefore, f is returned unchanged in this case.

2. In line 10 we check the case that f has the form $g \leftrightarrow h$. First, we eliminate the operator "$\leftrightarrow$" from g and h by recursively calling the function `elimBiconditional`. Then we use the equivalence

   $$(g \leftrightarrow h) \Leftrightarrow (g \rightarrow h) \wedge (h \rightarrow g).$$

3. In lines 8 and 9 we deal with the cases where f is equal to verum or falsum. Note that these formulas are also represented as nested lists. For example, verum is represented as the list `['⊤']`, while falsum is represented in *TypeScript* as the list `['⊥']`. In this case, f is returned unchanged.

4. In line 10, we consider the case where f is a negation. Then f has the form

   ['¬', g]

   and we have to recursively remove the operator "↔" from g.

5. In the remaining cases, f has the form

   ['o', g, h] with $o \in \{\rightarrow, \wedge, \vee\}$.

   In these cases, the operator "↔" has to be removed recursively from the subformulas g and h.

Next, we look at the function for eliminating the "→" operator. Figure 4.6 on page 57 shows the implementation of the function `eliminateConditional`. The underlying idea of the implementation is the same as for the elimination of the operator "↔". The only difference is that we now use the equivalence

$$g \rightarrow h \Leftrightarrow \neg g \vee h.$$

Furthermore, when implementing this function, we can assume that the operator "↔" has already been eliminated from the propositional formula `f`, which is passed as an argument. This eliminates one case in the implementation.

```
1  type Formula2 = Variable
2                | ['⊤' | '⊥']
3                | ['¬', Formula2]
4                | ['∧' | '∨', Formula2, Formula2];
5
6  function eliminateConditional(f: Formula1): Formula2 {
7      if (typeof f == 'string') { return f; }
8      switch (f[0]) {
9          case '⊤':
10         case '⊥': return f;
11         case '¬': const [_, g] = f; return ['¬', eliminateConditional(g)];
12         default:
13             const [op, h, k] = f;
14             const [hs, ks] = [h, k].map(eliminateConditional);
15             switch (op) {
16                 case '→': return ['∨', ['¬', hs], ks];
17                 default:  return [op, hs, ks];
18 }}}
```

Figure 4.6: Elimination of →

Next, we present the functions for calculating the negation normal form. Figure 4.7 on page 59 shows the implementation of the functions `nnf` and `neg`, which call each other. Here, `nnf(`f`)` calculates the negation normal form of f, while `neg(`f`)` calculates the negation normal form of $\neg f$, so the following holds

$$\texttt{neg(f)} = \texttt{nnf}(\neg \texttt{f}).$$

The actual work is done in the function `neg`, because this is where DeMorgan's laws

$$\neg(f \wedge g) \Leftrightarrow \neg f \vee \neg g \quad \text{and} \quad \neg(f \vee g) \Leftrightarrow \neg f \wedge \neg g$$

are applied. We describe the transformation into negation normal form by the following equations:

1. $\texttt{nnf}(p) = p \quad$ for all $p \in \mathcal{P}$,
2. $\texttt{nnf}(\top) = \top$,
3. $\texttt{nnf}(\bot) = \bot$,
4. $\texttt{nnf}(\neg \texttt{f}) = \texttt{neg(f)}$,
5. $\texttt{nnf}(f_1 \wedge f_2) = \texttt{nnf}(f_1) \wedge \texttt{nnf}(f_2)$,
6. $\texttt{nnf}(f_1 \vee f_2) = \texttt{nnf}(f_1) \vee \texttt{nnf}(f_2)$.

The auxiliary procedure `neg`, which calculates the negation normal form of $\neg f$, is also specified by recursive equations:

1. $\texttt{neg}(p) = \texttt{nnf}(\neg p) = \neg p$ for all propositional variables p.
2. $\texttt{neg}(\top) = \texttt{nnf}(\neg\top) = \texttt{nnf}(\bot) = \bot$,
3. $\texttt{neg}(\bot) = \texttt{nnf}(\neg\bot) = \texttt{nnf}(\top) = \top$,
4. $\texttt{neg}(\neg f) = \texttt{nnf}(\neg\neg f) = \texttt{nnf}(f)$.
5. $$\begin{aligned} & \texttt{neg}(f_1 \wedge f_2) \\ = \; & \texttt{nnf}(\neg(f_1 \wedge f_2)) \\ = \; & \texttt{nnf}(\neg f_1 \vee \neg f_2) \\ = \; & \texttt{nnf}(\neg f_1) \vee \texttt{nnf}(\neg f_2) \\ = \; & \texttt{neg}(f_1) \vee \texttt{neg}(f_2). \end{aligned}$$
   Hence we have:
   $$\texttt{neg}(f_1 \wedge f_2) = \texttt{neg}(f_1) \vee \texttt{neg}(f_2).$$
6. $$\begin{aligned} & \texttt{neg}(f_1 \vee f_2) \\ = \; & \texttt{nnf}(\neg(f_1 \vee f_2)) \\ = \; & \texttt{nnf}(\neg f_1 \wedge \neg f_2) \\ = \; & \texttt{nnf}(\neg f_1) \wedge \texttt{nnf}(\neg f_2) \\ = \; & \texttt{neg}(f_1) \wedge \texttt{neg}(f_2). \end{aligned}$$
   Therefore we have:
   $$\texttt{neg}(f_1 \vee f_2) = \texttt{neg}(f_1) \wedge \texttt{neg}(f_2).$$

```
type NNF = Variable
         | ['⊤' | '⊥']
         | ['¬', Variable]
         | ['∧' | '∨', NNF, NNF];

function nnf(f: Formula2): NNF {
    if (typeof f == 'string') { return f; }
    switch (f[0]) {
        case '⊤':
        case '⊥': return f;
        case '¬': const [_, g] = f; return neg(g);
        default:  const [op, h, k] = f; return [op, nnf(h), nnf(k)];
}}

function neg(f: Formula2): NNF {
    if (typeof f == 'string') { return ['¬', f]; }
    switch (f[0]) {
        case '⊤': return ['⊥'];
        case '⊥': return ['⊤'];
        case '¬': const [_, g] = f; return nnf(g);
        default:  const [op, h, k] = f;
                  return [op == '∧' ? '∨' : '∧', neg(h), neg(k)];
}}
```

Figure 4.7: Computing the negation normal form.

Finally, we present the function `cnf` that allows us to transform a propositional formula f from negation normal form into conjunctive normal form. Furthermore, `cnf` converts the resulting formula into set notation, i.e. the formulas are represented as sets of sets of literals. We interpret a set of literals as a disjunction of the literals and a set of clauses is interpreted as a conjunction of the clauses. Mathematically, our goal is therefore to find a function

$$\texttt{cnf} : \texttt{NNF} \to \texttt{CNF}$$

so that $\texttt{cnf}(f)$ for a formula f, which is in negation normal form, returns a set of clauses as a result whose conjunction is equivalent to f. The definition of $\texttt{cnf}(f)$ is given recursively.

1. If f is a propositional variable, we return as result a set that contains exactly one clause. This clause is itself a set of literals, the only literal of which is the propositional variable f:

   $$\texttt{cnf}(f) := \{\{f\}\} \quad \text{if } f \in \mathcal{P}.$$

2. We saw earlier that the empty set of *clauses* can be interpreted as $\top$. Therefore:

   $$\texttt{cnf}(\top) := \{\}.$$

3. We had also seen that the empty set of *literals* can be interpreted as $\bot$. Therefore:

$$\texttt{cnf}(\bot) := \{\{\}\}.$$

4. If f is a negation, then the following holds:

   $$f = \neg p \quad \text{with } p \in \mathcal{P},$$

   because f is in negation normal form and in such a formula the negation operator can only be applied to a propositional variable. Therefore, f is a literal and we return as a result a set that contains exactly one clause. This clause is itself a set of literals, which contains the formula f as the only literal:

   $$\texttt{cnf}(\neg p) := \{\{\neg p\}\} \quad \text{if } p \in \mathcal{P}.$$

5. If f is a conjunction and therefore $f = g \wedge h$, then we first transform the formulas g and h into CNF. We then obtain sets of clauses $\texttt{cnf}(g)$ and $\texttt{cnf}(h)$. Since we interpret a set of clauses as a conjunction of the clauses contained in the set, it is sufficient to form the union of the sets $\texttt{cnf}(g)$ and $\texttt{cnf}(h)$, so we have

   $$\texttt{cnf}(g \wedge h) = \texttt{cnf}(g) \cup \texttt{cnf}(h).$$

6. If $f = g \vee h$, we first transform g and h into CNF. This gives us

   $$\texttt{cnf}(g) = \{g_1, \cdots, g_m\} \quad \text{and} \quad \texttt{cnf}(h) = \{h_1, \cdots, h_n\}.$$

   The formulas g_i and the h_j are clauses. In order to form the CNF of $g \vee h$ we proceed as follows:

   $$\begin{array}{lll} & g \vee h & \\ \Leftrightarrow & (k_1 \wedge \cdots \wedge k_m) \vee (l_1 \wedge \cdots \wedge l_n) & \\ \Leftrightarrow & (k_1 \vee l_1) \quad \wedge \quad \cdots \quad \wedge \quad (k_m \vee l_1) \quad \wedge & \\ & \quad \vdots \qquad\qquad\qquad\qquad\quad \vdots & \\ & (k_1 \vee l_n) \quad \wedge \quad \cdots \quad \wedge \quad (k_m \vee l_n) & \\ \Leftrightarrow & \{k_i \vee l_j : i \in \{1, \cdots, m\}, j \in \{1, \cdots, n\}\} & \end{array}$$

   If we take into account that clauses in the set notation are understood as sets of literals that are implicitly linked disjunctively, then we can also write $k_i \vee l_j$ as the union $k_i \cup l_j$. Therefore, we obtain

   $$\texttt{cnf}(g \vee h) = \{k \cup l \mid k \in \texttt{cnf}(g) \ \wedge \ l \in \texttt{cnf}(h)\}.$$

Figure 4.8 on page 61 shows the implementation of the function `cnf`. (The name `cnf` is the abbreviation of conjunctive normal form).

Lastly, we show in Figure 4.9 on page 62 how the functions that have been shown above interact.

1. The function `normalize` first eliminates the operators "$\leftrightarrow$" with the help of the function `eliminateBiconditional`.
2. The operator "$\rightarrow$" is then replaced with the help of the function `eliminateConditional`.
3. Calling `nnf` converts the formula to negation-normal form.
4. The negation normal form is now converted to conjunctive normal form using the function `cnf`, whereby the formula is simultaneously converted to set notation.

```
type CNF = Set<Clause>;

function cnf(f: NNF): CNF {
    if (typeof f == 'string') { return set(set(f)); }
    switch (f[0]) {
        case '⊤': return set();
        case '⊥': return set(set());
        case '¬': const [_, p] = f; return set(set(tpl('¬', p)));
        default:
            const [op, g, h] = f;
            const [gs, hs] = [g, h].map(cnf)
            switch (op) {
                case '∧': return gs.union(hs);
                case '∨': return flatMap(gs, c => hs.map(d => c.union(d)));
}}}
```

Figure 4.8: Computing the Conjunctive Normal Form.

5. Finally, the function `simplify` removes all clauses from the set `N4` that are trivial.

6. The function `isTrivial` checks whether a clause C, which is in set notation, contains both a variable p and the negation $\neg p$ of this variable, because then this clause is equivalent to $\top$ and can be omitted.

The complete program for calculating the conjunctive normal form can be found as the file

`TypeScript/Chapter-4/04-CNF.ipynb`

on my GitHub repository.

```
function getComplement(l: Literal): Literal {
    if (typeof l == 'string') {
        return new Tuple('¬', l);
    } else {
        return l.get(1);
    }
}

function isTrivial(clause: Clause): boolean {
    return clause.some(l => clause.has(getComplement(l)));
}

function simplify(clauses: CNF): CNF {
    return clauses.filter(c => !isTrivial(c));
}

function normalize(f: Formula): CNF {
    const n1 = eliminateBiconditional(f);
    const n2 = eliminateConditional(n1);
    const n3 = nnf(n2);
    const n4 = cnf(n3);
    return simplify(n4);
}
```

Figure 4.9: Normalizing a Formula.

Exercise 11: Calculate the conjunctive normal forms of the following formulas and state your result in set notation.

(a) $p \vee q \rightarrow r$,

(b) $p \vee q \leftrightarrow r$,

(c) $p \rightarrow q \leftrightarrow \neg p \rightarrow \neg q$,

(d) $p \rightarrow q \leftrightarrow \neg q \rightarrow \neg p$,

(e) $\neg r \wedge (q \vee p \rightarrow r) \rightarrow \neg q \wedge \neg p$.

4.6 The Concept of a Derivation

Logic is concerned with the validity of proofs. In order to build an automatic theorem prover that can help us with the verification of a digital circuit or a program we need to investigate the concept of

a proof. In mathematics, a proof is a conclusive argument that starts from given axioms and applies certain rules of derivation to these axioms in order to derive theorems. Our aim is to formalize this process so that we are able to implement it. The formalized version of a proof will be called a derivation.

If $\{f_1, \cdots, f_n\}$ is a set of formulas and g is another formula, then we might ask whether the formula g is a logical consequence of the formulas $f_1, \cdots, f_n$, i.e. whether

$$\models f_1 \wedge \cdots \wedge f_n \rightarrow g$$

holds. There are various ways to answer this question. We already know one method: First, we transform the formula $f_1 \wedge \cdots \wedge f_n \rightarrow g$ into conjunctive normal form. Thereby we obtain a set of clauses $\{k_1, \cdots, k_m\}$, whose conjunction is equivalent to the formula

$$f_1 \wedge \cdots \wedge f_n \rightarrow g$$

This formula is a tautology if and only if each of the clauses $k_1, \cdots, k_m$ is trivial.

However, the above method is quite inefficient. We demonstrate this with an example and apply the method to decide whether $p \rightarrow r$ follows from the two formulas $p \rightarrow q$ and $q \rightarrow r$. We compute the conjunctive normal form of the formula

$$h := (p \rightarrow q) \wedge (q \rightarrow r) \rightarrow p \rightarrow r$$

and after a laborious calculation, we obtain

$$(p \vee \neg p \vee r \vee \neg r) \wedge (\neg q \vee \neg p \vee r \vee \neg r) \wedge (\neg q \vee \neg p \vee q \vee r) \wedge (p \vee \neg p \vee q \vee r).$$

Although we can now see that the formula h is a tautology, given the fact that we can see with the naked eye that $p \rightarrow r$ follows from the formulas $p \rightarrow q$ and $q \rightarrow r$, this calculation is far too inefficient.

Therefore, we now present another method that can help us decide whether a formula follows from a given set of formulas. The idea of this method is to derive the formula to be proved using inference rules from the given formulas. The concept of an inference rule is based on the following definition.

Definition 16 (Inference Rule)
A propositional inference rule is a pair of the form

$$\big\langle \langle f_1, f_2 \rangle, k \big\rangle.$$

Here, $\langle f_1, f_2 \rangle$ is a pair of propositional formulas, and k is a single propositional formula. The formulas f_1 and f_2 are referred to as premises, and the formula k is called the conclusion of the inference rule. If the pair $\big\langle \langle f_1, f_2 \rangle, k \big\rangle$ is an inference rule, then this is written as:

$$\frac{f_1 \qquad f_2}{k}.$$

We read this inference rule as: "From f_1 and f_2 we conclude k." ◇

Examples of inference rules:

Affirming the Antecedent	Denying the Consequent	Denying the Antecedent
$\dfrac{f \quad f \to g}{g}$	$\dfrac{\neg g \quad f \to g}{\neg f}$	$\dfrac{\neg f \quad f \to g}{\neg g}$

Initially, the definition of the notion of an inference rule does not limit the formulas that can be used as premises or conclusion. However, it is certainly not useful to allow arbitrary inference rules. If we want to use inference rules in proofs, they should be correct in the sense explained in the following definition.

Definition 17 (Correct Inference Rule) *An inference rule of the form*

$$\frac{f_1 \quad f_2}{k}$$

is correct if and only if

$$\models f_1 \wedge f_2 \to k.$$

◇

With this definition, we see that the rules of inference referred to above as "Affirming the Antecedent" and "Denying the Consequent" are correct, while the one called "Denying the Antecedent" is, in fact, a logical fallacy.

From now on, we assume that all formulas are clauses. On the one hand, this is not a real restriction, as we can transform any formula into an equivalent set of clauses. On the other hand, in many practical problems the formulas are already in the form of clauses. Therefore, we now present an inference rule where both the premises and the conclusion are clauses.

Definition 18 (Cut Rule) *If p is a propositional variable and k_1 and k_2 are sets of literals, which we interpret as clauses, then we refer to the following inference rule as the cut rule:*

$$\frac{k_1 \cup \{p\} \quad \{\neg p\} \cup k_2}{k_1 \cup k_2}.$$

◇

The cut rule is very general. If we set $k_1 = \{\}$ and $k_2 = \{q\}$ in the definition above, we obtain the following rule as a special case:

$$\frac{\{\} \cup \{p\} \qquad \{\neg p\} \cup \{q\}}{\{\} \cup \{q\}}$$

Interpreting the sets of literals as disjunctions, we have:

$$\frac{p \qquad \neg p \vee q}{q}$$

Taking into account that the formula $\neg p \vee q$ is equivalent to the formula $p \to q$, this instance of the cut rule is none other than Modus Ponens. The rule Modus Tollens is also a special case of the cut rule. We obtain this rule if we set $k_1 = \{\neg q\}$ and $k_2 = \{\}$ in the cut rule.

Theorem 19 *The cut rule is correct.*

Proof: We need to show that

$$\models (k_1 \vee p) \wedge (\neg p \vee k_2) \rightarrow k_1 \vee k_2$$

holds. To do this, we convert the above formula into conjunctive normal form:

$$\begin{aligned}
& (k_1 \vee p) \wedge (\neg p \vee k_2) \rightarrow k_1 \vee k_2 \\
\Leftrightarrow\ & \neg\big((k_1 \vee p) \wedge (\neg p \vee k_2)\big) \vee k_1 \vee k_2 \\
\Leftrightarrow\ & \neg(k_1 \vee p) \vee \neg(\neg p \vee k_2) \vee k_1 \vee k_2 \\
\Leftrightarrow\ & (\neg k_1 \overset{+}{\wedge} \neg p) \overset{*}{\vee} (p \overset{+}{\wedge} \neg k_2) \overset{*}{\vee} k_1 \overset{*}{\vee} k_2 \\
\Leftrightarrow\ & (\neg k_1 \vee p \vee k_1 \vee k_2) \wedge (\neg k_1 \vee \neg k_2 \vee k_1 \vee k_2) \wedge (\neg p \vee p \vee k_1 \vee k_2) \wedge (\neg p \vee \neg k_2 \vee k_1 \vee k_2) \\
\Leftrightarrow\ & \top \wedge \top \wedge \top \wedge \top \\
\Leftrightarrow\ & \top
\end{aligned}$$

□

Definition 20 (Derivation, $\vdash$)
Let M be a set of clauses and f a single clause. The formulas from M are referred to as our axioms. Our goal is to derive the formula f using the axioms from M. For this, we define the relation

$$M \vdash f.$$

We read "$M \vdash f$" as "M derives f". The inductive definition is as follows:

1. *From a set M of assumptions, any of the assumptions can be derived:*

   If $f \in M$, *then* $M \vdash f$.

2. *If $k_1 \cup \{p\}$ and $\{\neg p\} \cup k_2$ are clauses that can be derived from M, then using the cut rule, the clause $k_1 \cup k_2$ can also be derived from M:*

   If both $M \vdash k_1 \cup \{p\}$ *and* $M \vdash \{\neg p\} \cup k_2$ *hold, then* $M \vdash k_1 \cup k_2$ *holds, too.* ◇

Example: To illustrate the concept of a derivation, we provide an example and show

$$\{\ \{\neg p, q\},\ \{\neg q, \neg p\},\ \{\neg q, p\},\ \{q, p\}\ \} \vdash \bot.$$

At the same time, we demonstrate through this example how proofs are presented on paper:

1. From $\{\neg p, q\}$ and $\{\neg q, \neg p\}$ it follows by the cut rule that $\{\neg p, \neg p\}$ holds. Since $\{\neg p, \neg p\} = \{\neg p\}$, we write this as

   $$\{\neg p, q\}, \{\neg q, \neg p\} \ \vdash\ \{\neg p\}.$$

   Remark: This example shows that the clause $k_1 \cup k_2$ might contain fewer elements than the sum $\texttt{card}(k_1) + \texttt{card}(k_2)$. This situation occurs when there are literals that appear in both k_1 and k_2.

2. $\{\neg q, \neg p\},\ \{p, \neg q\} \ \vdash\ \{\neg q\}$.

3. $\{p, q\},\ \{\neg q\} \ \vdash\ \{p\}$.

4. $\{\neg p\},\ \{p\} \ \vdash\ \{\}$.

As another example, we demonstrate that $p \to r$ follows from $p \to q$ and $q \to r$. First, we convert all formulas into clauses:

$$\texttt{cnf}(p \to q) = \big\{\{\neg p, q\}\big\}, \quad \texttt{cnf}(q \to r) = \big\{\{\neg q, r\}\big\}, \quad \texttt{cnf}(p \to r) = \big\{\{\neg p, r\}\big\}.$$

Thus, we have $M = \big\{\ \{\neg p, q\},\ \{\neg q, r\}\ \big\}$ and must show that

$$M \vdash \{\neg p, r\}$$

holds. The proof consists of a single application of the Cut Rule:

$$\{\neg p, q\},\ \{\neg q, r\}\ \vdash\ \{\neg p, r\}.$$ ◇

4.6.1 Properties of the Concept of a Formal Derivation

The relation $\vdash$ has two important properties:

Theorem 21 (Correctness) *If $\{k_1, \cdots, k_n\}$ is a set of clauses and k is a single clause, then:*

$$\text{If } \{k_1, \cdots, k_n\} \vdash k \text{ holds, then } \models k_1 \wedge \cdots \wedge k_n \to k \text{ also holds.}$$

In other words: If we can prove a clause k using the assumptions $k_1, \cdots, k_n$, then the clause k logically follows from these assumptions.

Proof: The proof of the correctness theorem proceeds by induction on the definition of the relation $\vdash$.

1. Case: It holds that $\{k_1, \cdots, k_n\} \vdash k$ because $k \in \{k_1, \cdots, k_n\}$. Then there exists an $i \in \{1, \cdots, n\}$ such that $k = k_i$. In this case, we need to show

   $$\models k_1 \wedge \cdots \wedge k_i \wedge \cdots \wedge k_n \to k_i$$

   which is obvious.

2. Case: It holds that $\{k_1, \cdots, k_n\} \vdash k$ because there is a propositional variable p and clauses g and h such that

   $$\{k_1, \cdots, k_n\} \vdash g \cup \{p\} \quad \text{and} \quad \{k_1, \cdots, k_n\} \vdash h \cup \{\neg p\}$$

   hold and from this we have concluded using the cut rule that

   $$\{k_1, \cdots, k_n\} \vdash g \cup h$$

   where $k = g \cup h$. We have to show that

   $$\models k_1 \wedge \cdots \wedge k_n \to g \vee h$$

   holds. Let $\mathcal{I}$ be a propositional valuation such that

   $$\mathcal{I}(k_1 \wedge \cdots \wedge k_n) = \texttt{true}$$

   Then we need to show that we have

   $$\mathcal{I}(g) = \texttt{true} \quad \text{or} \quad \mathcal{I}(h) = \texttt{true}.$$

   By the induction hypothesis, we know

   $$\models k_1 \wedge \cdots \wedge k_n \to g \vee p \quad \text{and} \quad \models k_1 \wedge \cdots \wedge k_n \to h \vee \neg p.$$

   Since $\mathcal{I}(k_1 \wedge \cdots \wedge k_n) = \texttt{true}$, it follows that

$$\mathcal{I}(g \vee p) = \texttt{true} \quad \text{and} \quad \mathcal{I}(h \vee \neg p) = \texttt{true}.$$

Now there are two cases:

(a) Case: $\mathcal{I}(p) = \texttt{true}$.
Then $\mathcal{I}(\neg p) = \texttt{false}$ and hence from the fact that $\mathcal{I}(h \vee \neg p) = \texttt{true}$, we must have

$$\mathcal{I}(h) = \texttt{true}.$$

This immediately implies

$$\mathcal{I}(g \vee h) = \texttt{true}. \quad \surd$$

(b) Case: $\mathcal{I}(p) = \texttt{false}$.
Since $\mathcal{I}(g \vee p) = \texttt{true}$ we must have

$$\mathcal{I}(g) = \texttt{true}.$$

Thus, we also have

$$\mathcal{I}(g \vee h) = \texttt{true}. \quad \surd$$

□

The converse of this theorem only holds in a weakened form, specifically when k is the empty clause, which corresponds to falsum. Therefore, we say that the cut rule is refutation-complete. We will prove this later, but first we need to introduce the concept of satisfiability.

4.6.2 Satisfiability

To succinctly state the theorem of refutation completeness in propositional logic, we need the concept of satisfiability, which we introduce next.

Definition 22 (Satisfiability)
Let M be a set of propositional formulas. If there exists a propositional valuation $\mathcal{I}$ that satisfies all formulas from M, i.e. we have

$$\mathcal{I}(f) = \texttt{true} \quad \text{for all } f \in M,$$

then we call M satisfiable. In this case the propositional valuation $\mathcal{I}$ is called a solution of M. Furthermore, we say that M is unsatisfiable and write

$$M \models \bot,$$

if there is no propositional valuation $\mathcal{I}$ that simultaneously satisfies all formulas from M. Denoting the set of propositional valuations as $\textsc{Ali}$, we formally write

$$M \models \bot \quad \textit{iff} \quad \forall \mathcal{I} \in \textsc{Ali} : \exists C \in M : \mathcal{I}(C) = \texttt{false}.$$

If a set M of propositional formulas is satisfiable, we also write

$$M \not\models \bot.$$

◇

Remark: If $M = \{f_1, \cdots, f_n\}$ is a set of propositional formulas, it is straightforward to show that M is unsatisfiable if and only if

$$\models f_1 \wedge \cdots \wedge f_n \rightarrow \bot$$

holds. ◇

Definition 23 (Saturated Sets of Clauses)
A set M of clauses is saturated if every clause that can be derived from two clauses $C_1, C_2 \in M$ using the cut rule is already itself an element of the set M, i.e. if

$$C_1 \in M, \quad C_2 \in M, \quad \text{and} \quad C_1,\ C_2 \vdash C,$$

then we must also have

$$C \in M.$$

Remark: If M is a finite set of clauses, we can extend M to a saturated set of clauses $\overline{M}$ by initializing $\overline{M}$ as the set M and then continually applying the cut rule to clauses from $\overline{M}$ and adding them to the set $\overline{M}$ as long as this is possible. Since there is only a limited number of clauses in set notation that can be formed from a given finite set of propositional variables, this process must terminate, and the set of formulas we then obtain is saturated. ◇

4.6.3 Refutational Completeness

We are now ready to state and prove the refutational completeness of the cut rule for propositional logic.

Theorem 24 (Refutational Completeness)
Assume M is a set of clauses that is unsatisfiable. Then M derives the empty clause:

$$\text{If } M \models \bot, \text{ then } M \vdash \{\}.$$

Proof: We prove this by contraposition. Assume that M is a finite set of clauses such that

(a) M is unsatisfiable, thus

$$M \models \bot,$$

(b) but such that the empty clause cannot be derived from M, i.e. we have

$$M \not\vdash \{\}.$$

We will show that then there has to exist a propositional valuation $\mathcal{I}$ under which all clauses from M become `true`, which contradicts the unsatisfiability of M.

Following the previous remark, we saturate the set M to obtain the saturated set $\overline{M}$. Let

$$\{p_1, \cdots, p_N\}$$

be the set of all propositional variables appearing in clauses from M. We denote the set of propositional variables appearing in a clause C by $\texttt{var}(C)$. For all $k = 0, 1, \cdots, N$ we now define a propositional valuation $\mathcal{I}_k$ by induction on k. For all $k = 0, 1, \cdots, N$ the propositional valuation $\mathcal{I}_k$ has the property that for each clause $C \in \overline{M}$, which contains only the variables $p_1, \cdots, p_k$, we have that $\mathcal{I}_k(C) = \texttt{true}$ holds. This is formally written as follows:

$$\forall C \in \overline{M} : \Big(\texttt{var}(C) \subseteq \{p_1, \cdots, p_k\} \Rightarrow \mathcal{I}_k(C) = \texttt{true}\Big). \tag{$*$}$$

Additionally, the propositional valuation $\mathcal{I}_k$ only defines values for the variables $p_1, \cdots, p_k$, so

$$\texttt{dom}(\mathcal{I}_k) = \{p_1, \cdots, p_k\}.$$

I.A.: $k = 0$.

We define $\mathcal{I}_0$ as the empty propositional valuation, which assigns no values to any variables. To prove $(*)$, we must show that for each clause $C \in \overline{M}$ that contains no propositional variable, $\mathcal{I}_0(C) = \texttt{true}$ holds. The only clause that contains no variable is the empty clause. Since we assumed that $M \not\vdash \{\}$, $\overline{M}$ cannot contain the empty clause. Thus, there is nothing to prove.

I.S.: $k \mapsto k + 1$.

$\mathcal{I}_k$ is already defined by the induction hypothesis. We first set

$$\mathcal{I}_{k+1}(p_i) := \mathcal{I}_k(p_i) \quad \text{for all } i = 1, \cdots, k$$

and must now define $\mathcal{I}_{k+1}(p_{k+1})$. This is done by case analysis.

(a) There exists a clause

$$C \cup \{p_{k+1}\} \in \overline{M},$$

such that C contains at most the variables $p_1, \cdots, p_k$ and moreover $\mathcal{I}_k(C) = \texttt{false}$. In this case, we set

$$\mathcal{I}_{k+1}(p_{k+1}) := \texttt{true},$$

since otherwise $\mathcal{I}_{k+1}\big(C \cup \{p_{k+1}\}\big) = \texttt{false}$ would hold.

We must now show that for every clause $D \in \overline{M}$ that contains only the variables $p_1, \cdots, p_k, p_{k+1}$, the statement $\mathcal{I}_{k+1}(D) = \texttt{true}$ holds. There are three possibilities:

1. Case: $\texttt{var}(D) \subseteq \{p_1, \cdots, p_k\}$
   Then the claim holds by the induction hypothesis, since the valuations $\mathcal{I}_{k+1}$ and $\mathcal{I}_k$ agree on these variables.
2. Case: $p_{k+1} \in D$.
   Since we defined $\mathcal{I}_{k+1}(p_{k+1})$ as $\texttt{true}$, $\mathcal{I}_{k+1}(D) = \texttt{true}$ holds.
3. Case: $(\neg p_{k+1}) \in D$.
   Then D has the form

   $$D = E \cup \{\neg p_{k+1}\}.$$

   At this point, we need the fact that $\overline{M}$ is saturated. We apply the cut rule to the clauses $C \cup \{p_{k+1}\}$ and $E \cup \{\neg p_{k+1}\}$:

   $$C \cup \{p_{k+1}\}, \quad E \cup \{\neg p_{k+1}\} \quad \vdash \quad C \cup E$$

   Since $\overline{M}$ is saturated, $C \cup E \in \overline{M}$. $C \cup E$ contains only the variables $p_1, \cdots, p_k$. Therefore by the induction hypothesis,

   $$\mathcal{I}_{k+1}(C \cup E) = \mathcal{I}_k(C \cup E) = \texttt{true}.$$

   Since $\mathcal{I}_k(C) = \texttt{false}$, $\mathcal{I}_k(E) = \texttt{true}$ must hold. Thus we have

   $$\begin{array}{rcl} \mathcal{I}_{k+1}(D) & = & \mathcal{I}_{k+1}\big(E \cup \{\neg p_{k+1}\}\big) \\ & = & \mathcal{I}_k(E) \mathbin{\textcircled{\vee}} \mathcal{I}_{k+1}(\neg p_{k+1}) \\ & = & \texttt{true} \mathbin{\textcircled{\vee}} \texttt{false} = \texttt{true} \end{array}$$

   and that was to be shown.

(b) There is no clause

$$C \cup \{p_{k+1}\} \in \overline{M},$$

such that C contains at most the variables $p_1, \cdots, p_k$ and moreover $\mathcal{I}_k(C) = \texttt{false}$. In this case, we set

$$\mathcal{I}_{k+1}(p_{k+1}) := \texttt{false}.$$

In this case, for clauses $D \in \overline{M}$, there are three cases:

1. D does not contain the variable p_{k+1}.
   In this case, $\mathcal{I}_{k+1} = \texttt{true}$ already holds by the induction hypothesis.
2. $D = E \cup \{p_{k+1}\}$.
   Then we must have $\mathcal{I}_k(E) = \texttt{true}$ since otherwise we would be in case (a) of the outer case distinction. Obviously, $\mathcal{I}_k(E) = \texttt{true}$ implies $\texttt{I}_k(D) = \texttt{true}$.
3. $D = E \cup \{\neg p_{k+1}\}$.
   In this case, by definition of $\mathcal{I}$, $\mathcal{I}_{k+1}(p_{k+1}) = \texttt{false}$ holds and hence $\mathcal{I}_{k+1}(\neg p_{k+1}) = \texttt{true}$ follows, resulting in $\mathcal{I}_{k+1}(D) = \texttt{true}$.

Through the induction, we have thus shown that the propositional valuation $\mathcal{I}_N$ renders all clauses $C \in \overline{M}$ `true`, as only the propositional variables $p_1, \cdots, p_N$ occur in $\overline{M}$. Since $M \subseteq \overline{M}$, $\mathcal{I}_N$ also renders all clauses from M `true`, contradicting the assumption $M \models \bot$. □

4.6.4 Constructive Interpretation of the Proof of Refutation Completeness

In this section, we implement a program that takes a set of clauses M as its input. If the empty set of clauses cannot be derived from M, i.e. if

$$M \not\vdash \{\},$$

then it returns a propositional valuation $\mathcal{I}$ that satisfies all clauses in M. This program is shown in Figures 4.10, 4.11, and 4.12 on the following pages. You can find this program at the address

`github.com/karlstroetmann/Logic/blob/master/TypeScript/Chapter-4/05-Completeness.ipynb`

online.

The basic idea of this program is that we attempt to derive all clauses from a given set M of clauses that can be derived from M by using the cut rule. Let us call this set $\widetilde{M}$. If $\widetilde{M}$ contains the empty clause, then, due to the correctness of the cut rule, M has to be unsatisfiable. However, if we fail to derive the empty clause from M, we construct a propositional valuation $\mathcal{I}$ from $\widetilde{M}$ such that $\mathcal{I}$ satisfies all clauses from M. We first discuss the auxiliary procedures shown in Figure 4.10.

1. The function `complement` takes as argument a literal l and computes the complement $\overline{l}$ of this literal. If the literal l is a propositional variable p, which we recognize by l being a string, then we have $\overline{p} = \neg p$. If l is in the form $\neg p$ with a propositional variable p, then $\overline{\neg p} = p$.

2. The function `extractVariable` extracts the propositional variable contained in a literal l. The implementation is analogous to the previous implementation of the function `complement` with a case distinction, considering whether l is either in the form p or in the form $\neg p$, where p is the propositional variable to be extracted.

3. The function `collectVars` takes as an argument a set M of clauses, where each clause $C \in M$ is represented as a set of literals. The task of the function `collectVars` is to compute the set of all

```
function complement(l: Literal): Literal {
    if (typeof l == 'string') { return tpl('¬', l); }
    return l.get(1);
}

function extractVariable(l: Literal): Variable {
    if (typeof l == 'string') { return l; }
    return l.get(1);
}

function collectVariables(M: Set<Clause>): Set<Variable> {
    let result = set<Variable>();
    return result.flatMap(M, cl => cl.map(extractVariable));
}

function cutRule(C1: Clause, C2: Clause): Set<Clause> {
    return C1.filterMap(
        l => C2.has(complement(l)),
        l => {
            const lBar = complement(l);
            // (C1 \ {l}) U (C2 \ {lBar})
            return C1.difference(set(l)).union(C2.difference(set(lBar)));
        }
    );
}
```

Figure 4.10: Auxiliary functions that are used in Figure 4.11.

propositional variables that occur in any of the clauses C from M. We compute the union of the sets of variables occurring in any clause C of M using the method `flatMap`.

4. The function `cutRule` takes two clauses C_1 and C_2 as its arguments and calculates the set of all clauses that can be derived using an application of the cut rule from C_1 and C_2. For example, from the two clauses

   $$\{p, q\} \quad \text{and} \quad \{\neg p, \neg q\}$$

   we can derive both the clause

   $$\{q, \neg q\} \quad \text{as well as the clause} \quad \{p, \neg p\}$$

   using the cut rule.

Figure 4.11 shows the function `saturate`. This function takes as input a set `Clauses` of propositional clauses, represented as sets of literals. The task of the function is to derive all clauses that can be derived directly or indirectly from the set `Clauses` using the cut rule. Specifically, the set S of clauses returned by the function `saturate` is saturated under the application of the cut rule, meaning:

```
26 function saturate(Clauses: Set<Clause>): Set<Clause> {
27     let currentClauses = Clauses.clone();
28     while (true) {
29         const sizeBefore = currentClauses.size;
30         // 1. Generate all possible pairs of clauses (C1, C2)
31         const pairs = currentClauses.cartesianProduct(currentClauses);
32         // 2. Map each pair through the cutRule, flattening the results into one set
33         const derived = set<Clause>().flatMap(pairs, ([C1, C2]) => cutRule(C1, C2));
34         if (derived.some(D => D.isEmpty())) {
35             return set(set());
36         }
37         currentClauses = currentClauses.union(derived);
38         if (currentClauses.size == sizeBefore) {
39             return currentClauses;
40 }}}
```

Figure 4.11: The function `saturate`.

1. If S contains the empty clause $\{\}$, then S is saturated.
2. Otherwise, `Clauses` must be a subset of S and additionally the following must hold: If for a literal l both the clause $C_1 \cup \{l\}$ and the clause $C_2 \cup \{\overline{l}\}$ are contained in S, then the clause $C_1 \cup C_2$ is also an element of the set of clauses S:

$$C_1 \cup \{l\} \in S \ \wedge \ C_2 \cup \{\overline{l}\} \in S \ \Rightarrow \ C_1 \cup C_2 \in S$$

We now explain the implementation of the function `saturate`.

1. The `while` loop that starts in line 28 is tasked with applying the cut rule as long as possible to derive new clauses from the given clauses using the cut rule. Since this loop's condition has the value `true`, it can only be terminated by executing one of the two `return` commands in line 35 or line 39.
2. In line 33, the set `derived` is defined as the set of clauses that can be inferred using the cut rule from two of the clauses in the set `currentClauses`.
3. If the set `derived` contains the empty clause, then the set `Clauses` is contradictory, and the function `saturate` returns as a result the set $\{\{\}\}$. Note that the set $\{\{\}\}$ corresponds to falsum.
4. We add the newly found clauses to the set `currentClauses` in line 37.
5. If we find in line 38 that we have not derived any new clauses, then the set `currentClauses` is saturated and we return this set in line 39.

At this point, we must verify that the `while` loop will eventually terminate. This is due to two reasons:

1. In each iteration of the loop, the number of elements of the set `currentClauses` is increased by at least one, since we check that the size has changed.

2. The set `Clauses`, with which we originally start, contains a certain number n of propositional variables. However, the application of the cut rule does not generate any new variables. Therefore, the number of propositional variables that appear in `currentClauses` always remains the same. This limits the number of literals that can appear in `Clauses`: If there are only n propositional variables, then there can be at most $2 \cdot n$ different literals. However, each clause from `currentClauses` is a subset of the set of all literals. Since a set with k elements has a total of 2^k subsets, there can be at most $2^{2 \cdot n}$ different clauses that can appear in `currentClauses`.

From the two reasons given above, we can conclude that the `while` loop that starts in line 28 will terminate after at most $2^{2 \cdot n}$ iterations. Fortunately, it will require far fewer iterations in practical applications.

```
41  function findValuation(Clauses: Set<Clause>): false | Set<Literal> {
42      const Saturated = saturate(Clauses);
43      if (Saturated.some(c => c.isEmpty())) { return false; }
44      const Variables = collectVariables(Clauses);
45      const Literals = set<Literal>();
46      for (const p of Variables) {
47          // A clause forces 'p' to be true if it contains 'p',
48          // AND all OTHER literals in the clause evaluate to false.
49          // (i.e., their complements are already in our Literals set)
50          const forcesP = Saturated.some(
51              C => C.has(p) &&
52                   C.difference(set(p)).every(l => Literals.has(complement(l)))
53          );
54          Literals.add(forcesP ? p : complement(p));
55      }
56      return Literals;
57  }
```

Figure 4.12: The function `findValuation`.

Next, we discuss the implementation of the function `findValuation`, which is shown in Figure 4.12. This function receives a set `Clauses` of clauses as input. If this set is contradictory, the function should return the result `false`. Otherwise, the function should compute a propositional valuation $\mathcal{I}$ that satisfies all clauses from the set `Clauses`. In detail, the function `findValuation` works as follows.

1. In line 42, we saturate the set `Clauses` and compute all clauses that can be derived from the original set of clauses using the cut rule.

2. If the empty clause can be derived, then it follows from the correctness of the cut rule that the original set of clauses is contradictory, and we return `false` instead of a valuation, as a contradictory set of clauses is certainly not satisfiable.

3. In line 44, we compute the set of all propositional variables that appear in the set `Clauses`. We need this set because in the propositional valuation, which we want to return as a result, we

must map these variables to the set $\{\texttt{true}, \texttt{false}\}$.

4. Next, we compute a propositional valuation under which all clauses from the set `Clauses` become `true`. For this purpose, we first compute a set of literals, which we store in the variable `Literals`. The idea is that we include the propositional variable `p` in the set `Literals` exactly when the sought-after valuation $\mathcal{I}$ evaluates the propositional variable `p` to `true`. Otherwise, we include the literal $\neg$`p` in the set `Literals`. We return this set `Literals` as the result. The sought-after propositional valuation $\mathcal{I}$ can then be computed according to the formula

$$\mathcal{I}(\texttt{p}) = \begin{cases} \texttt{true} & \text{if } \texttt{p} \in \texttt{Literals} \\ \texttt{false} & \text{if } \neg\texttt{p} \in \texttt{Literals}. \end{cases}$$

5. The computation of the set `Literals` is done using a `for` loop. The idea is that for a propositional variable `p`, we add the literal `p` to the set `Literals` exactly when the valuation $\mathcal{I}$ has to map the variable `p` to `true` in order to satisfy the clauses. Otherwise, we add the literal $\neg$`p` to this set instead.

   The condition for adding the literal p is as follows: Suppose we have already found values for the variables $p_1, \cdots, p_n$ in the set `Literals`. The values of these variables are determined by the literals $l_1, \cdots, l_n$ in the set `Literals` as follows: If $l_i = p_i$, then $\mathcal{I}(p_i) = \texttt{true}$ and if $l_i = \neg p_i$, then we have $\mathcal{I}(p_i) = \texttt{false}$. Now assume that a clause C exists in the set `Clauses` such that

$$C \backslash \{p\} \subseteq \{\overline{l_1}, \cdots, \overline{l_n}\} \quad \text{and} \quad \texttt{p} \in C$$

   holds. If $\mathcal{I}(C) = \texttt{true}$ is to hold, then $\mathcal{I}(\texttt{p}) = \texttt{true}$ must hold, because by construction of $\mathcal{I}$ we have

$$\mathcal{I}(\overline{l_i}) = \texttt{false} \quad \text{for all } i \in \{1, \cdots, n\}$$

   and thus `p` is the only literal in the clause C that we can still make `true` with the help of the valuation $\mathcal{I}$. In this case, we thus add the literal `p` to the set `Literals`. Otherwise, the literal $\neg p$ is added to the set `Literals`.

The definition of the function `findValuation` implements the inductive definition of the valuations $\mathcal{I}_k$ that we specified in the proof of refutation completeness.

4.7 The Algorithm of Davis and Putnam

In practice, we often have to compute a propositional valuation $\mathcal{I}$ for a given set of clauses K such that

$$\texttt{evaluate}(C, \mathcal{I}) = \texttt{true} \quad \text{for all } C \in K$$

holds. In this case, the valuation $\mathcal{I}$ is a solution of the set of clauses K. In the last section, we introduced the function `findValuation` which can be used to compute such a solution of a set of clauses. Unfortunately, this function is too inefficient to be useful in practice. Therefore, in this section, we will introduce an algorithm that enables us to compute a solution for a set of propositional clauses in many practically relevant cases, even when the number of variables is large. This procedure is based on Davis and Putnam [DP60, DLL62]. Refinements of this procedure [MMZ+01] are used, for example, to verify the correctness of digital electronic circuits.

To motivate the algorithm, let us first consider those cases where it is immediately clear whether there is a valuation that solves a set of clauses K. Consider the following example:

$$K_1 = \{\ \{p\},\ \{\neg q\},\ \{r\},\ \{\neg s\},\ \{\neg t\}\ \}$$

The clause set K_1 corresponds to the propositional formula

$$p \wedge \neg q \wedge r \wedge \neg s \wedge \neg t.$$

Therefore, K_1 is solvable and the valuation

$$\mathcal{I} = \{\ p \mapsto \texttt{true},\ q \mapsto \texttt{false},\ r \mapsto \texttt{true},\ s \mapsto \texttt{false},\ t \mapsto \texttt{false}\ \}$$

is a solution of K_1. Consider another example:

$$K_2 = \{\ \{\}, \{p\},\ \{\neg q\},\ \{r\}\ \}$$

This clause set corresponds to the formula

$$\bot \wedge p \wedge \neg q \wedge r.$$

Obviously, K_2 is unsolvable. As a final example, consider

$$K_3 = \{\{p\}, \{\neg q\}, \{\neg p\}\}.$$

This clause set encodes the formula

$$p \wedge \neg q \wedge \neg p$$

and is obviously also unsolvable, as a solution $\mathcal{I}$ would have to make the propositional variable p both `true` and `false` simultaneously. We take the observations made in the last three examples as the basis for two definitions.

Definition 25 (Unit Clause) *A clause C is a unit clause if C consists of only one literal. Then either*

$$C = \{p\} \quad \text{or} \quad C = \{\neg p\}$$

for a propositional variable p. ◇

Definition 26 (Trivial Set of Clauses) *A set of clauses K is a trivial set of clauses if one of the following two cases applies:*

1. *K contains the empty clause, so $\{\} \in K$.*

   In this case, K is obviously unsolvable.

2. *K contains only unit clauses with different propositional variables. Denoting the set of propositional variables by $\mathcal{P}$, this condition is expressed as*

   (a) $\texttt{card}(C) = 1$ *for all* $C \in K$ *and*

   (b) There is no $p \in \mathcal{P}$ such that:

   $$\{p\} \in K \quad \text{and} \quad \{\neg p\} \in K.$$

   In this case, we can define the propositional valuation $\mathcal{I}$ as follows:

   $$\mathcal{I}(p) = \begin{cases} \texttt{true} & \text{if} \quad \{p\} \in K, \\ \texttt{false} & \text{if} \quad \{\neg p\} \in K. \end{cases}$$

   Then $\mathcal{I}$ is a solution of the set of clauses K. ◇

How can we transform a given set of clauses into a trivial set of clauses? There are three ways to simplify a set of clauses:

1. The Cut Rule,
2. Subsumption, and
3. Case Distinction.

We are already familiar with the cut rule. We will explain the other two methods in more detail later.

4.7.1 Simplification with the Cut Rule

A typical application of the resolution rule has the form:

$$\frac{C_1 \cup \{p\} \quad \{\neg p\} \cup C_2}{C_1 \cup C_2}$$

Usually, the clause $C_1 \cup C_2$ that is generated here will contain more literals than the premises $C_1 \cup \{p\}$ and $\{\neg p\} \cup C_2$. If the clause $C_1 \cup \{p\}$ contains a total of $m+1$ literals and the clause $\{\neg p\} \cup C_2$ contains a total of $n+1$ literals, then the conclusion $C_1 \cup C_2$ can contain up to $m+n$ literals. Of course, it can also contain fewer literals if there are literals that occur in both C_1 and C_2. Often, $m+n$ is greater than both $m+1$ and $n+1$. We are only certain that the clauses do not grow if we have $n = 0$ or $m = 0$. This case occurs when one of the two clauses consists of a single literal and is consequently a unit clause. Since our goal is to simplify the set of clauses, we allow only applications of the resolution rule where one of the clauses is a unit clause. Such cuts are referred to as unit cuts. To perform all possible cuts with a given unit clause $\{l\}$, we define a function

$$\texttt{unitCut} : 2^{\mathcal{K}} \times \mathcal{L} \to 2^{\mathcal{K}}$$

such that for a set of clauses K and a literal l the function $\texttt{unitCut}(K, l)$ simplifies the set of clauses K as much as possible with unit cuts with the clause $\{l\}$:

$$\texttt{unitCut}(K, l) = \Big\{ C \backslash \{\overline{l}\} \;\Big|\; C \in K \Big\}.$$

Note that the set $\texttt{unitCut}(K, l)$ contains the same number of clauses as the set K. However, those clauses from the set K that contain the literal $\overline{l}$ have been reduced. All other clauses from K remain unchanged.

Of course, we may only simplify a set of clauses K using the expression $\texttt{unitCut}(K, l)$ if the unit clause $\{l\}$ is an element of the set K.

4.7.2 Simplification via Subsumption

We start by demonstrating the principle of subsumption with an example. Consider the set of clauses

$$K = \big\{\{p, q, \neg r\}, \{p\}\big\} \cup M.$$

Clearly, the clause $\{p\}$ implies the clause $\{p, q, \neg r\}$, because whenever $\{p\}$ is satisfied, $\{q, p, \neg r\}$ is automatically satisfied as well. This is because

$$\models p \to q \vee p \vee \neg r$$

holds. Generally, we say that a clause C is subsumed by a unit clause U when

$$U \subseteq C$$

applies. If K is a set of clauses with $C \in K$, the unit clause $U \in K$, and C is subsumed by U, then we can reduce the set K by unit subsumption to the set $K - \{C\}$, thus we can delete the clause C from K. In order to implement this, we define a function

$$\texttt{subsume} : 2^{\mathcal{K}} \times \mathcal{L} \to 2^{\mathcal{K}},$$

which simplifies a given set of clauses K, containing the unit clause $\{l\}$, through subsumption by deleting all clauses subsumed by $\{l\}$ from K. The unit clause $\{l\}$ itself is, of course, retained. Therefore, we define:

$$\texttt{subsume}(K, l) := \big\{C \in K \mid l \notin C\big\} \cup \big\{\{l\}\big\}.$$

In the above definition, the unit clause $\{l\}$ has to be included in the result since the set

$$\big\{C \in K \mid l \notin C\big\}$$

does not contain the unit clause $\{l\}$. The two sets of clauses $\texttt{subsume}(K, l)$ and K are equivalent if and only if $\{l\} \in K$. Therefore, a set of clauses K is only simplified using the expression $\texttt{subsume}(K, l)$ if the unit clause $\{l\}$ is contained in the set K.

4.7.3 Simplification through Case Distinction

A calculus that only uses unit cuts and subsumption is not refutation-complete. Therefore, we need another method to simplify sets of clauses. Such a simplification is case distinction. This principle is based on the following theorem.

Proposition 27 *If K is a set of clauses and p is a propositional variable, then K is satisfiable if and only if $K \cup \big\{\{p\}\big\}$ or $K \cup \big\{\{\neg p\}\big\}$ is satisfiable.*

Proof:

"$\Rightarrow$": If K is satisfied by a valuation $\mathcal{I}$, then there are two possibilities for $\mathcal{I}(p)$, as $\mathcal{I}(p)$ is either `true` or `false`. If $\mathcal{I}(p) = \texttt{true}$, then the set $K \cup \big\{\{p\}\big\}$ is also satisfiable, otherwise $K \cup \big\{\{\neg p\}\big\}$ is satisfiable.

"$\Leftarrow$": Since K is a subset of both $K \cup \big\{\{p\}\big\}$ and of $K \cup \big\{\{\neg p\}\big\}$, it is clear that K is satisfiable if any of these sets is satisfiable. □

We can now simplify a set of clauses K by selecting a propositional variable p that occurs in K. Then we form the sets

$$K_1 := K \cup \big\{\{p\}\big\} \quad \text{and} \quad K_2 := K \cup \big\{\{\neg p\}\big\}$$

and recursively investigate whether K_1 is satisfiable. If we find a solution for K_1, this is also a solution for the original set of clauses K, and we have achieved our goal. Otherwise, we recursively investigate whether K_2 is satisfiable. If we find a solution, this is also a solution for K. If we find no solution for either K_1 or K_2, then K cannot have a solution either, since any solution $\mathcal{I}$ for K must make the variable p either `true` or `false`. The recursive examination of K_1 or K_2 is easier than the examination of K, because in K_1 and K_2, with the unit clauses $\{p\}$ and $\{\neg p\}$, we can perform both unit subsumptions and unit cuts, thereby simplifying these sets.

4.7.4 The Algorithm

We can now outline the Davis and Putnam algorithm . Given a set of clauses K, the goal is to find a solution for K. We are looking for a valuation $\mathcal{I}$, such that:

$$\mathcal{I}(C) = \texttt{true} \quad \text{for all } C \in K.$$

The algorithm of Davis and Putnam consists of the following steps.

1. Perform all unit cuts and unit subsumptions that are possible with clauses from K.
2. Then, if K is trivial, the procedure ends.
   (a) If $\{\} \in K$, then K is unsolvable.
   (b) Otherwise, the propositional valuation
   $$\mathcal{I} := \big\{p \mapsto \texttt{true} \mid p \in \mathcal{P} \wedge \{p\} \in K\big\} \cup \big\{p \mapsto \texttt{false} \mid p \in \mathcal{P} \wedge \{\neg p\} \in K\big\}$$
   is a solution for K.
3. If K is not trivial, select a propositional variable p that appears in K.
   (a) We then try recursively to solve the set of clauses
   $$K \cup \big\{\{p\}\big\}$$
   If successful, we have found a solution for K.
   (b) If $K \cup \big\{\{p\}\big\}$ is unsolvable, we instead try to solve the set of clauses
   $$K \cup \big\{\{\neg p\}\big\}$$
   If this also fails, K is unsolvable. Otherwise, we have found a solution for K.

In order to implement this algorithm, it is convenient to combine the two functions `unitCut`() and `subsume`() into a single function. Therefore, we define the function

$$\texttt{reduce} : 2^{\mathcal{K}} \times \mathcal{L} \to 2^{\mathcal{K}}$$

as follows:

$$\texttt{reduce}(K, l) = \Big\{ C \backslash \big\{\overline{l}\big\} \mid C \in K \wedge \overline{l} \in C \Big\} \cup \Big\{ C \in K \mid \overline{l} \notin C \wedge l \notin C \Big\} \cup \big\{\{l\}\big\}.$$

Thus, the set contains the results of cuts with the unit clause $\{l\}$ and we have removed those clauses that are subsumed by the unit clause $\{l\}$. The two sets K and $\texttt{reduce}(K, l)$ are logically equivalent if and only if $\{l\} \in K$. Therefore, K will only be replaced by $\texttt{reduce}(K, l)$ if $\{l\} \in K$.

4.7.5 An Example

To illustrate, we demonstrate the algorithm of Davis and Putnam with an example. The set K is defined as follows:

$$K := \Big\{\{p, q, s\},\ \{\neg p, r, \neg t\},\ \{r, s\},\ \{\neg r, q, \neg p\}, \{\neg s, p\},\\ \{\neg p, \neg q, s, \neg r\},\ \{p, \neg q, s\},\ \{\neg r, \neg s\},\ \{\neg p, \neg s\}\Big\}.$$

We will show that K is unsolvable. Since the set K contains no unit clauses, there is nothing to do in

the first step. Since K is not trivial, we are not finished yet. Thus, we proceed to step 3 and choose a propositional variable that occurs in K. At this point, it makes sense to choose a variable that appears in as many clauses of K as possible. We therefore choose the propositional variable p.

1. First, we form the set
   $$K_0 := K \cup \big\{\{p\}\big\}$$
   and attempt to solve this set. To do this, we form
   $$K_1 := \texttt{reduce}\big(K_0, p\big) = \Big\{\{r, \neg t\},\ \{r, s\},\ \{\neg r, q\},\ \{\neg q, s, \neg r\},\ \{\neg r, \neg s\},\ \{\neg s\},\ \{p\}\ \Big\}.$$
   The clause set K_1 contains the unit clause $\{\neg s\}$, so next we reduce using this clause:
   $$K_2 := \texttt{reduce}\big(K_1, \neg s\big) = \Big\{\{r, \neg t\},\ \{r\},\ \{\neg r, q\},\ \{\neg q, \neg r\},\ \{\neg s\},\ \{p\}\Big\}.$$
   We have found the new unit clause $\{r\}$. We use this unit clause to reduce K_2:
   $$K_3 := \mathit{reduce}\big(K_2, r\big) = \Big\{\{r\},\ \{q\},\ \{\neg q\},\ \{\neg s\},\ \{p\}\Big\}$$
   Since K_3 contains the unit clause $\{q\}$, we now reduce with q:
   $$K_4 := \mathit{reduce}\big(K_3, q\big) = \Big\{\{r\},\ \{q\},\ \{\},\ \{\neg s\},\ \{p\}\Big\}.$$
   The clause set K_4 contains the empty clause and is therefore unsolvable.

2. Thus, we now form the set
   $$K_5 := K \cup \big\{\{\neg p\}\big\}$$
   and attempt to solve this set. We form
   $$K_6 = \mathit{reduce}\big(K_5, \neg p\big) = \Big\{\{q, s\},\ \{r, s\},\ \{\neg s\},\ \{\neg q, s\},\ \{\neg r, \neg s\},\ \{\neg p\}\ \Big\}.$$
   The set K_6 contains the unit clause $\{\neg s\}$. We therefore form
   $$K_7 = \mathit{reduce}\big(K_6, \neg s\big) = \Big\{\{q\},\ \{r\},\ \{\neg s\},\ \{\neg q\},\ \{\neg p\}\ \Big\}.$$
   The set K_7 contains the new unit clause $\{q\}$. We use this unit clause to reduce K_7:
   $$K_8 = \mathit{reduce}\big(K_7, q\big) = \Big\{\{q\},\ \{r\},\ \{\neg s\},\ \{\},\ \{\neg p\}\ \Big\}.$$
   Since K_8 contains the empty clause, K_8 and thus the originally given set K is unsolvable.

In this example, we were fortunate as we only had to make a single case distinction. In more complex examples, it is often necessary to perform more than one case distinction.

4.7.6 Implementation

Next, we provide the implementation of the function `solve`, which can answer the question of whether a set of clauses is satisfiable. The implementation is shown in Figure 4.13 on page 80. The function receives one argument: The set `Clauses`. Here, `Clauses` is the set of clauses that needs to be solved. If the set `Clauses` is satisfiable, then the call

```
solve(Clauses)
```

returns a set of unit clauses `Result`, such that any valuation $\mathcal{I}$, which satisfies all unit clauses from `Result`, also satisfies all clauses from the set `Clauses`. If the set `Clauses` is not satisfiable, the call

```
solve(Clauses)
```

returns the set $\{\{\}\}$, as the empty clause represents the unsatisfiable formula $\bot$.

```
1  function solve(Clauses: Set<Clause>): Set<Clause> {
2      const Variables = flatMap(Clauses, clause => clause.map(extractVariable));
3      const Units     = Clauses.filter(C => C.size == 1);
4      return solveRecursive(Clauses, Units, Variables, set());
5  }
6
7  function solveRecursive(
8      Clauses:   Clauses,
9      Units:     Clauses,
10     Variables: Set<Variable>,
11     UsedVars:  Set<Variable>
12 ): Clauses {
13     const S = saturate(Clauses, Units);
14     const EmptyClause = set<Literal>();
15     if (S.has(EmptyClause))        { return set(EmptyClause); }
16     if (S.every(C => C.size == 1)) { return S;                }
17     const p = arb(Variables.difference(UsedVars));
18     const nextUsedVars = UsedVars.union(set(p));
19     const Result1 = solveRecursive(S, set(set(p)),
20                                    Variables,
21                                    nextUsedVars);
22     if (!Result1.has(EmptyClause)) { return Result1; }
23     const pBar = complement(p);
24     return solveRecursive(S, set(set(pBar)),
25                           Variables,
26                           nextUsedVars);
27 }
```

Figure 4.13: The function `solve`.

The implementations of `solve` and the auxiliary function `solveRecursive` are shown in Figure 4.13 on page 80.

1. The function `solve` serves as the entry point. In line 2, it first computes the set `Variables` containing all propositional variables that occur in the given set of `Clauses`. It does this by mapping `extractVariable` over all literals and computing the union of these sets.

2. In line 3, it computes the set `Units` of all unit clauses of `Clauses`.

3. In line 4, it calls the helper function `solveRecursive`, passing the original `Clauses`, the set

`Units`, the complete set of `Variables`, and an empty set `set()` to initialize `UsedVars`. This is the set of variables that have already been used in case distinctions. Initially, no unit clause has been used for a reduction and no variable has been used for a case distinction.

4. The real work is done in the function `solveRecursive`, which solves the set `Clauses` $\cup$ `Units`. Here, `Units` is a set of unit clauses that have not yet been used for unit cuts and unit subsumptions, while all other unit clauses of `Clauses` have already been used. In line 13, we reduce the set of clauses `Clauses` $\cup$ `Units` as much as possible by calling the function `saturate`. This applies all possible unit cuts and removes clauses subsumed by unit clauses, storing the result in `S`. The function `saturate` uses the unit clauses from the set `Units` and the unit clauses that are created by these reductions.

5. In line 14, we define `EmptyClause` to represent an empty set of literals.

6. In line 15, we check whether the saturated set `S` contains the empty clause. If it does, the set is contradictory, and we return the set containing the empty clause (which represents $\bot$).

7. In line 16, we check if all clauses in `S` are unit clauses (i.e., their size is exactly 1). If this is the case, the set `S` is trivial and represents a valid solution, so we return `S`.

8. If the set is neither contradictory nor a trivial solution, we must make a case distinction. In line 17, we select an arbitrary variable `p` from the set of `Variables` that has not yet been used (i.e., it is not in `UsedVars`). It is very important to use the function `arb` here, as `arb` picks a random element from a set. If we instead used a function that always picks the first element of a given set, the performance of the implementation would degrade seriously.

9. In line 18, we create the set `nextUsedVars` by adding the chosen variable `p` to the set of `UsedVars`.

10. In lines 19–21, we recursively try to solve the set of clauses assuming `p` is `true`. We do this by adding the unit clause `{p}` to `S` and calling `solveRecursive`. As `S` is saturated, all unit clauses of `S` have already been used. Hence, the new unit clause `{p}` is the only unit clause that has not been used yet and we pass the set `set(set(p))` as the argument `Units`. The result is stored in `Result1`.

11. In line 22, we check if this first attempt was successful. If `Result1` does not contain the empty clause, we have found a valid solution and we return `Result1`.

12. If the first attempt failed, we must try the opposite assignment. In line 23, we compute the complement of `p` (which is $\neg p$) and store it in `pBar`.

13. Finally, in lines 24–26, we recursively call `solveRecursive` again, this time adding the unit clause `{pBar}` to `S` to simulate the assignment where `p` is `false`. We return the result of this call; if this alternative also fails, the set of clauses is unsatisfiable.

We now discuss the auxiliary procedures used in the implementation of the function `solve`. First, we discuss the function `saturate`. This function receives a set `Clauses` of clauses and a set `Units` of unit clauses that have not yet been used for unit cuts and unit subsumptions. It performs all possible unit cuts and unit subsumptions on the set `Clauses` $\cup$ `Units`. Every unit clause $\{l\}$ of `Clauses` that is not an element of `Units` must already have been used, i.e. neither l nor $\overline{l}$ occurs in any other clause of `Clauses`. The function `solveRecursive` guarantees this. The function `saturate` is shown in Figure 4.14 on page 82.

```
1   function saturate(Clauses: Clauses, Units: Clauses): Clauses {
2       let S       = Clauses.union(Units);
3       let Pending = Units;
4       const EmptyClause = set<Literal>();
5       while (Pending.size > 0 && !S.has(EmptyClause)) {
6           const unit = first(Pending);
7           const [Reduced, NewUnits] = reduce(S, first(unit));
8           S       = Reduced;
9           Pending = Pending.difference(set(unit)).union(NewUnits);
10      }
11      return S;
12  }
```

Figure 4.14: The function `saturate`.

1. In line 2, we store the set `Clauses` $\cup$ `Units` in the variable `S`. The method `union` creates a new set, so the original set `Clauses` is not changed when we later assign new sets to `S`.

2. In line 3, we initialize the variable `Pending` with the set `Units`. The set `Pending` contains those unit clauses of `S` that have not yet been used for unit cuts and subsumptions.

3. In line 4, we define `EmptyClause` to represent the empty clause.

4. In line 5, we begin a `while` loop that is continued as long as there are unit clauses that have not been used and `S` does not contain the empty clause. As soon as `S` contains the empty clause, `S` is unsatisfiable and further reductions are pointless.

5. In line 6, we select an arbitrary unit clause from the set `Pending` and store it in the variable `unit`.

6. In line 7, the actual work is done by a call to the function `reduce`. By extracting the literal with `first(unit)`, this function performs all possible unit cuts with the unit clause and also removes all clauses that are subsumed by the unit clause. It returns a pair: The first component is the simplified set of clauses, which we store in `S` in line 8. The second component is the set `NewUnits` of those unit clauses that are created by the unit cuts.

7. In line 9, we remove the clause `unit` from the set `Pending`, since it has now been used, and add the new unit clauses. Note that we never have to search the set `S` for new unit clauses, since all of them are returned by `reduce`. This is important for efficiency, as `S` may contain thousands of clauses. Furthermore, every unit clause has to be used only once: After the reduction with a unit clause $\{l\}$, neither l nor $\overline{l}$ occurs in any other clause of `S`. As later reductions only remove or shorten clauses, this remains true.

8. At the end, in line 11, the remaining set of clauses `S` is returned.

The function `reduce` is shown in Figure 4.15. In the previous section, we defined the function $\mathit{reduce}(S, l)$, which reduces a set of clauses `Cs` using the literal l, as

$$\mathit{reduce}(\texttt{Cs}, l) = \Big\{ C \backslash \{\overline{l}\} \mid C \in \texttt{Cs} \wedge \overline{l} \in C \Big\} \cup \Big\{ C \in \texttt{Cs} \mid \overline{l} \notin C \wedge l \notin C \Big\} \cup \Big\{ \{l\} \Big\}$$

```
1  function reduce(Clauses: Clauses, l: Literal): [Clauses, Clauses] {
2      const lBar     = complement(l);
3      const result   = Clauses.filterMap(
4          cl => !cl.has(l),
5          cl => cl.has(lBar) ? cl.difference(set(lBar)) : cl
6      );
7      result.add(set(l));
8      const newUnits = Clauses.filterMap(
9          cl => cl.size == 2 && cl.has(lBar) && !cl.has(l),
10         cl => cl.difference(set(lBar))
11     );
12     return [result, newUnits];
13 }
```

Figure 4.15: The function `reduce`.

1. The function `reduce` takes a set of clauses `Clauses` and a literal `l` as arguments. It returns a pair: The first component is the reduced set of clauses, the second component is the set of those unit clauses that are created by the unit cuts.

2. In line 2, it computes the complement of the literal `l` using the helper function `complement` and stores it in the constant `lBar`.

3. In lines 3 to 6, the method `filterMap` is applied to the set `Clauses` to perform both unit subsumptions and unit cuts simultaneously.

4. The first argument to `filterMap`, `cl => !cl.has(l)`, acts as a filter. It retains only those clauses that do not contain the literal `l`. This implements unit subsumption, as any clause containing `l` is already satisfied and can be discarded.

5. The second argument,

   ```
   cl => cl.has(lBar) ? cl.difference(set(lBar)) : cl,
   ```

   maps the remaining clauses. If a clause contains the complementary literal `lBar`, this literal is removed using the `difference` method, which implements the unit cut. Otherwise, the clause remains unchanged. The modified set of clauses is stored in the constant `result`.

6. In line 7, the unit clause containing just the literal `l` is explicitly added back to the `result` set.

7. In lines 8 to 11, we compute the set `newUnits`. A unit cut with the unit clause $\{l\}$ creates the unit clause $\{k\}$ from every clause of the form $\{\overline{l}, k\}$ where $k \neq l$. These clauses are selected by the filter `cl => cl.size == 2 && cl.has(lBar) && !cl.has(l)` and the literal `lBar` is removed

from them. As all other clauses either remain unchanged or are removed, `newUnits` contains exactly those unit clauses that are created by the unit cuts.

8. Finally, in line 12, the function returns the fully reduced set of clauses `result` together with the set `newUnits`.

```
1  function complement(l: Literal): Literal {
2      if (typeof l == 'string') { return new Tuple('¬', l); }
3      return l.get(1);
4  }
5
6  function extractVariable(l: Literal): Variable {
7      if (typeof l == 'string') { return l; }
8      return l.get(1);
9  }
```

Figure 4.16: The functions `complement` and `extractVariable`.

Figure 4.16 shows the auxiliary functions `complement` and `extractVariable`.

1. The first function, `complement`, takes a literal `l` as its argument and computes its complement.
2. In line 2, it checks whether the literal `l` is a positive propositional variable by checking if its type is a string. If so, it returns the negated literal by creating a new `Tuple` containing the negation symbol '¬' and the variable `l`.
3. In line 3, if `l` is not a string, it is a negated literal represented as a tuple. The function returns the positive complement by extracting the second element of the tuple using `l.get(1)`, which corresponds to the underlying propositional variable.
4. The second function, `extractVariable`, takes a literal `l` as its argument and extracts the base propositional variable it contains.
5. In line 7, it checks if `l` is a string. If it is, the literal is already a positive propositional variable, so it simply returns `l` unchanged.
6. In line 8, if `l` is a negated literal (a tuple), it extracts and returns the underlying propositional variable by accessing the second element of the tuple using `l.get(1)`.

You can find the Jupyter notebook implementing the Davis-Putnam algorithm at:

github.com/karlstroetmann/Logic/blob/master/TypeScript/Chapter-4/06-Davis-Putnam.ipynb.

4.7.7 The Jeroslow-Wang Heuristic

In our previous implementation of the Davis-Putnam algorithm, we selected the variable for the case distinction arbitrarily. However, the choice of the splitting variable has a significant impact on

the efficiency of the algorithm. To minimize the search tree, it is highly advantageous to choose a literal that frequently occurs in short clauses, as this quickly leads to unit clauses and triggers further simplifications via unit propagation.

```
1  function scoreLiteral(Cls: Clauses, l: Literal): number {
2      return Cls.reduce((s, c) => c.has(l) ? s + 1 / 2 ** c.size: s, 0);
3  }
4
5  function selectLiteral(
6      Clauses:   Clauses,
7      Variables: Set<Variable>,
8      UsedVars:  Set<Variable>
9  ): Literal {
10     let maxLiteral: Literal = first(Variables);
11     let maxScore = -Infinity;
12     for (const variable of Variables) {
13         if (UsedVars.has(variable)) { continue; }
14         for (const literal of [variable, tpl('¬', variable)]) {
15             const score = scoreLiteral(Clauses, literal);
16             if (score > maxScore) {
17                 maxScore   = score;
18                 maxLiteral = literal;
19     }}}
20     return maxLiteral;
21 }
```

Figure 4.17: Selecting a literal using the Jeroslow-Wang heuristic.

The Jeroslow-Wang heuristic assigns a weight to each literal based on its occurrences in the current set of clauses. For a set of clauses `Clauses` and a literal l, the score $\texttt{JW}(\texttt{Clauses}, l)$ is defined as:

$$\texttt{JW}(\texttt{Clauses}, l) = \sum_{\{\, C \in \texttt{Clauses} \mid l \in C \,\}} \frac{1}{2^{|C|}}$$

Here, $|C|$ denotes the number of literals in the clause C. The idea is to choose a literal that occurs in many clauses and therefore subsumes a lot of clauses. Clauses with fewer literals are much more important than clauses with many literals, as it is much easier to satisfy a clause with many literals. If a clause has n different literals, then the probability that the clause is `false` is only $\frac{1}{2^n}$ if the propositional variables are assigned randomly.

Figure 4.17 shows the *TypeScript* functions `scoreLiteral` and `selectLiteral`, which compute this heuristic. The function `selectLiteral` works as follows:

1. The helper function `scoreLiteral` (lines 1–3) computes the Jeroslow-Wang score for a given literal `l`. It uses the `reduce` method to iterate over all clauses. If the literal is present in the

clause `c`, it adds $1/2^{|C|}$ to the running sum `s`; otherwise, it leaves `s` unchanged.

2. In lines 10 and 11, we initialize `maxLiteral` with an arbitrary variable and `maxScore` with negative infinity to keep track of the literal that achieves the highest score.
3. In line 12, we iterate over all `Variables`.
4. In line 13, we immediately skip the variable if it has already been assigned (i.e., it is present in `UsedVars`).
5. In line 14, we loop over both the positive literal `variable` and the negative literal `tpl('¬', variable)`.
6. In line 15, we calculate the score for the current `literal` using the `scoreLiteral` helper function.
7. In lines 16–19, if the computed `score` is greater than `maxScore`, we update both `maxScore` and `maxLiteral` to reflect the new highest-scoring candidate.
8. Finally, in line 20, the function returns the literal that accumulated the maximum score.

To utilize this heuristic, we must integrate it into our main recursive solving procedure. Figure 4.18 displays the updated *TypeScript* implementation of `solveRecursive`.

```
1  function solveRecursive(
2      Clauses:   Clauses,
3      Units:     Clauses,
4      Variables: Set<Variable>,
5      UsedVars:  Set<Variable>): Clauses
6  {
7      const S = saturate(Clauses, Units);
8      const EmptyClause = set<Literal>();
9      if (S.has(EmptyClause)) { return set(EmptyClause);  }
10     if (S.every(C => C.size == 1)) { return S; }
11     const l = selectLiteral(S, Variables, UsedVars);
12     const nextUsedVars = UsedVars.union(set(extractVariable(l)));
13     const Result1 = solveRecursive(S, set(set(l)), Variables, nextUsedVars);
14     if (!Result1.has(EmptyClause)) { return Result1; }
15     const lBar = complement(l);
16     return solveRecursive(S, set(set(lBar)), Variables, nextUsedVars);
17 }
```

Figure 4.18: The updated `solveRecursive` function using the heuristic.

The base cases for handling contradiction and trivial solutions remain identical to our previous implementation. The significant changes occur during the case distinction phase:

1. In line 11, instead of picking an arbitrary variable, we now call `selectLiteral` to find the most promising literal `l` based on the Jeroslow-Wang score. The scores are computed for the saturated set `S`, i.e. for the clauses that remain after all unit cuts and unit subsumptions have been performed.
2. In line 12, we compute the set `nextUsedVars`. Since `selectLiteral` returns a literal, we must extract the underlying propositional variable using `extractVariable(l)` before adding it to `UsedVars`.
3. In line 13, we perform the first recursive descent. We pass the saturated set `S` together with the unit clause `set(l)`, which is the only unit clause that has not been used yet. Because `selectLiteral` returns a specific literal (which could already be negative), we directly enforce this literal rather than simply assuming a positive variable.
4. In line 14, we check if the first recursive call yielded a valid solution. If it did, we return it.
5. In line 15, if the first path failed to find a solution, we compute the complement of our chosen literal, storing it in `lBar`.
6. Finally, in line 16, we recursively explore the alternative path by enforcing the complementary literal `lBar`.

You can find the Jupyter notebook implementing the Davis-Putnam algorithm with the Jeroslow-Wang heuristic at:

`github.com/karlstroetmann/.../Chapter-4/07-Davis-Putnam-JW.ipynb`.

4.8 Solving Logical Puzzles

In this section, we demonstrate how certain combinatorial problems can be reduced to the satisfiability of a set of formulas from propositional logic. These problems can then be solved using the algorithm of Davis and Putnam. We will discuss the solution of two puzzles:

1. We start with the 8-Queens Problem.
2. As a second example we discuss the Zebra Puzzle. This puzzle is also known as Einstein's Riddle, even though there is no evidence that it is related to Albert Einstein in any way.

4.8.1 The 8-Queens Problem

As our first example, we consider the 8-Queens Problem. This problem asks us to place 8 queens on a chessboard such that no queen can attack another queen. In the game of chess, a queen can attack another piece if that piece is either

- in the same row,
- in the same column, or
- on the same diagonal

as the queen. Figure 4.19 on page 88 shows a chessboard where a queen is placed in the third row of the fourth column. This queen can move to all the fields marked with arrows, and thus can attack pieces that are placed on these fields.

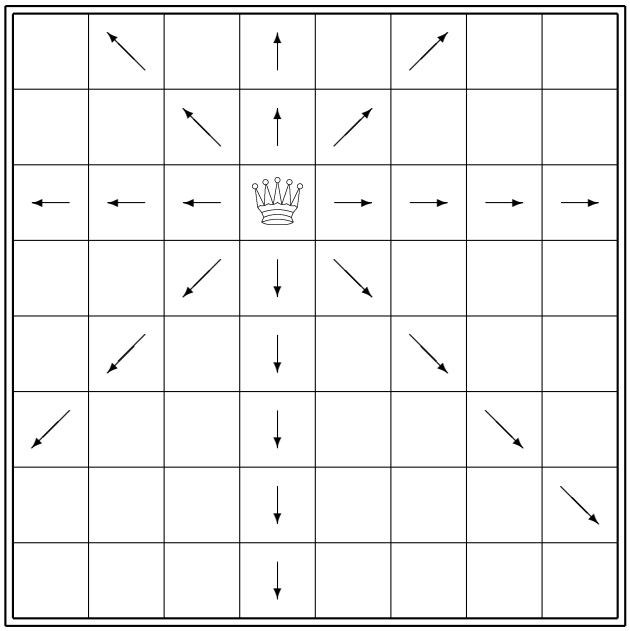


Figure 4.19: The 8-Queens-Problem.

First, let us consider how we can represent a chessboard with queens placed on it in propositional logic. One possibility is to introduce a propositional logic variable for each square. This variable expresses that there is a queen on the corresponding square. We assign names to these variables as follows: The variable that denotes the j-th column of the i-th row is represented by the string

"Q<i,j>" with $i, j \in \{1, \cdots, 8\}$

We number the rows from top to bottom, while the columns are numbered from left to right. Figure 4.21 on page 89 shows the assignment of variables to the squares. The function shown in Figure 4.20 varName(r, c) calculates the variable that expresses that there is a queen in row r and column c.

```
1  function varName(row: number, col: number): Variable {
2      return `Q<${row},${col}>`;
3  }
```

Figure 4.20: The function varName to compute a propositional variable.

Next, we consider how to encode the individual conditions of the 8-Queens problem as formulas from propositional logic. Ultimately, all statements of the form

- "at most one queen in a row",
- "at most one queen in a column", or

Q<1,1>	Q<1,2>	Q<1,3>	Q<1,4>	Q<1,5>	Q<1,6>	Q<1,7>	Q<1,8>
Q<2,1>	Q<2,2>	Q<2,3>	Q<2,4>	Q<2,5>	Q<2,6>	Q<2,7>	Q<2,8>
Q<3,1>	Q<3,2>	Q<3,3>	Q<3,4>	Q<3,5>	Q<3,6>	Q<3,7>	Q<3,8>
Q<4,1>	Q<4,2>	Q<4,3>	Q<4,4>	Q<4,5>	Q<4,6>	Q<4,7>	Q<4,8>
Q<5,1>	Q<5,2>	Q<5,3>	Q<5,4>	Q<5,5>	Q<5,6>	Q<5,7>	Q<5,8>
Q<6,1>	Q<6,2>	Q<6,3>	Q<6,4>	Q<6,5>	Q<6,6>	Q<6,7>	Q<6,8>
Q<7,1>	Q<7,2>	Q<7,3>	Q<7,4>	Q<7,5>	Q<7,6>	Q<7,7>	Q<7,8>
Q<8,1>	Q<8,2>	Q<8,3>	Q<8,4>	Q<8,5>	Q<8,6>	Q<8,7>	Q<8,8>

Figure 4.21: Interpretation of the variables.

- "at most one queen in a diagonal"

can be reduced to the same basic pattern: Given a set of propositional variables

$$V = \{x_1, \cdots, x_n\}$$

we need a formula that states that at most one of the variables in V has the value `true`. This is equivalent to the statement that for every pair $x_i, x_j \in V$ with $x_i \neq x_j$, the following formula holds:

$$\neg(x_i \wedge x_j).$$

This formula expresses that the variables x_i and x_j cannot both take the value `true` at the same time. According to De Morgan's laws, we have

$$\neg(x_i \wedge x_j) \Leftrightarrow \neg x_i \vee \neg x_j$$

and the clause on the right side of this equivalence can be written in set notation as

$$\{\neg x_i, \neg x_j\}.$$

```
function atMostOne(V: Set<Variable>): Set<Clause> {
    return V.cartesianProduct(V)
            .filterMap(([p, q]) => p != q,
                       ([p, q]) => set(tpl('¬', p), tpl('¬', q)));
}
```

Figure 4.22: The function `atMostOne`.

Therefore, the formula for a set of variables V that expresses that no two different variables are simultaneously `true` can be written as a set of clauses in the form

$$\big\{\{\neg p, \neg q\} \mid \langle p, q\rangle \in V \times V \wedge p \neq q\big\}$$

We implement these ideas in a *TypeScript* function. The function `atMostOne` shown in Figure 4.22 receives as input a set V of propositional variables. The call `atMostOne`(V) calculates a set of clauses. These clauses are `true` if and only if at most one of the variables in V has the value `true`.

With the help of the function `atMostOne`, we can now implement the function `atMostOneInRow`. The call

```
atMostOneInRow(row, n)
```

calculates for a given row `row` and a board size of `n` a formula that expresses that at most one queen is in the given `row`. Figure 4.23 shows the function `atMostOneInRow`: We collect all the variables of the row specified by `row` in the set

$$\big\{\texttt{varName(row,col)} \mid \texttt{col} \in \{1, \cdots, n\}\big\}$$

and call the function `atMostOne` with this set, which delivers the result as a set of clauses.

```
function atMostOneInRow(row: number, n: number): Set<Clause> {
    return atMostOne(range(1, n).map(col => varName(row, col)));
}
```

Figure 4.23: The function `atMostOneInRow`.

Next, we compute a formula that states that at least one queen is in a given column. For the first column, this formula would look like

$$\texttt{Q<1,1>} \vee \texttt{Q<2,1>} \vee \texttt{Q<3,1>} \vee \texttt{Q<4,1>} \vee \texttt{Q<5,1>} \vee \texttt{Q<6,1>} \vee \texttt{Q<7,1>} \vee \texttt{Q<8,1>}$$

and generally, for a column c where $c \in \{1, \cdots, 8\}$, the formula is

$$\texttt{Q<1,c>} \vee \texttt{Q<2,c>} \vee \texttt{Q<3,c>} \vee \texttt{Q<4,c>} \vee \texttt{Q<5,c>} \vee \texttt{Q<6,c>} \vee \texttt{Q<7,c>} \vee \texttt{Q<8,c>}.$$

When we write this formula in set notation as a set of clauses, we obtain

$\{\{$`Q<1,c>,Q<2,c>,Q<3,c>,Q<4,c>,Q<5,c>,Q<6,c>,Q<7,c>,Q<8,c>`$\}\}$.

Figure 4.24 shows a *TypeScript* function that calculates the corresponding set of clauses for a given column `col` and a given board size `n`. We return a set containing a single clause because our implementation of the Davis and Putnam algorithm works with a set of clauses.

```
function oneInColumn(col: number, n: number): Set<Clause> {
    return set(range(1, n).map(row => varName(row, col)));
}
```

Figure 4.24: The Function `oneInColumn`

At this point, you might expect us to provide formulas that express that there is at most one queen in a given column and that there is at least one queen in every row. However, such formulas are unnecessary because if we know that there is at least one queen in every column, we already know that there are at least 8 queens on the board. If we additionally know that there is at most one queen in each row, it follows that there are at most 8 queens on the board. Thus, there are exactly 8 queens on the board. Therefore, there can only be at most one queen per column, otherwise, we would have more than 8 queens on the board, and similarly, there must be at least one queen in each row, otherwise, we would not reach a total of 8 queens.

Next, let's consider how we can characterize the variables that are on the same diagonal. There are basically two types of diagonals: descending diagonals and ascending diagonals. We first consider the ascending diagonals. The longest ascending diagonal, also called the main diagonal, in the case of an 8×8 board consists of the variables

`Q<8,1>, Q<7,2>, Q<6,3>, Q<5,4>, Q<4,5>, Q<3,6>, Q<2,7>, Q<1,8>`.

The indices r and c of the variables `Q`(r, c) clearly satisfy the equation

$$r + c = 9.$$

In general, the indices of the variables of an ascending diagonal that contains more than one square satisfy the equation

$$r + c = k,$$

where k for an 8×8 chess board takes a value from the set $\{3, \cdots, 15\}$. We pass the value k as an argument to the function `atMostOneInRisingDiagonal`. This function is shown in Figure 4.25.

To see how the variables of a descending diagonal can be characterized, let's consider the main descending diagonal, which consists of the variables

`Q<1,1>, Q<2,2>, Q<3,3>, Q<4,4>, Q<5,5>, Q<6,6>, Q<7,7>, Q<8,8>`

The indices r and c of these variables clearly satisfy the equation

$$r - c = 0.$$

Generally, the indices of the variables on a descending diagonal satisfy the equation

$$r - c = k,$$

where k takes a value from the set $\{-6, \cdots, 6\}$. We pass the value k as an argument to the function

```
1  function atMostOneInRisingDiagonal(k: number, n: number): Set<Clause> {
2      const Numbers = range(1, n);
3      return atMostOne( Numbers.cartesianProduct(Numbers)
4                        .filterMap(
5                            ([row, col]) => row + col == k,
6                            ([row, col]) => varName(row, col)));
7  }
```

Figure 4.25: The function `atMostOneInRisingDiagonal`.

`atMostOneInFallingDiagonal`. This function is shown in Figure 4.26.

```
1  function atMostOneInFallingDiagonal(k: number, n: number): Set<Clause> {
2      const Numbers = range(1, n);
3      return atMostOne( Numbers.cartesianProduct(Numbers)
4                        .filterMap(
5                            ([row, col]) => row - col == k,
6                            ([row, col]) => varName(row, col)));
7  }
```

Figure 4.26: The function `atMostOneInFallingDiagonal`.

Now we are able to summarize our results: We can construct a set of clauses that fully describe the 8-Queens problem. Figure 4.27 shows the implementation of the function `allClauses`. The call

$$\texttt{allClauses}(n)$$

computes for a chessboard of size n a set of clauses that are satisfied if and only if on the chessboard

1. there is at most one queen in each row (line 2),
2. there is at most one queen in each rising diagonal (line 3),
3. there is at most one queen in each falling diagonal (line 4), and
4. there is at least one queen in each column (line 5).

The expressions in the individual lines yield sets of clauses. This is achieved by `flatMap`. In each line, `flatMap` takes a set as its first argument. The second argument is a function that maps each of the elements of the set to a set of clauses. `flatMap` then takes the union of these sets of clauses so that the end result is a set of clauses. The nice thing about `flatMap` is that it can be chained since it is a method of the class `Set`. It starts with the empty set of clauses and then each call of `flatMap` adds more sets of clauses to build up the resulting set.

Finally, we show in Figure 4.28 the function `queens`, with which we can solve the 8-Queens problem.

```
function allClauses(n: number): Set<Clause> {
    return set<Clause>()
           .flatMap(range(   1,   n), row => atMostOneInRow(row, n))
           .flatMap(range(   3, 2*n), k   => atMostOneInRisingDiagonal(k, n))
           .flatMap(range(-n+2, n-2), k   => atMostOneInFallingDiagonal(k, n))
           .flatMap(range(   1,   n), col => oneInColumn(col, n));
}
```

Figure 4.27: The function `allClauses`.

1. First, we encode the problem as a set of clauses that is solvable only if the problem has a solution.
2. Then, we compute the solution using the function `solve` from the notebook that implements the algorithm of Davis and Putnam.
3. If the problem is solvable, the solution is returned.

The complete program is available as a Jupyter Notebook on GitHub as

karlstroetmann/Logic/blob/master/TypeScript/Chapter-4/08-N-Queens.ipynb.

```
function queens(n: number): Set<Clause> {
    const Clauses     = allClauses(n);
    const Solution    = solve(Clauses);
    const EmptyClause = set<Literal>();
    if (Solution.has(EmptyClause)) {
        console.log(`The problem is not solvable for ${n} queens!`);
    }
    return Solution;
}
```

Figure 4.28: The function `queens` that solves the n-queens-problem.

A graphical representation of a computed solution can be seen in Figure 4.29.

Jessica Roth and Koen Loogman (who are two former DHBW students) implemented an animation of the Davis and Putnam procedure. You can find and try this animation at the address

https://koenloogman.github.io/Animation-Of-N-Queens-Problem-In-JavaScript/

on the internet.

The 8-Queens problem is of course only a playful application of propositional logic. Nevertheless, it demonstrates the effectiveness of the Davis and Putnam algorithm very well, as the set of clauses

computed by the function `allClauses` consists of 512 different clauses. In this set of clauses, 64 different variables appear.

In practice, there are many problems that can be similarly reduced to solving a set of clauses. For example, creating a timetable that meets certain constraints is one such problem. Generalizations of the timetable problem are referred to in the literature as Scheduling Problems. The efficient solution of such problems is a subject of current research.

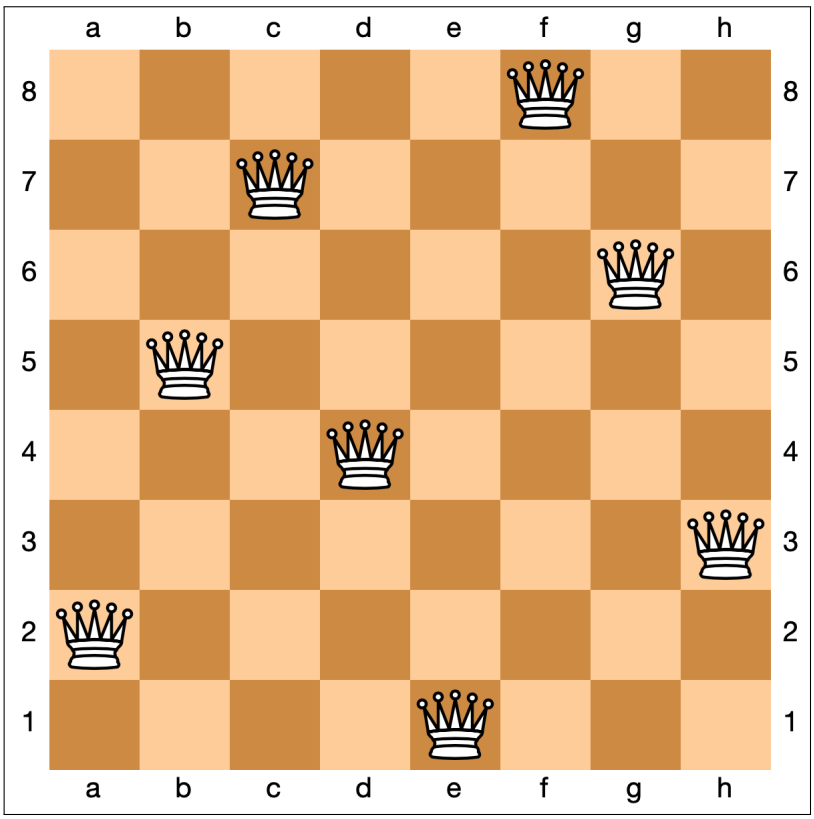


Figure 4.29: A solution of the 8-queens-problem.

Exercise 12: The following exercise is taken from the book 99 Logeleien von Zweistein [vR68]. This book was published in 1968. It was written by Thomas von Randow.

The gentlemen Amann, Bemann, Cemann *and* Demann *are called — not necessarily in the same order — by their first names* Erich, Fritz, Gustav *and* Heiner. *They are all married to exactly one woman. We also know the following about them and their wives:*

1. *Either* Amann's *first name is* Heiner, *or* Bemann's *wife is* Inge.
2. *If* Cemann *is married to* Josefa, *then — **and only in this case** —* Klara's *husband is **not** called* Fritz.
3. *If* Josefa's *husband is **not** called* Erich, *then* Inge *is married to* Fritz.
4. *If* Luise's *husband is called* Fritz, *then* Klara's *husband's first name is **not*** Gustav.
5. *If the wife of* Fritz *is called* Inge, *then* Erich *is **not** married to* Josefa.
6. *If* Fritz *is **not** married to* Luise, *then* Gustav's *wife's name is* Klara.
7. *Either* Demann *is married to* Luise, *or* Cemann *is called* Gustav.

What are the full names of these gentlemen, and what are their wives' first names?

Hint: You should use the following propositional variables to solve the problem.

1. `MaleName<`x`,`z`>` for any male first name x and any surname z expresses that the gentleman with first name x has surname z.
2. `Married<`x`,`y`>` for any male first name x and any female first name y expresses that the gentleman with first name x is married to the lady with first name y.
3. `FemaleName<`y`,`z`>` for any female first name y and any last name z expresses that the lady with first name y has surname z. ◇

I have provided the framework of a notebook at the following address:

`Chapter-4/Zweistein-Frame.ipynb`

You should edit this notebook in order to solve the given problem. ◇

4.8.2 The Zebra Puzzle

The following logical puzzle appeared in the magazine Life International on December 17th, 1962:

- There are five houses.
- The Englishman lives in the red house.
- The Spaniard owns the dog.
- Coffee is drunk in the green house.

- The Ukrainian drinks tea.
- The green house is immediately to the right of the ivory house.
- The Old Gold smoker owns snails.
- Kools are smoked in the yellow house.
- Milk is drunk in the middle house.
- The Norwegian lives in the first house.
- The man who smokes Chesterfields lives in the house next to the man with the fox.
- Kools are smoked in the house next to the house where the horse is kept.
- The Lucky Strike smoker drinks orange juice.
- The Japanese smokes Parliaments.
- The Norwegian lives next to the blue house.

Furthermore, each of the five houses is painted in a different colour, their inhabitants are of different nationalities, own different pets, drink different beverages, and smoke different brands of cigarettes.

Our task is to answer the following questions:

1. Who drinks water?
2. Who owns the zebra?

In order to formulate this puzzle as a propositional formula we first have to choose the appropriate propositional variables. In order to express the fact that the Englishman lives in the ith house we can use the variables

$$\texttt{English}\langle i\rangle \quad \text{for } i = 1, \cdots, 5.$$

The idea is that the variable $\texttt{English}\langle i\rangle$ is `true` iff the Englishman lives in the ith house. In a similar way we use the variables

$$\texttt{Spanish}\langle i\rangle, \ \texttt{Ukrainian}\langle i\rangle, \ \texttt{Norwegian}\langle i\rangle, \text{ and } \texttt{Japanese}\langle i\rangle$$

to encode where the people of different nationalities live. The different drinks are encoded by the variables

$$\texttt{Coffee}\langle i\rangle, \ \texttt{Milk}\langle i\rangle, \ \texttt{OrangeJuice}\langle i\rangle, \ \texttt{Tea}\langle i\rangle, \text{ and } \texttt{Water}\langle i\rangle.$$

The colours are encoded using the variables

$$\texttt{Red}\langle i\rangle, \ \texttt{Green}\langle i\rangle, \ \texttt{Ivory}\langle i\rangle, \ \texttt{Yellow}\langle i\rangle, \text{ and } \texttt{Blue}\langle i\rangle.$$

The pets are encoded using the variables

$$\texttt{Dog}\langle i\rangle, \ \texttt{Snails}\langle i\rangle, \ \texttt{Fox}\langle i\rangle, \ \texttt{Horse}\langle i\rangle, \text{ and } \texttt{Zebra}\langle i\rangle.$$

The cigarette brands are encoded as

$$\texttt{OldGold}\langle i\rangle, \ \texttt{Kools}\langle i\rangle, \ \texttt{Chesterfields}\langle i\rangle, \ \texttt{Luckies}\langle i\rangle, \text{ and } \texttt{Parliaments}\langle i\rangle.$$

We are using the angular brackets "⟨" and "⟩" as part of the variable names because our parser for propositional logic accepts these symbols as part of variable names. The parser would be confused if we used normal parentheses.

```
function varName(f: string, i: number): string {
    return `${f}<${i}>`;
}
```

Figure 4.30: The function varName(f, i).

In order to be able to write the conditions concisely, we introduce the function varName(f, i), which is shown in Figure 4.30. This function takes a string f, where f is a property like, e.g. Japanese and an integer $i \in \{1, \cdots, 5\}$ and returns the string $f\langle i \rangle$.

```
function somewhere(x: string): Set<Literal> {
    return range(1, 5).map(i => varName(x, i))
}
```

Figure 4.31: The function somewhere.

The function somewhere(x) that is shown in Figure 4.31 takes a string x as its argument, where x denotes a property like, e.g. Japanese. It returns a clause that specifies that x has to be located in one of the five houses. For example, the call somewhere('Japanese') returns the clause

```
{'Japanese<1>', 'Japanese<2>', 'Japanese<3>', 'Japanese<4>', 'Japanese<5>'}
```

as a Set.

```
function atMostOneAt(S: Set<string>, i: number): Set<Clause> {
    return atMostOne(S.map(x => varName(x, i)));
}
```

Figure 4.32: The function atMostOneAt(S, i)

Given a set of properties S and a house number i, the function atMostOneAt(S, i) returns a set of clauses that specifies that the person living in house number i has at most one of the properties from the set S. For example, if

$$S = \{\texttt{"Japanese"}, \texttt{"Englishman"}, \texttt{"Spaniard"}, \texttt{"Norwegian"}, \texttt{"Ukrainian"}\},$$

then atMostOneAt$(S, 3)$ specifies that the inhabitant of house number 3 has at most one of the nationalities from the set S. The implementation makes use of the function atMostOne that is shown in Figure 4.22 on page 90.

```
function onePerHouse(S: Set<Variable>): Set<Clause> {
    const Result = S.map(x => somewhere(x));
    return Result.flatMap(range(1, 5), i => atMostOneAt(S, i));
}
```

Figure 4.33: The function `onePerHouse`.

The function `onePerHouse`(S) takes a set S of properties. For example, S could be the set of the five nationalities. In this case, the function would create a set of clauses that expresses that there has to be a house where the Japanese lives, a house where the Englishman lives, a house where the Spaniard lives, a house where the Norwegian lives, and a house where the Ukrainian lives. Furthermore, the set of clauses would contain clauses that express the fact that these five persons live in different houses.

```
function sameHouse(a: Variable, b: Variable): Set<Clause> {
    return flatMap(range(1, 5),
                   i => parseKNF(`${varName(a, i)} ↔ ${varName(b, i)}`));
}
```

Figure 4.34: The function `sameHouse`(a, b).

Given two properties a and b the function `sameHouse`(a, b) that is shown in Figure 4.34 computes a set of clauses that specifies that if the inhabitant of house number i has the property a, then he also has the property b and vice versa. For example, `sameHouse`$($`"Japanese"`, `"Dog"`$)$ specifies that the Japanese guy keeps a dog. The function `parseKNF` that is used here takes a string, converts it into a formula from propositional logic, and finally converts this formula into CNF.

```
function nextTo(a: Variable, b: Variable): Set<Clause> {
    const Formulas = [
        `${varName(a, 1)} → ${varName(b, 2)}`, // House 1
        ...[2, 3, 4].map(
          i => `${varName(a, i)} → ${varName(b, i-1)} ∨ ${varName(b, i+1)}`),
        `${varName(a, 5)} → ${varName(b, 4)}`  // House 5
    ];
    const Result = set<Clause>();
    return Result.flatMap(Formulas, formula => parseKNF(formula));
}
```

Figure 4.35: nextTo

Given two properties a and b the function `nextTo`(a, b) computes a set of clauses that specifies that the inhabitants with properties a and b are neighbours. For example, `nextTo('Japanese','Dog')` specifies that the Japanese guy lives next to the guy who keeps a dog. This is specified as follows:

1. If the person with property a lives in the first house, then the person with property b has to live in the second house.
2. If the person with property a lives in one of the houses i where $i \in \{2, 3, 4\}$, then the person with property b either lives to the left in house $i - 1$ or to the right in house $i + 1$.
3. If the person with property a lives in the fifth house, then the person with property b has to live in the fourth house.

```
function leftTo(a: Variable, b: Variable): Set<Clause> {
    const Formulas = [
        ...[1, 2, 3, 4].map(i => `${varName(a, i)} → ${varName(b, i + 1)}`),
        `¬${varName(a, 5)}`
    ];
    const Result = set<Clause>();
    return Result.flatMap(Formulas, formula => parseKNF(formula));
}
```

Figure 4.36: The function `leftTo`.

Given two properties a and b the function `leftTo`(a, b) computes a list of clauses that specifies that the inhabitant with property a lives in the house to the left of the inhabitant who has property b. For example, `leftTo('Japanese','Dog')` specifies that the Japanese guy lives in the house to the left of the house where there is a dog.

```
const Nations = set("English", "Spanish", "Ukrainian", "Norwegian", "Japanese");
const Drinks  = set("Coffee", "Tea", "Milk", "OrangeJuice", "Water");
const Pets    = set("Dog", "Snails", "Horse", "Fox", "Zebra");
const Brands  = set("Luckies", "Parliaments", "Kools", "Chesterfields", "OldGold");
const Colours = set("Red", "Green", "Ivory", "Yellow", "Blue");
```

Figure 4.37: Definition of the properties.

In order to be able to formulate the different conditions conveniently, we define the sets shown in Figure 4.37.

The function `allClauses` in Figure 4.38 computes a set of clauses that encodes the conditions of the zebra puzzle. If we solve these clauses with the algorithm of Davis and Putnam, we get the solution that is shown in Figure 4.39.

```
function allClauses(): Set<Clause> {
    const Result = set<Clause>();
    // Every property category gets exactly one house
    Result.flatMap([Nations, Drinks, Pets, Brands, Colours], S => onePerHouse(S));
    const puzzleClues = [
        // The Englishman lives in the red house.
        sameHouse("English", "Red"),
        // The Spaniard owns the dog.
        sameHouse("Spanish", "Dog"),
        // Coffee is drunk in the green house.
        sameHouse("Coffee", "Green"),
        // The Ukrainian drinks tea.
        sameHouse("Ukrainian", "Tea"),
        // The green house is immediately to the right of the ivory house.
        leftTo("Ivory", "Green"),
        // The Old Gold smoker owns snails.
        sameHouse("OldGold", "Snails"),
        // Kools are smoked in the yellow house.
        sameHouse("Kools", "Yellow"),
        // Milk is drunk in the middle house.
        parseKNF(varName("Milk", 3)),
        // The Norwegian lives in the first house.
        parseKNF(varName("Norwegian", 1)),
        // Chesterfields smoker lives next to the fox owner.
        nextTo("Chesterfields", "Fox"),
        // Kools are smoked next to the horse.
        nextTo("Kools", "Horse"),
        // The Lucky Strike smoker drinks orange juice.
        sameHouse("Luckies", "OrangeJuice"),
        // The Japanese smokes Parliaments.
        sameHouse("Japanese", "Parliaments"),
        // The Norwegian lives next to the blue house.
        nextTo("Norwegian", "Blue")
    ];
    return Result.flatMap(puzzleClues, S => S);
}
```

Figure 4.38: The function `allClauses`.

House	Nationality	Drink	Animal	Brand	Colour
1	Norwegian	Water	Fox	Kools	Yellow
2	Ukrainian	Tea	Horse	Chesterfields	Blue
3	English	Milk	Snails	OldGold	Red
4	Spanish	OrangeJuice	Dog	LuckyStrike	Ivory
5	Japanese	Coffee	Zebra	Parliaments	Green

Figure 4.39: The solution of the zebra puzzle.

Exercise 13: **The Trial of the Nine Portals (by Raymond M. Smullyan [Smu82])**
In an era of high-stakes romantic gambles and questionable parenting, there reigned a King—a man whose passion for discrete mathematics far outweighed his concern for his daughter's emotional well-being.

Determined that his heir-in-law be a man of cold, calculating intellect rather than mere royal vibes, the King broadcast a decree across the realm. It was essentially a LinkedIn job posting for a Prince, but with significantly higher mortality rates.

Eventually, a Prince arrived. The King led him to a corridor containing **nine identical doors**, behind which lay one of three fates:

1. **The Princess**: The ultimate prize (and a lifetime of complex family dinners).
2. **A Ravenous Tiger**: An immediate career change to cat food.
3. **Void & Dust**: Empty rooms for those who enjoy the solace of disappointment.

The Royal Logic Protocol

The King, smirking with the malice of a tenured professor, pointed to the cryptic signs etched into each door. "To ensure you aren't a total dullard," he barked, "you must navigate this gauntlet using the **Infallible Laws of Logic**:"

1. **The Path of Teeth**: If a room contains a Tiger, the inscription on that door is a bold-faced lie, as blatantly false as the answer you get when you ask a tenured professor the stupid question "Was kommt in der Klausur?".
2. **The Path of Matrimony:** If the room contains the Princess, the inscription is absolute truth.
3. **The Path of the Void**: If the room is empty, we enter the realm of bureaucratic inconsistency. Either **all** signs on empty rooms are true, or **all** of them are false. There is no middle ground, only total logical uniformity or total deception.

"Choose wisely," the King added, adjusting his crown. "For in this kingdom, love isn't just a battlefield—it's a multiple-choice logic exam where the penalty for a 'C' is being digested."

The Prince looked at the doors. The doors looked back. Somewhere, a tiger sharpened its claws, and the Princess wondered if she should have just joined a convent.

The prince then read the inscriptions. These were as follows:

1. Room: The princess is in a room with an odd number and there is no tiger in the rooms with an even number.
2. Room: This room is empty.
3. Room: The inscription on Room No. 5 is `true`, the inscription on Room No. 7 is `false`, and there is a tiger in Room No. 3.
4. Room: The inscription on Room No. 1 is `false`, there is no tiger in Room No. 8, and the inscription on Room No. 9 is `true`.
5. Room: If the inscription on Room No. 2 or on Room No. 4 is `true`, then there is no tiger in Room No. 1.
6. Room: The inscription on Room No. 3 is `false`, the princess is in Room No. 2, and there is no tiger in Room No. 2.
7. Room: The princess is in Room No. 1 and the inscription on Room No. 5 is `true`.
8. Room: There is no tiger in this room and Room No. 9 is empty.
9. Room: Neither in this room nor in Room No. 1 is there a tiger, and moreover, the inscription on Room No. 6 is `true`.

I have provided the framework of a notebook at the following address:

`Chapter-4/Prince-Tiger-Frame.ipynb`

You should edit this notebook in order to solve the given problem. ◇

4.9 Sudoku

As another example we will solve a Sudoku puzzle. In particular, we will solve an instance of a Sudoku puzzle that was created by Arto Inkala. He claims that this is the hardest solvable instance of a Sudoku puzzle. Figure 4.40 shows the Sudoku created by Arto Inkala.

A Sudoku puzzle is a logic-based, combinatorial number-placement puzzle. The objective is to fill a 9×9 grid with digits from the set $\{1, \cdots, 9\}$ so that each column, each row, and each of the nine 3×3 subgrids that compose the grid (also called "boxes") contains all of these digits exactly once. The puzzle is given as a partially completed grid. A well-posed puzzle must have a single unique solution.

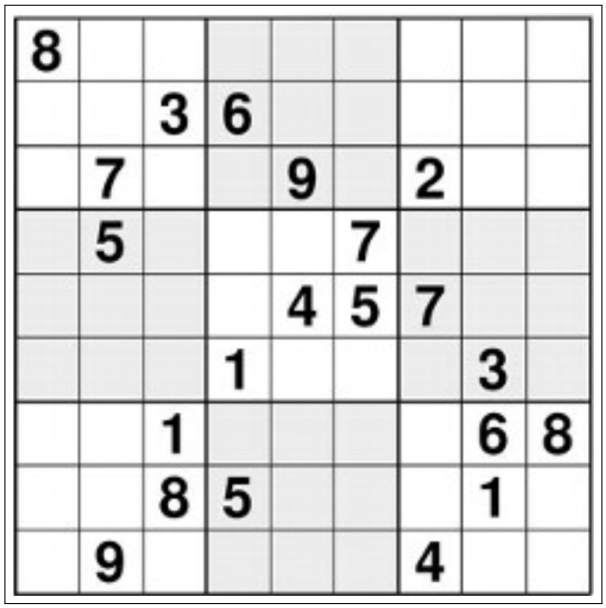


Figure 4.40: The Sudoku puzzle created by Arto Inkala.

In order to formulate a Sudoku as a satisfiability problem from propositional logic, we will use the following set of propositional variables:

$$\mathcal{P} = \{Q_{r,c,d} \mid r \in \{1,\cdots,9\} \wedge c \in \{1,\cdots,9\} \wedge d \in \{1,\cdots,9\}\}$$

For a given digit $d \in \{1,\cdots,9\}$, a given row $r \in \{1,\cdots,9\}$, and a given column $c \in \{1,\cdots,9\}$, the variable $Q_{r,c,d}$ is supposed to be `true` if d is the digit in row r and column c of the Sudoku. Note that this means that we have $9^3 = 729$ different propositional variables. In our implementation we will use the function `varName` shown in Figure 4.41 to create these variables.

```
1  function varName(row: number, col: number, digit: number): string {
2      return `Q<${row},${col},${digit}>`;
3  }
```

Figure 4.41: The function `varName`.

The Sudoku itself is created by the function `createPuzzle` shown in Figure 4.42 on page 104. In this function, the puzzle is represented as a list of lists. Each of these lists represents a row. The inner lists contain digits and the character `"*"`. This character represents those digits that are initially unknown and whose value has to be computed.

In our implementation we will use the function `atMostOne` that we have already seen in Figure 4.22 on page 90 that expresses that at most one of a given set of propositional variables is `true`. Furthermore, we will use the auxiliary function `atLeastOne`(V) which takes a set of propositional variables V and which returns a formula in conjunctive normal form in set notation that is `true` if and only if at least one of the variables in V is `true`. Note that in set notation we have

```
function createPuzzle(): (number | '*')[][] {
   return [[ 8 , '*', '*', '*', '*', '*', '*', '*', '*'],
           ['*', '*',  3,   6 , '*', '*', '*', '*', '*'],
           ['*',  7 , '*', '*',  9 , '*',  2 , '*', '*'],
           ['*',  5 , '*', '*', '*',  7 , '*', '*', '*'],
           ['*', '*', '*', '*',  4 ,  5 ,  7 , '*', '*'],
           ['*', '*', '*',  1 , '*', '*', '*',  3 , '*'],
           ['*', '*',  1 , '*', '*', '*', '*',  6 ,  8 ],
           ['*', '*',  8 ,  5 , '*', '*', '*',  1 , '*'],
           ['*',  9 , '*', '*', '*', '*',  4 , '*', '*']
   ];
}
```

Figure 4.42: The function `createPuzzle`.

$\texttt{atLeastOne}(V) = \{V\}$.

The reason is that in set notation V is interpreted as a clause, i.e. as a disjunction of its variables. Using these two functions we can then define a function $\texttt{exactlyOne}(V)$ that takes a set of variables and returns `true` iff exactly one of the variables in V is `true`. The implementation is shown in Figure 4.43.

```
function atLeastOne(S: Set<Variable>): Set<Clause> {
    return set(S);
}
function exactlyOne(S: Set<Variable>): Set<Clause> {
    return atMostOne(S).union(atLeastOne(S));
}
```

Figure 4.43: The functions `atLeastOne` and `exactlyOne`.

```
type Position = Tuple<[number, number]>;

function exactlyOneForDigit(L: Set<Position>, digit: number): Set<Clause> {
    return exactlyOne(L.map(([row, col]) => varName(row, col, digit)));
}
```

Figure 4.44: The function `exactlyOneForDigit`.

Figure 4.44 shows the implementation of the function `exactlyOneForDigit`(L, d).

First, we define the type `Position`, which is a pair of the form (r, c), where $r \in \{1, \cdots, 9\}$ specifies the row, while $c \in \{1, \cdots, 9\}$ specifies the column of the position.

The function `exactlyOneForDigit`(L, d) takes a set L of positions and a digit d as its arguments. The function `exactlyOneForDigit` computes a set of formulas that express that the digit d occurs in exactly one of the positions in L.

```
function exactlyOneForEveryDigit(L: Set<Position>): Set<Clause> {
    return flatMap(range(1, 9), d => exactlyOneForDigit(L, d));
}
```

Figure 4.45: The function `exactlyOneForEveryDigit`.

The function `exactlyOneForEveryDigit`(L) takes a set L of cell positions as its argument. It returns a set of formulas expressing that all of the nine digits occur at exactly one position in the list `L`.

```
function exactlyOneDigit(row: number, col: number): Set<Clause> {
    return exactlyOne(range(1, 9).map(d => varName(row, col, d)));
}
```

Figure 4.46: The function `exactlyOneDigit`.

The function `exactlyOneDigit`(r, c) takes a row r and a column c as its arguments. It returns a set of clauses expressing that there is exactly one digit in the given cell.

Furthermore, we need a function encoding the constraints from the puzzle itself. This function is called `constraintsFromPuzzle` and its implementation is shown in Figure 4.47 on page 106. For example, the given puzzle demands that the digit in row 2 and column 4 is the digit 6. This can be expressed as the formula $Q_{2,4,6}$. In set notation this formula takes the form

$$\big\{\{Q_{2,4,6}\}\big\}.$$

Of course, we have to take the union of all these formulas. This leads to the code shown in Figure 4.47 on page 106. Note that we have to subtract 1 from the row r and the column c when we access the array `Puzzle`, since in TypeScript array indices start with 0.

Finally, we have to combine all constraints of the puzzle. This is done with the function `allConstraints` shown in Figure 4.48 on page 106.

The Jupyter notebook showing the resulting program is available at:

`Chapter-4/10-Sudoku.ipynb`

```
function constraintsFromPuzzle(): Set<Clause> {
    const Puzzle  = createPuzzle();
    const Indices = range(1, 9);
    return Indices.cartesianProduct(Indices).filterMap(
        ([r, c]) => Puzzle[r-1][c-1] != '*',
        ([r, c]) => set(varName(r, c, Puzzle[r-1][c-1] as number)));
}
```

Figure 4.47: The function `constraintsFromPuzzle`

```
function getBlockCells(r: number, c: number): Set<Tuple<[number, number]>> {
    return range(1, 3).cartesianProduct(range(1, 3)).map(
        ([row, col]) => tpl(r * 3 + row, c * 3 + col)
    );
}
function allConstraints(): Set<Clause> {
    // 1. Start with constraints from the puzzle
    const Clauses = constraintsFromPuzzle();
    // 2. There is exactly one digit in every field
    Clauses.flatMap(
        range(1, 9).cartesianProduct(range(1, 9)),
        ([row, col]) => exactlyOneDigit(row, col)
    );
    // 3. All entries in a row are unique
    Clauses.flatMap(
        range(1, 9),
        row => exactlyOneForEveryDigit(range(1, 9).map(col => tpl(row, col)))
    );
    // 4. All entries in a column are unique
    Clauses.flatMap(
        range(1, 9),
        col => exactlyOneForEveryDigit(range(1, 9).map(row => tpl(row, col)))
    );
    // 5. All entries in a 3 times 3 square are unique
    const block = range(0, 2).cartesianProduct(range(0, 2));
    Clauses.flatMap(block,
                    ([r, c]) => exactlyOneForEveryDigit(getBlockCells(r, c)));
    return Clauses;
}
```

Figure 4.48: The function `allConstraints`.

4.10 Sudoku and Logic-Solver

In our previous examples, we modeled the Sudoku puzzle using propositional logic and solved it using our own implementation of the Davis-Putnam algorithm in the notebook. Although our algorithm is logically correct, solving the resulting set of over 10,000 clauses takes about three and a half minutes on a modern computer. To solve such problems efficiently, we can rely on industrial-strength SAT solvers.

As described in the `README.md` file provided with its distribution, `Logic-Solver` is a boolean satisfiability solver available as an `npm` package. It serves as a JavaScript wrapper around MiniSat, an advanced C++ solver compiled to JavaScript via Emscripten. `Logic-Solver` abstracts away the manual conversion to Conjunctive Normal Form (CNF) by allowing us to build formulas using high-level methods like `Logic.or`, `Logic.and`, and `Logic.exactlyOne`.

This section extends the concepts previously discussed in this chapter by demonstrating how to encode and solve the same Sudoku puzzle using this optimized library. We will explore the implementation found in the notebook `11-Logic-Solver-Sudoku.ipynb`.

4.10.1 Implementation of the Sudoku Solver

We begin by defining the basic types and a utility function to generate sequences of numbers. This is shown in Figure 4.49.

```
type Variable        = string;
type NegatedVariable = `-${string}`;
type Literal         = Variable | NegatedVariable;

function range(start: number, stop: number): number[] {
    return Array.from({ length: stop - start + 1 }, (_, index) => start + index);
}
```

Figure 4.49: Type definitions and the `range` function.

The TypeScript types `Variable`, `NegatedVariable`, `Literal`, and `Clause` establish how boolean logic primitives are formatted as strings. The `range` function acts as a helper to easily generate arrays of numbers, such as $[1, \ldots, 9]$, which we need for iterating over the Sudoku grid.

The `createPuzzle` function (Figure 4.50) returns the puzzle as a two-dimensional array. Given numbers are integers, and unknown fields are marked with a `"*"`.

To encode the board in propositional logic, we generate variable names using the `varName` function, and we can negate them using the `complement` function, both shown in Figure 4.51.

The `varName` function creates strings like `"Q<1,2,3>"` to represent the boolean variable stating that row 1, column 2 holds the digit 3. The `complement` function returns the logical negation of a literal by either adding or removing a leading minus sign (`-`), satisfying the format expected by `Logic-Solver`.

We now define the logical constraints, starting with the requirement that a specific digit must appear exactly once within a given set of board positions.

```
function createPuzzle(): (number | string)[][] {
  return [[ 8 , "*", "*", "*", "*", "*", "*", "*", "*"],
          ["*", "*",  3 ,  6 , "*", "*", "*", "*", "*"],
          ["*",  7 , "*", "*",  9 , "*",  2 , "*", "*"],
          ["*",  5 , "*", "*", "*",  7 , "*", "*", "*"],
          ["*", "*", "*", "*",  4 ,  5 ,  7 , "*", "*"],
          ["*", "*", "*",  1 , "*", "*", "*",  3 , "*"],
          ["*", "*",  1 , "*", "*", "*", "*",  6 ,  8 ],
          ["*", "*",  8 ,  5 , "*", "*", "*",  1 , "*"],
          ["*",  9 , "*", "*", "*", "*",  4 , "*", "*"]
         ];
}
```

Figure 4.50: The function `createPuzzle`.

```
function varName(row: number, col: number, digit: number): string {
    return `Q<${row},${col},${digit}>`;
}

function complement(l: Literal): Literal {
    return l.startsWith('-') ? l.substring(1) :  '-' + l;
}
```

Figure 4.51: The functions `varName` and `complement`.

```
type Position = [number, number];

function exactlyOneForDigit(L: Position[], digit: number): Logic.Formula {
    const variables = L.map(([row, col]) => varName(row, col, digit));
    return Logic.exactlyOne(variables) as Logic.Formula;
}
```

Figure 4.52: The function `exactlyOneForDigit`.

In Figure 4.52, we define `Position` as a tuple of row and column. The `exactlyOneForDigit` function leverages `Logic.exactlyOne`, a powerful cardinality constraint built directly into `Logic-Solver`. It takes a list of positions `L` and ensures that the specified `digit` is placed in exactly one of those positions. To apply this to all digits, we use the function `exactlyOneForEveryDigit`.

```
function exactlyOneForEveryDigit(L: Position[]): Logic.Formula[] {
    return range(1, 9).map(d => exactlyOneForDigit(L, d));
}
```

Figure 4.53: The function `exactlyOneForEveryDigit`.

The `exactlyOneForEveryDigit` function (Figure 4.53) maps over the numbers 1 through 9, returning an array of formulas. This effectively states that within the given set of coordinates `L` (which will later represent a full row, column, or block), every digit from 1 to 9 must occur exactly once.

```
function exactlyOneDigit(row: number, col: number): Logic.Formula {
    return Logic.exactlyOne(
        range(1, 9).map(d => varName(row, col, d))
    ) as Logic.Formula;
}
```

Figure 4.54: The function `exactlyOneDigit`.

Furthermore, we must ensure that every cell contains exactly one digit. The `exactlyOneDigit` function in Figure 4.54 enforces that for a fixed `row` and `col`, exactly one of the variables corresponding to digits 1 through 9 evaluates to `true`.

```
function constraintsFromPuzzle(): Logic.Formula {
    const Puzzle = createPuzzle();
    const Facts  = [];
    for (let r = 1; r <= 9; ++r) {
        for (let c = 1; c <= 9; ++c) {
            if (Puzzle[r-1][c-1] != '*') {
                Facts.push(varName(r, c, Puzzle[r-1][c-1] as number));
            }
        }
    }
    return Logic.and(Facts) as Logic.Formula;
}
```

Figure 4.55: The function `constraintsFromPuzzle`.

We must also convert the initial numbers from the puzzle grid into logical facts. The function `constraintsFromPuzzle` (Figure 4.55) iterates over the 9×9 array. For every non-asterisk entry, it

generates the corresponding variable string and adds it to the `Facts` array. It then combines all these facts using `Logic.and`, creating a single formula that anchors the starting state of the board.

```
function getBlockCells(r: number, c: number): Position[] {
    let Result = [];
    for (let row = 1; row <= 3; ++row) {
        for (let col = 1; col <= 3; ++col) {
            Result.push([r * 3 + row, c * 3 + col]);
        }
    }
    return Result;
}
```

Figure 4.56: The function `getBlockCells`.

To validate the 3×3 macro-blocks, we need a helper function to calculate coordinate translations. The `getBlockCells` function in Figure 4.56 takes the 0-based index of a macro-block (`r` and `c`, both spanning 0 to 2) and returns the nine 1-based coordinates `[row, col]` on the board that belong to that block.

With all helpers in place, we construct the master function that binds everything together. The `allConstraints` function (Figure 4.57) takes a `Logic.Solver` instance and sequentially feeds it all our rules via `solver.require(...)`. It ensures:

1. The initial grid values are respected.
2. Each cell contains exactly one digit.
3. Rows contain no duplicates.
4. Columns contain no duplicates.
5. 3×3 macro-blocks contain no duplicates.

Finally, it calls `solver.solve()`, executing MiniSat internally, and returns the `Logic.Solution` object.

Executing this entire program yields a huge performance improvement. While the Davis-Putnam algorithm spent about three and a half minutes laboring through the SAT derivation, the Jupyter notebook that uses `Logic-Solver` accomplishes the same goal in roughly 60 milliseconds. The Jupyter notebook showing the resulting program is available at:

`Chapter-4/11-Logic-Solver-Sudoku.ipynb`

```
function allConstraints(solver: Logic.Solver): Logic.Solution {
    // 1. Start with constraints from the puzzle
    solver.require(constraintsFromPuzzle());
    // 2. There is exactly one digit in every field
    for (let row = 1; row <= 9; ++row) {
        for (let col = 1; col <= 9; ++col) {
            solver.require(exactlyOneDigit(row, col));
        }
    }
    // 3. All entries in a row are unique
    for (let row = 1; row <= 9; ++row) {
        solver.require(
            exactlyOneForEveryDigit(range(1, 9).map(col => [row, col]))
        );
    }
    // 4. All entries in a column are unique
    for (let col = 1; col <= 9; ++col) {
        solver.require(
            exactlyOneForEveryDigit(range(1, 9).map(row => [row, col]))
        );
    }
    // 5. All entries in a 3x3 square are unique
    for (let row = 0; row <= 2; ++row) {
        for (let col = 0; col <= 2; ++col) {
            solver.require(
                exactlyOneForEveryDigit(getBlockCells(row, col))
            );
        }
    }
    return solver.solve();
}
```

Figure 4.57: The function `allConstraints`.

4.11 Check Your Comprehension

(a) How have we defined the set of propositional logic formulas?

(b) How have we defined the semantics of propositional logic formulas?

(c) How do we represent propositional logic formulas in *TypeScript*?

(d) What is a tautology?

(e) How can we transform a propositional logic formula into conjunctive normal form?

(f) Define the notion of an inference rule.

(g) When is an inference rule correct?

(h) Define the cut rule.

(i) How did we define the proof concept $M \vdash C$?

(j) What is a saturated set of clauses?

(k) What are the two most important properties of the proof concept $\vdash$?

(l) Assume that M is a saturated set of clauses. How can we define a propositional valuation $\mathcal{I}$ such that $\mathcal{I}(C) = \texttt{true}$ for all $C \in M$?

(m) How did we define the notion of a set of clauses being solvable?

(n) Do you know how the algorithm of Davis and Putnam works?

(o) Can you code a logical puzzle as a propositional formula?

Chapter 5

First-Order Logic

In propositional logic, we have studied the combination of atomic propositions using the propositional connectives "$\neg$", "$\vee$", "$\wedge$", "$\rightarrow$" and "$\leftrightarrow$". In first-order logic we additionally examine the structure of these atomic propositions. The following additional concepts are introduced in first-order logic:

1. Terms are used as designations for objects.
2. These terms are composed of object variables and function symbols. In the following examples, "x" is an object variable, while "*father*" and "*mother*" are unary function symbols and "*isaac*" is a nullary function symbol:

   $$\mathit{father}(x), \quad \mathit{mother}(\mathit{isaac}).$$

   Nullary function symbols are also referred to as constants and instead of object variables we simply talk about variables.
3. Different objects are related by predicate symbols. In the following example, we use the predicate symbols "*isBrother*" and "$<$". Additionally, "*albert*" and "*bruno*" are constants:

   $$\mathit{isBrother}\big(\mathit{albert}, \mathit{father}(\mathit{bruno})\big), \quad x + 7 < x \cdot 7.$$

   The resulting formulas are referred to as atomic formulas , since they do not contain subformulas.
4. Atomic formulas can be combined using the propositional connectives as shown in the following example:

   $$x > 1 \rightarrow x + 7 < x \cdot 7.$$

5. Finally, the quantifiers "$\forall$" (*for all*) and "$\exists$" (*there exists*) are introduced to distinguish between variables that are existentially quantified and those variables that are universally quantified. For example, the formula

   $$\forall x \in \mathbb{R} : \exists n \in \mathbb{N} : x < n$$

   is read as: "*For all real numbers x there exists a natural number n such that x is less than n.*"

This chapter is structured as follows:

(a) In the next section, we will define the syntax of first-order logic formulas, i.e., we will specify those strings that are regarded as first-order formulas.

(b) In the following section, we deal with the semantics of these formulas, that is we specify the meaning of first-order formulas.

(c) After that, we show how to implement syntax and semantics in *TypeScript*.

(d) As an application of first-order logic we discuss constraint programming. In constraint programming, a given problem is described using first-order logic formulas. To solve the problem, a so-called constraint solver is used.

(e) We develop a simple constraint solver that works by means of backtracking.

(f) Then we discuss the constraint solver `Z3`, which is a state-of-the-art constraint solver developed by Microsoft Corporation.

5.1 Syntax of First-Order Logic

First, we define the concept of a signature. Essentially, this is a structured summary of variables, function symbols, and predicate symbols, along with a specification of the arity of these symbols.

Definition 28 (Signature) *A signature is a 4-tuple*

$$\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle,$$

such that the following holds:

1. $\mathcal{V}$ *is the set of object variables, which we often simply refer to as variables for brevity.*
2. $\mathcal{F}$ *is the set of function symbols.*
3. $\mathcal{P}$ *is the set of predicate symbols.*
4. *arity is a function that assigns each function and predicate symbol its arity:*

   $$\mathit{arity} : \mathcal{F} \cup \mathcal{P} \to \mathbb{N}.$$

   We say that the function or predicate symbol f is an n-ary symbol if $\mathit{arity}(f) = n$.

   A function symbol c such that $\mathit{arity}(c) = 0$ is called a constant.

   A predicate symbol p such that $\mathit{arity}(p) = 0$ is called a propositional variable.
5. *Since we need to be able to distinguish between variables, function symbols, and predicate symbols, we agree that the sets $\mathcal{V}$, $\mathcal{F}$, and $\mathcal{P}$ have to be pairwise disjoint:*

   $$\mathcal{V} \cap \mathcal{F} = \{\}, \quad \mathcal{V} \cap \mathcal{P} = \{\}, \quad \text{and} \quad \mathcal{F} \cap \mathcal{P} = \{\}.$$ ◇

Example: Next, we define the signature Σ_G of group theory. Let

1. $\mathcal{V} := \{x, y, z\}$ be the set of variables,
2. $\mathcal{F} := \{\mathtt{e}, \circ\}$ be the set of function symbols,

3. $\mathcal{P} := \{=\}$ be the set of predicate symbols,
4. $\mathit{arity} := \{\mathsf{e} \mapsto 0, \circ \mapsto 2, = \mapsto 2\}$ specifies the arities of function and predicate symbols.

Then the signature Σ_G of group theory is defined as follows:

$$\Sigma_G := \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle. \qquad \diamond$$

We use expressions built from variables and function symbols as identifiers for objects. These expressions are called terms. The formal definition follows.

Definition 29 (Terms, $\mathcal{T}_\Sigma$)
If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle$ is a signature, then we define the set $\mathcal{T}_\Sigma$ of Σ-terms, inductively:

1. *For every variable $x \in \mathcal{V}$, we have that $x \in \mathcal{T}_\Sigma$. Thus, every variable is also a term.*
2. *If $c \in \mathcal{F}$ is a 0-ary function symbol, i.e. $\mathit{arity}(c) = 0$, then $c \in \mathcal{T}_\Sigma$.*
   Therefore, every constant is a term.
3. *If $f \in \mathcal{F}$ is an n-ary function symbol such that $n > 0$ and $t_1, \cdots, t_n \in \mathcal{T}_\Sigma$, then*
   $$f(t_1, \cdots, t_n) \in \mathcal{T}_\Sigma,$$
   thus the expression $f(t_1, \cdots, t_n) \in \mathcal{T}_\Sigma$ is a term. $\diamond$

Example: Let

1. $\mathcal{V} := \{x, y, z\}$ be the set of variables,
2. $\mathcal{F} := \{0, 1, +, -, *\}$ be the set of function symbols,
3. $\mathcal{P} := \{=, \leq\}$ be the set of predicate symbols,
4. $\mathit{arity} := \{0 \mapsto 0, 1 \mapsto 0, + \mapsto 2, - \mapsto 2, * \mapsto 2, = \mapsto 2, \leq \mapsto 2\}$ specifies the arity of function and predicate symbols, and
5. $\Sigma_{\text{arith}} := \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle$ be a signature.

Then we can construct Σ_{arith}-terms as follows:

1. $x, y, z \in \mathcal{T}_{\Sigma_{\text{arith}}}$,
   since all variables are also Σ_{arith}-terms.
2. $0, 1 \in \mathcal{T}_{\Sigma_{\text{arith}}}$,
   since 0 and 1 are nullary function symbols.
3. $+(0, x) \in \mathcal{T}_{\Sigma_{\text{arith}}}$,
   since $0 \in \mathcal{T}_{\Sigma_{\text{arith}}}$, $x \in \mathcal{T}_{\Sigma_{\text{arith}}}$ and $+$ is a binary function symbol.
4. $*(+(0, x), 1) \in \mathcal{T}_{\Sigma_{\text{arith}}}$,
   since $+(0, x) \in \mathcal{T}_{\Sigma_{\text{arith}}}$, $1 \in \mathcal{T}_{\Sigma_{\text{arith}}}$ and $*$ is a binary function symbol.

In practice, for certain binary functions, we use an infix notation, i.e., we write binary function symbols between their arguments. For example, we write $x + y$ instead of $+(x, y)$. The infix notation is then to be understood as an abbreviation for the above-defined representation. This, of course, works only if we also define the precedence levels and the associativities for the various operators. ◇

Next, we define the concept of atomic formulas. These are formulas that cannot be decomposed into smaller formulas: atomic formulas thus contain neither propositional connectives nor quantifiers.

Definition 30 (Atomic Formulas, $\mathcal{A}_\Sigma$) *Given a signature $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity \rangle$. The set of atomic Σ-formulas $\mathcal{A}_\Sigma$ is defined as follows:*

1. *$\top \in \mathcal{A}_\Sigma$ and $\bot \in \mathcal{A}_\Sigma$.*
2. *If $p \in \mathcal{P}$ and $arity(p) = 0$, then $p \in \mathcal{A}_\Sigma$.*

   Therefore, propositional variables are atomic formulas.
3. *If $p \in \mathcal{P}$ is an n-ary predicate symbol such that $n > 0$ and, furthermore, n Σ-terms $t_1, \cdots, t_n$ are given, then $p(t_1, \cdots, t_n)$ is an atomic Σ-formula:*

   $$p(t_1, \cdots, t_n) \in \mathcal{A}_\Sigma.$$

□

Example: Continuing from the last example, we can see that

$$=(*(+(0, x), 1), 0)$$

is an atomic Σ_{arith}-formula. Note that we have not yet discussed the truth value of such formulas. The question of whether a formula is considered true or false will be explored in the next section.

In practice we will often use infix notation for binary predicate symbols. The atomic formula given above is then written as

$$(0 + x) * 1 = 0.$$

◇

5.1.1 Bound Variables and Free Variables

In defining first-order logic formulas, it is necessary to distinguish between so-called bound and free variables. We introduce these concepts informally using an example from calculus. Consider the following equation:

$$\int_0^x y \cdot t \, dt = \frac{1}{2} \cdot x^2 \cdot y$$

In this equation, the variables x and y occur free, while the variable t is bound by the integral. This means the following: We can substitute arbitrary values for x and y in this equation without changing the validity of the formula. For example, substituting 2 for x, we get

$$\int_0^2 y \cdot t \, dt = \frac{1}{2} \cdot 2^2 \cdot y$$

and this equation is also valid. Conversely, it makes no sense to substitute a number for the bound variable t. The left side of the resulting equation would simply be undefined. We can only replace t with another variable. For example, replacing variable t with u, we get

$$\int_0^x y \cdot u \, du = \frac{1}{2} \cdot x^2 \cdot y$$

and this is effectively the same statement as above. However, this does not work with every variable. If we substitute the variable y for t, we get

$$\int_0^x y \cdot y \, dy = \frac{1}{2} \cdot x^2 \cdot y.$$

This statement is incorrect! The problem is that the previously free variable y becomes bound during the substitution.

A similar problem arises when we substitute arbitrary terms for y. As long as these terms do not contain the variable t, everything is fine. For example, if we substitute the term x^2 for y, we get

$$\int_0^x x^2 \cdot t \, dt = \frac{1}{2} \cdot x^2 \cdot x^2$$

and this formula is valid. However, if we substitute the term t^2 for y, we get

$$\int_0^x t^2 \cdot t \, dt = \frac{1}{2} \cdot x^2 \cdot t^2$$

and this formula is no longer valid. These examples show that we have to distinguish between bound and free variables.

In first-order logic, the quantifiers "$\forall$" (for all) and "$\exists$" (there exists) bind variables in a similar way as the integral operator "$\int \cdot \, \mathrm{d}t$". The above explanations show that there are two different types of variables: free variables and bound variables. To clarify these concepts, we first define for a Σ-term t the set of variables contained in t.

Definition 31 (*Var*(t)) *Given a signature $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity \rangle$ and t a Σ-term, we define the set $Var(t)$ of variables that occur in t by induction on the structure of the term:*

1. $Var(x) := \{x\}$ *for all* $x \in \mathcal{V}$,
2. $Var(c) := \{\}$ *for all* $c \in \mathcal{F}$ *such that* $arity(c) = 0$,
3. $Var\big(f(t_1, \cdots, t_n)\big) := Var(t_1) \cup \cdots \cup Var(t_n)$. ◇

5.1.2 Σ-Formulas

Definition 32 (Σ-Formula, $\mathbb{F}_\Sigma$, bound and free variables, *BV*(F), *FV*(F))
Let $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity \rangle$ be a signature. We denote the set of Σ-Formulas by $\mathbb{F}_\Sigma$. We define this set inductively. Simultaneously, for each formula $F \in \mathbb{F}_\Sigma$, we define the set $BV(F)$ of variables that occur bound in F and the set $FV(F)$ of variables that occur free in F.

1. $\bot \in \mathbb{F}_\Sigma$ *and* $\top \in \mathbb{F}_\Sigma$, *and we define*
$$FV(\bot) := FV(\top) := BV(\bot) := BV(\top) := \{\}.$$
2. *If* $p \in \mathcal{A}_\Sigma$ *and* $arity(p) = 0$, *then* $p \in \mathbb{F}_\Sigma$ *and*
$$FV(p) := \{\} \quad \text{and} \quad BV(p) := \{\}.$$
3. *If* $F = p(t_1, \cdots, t_n)$ *is an atomic Σ-formula, then* $F \in \mathbb{F}_\Sigma$. *Furthermore, we have*

   (a) $FV\big(p(t_1, \cdots, t_n)\big) := Var(t_1) \cup \cdots \cup Var(t_n)$,
   (b) $BV\big(p(t_1, \cdots, t_n)\big) := \{\}$.

4. *If $F \in \mathbb{F}_\Sigma$, then $\neg F \in \mathbb{F}_\Sigma$. Furthermore, we have*

   (a) $FV\big(\neg F\big) := FV(F)$,
   (b) $BV\big(\neg F\big) := BV(F)$.

5. *If $F, G \in \mathbb{F}_\Sigma$ and additionally*

$$\big(FV(F) \cup FV(G)\big) \cap \big(BV(F) \cup BV(G)\big) = \{\},$$

   then it also holds that

   (a) $(F \wedge G) \in \mathbb{F}_\Sigma$,
   (b) $(F \vee G) \in \mathbb{F}_\Sigma$,
   (c) $(F \to G) \in \mathbb{F}_\Sigma$,
   (d) $(F \leftrightarrow G) \in \mathbb{F}_\Sigma$.

   Furthermore, we define for all propositional connectives $\odot \in \{\wedge, \vee, \to, \leftrightarrow\}$:

   (a) $FV\big((F \odot G)\big) := FV(F) \cup FV(G)$.
   (b) $BV\big((F \odot G)\big) := BV(F) \cup BV(G)$.

6. *Let $x \in \mathcal{V}$ and $F \in \mathbb{F}_\Sigma$ with $x \notin BV(F)$. Then:*

   (a) $(\forall x : F) \in \mathbb{F}_\Sigma$,
   (b) $(\exists x : F) \in \mathbb{F}_\Sigma$.

   Furthermore, we define

   (a) $FV\big((\forall x : F)\big) := FV\big((\exists x : F)\big) := FV(F) \backslash \{x\}$,
   (b) $BV\big((\forall x : F)\big) := BV\big((\exists x : F)\big) := BV(F) \cup \{x\}$.

If the signature Σ is clear from the context or irrelevant, we may also write $\mathbb{F}$ instead of $\mathbb{F}_\Sigma$ and simply refer to formulas instead of Σ-formulas. ◇

In the definition given above, we have taken care that a variable cannot appear both free and bound in a formula simultaneously, because a straightforward induction on the structure of the formulas shows that for all $F \in \mathbb{F}_\Sigma$ the following holds:

$$FV(F) \cap BV(F) = \{\}.$$

Example: Continuing the example started above, we see that

$$(\exists x\colon\ \leq(+(y, x), y))$$

is a formula from $\mathbb{F}_{\Sigma_{\text{arith}}}$. The set of bound variables is $\{x\}$, the set of free variables is $\{y\}$. ◇

If we always wrote formulas in the prefix notation defined above, then readability would suffer disproportionately. To abbreviate, we agree that in first-order logic the same rules for saving parentheses apply that we have already used in propositional logic. In addition, the same quantifiers are

combined: for example, we write

$$\forall x, y\colon p(x,y) \quad \text{instead of} \quad \forall x\colon (\forall y\colon p(x,y)).$$

Moreover, we agree that we may also indicate binary predicate and function symbols in infix notation. In order to guarantee a unique readability, we must then define the precedence and the associativity of the function symbols. For example, we write

$$\mathtt{n}_1 = \mathtt{n}_2 \quad \text{instead of} \quad =(\mathtt{n}_1, \mathtt{n}_2).$$

The formula $(\exists x\colon \leq (+(y,x),y))$ becomes more readable as

$$\exists x\colon y + x \leq y$$

Moreover, in the literature, you will often find expressions of the form

$$\forall x \in M : F \quad \text{or} \quad \exists x \in M : F.$$

These are abbreviations defined as

$$\big(\forall x \in M : F\big) \overset{\text{def}}{\Longleftrightarrow} \forall x : \big(x \in M \to F\big) \quad \text{and} \quad \big(\exists x \in M : F\big) \overset{\text{def}}{\Longleftrightarrow} \exists x : \big(x \in M \wedge F\big).$$

Finally, we agree that quantifiers bind more tightly than the operators $\wedge$, $\vee$, $\to$, and $\leftrightarrow$.

Exercise 14: **Consider the following signature for formalizing family relationships:**

- **Variables:** $\{u, v, w, x, y, z\}$.
- **Function Symbols:**
  - $\mathtt{mother}(x)$: the mother of x.
  - $\mathtt{father}(x)$: the father of x.
- **Unary Predicates:**
  - $\mathtt{female}(x)$: x is female.
  - $\mathtt{male}(x)$: x is male.
- **Binary Predicates:**
  - `brother`,
  - `sister`,
  - `sibling`,
  - `grandmother`,
  - `grandfather`,
  - `uncle`,
  - `aunt`,
  - `cousin`,

In the following, your task is to define certain predicates. In general, a definition of an n-ary predicate p has the form

$$\forall x_1, \cdots, x_n : \big(p(x_1, \cdots, x_n) \leftrightarrow F\big),$$

where F is a formula that does not contain the predicate p. In this exercise, this formula may only contain the function symbols `mother` and `father` and the predicate symbols `male` and `female`. For the purpose of this exercise we make the simplifying assumption that every person is either female or male.

As an example, to `define` the predicate `brother` we can use the following formula:

$$\forall x, y : \big(\texttt{brother}(x, y) \leftrightarrow \texttt{male}(x) \wedge \texttt{father}(x) = \texttt{father}(y) \wedge \texttt{mother}(x) = \texttt{mother}(y)\big)$$

Define the following predicates:

(a) $\texttt{sister}(x, y)$: x is a sister of y,

(b) $\texttt{aunt}(x, y)$: x is an aunt of y,

(c) $\texttt{uncle}(x, y)$: x is an uncle of y,

(d) $\texttt{sibling}(x, y)$: x is a sibling of y,

(e) $\texttt{grandfather}(x, y)$: x is a grandfather of y,

(f) $\texttt{grandmother}(x, y)$: x is a grandmother of y,

(g) $\texttt{cousin}(x, y)$: x is a cousin of y. ◇

5.2 Semantics of First-Order Logic

Next, we define the meaning of the formulas. For this purpose, we introduce the concept of a Σ-structure. This kind of structure specifies how the function and predicate symbols of the signature Σ are to be interpreted.

Definition 33 (Structure) *Let a signature*

$$\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle$$

be given. A Σ-structure $\mathcal{S}$ is a pair $\langle \mathcal{U}, \mathcal{J} \rangle$, such that the following holds:

1. *$\mathcal{U}$ is a non-empty set. This set is also called the universe of the Σ-structure. This universe contains the values. Later, when we evaluate a term we will get a value.*
2. *$\mathcal{J}$ is the interpretation of the function and predicate symbols. Formally, we define $\mathcal{J}$ as a mapping with the following properties:*
   (a) *Every function symbol $c \in \mathcal{F}$ with $\mathit{arity}(c) = 0$ is mapped to an element of $\mathcal{U}$, i.e. we have $\mathcal{J}(c) \in \mathcal{U}$. Instead of writing $\mathcal{J}(c)$ we will write $c^{\mathcal{J}}$.*
   (b) *Every function symbol $f \in \mathcal{F}$ with $\mathit{arity}(f) = m$ and $m > 0$ is mapped to an m-ary function*

   $$f^{\mathcal{J}} : \mathcal{U}^m \to \mathcal{U}$$

   that maps m-tuples of the universe $\mathcal{U}$ into the universe $\mathcal{U}$.
   (c) *Every predicate symbol $p \in \mathcal{P}$ with $\mathit{arity}(p) = 0$ is mapped to the set $\mathbb{B}$ of truth values, i.e. $p^{\mathcal{J}} \in \{\texttt{true}, \texttt{false}\}$.*

(d) *Every predicate symbol $p \in \mathcal{P}$ with $arity(p) = n$ such that $n > 0$ is mapped to a subset*

$$p^{\mathcal{J}} \subseteq \mathcal{U}^n.$$

The idea is that an atomic formula of the form $p(t_1, \cdots, t_n)$ is interpreted as `true` *exactly if the interpretation of the tuple $\langle t_1, \cdots, t_n \rangle$ is an element of the set $p^{\mathcal{J}}$.*

(e) *If the symbol "=" is a member of the set of predicate symbols $\mathcal{P}$, then the interpretation of the equality symbol "=" has to be canonical, i.e. we must have*

$$=^{\mathcal{J}} = \big\{\langle u, u\rangle \mid u \in \mathcal{U}\big\}.$$

A formula of the type $s = t$ is thus interpreted as true exactly when the interpretation of the term s yields the same value as the interpretation of the term t. ◇

Example: The signature Σ_G of group theory is defined as

$$\Sigma_G = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity\rangle \quad \text{with}$$

1. $\mathcal{V} := \{x, y, z\}$
2. $\mathcal{F} := \{\mathrm{e}, *\}$
3. $\mathcal{P} := \{=\}$
4. $arity = \big\{\mathrm{e} \mapsto 0, * \mapsto 2, = \mapsto 2\big\}$

We can then define a Σ_G structure $\mathcal{Z} = \langle \{0, 1\}, \mathcal{J}\rangle$ by defining the interpretation $\mathcal{J}$ as follows:

1. $\mathrm{e}^{\mathcal{J}} := 0$,
2. $*^{\mathcal{J}} := \Big\{\langle 0,0\rangle \mapsto 0,\ \langle 0,1\rangle \mapsto 1,\ \langle 1,0\rangle \mapsto 1,\ \langle 1,1\rangle \mapsto 0\Big\}$,
3. $=^{\mathcal{J}} := \big\{\langle 0,0\rangle, \langle 1,1\rangle\big\}$.

   Note that we have no leeway in the interpretation of the equality symbol! ◇

If we want to evaluate terms that contain variables, we have to substitute values from the universe for these variables. Which values we substitute is determined by a variable assignment. We define this concept next.

Definition 34 (Variable Assignment) *Assume a signature*

$$\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity\rangle$$

is given. Furthermore, $\mathcal{S} = \langle \mathcal{U}, \mathcal{J}\rangle$ is a Σ-structure. Then a function of the form

$$\mathcal{I} : \mathcal{V} \to \mathcal{U}$$

is called an $\mathcal{S}$ variable assignment.

If $\mathcal{I}$ is an $\mathcal{S}$ variable assignment, $x \in \mathcal{V}$ and $c \in \mathcal{U}$, then $\mathcal{I}[x/c]$ denotes the variable assignment that maps the variable x to the value c and that otherwise agrees with $\mathcal{I}$:

$$\mathcal{I}[x/c](y) := \begin{cases} c & \text{if } y = x; \\ \mathcal{I}(y) & \text{otherwise.} \end{cases}$$ ◇

Definition 35 (Interpretation of Terms) *If $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity \rangle$ is a signature $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ is a Σ-structure and $\mathcal{I}$ is an $\mathcal{S}$ variable assignment, then for every term t the value $\mathcal{S}(\mathcal{I}, t)$ is defined by induction on t:*

1. *If $x \in \mathcal{V}$ we define:*
$$\mathcal{S}(\mathcal{I}, x) := \mathcal{I}(x).$$
2. *If $c \in \mathcal{F}$ and $arity(c) = 0$, then $\mathcal{S}(\mathcal{I}, c) := c^{\mathcal{J}}$.*
3. *If t is a term of the form $f(t_1, \cdots, t_n)$, then we define*
$$\mathcal{S}\big(\mathcal{I}, f(t_1, \cdots, t_n)\big) := f^{\mathcal{J}}\big(\mathcal{S}(\mathcal{I}, t_1), \cdots, \mathcal{S}(\mathcal{I}, t_n)\big).$$ ◇

Example: Given the Σ_G-structure $\mathcal{Z}$ we define a $\mathcal{Z}$ variable assignment $\mathcal{I}$ as follows:

$$\mathcal{I} := \big\{x \mapsto 0,\ y \mapsto 1,\ z \mapsto 0\big\}.$$

The previous line is interpreted as follows:

$$\mathcal{I}(\mathtt{x}) := 0, \quad \mathcal{I}(\mathtt{y}) := 1, \quad \text{and} \quad \mathcal{I}(\mathtt{z}) := 0.$$

Then we have

$$\mathcal{Z}\big(\mathcal{I}, \mathtt{x} * \mathtt{y}\big) = 1.$$ ◇

Example: If we continue our previous example we have

$$\mathcal{Z}(\mathcal{I}, x * y = y * x) = \mathtt{true}.$$ ◇

To define the semantics of arbitrary Σ-formulas, we assume that we have the following functions available, just like in propositional logic:

1. $\ominus: \mathbb{B} \to \mathbb{B}$,
2. $\oslash: \mathbb{B} \times \mathbb{B} \to \mathbb{B}$,
3. $\oplus: \mathbb{B} \times \mathbb{B} \to \mathbb{B}$,
4. $\otimes : \mathbb{B} \times \mathbb{B} \to \mathbb{B}$,
5. $\odot : \mathbb{B} \times \mathbb{B} \to \mathbb{B}$.

The definitions of these functions have been given by the table in Figure 4.1 on page 40.

Definition 36 (Semantics of Σ-formulas) *Let $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ be a Σ-structure and $\mathcal{I}$ a $\mathcal{S}$-variable assignment. For each Σ-formula F the value $\mathcal{S}(\mathcal{I}, F)$ is defined by induction:*

1. *$\mathcal{S}(\mathcal{I}, \top) := \mathtt{true}$ and $\mathcal{S}(\mathcal{I}, \bot) := \mathtt{false}$.*
2. *If p is a propositional variable, then we define the value of p as follows:*
$$\mathcal{S}(\mathcal{I}, p) := p^{\mathcal{J}}.$$
3. *For each atomic Σ-formula of the form $p(t_1, \cdots, t_n)$ we define the value as follows:*
$$\mathcal{S}\big(\mathcal{I}, p(t_1, \cdots, t_n)\big) := \Big(\big\langle \mathcal{S}(\mathcal{I}, t_1), \cdots, \mathcal{S}(\mathcal{I}, t_n)\big\rangle \in p^{\mathcal{J}}\Big).$$

4. $\mathcal{S}(\mathcal{I}, \neg F) := \ominus\big(\mathcal{S}(\mathcal{I}, F)\big)$.
5. $\mathcal{S}(\mathcal{I}, F \wedge G) := \oslash\big(\mathcal{S}(\mathcal{I}, F), \mathcal{S}(\mathcal{I}, G)\big)$.
6. $\mathcal{S}(\mathcal{I}, F \vee G) := \otimes\big(\mathcal{S}(\mathcal{I}, F), \mathcal{S}(\mathcal{I}, G)\big)$.
7. $\mathcal{S}(\mathcal{I}, F \to G) := \ominus\big(\mathcal{S}(\mathcal{I}, F), \mathcal{S}(\mathcal{I}, G)\big)$.
8. $\mathcal{S}(\mathcal{I}, F \leftrightarrow G) := \ominus\big(\mathcal{S}(\mathcal{I}, F), \mathcal{S}(\mathcal{I}, G)\big)$.
9. $\mathcal{S}\big(\mathcal{I}, \forall x\colon F\big) := \begin{cases} \texttt{true} & \text{if } \mathcal{S}(\mathcal{I}[x/c], F) = \texttt{true} \quad \textit{for all } c \in \mathcal{U} \textit{ holds;} \\ \texttt{false} & \textit{otherwise.} \end{cases}$
10. $\mathcal{S}\big(\mathcal{I}, \exists x\colon F\big) := \begin{cases} \texttt{true} & \text{if } \mathcal{S}(\mathcal{I}[x/c], F) = \texttt{true} \quad \textit{for some } c \in \mathcal{U} \textit{ holds;} \\ \texttt{false} & \textit{otherwise.} \end{cases}$ ◇

Example: Continuing from the above example, we have

$$\mathcal{Z}\big(\mathcal{I}, \forall \mathtt{x} : \mathrm{e} * x = x\big) = \texttt{true}.$$ ◇

Definition 37 (Universally valid) *If F is a Σ-formula such that for every Σ-structure $\mathcal{S}$ and for every $\mathcal{S}$-variable assignment $\mathcal{I}$*

$$\mathcal{S}(\mathcal{I}, F) = \texttt{true}$$

holds, we call F universally valid. In this case, we write

$$\models F.$$ ◇

If F is a formula such that $FV(F) = \{\}$, then the value $\mathcal{S}(\mathcal{I}, F)$ obviously does not depend on the interpretation $\mathcal{I}$. We also refer to such formulas as closed formulas. In this case, we write $\mathcal{S}(F)$ instead of $\mathcal{S}(\mathcal{I}, F)$. If additionally $\mathcal{S}(F) = \texttt{true}$, we also say that $\mathcal{S}$ is a model of F. This is written as

$$\mathcal{S} \models F.$$

The definitions of the terms "satisfiable" and "equivalent" can now be transferred from propositional logic. To avoid unnecessary complexity in the definitions, we assume a fixed signature Σ as given. Hence, in the following definitions when we talk about terms, formulas, structures, etc., we mean Σ-terms, Σ-formulas, and Σ-structures.

Definition 38 (Equivalent) *Two formulas F and G, in which the variables $x_1, \cdots, x_n$ appear free, are called equivalent if and only if*

$$\models \forall x_1 : \cdots \forall x_n : (F \leftrightarrow G)$$

holds. If no variables appear free in F and G, then F is equivalent to G if and only if

$$\models F \leftrightarrow G$$

holds. If F and G are equivalent, then this will be written as

$$F \Leftrightarrow G.$$ ◇

Remark: All of the equivalences that we have found in propositional logic are also equivalences in first-order logic. ◇

Definition 39 (Satisfiable) *A set $M \subseteq \mathbb{F}_\Sigma$ is satisfiable, if there exists a structure $\mathcal{S}$ and a variable assignment $\mathcal{I}$ such that*

$$\mathcal{S}(\mathcal{I}, F) = \texttt{true} \quad \text{for all } F \in M$$

holds. Otherwise, M is called unsatisfiable or contradictory. This is written as

$$M \models \bot$$ ◇

Our goal is to provide a method that allows us to check whether a set M of formulas is contradictory, i.e., whether $M \models \bot$ holds. It turns out that this is generally not possible; the question of whether $M \models \bot$ holds is undecidable. A proof of this fact is based on the unsolvability of the halting problem but the details are beyond the scope of this lecture. However, as in propositional logic, it is possible to specify a calculus $\vdash$ such that:

$$M \vdash \bot \quad \text{iff} \quad M \models \bot.$$

This calculus can then be used to implement a semi-decision procedure: To check if $M \models \bot$ holds, we try to derive the formula $\bot$ from the set M. If we proceed systematically by trying all possible proofs, if indeed $M \models \bot$ holds, we will eventually find a proof that demonstrates $M \vdash \bot$. However, if M is satisfiable, i.e. if we have

$$M \not\models \bot$$

we generally will not be able to detect this, because the set of all possible proofs is infinite and we can never try all proofs. We can only ensure that we attempt every proof eventually. But if there is no proof, we can never be certain of this fact, because at any given time we have only tried a part of the possible proofs.

The situation is similar to that in verifying certain number-theoretical questions. Consider the following specific example: A number n is called a perfect number, if the sum of all proper divisors of n equals n itself. For example, the number 6 is perfect, since the set of proper divisors of 6 is $\{1, 2, 3\}$ and

$$1 + 2 + 3 = 6.$$

So far, all known perfect numbers are divisible by 2. The question of whether there are odd numbers that are perfect is an open problem. To solve this problem, we could write a program that sequentially checks whether each odd number is perfect. Figure 5.1 on page 125 shows such a program. If there is an odd perfect number, then this program will eventually find it. However, if no odd perfect number exists, then the program will run indefinitely and we will never know for sure that there are no odd perfect numbers.

5.3 Implementing Σ-Structures in *TypeScript*

I have implemented a *Jupyter* notebook that can be used to evaluate formulas from first-order logic. This program is available as a Jupyter Notebook in the GitHub repository for this lecture:

`Chapter-5/01-FOL-Evaluation.ipynb`

This program can be used to check whether a given formula from first-order logic is satisfied in a finite Σ-structure $\mathcal{S}$.

```typescript
function divisors(n: number): Set<number> {
    return range(1, n-1).filter(i => n % i == 0);
}
function perfect(n: number): boolean {
    return divisors(n).reduce((s, i) => s + i, 0) == n;
}
function findOddPerfect(): number {
    let n = 1;
    while (true) {
        if (perfect(n)) { return n; }
        n += 2;
    }
}
```

Figure 5.1: The search for an odd perfect number.

However, we cannot use it to check whether a formula is universally valid because, on the one hand, we cannot apply the program if the Σ-structure has an infinite universe, and on the other hand, even the number of different finite Σ-structures we would need to test is infinitely large. ◇

Exercise 15: In the following exercises, you are given a number of different closed Σ-formulas F. For each of these formulas F your task is to show that F is not universally valid. In order to do so, you have to construct a Σ-structure $\mathcal{S}$ such that $\mathcal{S}(F) = \texttt{false}$.

(a) $\forall x : \exists y : p(x,y) \rightarrow \exists y : \forall x : p(x,y)$

(b) $\forall x : \big(p(x) \vee q(x)\big) \rightarrow \big(\forall x : p(x) \vee \forall x : q(x)\big)$

(c) $\big(\exists x : p(x) \wedge \exists x : q(x)\big) \rightarrow \exists x : \big(p(x) \wedge q(x)\big)$

(d) $\exists x : \big(p(x) \rightarrow q(x)\big) \rightarrow \exists x : p(x) \rightarrow \exists x : q(x)$

In each of these cases, use the program from the *Jupyter* notebook `01-FOL-Evaluation.ipynb` in order to verify your claim that $\mathcal{S}(F) = \texttt{false}$.

Exercise 16:

(a) How many different structures exist for the signature of group theory if we assume that the universe is of the form $\{1, \cdots, n\}$?

(b) Provide a satisfiable first-order formula F that is always false in a Σ-structure $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ if the universe $\mathcal{U}$ is finite.

Hint: Let $f : U \rightarrow U$ be a function. Think about how the statements "f *is injective*" and "f *is surjective*" are related when the universe is finite. ◇

5.4 Constraint Programming

It is time to see a practical application of first order logic. One of these practical applications is constraint programming. Constraint programming is an example of the declarative programming paradigm. In declarative programming, the idea is that in order to solve a given problem, this problem is specified and this specification is given as input to a problem solver which will then compute a solution to the problem. Hence, the task of the programmer is much easier than it normally is: Instead of implementing a program that solves a given problem, the programmer only has to specify the problem precisely, she does not have to explicitly code an algorithm to find the solution. Usually, the specification of a problem is much easier than the coding of an algorithm to solve the problem. This approach works well for some problems that can be specified using first order logic. The remainder of this section is structured as follows:

1. We first define constraint satisfaction problems and provide two examples:
   - We introduce the map coloring problem for the case of Australia.
   - We formulate the eight queens puzzle as a constraint satisfaction problem.
2. We discuss a simple constraint solver that is based on backtracking.

   More efficient constraint solvers will be discussed in the lecture on artificial intelligence in the 6th semester. Furthermore, later in this chapter we will introduce Z3, which is a very powerful constraint solver developed by Microsoft.
3. Finally, we demonstrate a number of puzzles that can be solved using constraint programming.

5.4.1 Constraint Satisfaction Problems

Conceptually, a constraint satisfaction problem is given by a set of first order logic formulas that contain a set of free variables. Furthermore, a Σ-structure $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ consisting of a universe $\mathcal{U}$ and the interpretation $\mathcal{J}$ of the function and predicate symbols used in these formulas is assumed to be specified by the context of the problem. The goal is to find a variable assignment such that the given first-order formulas are evaluated as true. The formal definition follows.

Definition 40 (CSP)
A constraint satisfaction problem (abbreviated as CSP) is defined as a triple

$$\mathbb{P} := \langle \Sigma, \mathcal{S}, \mathcal{C} \rangle$$

where

1. $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle$ *is signature,*
2. $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ *is* Σ*-structure,*
3. $\mathcal{C} \subseteq \mathbb{F}_\Sigma$*, i.e.* $\mathcal{C}$ *is a set of* Σ*-formulas from first order logic. These formulas are called the constraints of* $\mathbb{P}$*.* ◇

In the following, we will often specify a constraint satisfaction problem by just specifying the variables $\mathcal{V}$, the set of values $\mathcal{U}$ that these variables can take, and the set $\mathcal{C}$ of constraints. In this case, we assume that the sets $\mathcal{F}$ and $\mathcal{P}$ of function and predicate symbols and their interpretation $\mathcal{J}$ are implicitly given. In this case the constraint satisfaction problem is given as the triple

$$\mathbb{P} = \langle \mathcal{V}, \mathcal{U}, \mathcal{C} \rangle.$$

Given a CSP

$$\mathbb{P} = \langle \mathcal{V}, \mathcal{U}, \mathcal{C} \rangle,$$

a variable assignment for $\mathbb{P}$ is a function

$$\mathcal{I} : \mathcal{V} \to \mathcal{U}.$$

A variable assignment $\mathcal{I}$ is a solution of the CSP $\mathbb{P} = \langle \mathcal{V}, \mathcal{U}, \mathcal{C} \rangle$ if, given the assignment $\mathcal{I}$, all constraints from $\mathcal{C}$ are satisfied, i.e. we have

$$\mathcal{S}(\mathcal{I}, f) = \texttt{true} \quad \text{for all } f \in \mathcal{C}.$$

Finally, a partial variable assignment $\mathcal{B}$ for $\mathbb{P}$ is a function

$$\mathcal{B} : \mathcal{V} \to \mathcal{U} \cup \{\Omega\} \quad \text{where } \Omega \text{ denotes the undefined value.}$$

Hence, a partial variable assignment does not assign values to all variables. Instead, it assigns values only to a subset of the set $\mathcal{V}$. The domain $\texttt{dom}(\mathcal{B})$ of a partial variable assignment $\mathcal{B}$ is the set of those variables that are assigned a value different from Ω, i.e. we define

$$\texttt{dom}(\mathcal{B}) := \{x \in \mathcal{V} \mid \mathcal{B}(x) \neq \Omega\}.$$

We proceed to illustrate the definitions given so far by presenting two examples.

5.4.2 Example: Map Colouring

In map colouring a map showing different state borders is given and the task is to colour the different states such that no two states that have a common border share the same colour. Figure 5.2 on page 128 shows a map of Australia. There are seven different states in Australia:

1. Western Australia, abbreviated as WA,
2. Northern Territory, abbreviated as NT,
3. South Australia, abbreviated as SA,
4. Queensland, abbreviated as Q,
5. New South Wales, abbreviated as NSW,
6. Victoria, abbreviated as V, and
7. Tasmania, abbreviated as T.

Figure 5.2 would certainly look better if different states had been coloured with different colours. For the purpose of this example let us assume that we have only the three colours red, green, and blue available. The question then is whether it is possible to colour the different states in a way that

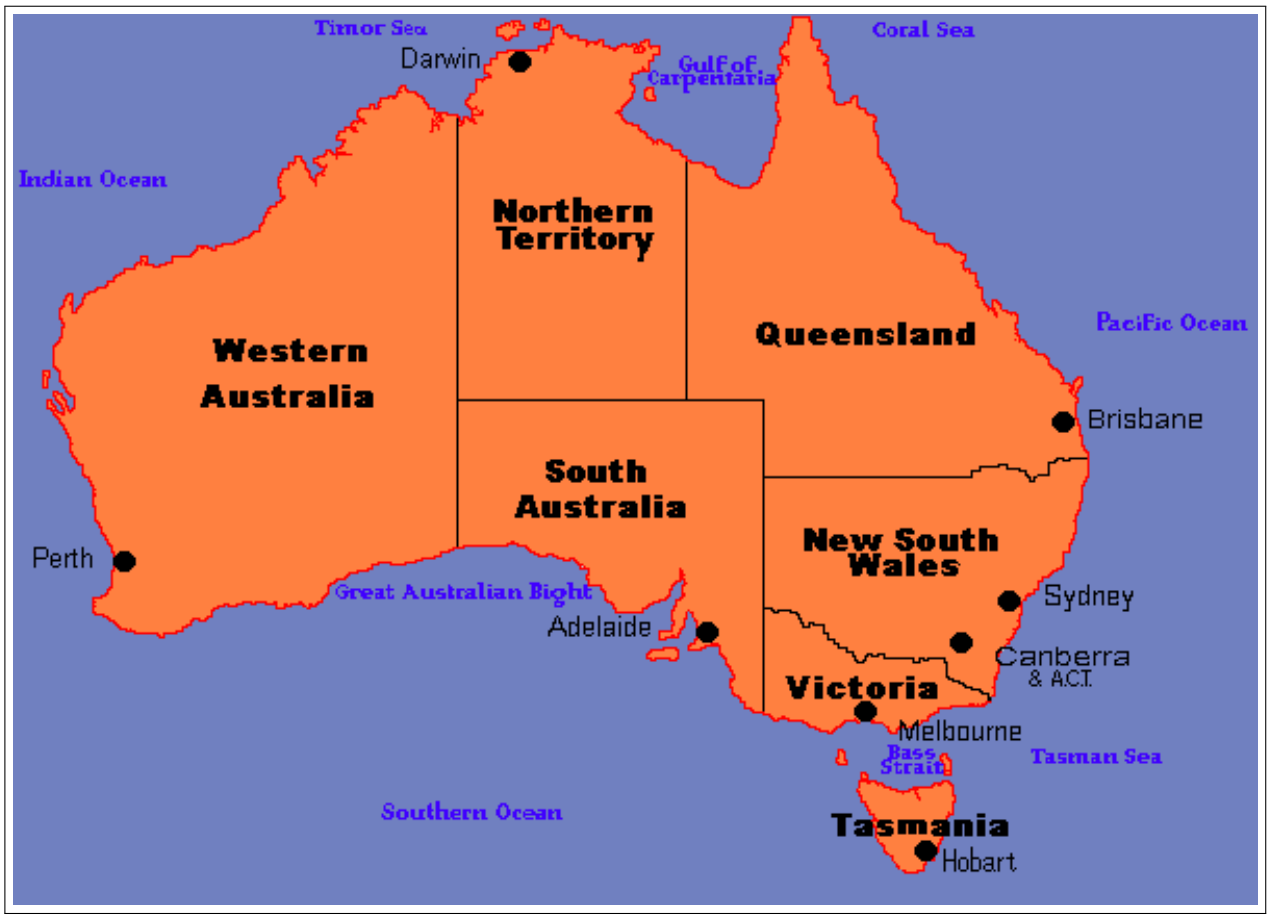


Figure 5.2: A map of Australia.

no two neighbouring states share the same colour. This problem can be formalized as a constraint satisfaction problem. To this end we define:

1. $\mathcal{V} := \{\texttt{WA}, \texttt{NT}, \texttt{SA}, \texttt{Q}, \texttt{NSW}, \texttt{V}, \texttt{T}\}$,
2. $\mathcal{U} := \{\texttt{red}, \texttt{green}, \texttt{blue}\}$,
3. $\mathcal{C} := \{\texttt{WA} \neq \texttt{NT}, \texttt{WA} \neq \texttt{SA}, \texttt{NT} \neq \texttt{SA}, \texttt{NT} \neq \texttt{Q}, \texttt{SA} \neq \texttt{Q}, \texttt{SA} \neq \texttt{NSW}, \texttt{SA} \neq \texttt{V}, \texttt{Q} \neq \texttt{NSW}, \texttt{NSW} \neq \texttt{V}\}$. The constraints do not mention the variable $\texttt{T}$ for Tasmania, as Tasmania does not share a common border with any of the other states.

Then $\mathbb{P} := \langle \mathcal{V}, \mathcal{U}, \mathcal{C} \rangle$ is a constraint satisfaction problem. If we define the assignment $\mathcal{I}$ such that

1. $\mathcal{I}(\texttt{WA}) = \texttt{red}$,
2. $\mathcal{I}(\texttt{NT}) = \texttt{blue}$,
3. $\mathcal{I}(\texttt{SA}) = \texttt{green}$,
4. $\mathcal{I}(\texttt{Q}) = \texttt{red}$,

5. $\mathcal{I}(\texttt{NSW}) = \texttt{blue}$,
6. $\mathcal{I}(\texttt{V}) = \texttt{red}$,
7. $\mathcal{I}(\texttt{T}) = \texttt{green}$,

then you can check that this assignment is indeed a solution to the constraint satisfaction problem $\mathbb{P}$. Figure 5.3 on page 129 shows this solution.

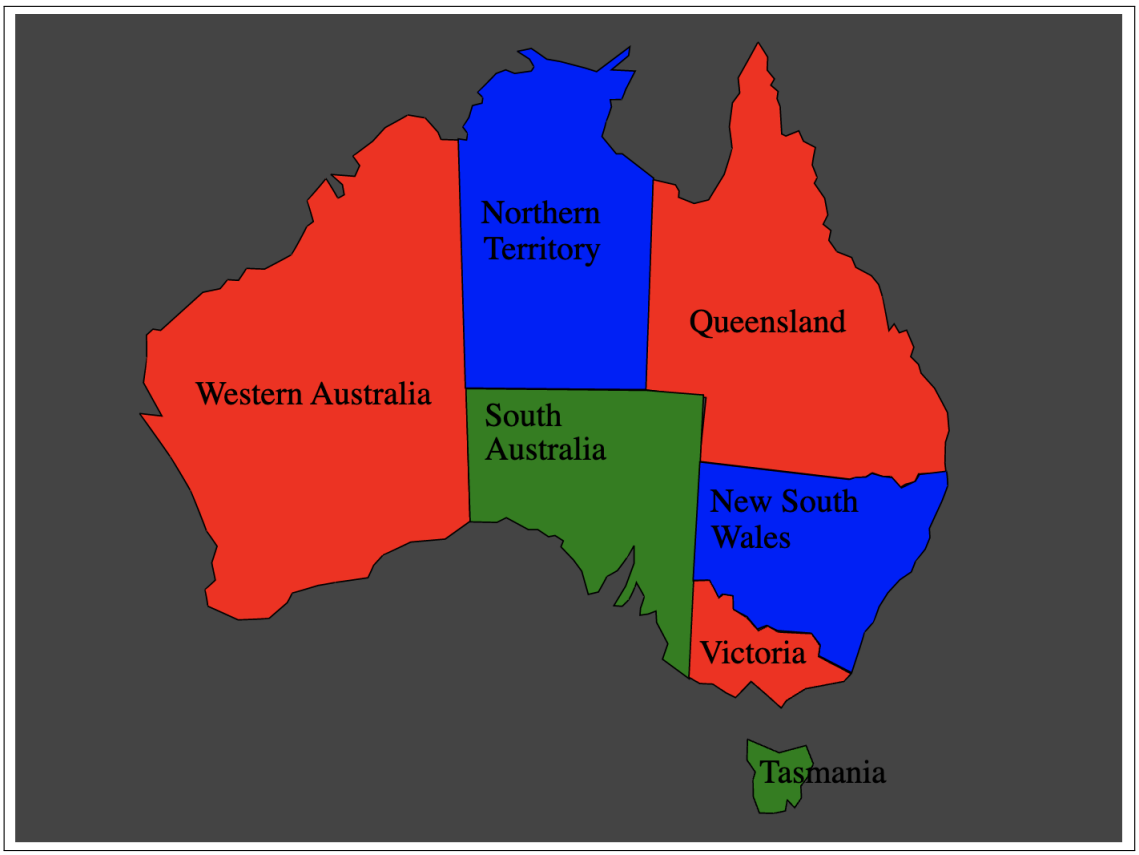


Figure 5.3: A map coloring for Australia.

5.4.3 Example: The Eight Queens Puzzle

The eight queens problem asks to put 8 queens onto a chessboard such that no queen can attack another queen. We have already discussed this problem in the previous chapter. Let us recapitulate: In chess, a queen can attack all pieces that are either in the same row, the same column, or the same diagonal. If we want to put 8 queens on a chessboard such that no two queens can attack each other, we have to put exactly one queen in every row: If we put more than one queen in a row, the queens in that row could attack each other. If we left a row empty, then, given that the other rows contain at most one queen, there would be fewer than 8 queens on the board. Therefore, in order to model the eight queens problem as a constraint satisfaction problem, we will use the following set of variables:

$$\mathcal{V} := \{\mathtt{Q}_1, \mathtt{Q}_2, \mathtt{Q}_3, \mathtt{Q}_4, \mathtt{Q}_5, \mathtt{Q}_6, \mathtt{Q}_7, \mathtt{Q}_8\},$$

where for $i \in \{1, \cdots, 8\}$ the variable $\mathtt{Q}_i$ specifies the column of the queen that is placed in row i. As the columns run from one to eight, we define the set $\mathcal{U}$ as

$$\mathcal{U} := \{1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8\}.$$

Next, let us define the constraints. There are two different types of constraints.

1. We have constraints that express that no two queens positioned in different rows share the same column. To capture these constraints, we define

   $$\mathtt{SameColumn} := \big\{\mathtt{Q}_i \neq \mathtt{Q}_j \;\big|\; i, j \in \{1, \cdots, 8\} \wedge j < i\big\}.$$

   Here the condition $j < i$ ensures that, for example, we have the constraint $\mathtt{Q}_2 \neq \mathtt{Q}_1$ but not the constraint $\mathtt{Q}_1 \neq \mathtt{Q}_2$, as the latter constraint would be redundant if the former constraint has already been established.

2. We have constraints that express that no two queens positioned in different rows share the same diagonal. The queens in row i and row j share the same diagonal iff the equation

   $$|i - j| = |\mathtt{Q}_i - \mathtt{Q}_j|$$

   holds. The expression $|i - j|$ is the absolute value of the difference of the rows of the queens in row i and row j, while the expression $|\mathtt{Q}_i - \mathtt{Q}_j|$ is the absolute value of the difference of the columns of these queens. To capture these constraints, we define

   $$\mathtt{SameDiagonal} := \big\{|i - j| \neq |\mathtt{Q}_i - \mathtt{Q}_j| \;\big|\; i, j \in \{1, \cdots, 8\} \wedge j < i\big\}.$$

Then, the set of constraints is defined as

$$\mathcal{C} := \mathtt{SameColumn} \cup \mathtt{SameDiagonal}$$

and the eight queens problem can be stated as the constraint satisfaction problem

$$\mathbb{P} := \langle \mathcal{V},\ \mathcal{U},\ \mathcal{C} \rangle.$$

If we define the assignment $\mathcal{I}$ such that

$$\mathcal{I}(\mathtt{Q}_1) := 4,\ \mathcal{I}(\mathtt{Q}_2) := 8,\ \mathcal{I}(\mathtt{Q}_3) := 1,\ \mathcal{I}(\mathtt{Q}_4) := 3,\ \mathcal{I}(\mathtt{Q}_5) := 6,\ \mathcal{I}(\mathtt{Q}_6) := 2,$$
$$\mathcal{I}(\mathtt{Q}_7) := 7,\ \mathcal{I}(\mathtt{Q}_8) := 5,$$

then it is easy to see that this assignment is a solution of the eight queens problem. This solution is shown in Figure 5.4 on page 131.

Later, when we implement procedures to solve CSPs, we will represent variable assignments and partial variable assignments as dictionaries. For example, the variable assignment $\mathcal{I}$ defined above would then be represented in *TypeScript* as the `Map`

$$\mathcal{I} = \mathtt{map}([[\mathtt{Q1}, 4],\ [\mathtt{Q2}, 8],\ [\mathtt{Q3}, 1],\ [\mathtt{Q4}, 3],\ [\mathtt{Q5}, 6],\ [\mathtt{Q6}, 2],\ [\mathtt{Q7}, 7],\ [\mathtt{Q8}, 5]]).$$

If we define

$$\mathcal{B} := \mathtt{map}([[\mathtt{Q1}, 4],\ [\mathtt{Q2}, 8],\ [\mathtt{Q3}, 1]]).$$

then $\mathcal{B}$ is a partial assignment and $\mathtt{dom}(\mathcal{B}) = \{\mathtt{Q}_1, \mathtt{Q}_2, \mathtt{Q}_3\}$. This partial assignment is shown in Figure 5.5 on page 131.

Figure 5.6 on page 132 shows a *TypeScript* program that can be used to create the eight queens puzzle as a CSP.

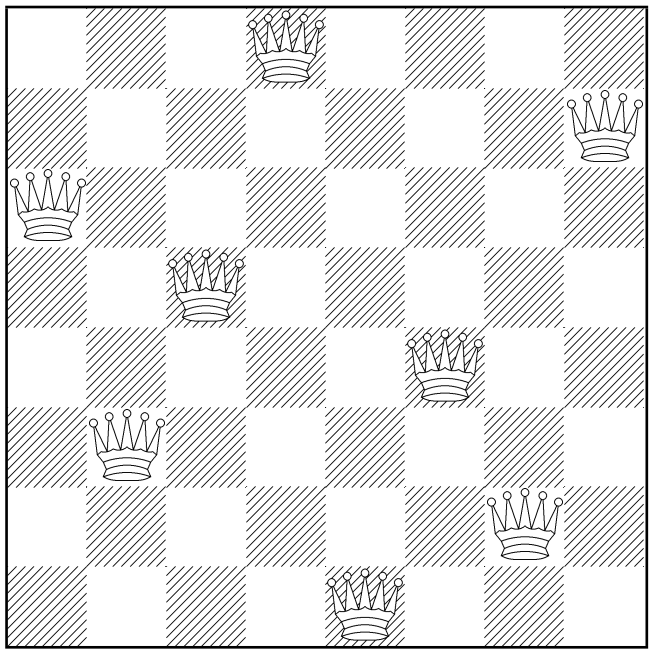


Figure 5.4: A solution of the eight queens problem.

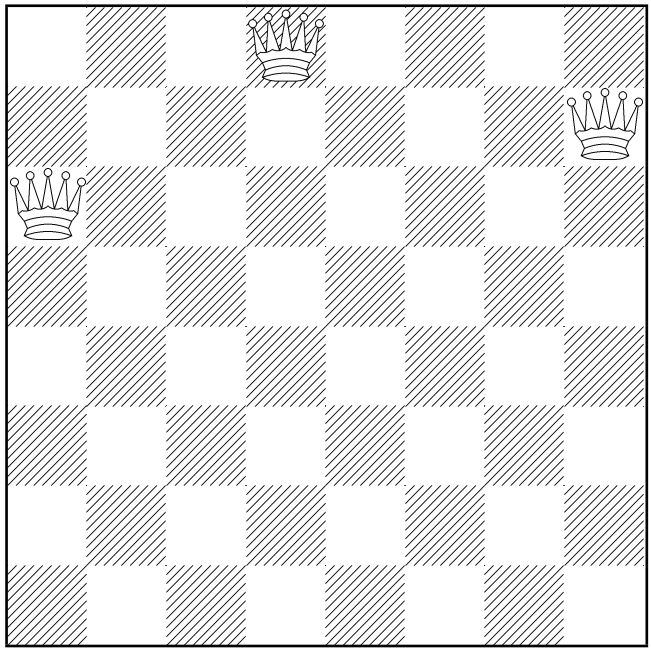


Figure 5.5: The partial assignment `map([[Q1,4], [Q2,8], [Q3,1]])`.

5.4.4 A Backtracking Constraint Solver

One approach to solve a CSP that is both conceptually simple and reasonably efficient is backtracking. The idea is to try to build variable assignments incrementally: We start with an empty dictionary and pick a variable x_1 that needs to have a value assigned. For this variable, we choose a value v_1 and

```
function queensCSP(n: number = 8): CSP {
    const indices = range(1, n);
    const Vars    = indices.map(i => `Q${i}`);
    const Values  = indices;
    const Constrs = indices.flatMap(
        i => range(i + 1, n).flatMap(
            j => [`Q${i} != Q${j}`,                          // Not on same column
                  `Math.abs(Q${i} - Q${j}) != ${j - i}`      // Not on same diagonal
                 ])
    );
    return { Vars, Values, Constrs };
}
```

Figure 5.6: *TypeScript* code to create the CSP representing the eight queens puzzle.

assign it to this variable. This yields the partial assignment $\{x_1 \mapsto v_1\}$. Next, we evaluate all those constraints that mention only the variable x_1 and check whether these constraints are satisfied. If any of these constraints is evaluated as `false`, we try to assign another value to x_1 until we find a value that satisfies all constraints that mention only x_1.

In general, if we have a partial variable assignment $\mathcal{B}$ of the form

$$\mathcal{B} = \{x_1 \mapsto v_1, \cdots, x_k \mapsto v_k\}$$

and we already know that all constraints that mention only the variables $x_1, \cdots, x_k$ are satisfied by $\mathcal{B}$, then in order to extend $\mathcal{B}$ we pick another variable x_{k+1} and choose a value v_{k+1} such that all those constraints that mention only the variables $x_1, \cdots, x_k, x_{k+1}$ are satisfied. If we discover that there is no such value v_{k+1}, then we have to undo the assignment $x_k \mapsto v_k$ and try to find a new value v_k such that, first, those constraints mentioning only the variables $x_1, \cdots, x_k$ are satisfied, and, second, it is possible to find a value v_{k+1} that can be assigned to x_{k+1}. This step of going back and trying to find a new value for the variable x_k is called backtracking. It might be necessary to backtrack more than one level and to also undo the assignment of the value v_{k-1} to the variable x_{k-1} or, indeed, we might be forced to undo the assignments of all variables $x_i, \cdots, x_k$ for some $i \in \{1, \cdots, n\}$. The details of this search procedure are best explained by looking at its implementation. The backtracking constraint solver is shown in Figure 5.7 on page 133 and Figure 5.8 on page 136. We discuss this program line by line.

1. Variables and Formulas are strings.
2. A `CSP` is a list of length 3.
   (a) The first element is a list of the variables.
   (b) The second element is a list of the values.
   We assume that all values are numbers.
   (c) The third element is a list of formulas. These formulas are the constraints.

```typescript
type Variable            = string;
type Formula             = string;
type CSP                 = [Variable[], number[], Formula[]];
type Assignment          = Map<Variable, number>;
type AnnotatedConstraint = [Formula, Set<Variable>];

function collectVariables(formula: Formula): Set<Variable> {
    /* not yet interesting, wait for the lecture on formal languages */
}
function evaluateExpression(
    expr:   Formula,
    assign: Assignment,
    vars:   Set<Variable>>
): boolean {
    const argNames:  string[] = [];
    const argValues: number[] = [];
    for (const v of vars) {
        argNames .push(v);
        argValues.push(assign.get(v));
    }
    const func = new Function(...argNames, `return (${expr});`);
    return func(...argValues);
}

function isConsistent(
    variable:    Variable,
    value:       number,
    assignment:  Assignment,
    assignedVars: Set<Variable>,
    constraints:  AnnotatedConstraint[]
): boolean {
    const newAssignment = assignment.mutableCopy();
    newAssignment.set(variable, value);
    const newAssignedVars = assignedVars.clone();
    newAssignedVars.add(variable);
    return constraints.every(([formula, vars]) => {
        const canEvaluate = vars.has(variable) && vars.isSubset(newAssignedVars);
        return !canEvaluate || evaluateExpression(formula, newAssignment, vars);
    });
}
```

Figure 5.7: A backtracking CSP solver, part 1.

3. A variable assignment is represented as a `Map` (remember that `Map` is really a `RecursiveMap`). This map maps the variables to the values, which are numbers.

4. An annotated constraint is a list of length 2. The first element of this list is a formula, while the second element is the set of variables that occur in this formula.

5. The function `collectVariables` takes a formula. It returns the set of all variables that occur in this formula.

   The details of the implementation of this function are not relevant. Since the function uses regular expressions and we will discuss these only in the lecture on Formal Languages, we are not yet able to discuss this implementation.

6. The function `evaluateExpression` takes three arguments:

   (a) `expr` is a formula. This formula is interpreted as TypeScript code that represents a Boolean expression. For example, `expr` could be the string `'x != y'`.

   (b) `assign` is a variable assignment. The domain of this variable assignment is guaranteed to contain all the variables that occur in `expr`.

   (c) `vars` is the set of all variables that occur in `expr`.

   The task of the function `evaluateExpression` is to evaluate `expr` with the variable assignment `assign`.

   The implementation proceeds as follows:

   (a) It creates two lists `argNames` and `argValues`. `argNames` is a list of the variable names, while `argValues` contains the corresponding values that are given in the variable assignment `assign`.

   (b) It dynamically creates a new function `func` that returns the given expression `expr`.

   (c) Finally, it calls this function with the corresponding arguments and returns the value computed by the dynamically created function. This way, we can evaluate arbitrary expressions.

7. The function `isConsistent` takes five arguments:

   (a) `variable` represents the name of the variable we currently want to assign a value to.

   (b) `value` is a number. This represents the proposed value we want to assign to the `variable`.

   (c) `assignment` is a partial variable assignment that is represented as a `Map`.

   (d) `assignedVars` is the set of all variables that have already been assigned a value. Hence this set is the domain of the partial variable assignment `assign`.

   (e) `constraints` is a list of annotated constraints that our assignment must satisfy.

   The task of the function `isConsistent` is to determine whether assigning the given `value` to the `variable` satisfies all the constraints that can be evaluated if we extend the partial assignment `assign` so that `variable` is mapped to `value`.

   The implementation proceeds as follows:

   (a) It copies the given variable assignment to `newAssignment`, since the old variable assignment must not be changed as we need it if we have to backtrack.

(b) It extends `newAssignment` so that `variable` is mapped to `value`.

(c) It computes the domain of `newAssignment` by first creating a copy of the old `assignment` and then adding `variable` to this set.

(d) Next, it checks that `newAssignment` does not violate any of the constraints that can be evaluated with `newAssignment`.

Given an annotated constraint `formula` that contains only variables from the set `vars` it first determines if the constraint is ready to be evaluated. The variable `canEvaluate` is `true` if and only if the current `variable` is part of the constraint, which is checked by `vars.has(variable)` and, furthermore, all variables required by this constraint have already been assigned values, which is checked by `vars.isSubset(newAssignedVars)`.

If the constraint cannot be evaluated yet, we do not need to bother with it. Otherwise, the constraint is evaluated.

The function `isConsistent` only returns `true` if all constraints that can be evaluated are evaluated as `true`.

1. The function `backtrackSearch` that is shown in Figure 5.8 on page 136 is the core recursive algorithm that explores possible variable assignments. It takes five arguments:

   (a) `assignment` is the current partial variable assignment.

   This assignment is known to be consistent: All constraints that only mention variables that are part of the domain of `assignment` can be evaluated.

   (b) `assignedVars` is the set of variables that have already been assigned a value.

   (c) `variables` is the complete list of all variables of our `CSP`.

   (d) `values` is the list of all possible values that can be assigned to a variable.

   (e) `constraints` is the list of annotated constraints.

   The task of this function is to extend the current partial assignment to a complete, consistent assignment, or return `null` if no such assignment exists.

   The implementation proceeds as follows:

   (a) It checks if the size of the current `assignment` equals the total length of the `variables` array. If they are equal, the assignment is complete. Since it is also consistent, it is a solution of our CSP and hence is returned.

   (b) It selects the next unassigned variable, `nextVar`, by finding the first variable in the `variables` list that is not present in the `assignedVars` set.

   (c) It loops through every possible `value` in the `values` array. For each value:

      (i) It checks if assigning this `value` to `nextVar` is mathematically valid by calling the `isConsistent` function.

      (ii) If it is consistent, it prepares for the next recursive step by creating safe, independent copies of the current state. It creates `newAssignment` using `mutableCopy` and assigns the new value. It also creates `newAssignedVars` using `clone()` and adds `nextVar`.

      (iii) It recursively calls `backtrackSearch` with these newly updated collections.

      (iv) If the recursive call is successful (i.e., `result != null`), it immediately propagates this success upwards by returning the `result`.

```
    function backtrackSearch(
        assignment:   Assignment,
        assignedVars: Set<Variable>>,
        variables:    Variables[],
        values:       number[],
        constraints:  AnnotatedConstraint[]
    ): Assignment | null {
        if (assignment.size == variables.length) { return assignment; }
        const nextVar = variables.find(v => !assignedVars.has(v));
        for (const value of values) {
            if (isConsistent(nextVar, value, assignment,
                             assignedVars, constraints))
            {
                const newAssignment = assignment.mutableCopy();
                newAssignment.set(nextVar, value);
                const newAssignedVars = assignedVars.clone();
                newAssignedVars.add(nextVar);
                const result =
                      backtrackSearch(
                          newAssignment,
                          newAssignedVars,
                          variables,
                          values,
                          constraints);
                if (result != null) { return result; }
            }
        }
        return null;
    }

    function solve(csp: CSP): Assignment | null {
        const [Vars, Values, Constrs] = csp;
        const annotatedConstraints: AnnotatedConstraint[] =
              Constrs.map(f => [f, collectVariables(f)]);
        const initialAssignment = map<Variable, number>();
        const initialAssignedVars = set<Variable>(); // Empty RecursiveSet
        return backtrackSearch(initialAssignment, initialAssignedVars,
                               Vars, Values, annotatedConstraints);
    }
```

Figure 5.8: A backtracking CSP solver, part 2.

(d) **Backtracking:** If the loop finishes checking all values and none led to a successful complete

assignment, the current branch is a dead end. It returns `null`, triggering the algorithm to backtrack to previous variables.

2. The function `solve` acts as the entry point to our solver. It takes a single argument, `csp`, which contains the complete definition of the problem.

   The implementation proceeds as follows:

   (a) It uses destructuring assignment to unpack the `csp` list into three separate variables: `Vars` (the variables), `Values` (the domain), and `Constrs` (the string formulas).

   (b) It transforms the raw string formulas into a list of `AnnotatedConstraints`. It does this by mapping over `Constrs` and pairing each formula `f` with its associated variables, which are computed using the `collectVariables` function.

   (c) It initializes the starting state for the search: an empty `Map` for `initialAssignment` and an empty `Set` for `initialAssignedVars`.

   (d) Finally, it initiates the search by calling the `backtrackSearch` function with these initial empty states along with the problem definitions, and returns the final result.

If we use backtracking, we can solve the 8 queens puzzle in less than a second. For the 8 queens puzzle the order in which variables are tried is not particularly important. The reason is that all variables are connected to all other variables. For other problems the ordering of the variables can be very important. The general strategy is that variables that are strongly related to each other should be grouped together in the list `Variables`.

Exercise 17: We have already discussed the Zebra Puzzle in Section 4.8.2 of the previous chapter. Your task is to reformulate this puzzle as a constraint programming problem. You should start from the following notebook:

`Chapter-5/Zebra-Frame.ipynb`

While it is important to order the variables in a sensible way, you shouldn't spend too much time with this task. ◇

Exercise 18: Figure 5.9 shows a cryptarithmetic puzzle. The idea is that the letters "`S`", "`E`", "`N`", "`D`", "`M`", "`O`", "`R`", "`Y`" are interpreted as variables ranging over the set of decimal digits, i.e. these variables can take values in the set $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Then, the string "`SEND`" is interpreted as a decimal number, i.e. it is interpreted as the number

$$\texttt{S} \cdot 10^3 + \texttt{E} \cdot 10^2 + \texttt{N} \cdot 10^1 + \texttt{D} \cdot 10^0.$$

The strings "`MORE`" and "`MONEY`" are interpreted similarly. To make the problem interesting, the assumption is that different variables have different values. Furthermore, the digits at the beginning of a number should be different from 0.

Your task is to reformulate this puzzle as a constraint programming problem. You should start from the following notebook:

`Chapter-5/Crypto-Arithmetic-Frame.ipynb`

It is important that you add the digits one by one as in elementary school. This way, the addition is separated into a number of constraints that can then be solved by backtracking. ◇



Figure 5.9: A cryptarithmetic puzzle

5.5 Solving Search Problems by Constraint Programming

In this section we show how we can formulate certain search problems as CSPs. We will explain our method by solving the missionaries and cannibals problem, which is explained in the following: Three missionaries and three infidels have to cross a river in order to get to a church where the infidels can be baptized. According to ancient Catholic mythology, baptizing the infidels is necessary to save them from the eternal tortures of hell fire. In order to cross the river, the missionaries and infidels have a small boat available that can take at most two passengers. If at any moment on either shore there are more infidels than missionaries, then the missionaries have a problem, since the infidels have a diet that is rather unhealthy for the missionaries.

In order to solve this problem via constraint programming, we first introduce the notion of a symbolic transition system.

Definition 41 (Symbolic Transition System) *A Symbolic Transition System (STS) is a 6-tuple*

$$\mathcal{T} = \langle \mathcal{V}, \mathcal{U}, \texttt{Start}, \texttt{Goal}, \texttt{Invariant}, \texttt{Transition} \rangle$$

such that:

(a) $\mathcal{V}$ *is a set of variables.*

These variables are strings. For every variable $x \in \mathcal{V}$ there is a primed variable x' which does not occur in $\mathcal{V}$. The set of these primed variables is denoted as $\mathcal{V}'$.

(b) $\mathcal{U}$ *is a set of values that these variables can take.*

(c) `Start`, `Goal`, *and* `Invariant` *are first-order formulas such that all free variables occurring in these formulas are elements from the set $\mathcal{V}$.*

- `Start` *describes the initial state of the transition system.*
- `Goal` *describes a state that should be reached by the transition system.*
- `Invariant` *is a formula that has to be true for every state of the transition system.*

(d) `Transition` *is a first-order formula. The free variables of this formula are elements of the set $\mathcal{V} \cup \mathcal{V}'$, i.e. they are either variables from the set $\mathcal{V}$ or they are primed variables from the set $\mathcal{V}'$.*

The formula `Transition` *describes how the variables in the transition system change during a state transition. The primed variables refer to the values of the original variables after the state transition.* ◇

Every state of a transition system is a mapping of the variables to values. The idea is that the formula `Start` describes the start state of our search problem, `Goal` describes the state that we want to reach, while `Invariant` is a formula that must be true initially and that has to remain true after every transition of our system.

Remark: In the definition given above, we have not specified the signature Σ that is used for the formula. In practice, this signature is given implicitly and consists of the function and predicate symbols provided by the programming language that is used. Note that we can also define our own functions if the underlying problem demands it. ◇

Example: In order to clarify the definition given above, we show how the *missionaries and cannibals* problem can be formulated as a symbolic transition system.

(a) $\mathcal{V} := \{\mathtt{M}, \mathtt{C}, \mathtt{B}\}$.

The value of `M` is the number of missionaries on the western shore, the value of `C` is the number of infidels on that shore, while the value of `B` is the number of boats on the western shore.

(b) $\mathcal{U} := \{0, 1, 2, 3\}$.

(c) $\mathtt{Start} := (\mathtt{M} = 3 \wedge \mathtt{C} = 3 \wedge \mathtt{B} = 1)$.

At the beginning, there are 3 missionaries, 3 infidels, and 1 boat on the western shore.

(d) $\mathtt{Goal} := (\mathtt{M} = 0 \wedge \mathtt{C} = 0 \wedge B = 0)$.

The goal is to transfer everybody to the eastern shore. If the goal is reached, the number of missionaries, infidels, and boats on the western shore will be 0.

(e) $\mathtt{Invariant} := \big((\mathtt{M} = 3 \vee \mathtt{M} = 0 \vee \mathtt{M} = \mathtt{C}) \ \wedge \ \mathtt{B} \leq 1\big)$.

The first part of the invariant, i.e. the formula $\mathtt{M} = 3 \vee \mathtt{M} = 0 \vee \mathtt{M} = \mathtt{C}$, describes those states where the missionaries are not threatened by the infidels.

- $\mathtt{M} = 3$: All missionaries are on the western shore.
- $\mathtt{M} = 0$: All missionaries are together on the eastern shore.
- $\mathtt{M} = \mathtt{C}$: On both shores the numbers of missionaries and cannibals are the same.
  If neither $\mathtt{M} = 3$ nor $\mathtt{M} = 0$ holds, then the number of missionaries and cannibals has to be the same on the western shore, because if, for example, there were more missionaries on the western shore than cannibals, then there would be fewer missionaries than cannibals on the eastern shore and hence there would be a problem.

The condition $\mathtt{B} \leq 1$ is needed because there is just one boat, but the set $\mathcal{U}$ of values also includes the numbers 2 and 3.

(f) $$\begin{aligned}\mathtt{Transition} := \quad & \mathtt{B}' = 1 - \mathtt{B} \\ \wedge \quad & \big(\mathtt{B} = 1 \ \rightarrow \ 1 \leq \mathtt{M} - \mathtt{M}' + \mathtt{C} - \mathtt{C}' \leq 2 \ \wedge \ \mathtt{M}' \leq \mathtt{M} \ \wedge \ \mathtt{C}' \leq \mathtt{C}\big) \\ \wedge \quad & \big(\mathtt{B} = 0 \ \rightarrow \ 1 \leq \mathtt{M}' - \mathtt{M} + \mathtt{C}' - \mathtt{C} \leq 2 \ \wedge \ \mathtt{M}' \geq \mathtt{M} \ \wedge \ \mathtt{C}' \geq \mathtt{C}\big)\end{aligned}$$

Let us explain the details of this formula:

- $\mathtt{B}' = 1 - \mathtt{B}$
  If the boat is initially on the western shore, i.e. $\mathtt{B} = 1$, it will be on the eastern shore afterwards, i.e. we will then have $\mathtt{B}' = 0$. If, instead, the boat is initially on the eastern shore, i.e. $\mathtt{B} = 0$, it will be on the western shore afterwards and then we have $\mathtt{B}' = 1$.
- $\mathtt{B} = 1 \;\rightarrow\; 1 \leq \mathtt{M} - \mathtt{M}' + \mathtt{C} - \mathtt{C}' \leq 2 \;\wedge\; \mathtt{M}' \leq \mathtt{M} \;\wedge\; \mathtt{C}' \leq \mathtt{C}$
  If the boat is initially on the western shore, then afterwards the number of missionaries and infidels will decrease, as they leave for the eastern shore. In this case $\mathtt{M} - \mathtt{M}'$ is the number of missionaries on the boat, while $\mathtt{C} - \mathtt{C}'$ is the number of infidels. The sum of these numbers has to be between 1 and 2 because the boat cannot travel empty and can take at most two passengers.
  The condition $\mathtt{M}' \leq \mathtt{M}$ is true because when missionaries are traveling from the western shore to the eastern shore, the number of missionaries on the western shore cannot increase. Similarly, $\mathtt{C}' \leq \mathtt{C}$ has to be true.
- $\mathtt{B} = 0 \;\rightarrow\; 1 \leq \mathtt{M}' - \mathtt{M} + \mathtt{C}' - \mathtt{C} \leq 2 \;\wedge\; \mathtt{M}' \geq \mathtt{M} \;\wedge\; \mathtt{C}' \geq \mathtt{C}$
  This formula describes the transition from the eastern shore to the western shore and is analogous to the previous formula. ◇

Figure 5.10 shows how the *missionaries and cannibals problem* can be represented as a symbolic transition system in *TypeScript*.

CSP.

1. The function `invariant` checks that the invariant
   $$M = 0 \vee M = 3 \vee M = C$$
   is satisfied for the given number of missionaries and cannibals.

2. In the function `transition` we use the following convention: Since variables cannot be primed in *TypeScript* we append the character `a` to the names of the original variables from the set $\mathcal{V}$, while we append `b` to these names to get the primed versions of the corresponding variable.

3. The function $\mathtt{missionariesCSP}(n)$ receives a natural number n as its argument. It returns a CSP that has a solution iff there is a solution of the *missionaries and cannibals* problem that crosses the river exactly n times. It uses the variables
   $$\mathtt{M}_i, \mathtt{C}_i, \text{ and } \mathtt{B}_i, \text{ where } i = 0, \cdots, n.$$
   $\mathtt{M}_i$ is the number of missionaries on the western shore after the boat has crossed the river i times. The variables $\mathtt{C}_i$ and $\mathtt{B}_i$ denote the number of infidels and boats respectively.
   Line 25 ensures that the invariant of the transition system is valid after every crossing of the boat. Line 26 and line 27 describe the mechanics of the crossing.

4. The function `findSolution` tries to find a natural number n such that the problem can be solved with n crossings. As the number of crossings has to be odd, we increment n by two at the end of the `while` loop.
   The function `findSolution` takes about 4 seconds to find the solution.

Exercise 19: An agricultural economist has to sell a wolf, a goat, and a cabbage at a market place. In order to reach the market place, she has to cross a river. The boat that she can use is so small

```
function invariant(M: Variable, C: Variable, B: Variable): Formula {
    return `${M} == 0 || ${M} == 3 || ${M} == ${C}`;
}

function transition(Ma: Variable, Ca: Variable, Ba: Variable,
                    Mb: Variable, Cb: Variable, Bb: Variable): Formula
{
    const westToEast = [`(1 <= (${Ma} - ${Mb}) + (${Ca} - ${Cb})`,
                        `(${Ma} - ${Mb}) + (${Ca} - ${Cb}) <= 2`,
                        `${Mb} <= ${Ma} && ${Cb} <= ${Ca})`];
    const eastToWest = [`(1 <= (${Mb} - ${Ma}) + (${Cb} - ${Ca})`,
                        `(${Mb} - ${Ma}) + (${Cb} - ${Ca}) <= 2`,
                        `${Mb} >= ${Ma} && ${Cb} >= ${Ca})`];

    return `(${Bb} == 1 - ${Ba}) && ((${Ba} == 1) ? ${westToEast.join(' &&')}
                                                  : ${eastToWest.join(' &&')})`;
}

function missionariesCSP(n: number): CSP {
    const Vars    = range(0, n).flatMap(i => [`M${i}`, `C${i}`, `B${i}`]);
    const Values  = range(0, n);
    const Constrs =
        ['M0 == 3 && C0 == 3 && B0 == 1',          // Start
         `M${n} == 0 && C${n} == 0 && B${n} == 0`, // Goal
         ...range(0, n-1).map(i => invariant( `M${i}`, `C${i}`, `B${i}`)),
         ...range(0, n-1).map(i => transition(`M${i}`, `C${i}`, `B${i}`,
                                              `M${i+1}`, `C${i+1}`, `B${i+1}`))];
    return [Vars, Values, Constrs];
}

function findSolution(): [number, Assignment] {
    let n = 1;
    while (true) {
        console.log(`Checking n = ${n}...`);
        const csp = missionariesCSP(n);
        const result = solve(csp);
        if (result != null) { return [n, result]; }
        n += 2;
    }
}
```

Figure 5.10: Turning the symbolic transition system into a CSP.

that it can only accommodate either the goat, the wolf, or the cabbage in addition to the agricultural economist herself. Now if the agricultural economist leaves the wolf alone with the goat, the wolf will eat the goat. If, instead, the agricultural economist leaves the goat with the cabbage, the goat will eat the cabbage. Is it possible for the agricultural economist to develop a schedule that allows her to cross the river without either the goat or the cabbage being eaten?

If you want to solve the puzzle yourself, you can do so at the following link:

http://www.mathcats.com/explore/river/crossing.html

Encode this problem as a symbolic transition system and then solve it with the help of the constraint solver developed earlier. Assume that the problem can be solved with $n \in \mathbb{N}$ crossings of the river. Use the following variables:

- $\mathtt{F}i$ for $i \in \{0, \cdots, n\}$ is the number of farmers on the western shore after the ith crossing.
- $\mathtt{W}i$ for $i \in \{0, \cdots, n\}$ is the number of wolves on the western shore after the ith crossing.
- $\mathtt{G}i$ for $i \in \{0, \cdots, n\}$ is the number of goats on the western shore after the ith crossing.
- $\mathtt{C}i$ for $i \in \{0, \cdots, n\}$ is the number of cabbages on the western shore after the ith crossing.

You should start from the following notebook:

`Chapter-5/Wolf-Goat-Cabbage-Frame.ipynb` ◇

5.6 The Z3 Solver

We proceed with a discussion of the solver Z3, which has been developed at Microsoft. Z3 implements most of the state-of-the-art constraint solving algorithms and is exceptionally powerful. We introduce Z3 via a series of examples.

5.6.1 A Simple Text Problem

The following is a simple text problem from my old 8th grade math book.

- *I have as many brothers as I have sisters.*
- *My sister has twice as many brothers as she has sisters.*
- *How many children does my father have?*

In order to solve this puzzle we need two additional assumptions.

1. My father has no illegitimate children.
2. All of my father's children identify themselves as either male or female.

Strangely, in my old math book these assumptions were not mentioned.

We can now infer the number of children. If we denote the number of boys by the variable b and the number of girls by the variable g, the problem statements are equivalent to the following two equations:

$$b - 1 = g \quad \text{and} \quad 2 \cdot (g - 1) = b.$$

```
1  import { init } from 'z3-solver';
2  const { Context } = await init();
3  const Z3 = Context("main");
4
5  const boys  = Z3.Int.const('boys');
6  const girls = Z3.Int.const('girls');
7
8  const S = new Z3.Solver();
9
10 S.add(boys.sub(1).eq(girls));         // boys - 1 == girls
11 S.add(girls.sub(1).mul(2).eq(boys)); // 2 * (girls - 1) == boys
12
13 await S.check();
14 const model = S.model();
15
16 const b = parseInt(model.eval(boys).toString());
17 const g = parseInt(model.eval(girls).toString());
18 console.log(`My father has ${b + g} children: ${b} boys, ${g} girls.`);
```

Figure 5.11: Solving a simple text problem.

Figure 5.11 on page 143 shows how we can solve the given problem using the *TypeScript* interface of Z3.

1. In line 1 we import the module `z3-solver` so that we can use the TypeScript API of Z3. The documentation of this API is available at the following address:

   https://microsoft.github.io/z3guide/programming/Z3%20JavaScript%20Examples/

   However, there is no need for you to consult this documentation as we will only use a very small part of the API of Z3 in this lecture.

2. Next, we have to initialize this solver using the function `init`. Since this is an asynchronous function, we have to `await` the result. The result of the call `init()` is an object with a property `Context`, which is a function.

3. In line 3 we use this function to create the object `Z3`.

   In the following examples we will omit these three lines, as they are always the same.

4. Lines 5 and 6 create the Z3 variables `boys` and `girls` as integer valued variables. The function `Int` takes one argument, which has to be a string. This string is the name of the variable. We store these variables in TypeScript variables of the same name. It would be possible to use different names for the TypeScript variables, but that would be very confusing.

5. Line 8 creates an object of the class `Solver`. This is the constraint solver provided by Z3.

6. Lines 10 and 11 add the constraints expressing that

   (a) the number of girls is one less than the number of boys `boys - 1 == girls`,

   (b) that my sister has twice as many brothers as she has sisters `2 * (girls - 1) == boys`.

   as constraints to the solver `S`.

   Unfortunately, *TypeScript* does not support operator overloading. Therefore, we have to use the following methods:

   (a) x`.add(`y`)` computes $x + y$,

   (b) x`.sub(`y`)` computes $x - y$,

   (c) x`.mul(`y`)` computes $x * y$,

   (d) x`.div(`y`)` computes x / y,

   (e) x`.eq(`y`)` computes $x = y$,

   (f) x`.neq(`y`)` computes $x \neq y$,

   (g) x`.le(`y`)` computes $x \leq y$,

   (h) x`.ge(`y`)` computes $x \geq y$,

   (i) x`.lt(`y`)` computes $x < y$,

   (j) x`.gt(`y`)` computes $x > y$,

7. In line 10 the method `check` examines whether the given set of constraints is satisfiable. In general, this method returns one of the following results:

   (a) `sat` is returned if the problem is solvable, (`sat` is short for *satisfiable*)

   (b) `unsat` is returned if the problem is unsolvable,

   (c) `unknown` is returned if Z3 is not powerful enough to solve the given problem.

   Note that `check` is an asynchronous method, so we have to `await` its result.

8. Since in our case the method `check` returns `sat`, we can extract the solution that is computed via the method `model` in line 14.

9. In order to extract the values that have been computed by Z3 for the variables `boys` and `girls`, we can use the method `eval` and convert the result to a string.

Exercise 20: Solve the following text problem using `Z3`.

(a) A Japanese deli offers both penguins and parrots.

(b) A parrot and a penguin together cost 666 bucks.

(c) The penguin costs 600 bucks more than the parrot.

What is the price of the parrot? You may assume that the prices of these delicacies are integer valued. You should start from the following notebook:

`Chapter-5/Parrot-and-Penguin-Frame.ipynb` ◇

Exercise 21: Solve the following text problem using `Z3`.

(a) A train travels at a uniform speed for 360 miles.

(b) The train would have taken 48 minutes less to travel the same distance if it had been faster by 5 miles per hour.

Find the speed of the train!

(a) As the speed is a real number you should declare this variable via the `Z3` function `Real` instead of using the function `Int`.

(b) Be careful to not mix up different units. In particular, take care not to mix up minutes and hours.

(c) When you formulate the information given above, you will get a system of **non-linear** equations, which is equivalent to a quadratic equation. This quadratic equation has two different solutions. One of these solutions is negative. In order to exclude the negative solution you need to add a constraint stating that the speed of the train has to be greater than zero.

You should start from the following notebook:

`Chapter-5/Train-Speed-Frame.ipynb` ◇

Exercise 22: Solve the following text problem using `Z3`.

Alice is twice as old as Bob was when Alice was as old as Bob is now. When Bob is as old as Alice is now, the sum of their ages will be 90. How old are Alice and Bob now?

You should start from the following notebook:

`Chapter-5/Alice-and-Bob-Frame.ipynb` ◇

5.6.2 The Knight's Tour

In this subsection we will solve the puzzle The Knight's Tour using `Z3`. This puzzle asks whether it is possible for a knight to visit all 64 squares of a chess board in 63 moves. We will start the tour in the upper left corner of the board.

In order to model this puzzle as a constraint satisfaction problem we first have to decide on the variables that we want to use. The idea is to have 64 variables that describe the position of the knight

after its ith move where $i = 0, 1, \cdots, 63$. However, it turns out that it is best to split the values of these positions up into a row and a column. If we do this, we end up with 128 variables of the form

$$\mathtt{R}_i \text{ and } \mathtt{C}_i \quad \text{for } i \in \{0, 1, \cdots, 63\}.$$

Here $\mathtt{R}_i$ denotes the row of the knight after its ith move, while $\mathtt{C}_i$ denotes the corresponding column. Next, we have to formulate the constraints. In this case, there are two kinds of constraints:

1. We have to specify that the move from the position $\langle \mathtt{R}_i, \mathtt{C}_i \rangle$ to the position $\langle \mathtt{R}_{i+1}, \mathtt{C}_{i+1} \rangle$ is a legal move for a knight. In chess, there are two ways for a knight to move:

   (a) The knight can move two squares horizontally left or right followed by moving vertically one square up or down, or

   (b) the knight can move two squares vertically up or down followed by moving one square left or right.

   Figure 5.12 shows all legal moves of a knight that is positioned in the square `e4`. Therefore, a formula that expresses that the ith move is a legal move of the knight is a disjunction of the following eight formulas that each describe one possible way for the knight to move:

   (a) $\mathtt{R}_{i+1} = \mathtt{R}_i + 2 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i + 1$,
   (b) $\mathtt{R}_{i+1} = \mathtt{R}_i + 2 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i - 1$,
   (c) $\mathtt{R}_{i+1} = \mathtt{R}_i - 2 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i + 1$,
   (d) $\mathtt{R}_{i+1} = \mathtt{R}_i - 2 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i - 1$,
   (e) $\mathtt{R}_{i+1} = \mathtt{R}_i + 1 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i + 2$,
   (f) $\mathtt{R}_{i+1} = \mathtt{R}_i + 1 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i - 2$,
   (g) $\mathtt{R}_{i+1} = \mathtt{R}_i - 1 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i + 2$,
   (h) $\mathtt{R}_{i+1} = \mathtt{R}_i - 1 \;\wedge\; \mathtt{C}_{i+1} = \mathtt{C}_i - 2$.

2. Furthermore, we have to specify that the position $\langle \mathtt{R}_i, \mathtt{C}_i \rangle$ is different from the position $\langle \mathtt{R}_j, \mathtt{C}_j \rangle$ if $i \neq j$.

Figure 5.13 shows how we can formulate the puzzle using `Z3`.

1. In line 1 we import the library `z3-solver` and import the types `Bool` and `BitVec` from this library.

   (a) `Bool` represents Boolean values.

   (b) `BitVec` represents vectors of bits of a given length. These vectors can be used to represent unsigned integers.

2. In line 2 we initialize the solver and use it to create `Z3`. This is the same code that we have used previously in Figure 5.11 on page 143.

3. We define the auxiliary functions `row` and `col` in lines 5 and 6. Given a natural number i, the expression `row`(i) returns the string `'R`i`'` and `col`(i) returns the string `'C`i`'`. These strings are the names of the variables $\mathtt{R}_i$ and $\mathtt{C}_i$.

4. The function `toBitVec` takes a natural number and turns it into a bit vector of length 4.

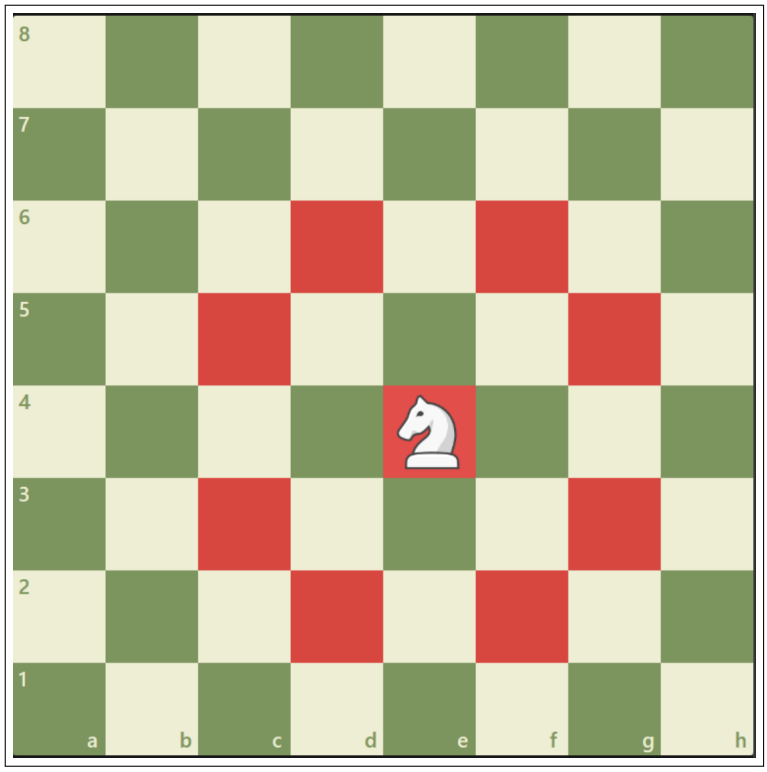


Figure 5.12: The moves of a knight, courtesy of chess.com.

5. The function `toBitVar` creates a `Z3` variable that represents a bit vector of length 4.

   The variables $\texttt{R}_i$ and $\texttt{C}_i$ will be represented as bit vectors of length 4. While the values of $\texttt{R}_i$ and $\texttt{C}_i$ are elements from the set $\{0, \cdots, 7\}$ and these values could be represented with 4 bits, we need to be able to add the numbers 1 and 2 to these values later. If we had only 3 bits, this would result in an overflow.

6. The function `isKnightMove` takes four parameters:

   (a) `row` is a `Z3` variable that specifies the row of the position of the knight before the move.

   (b) `col` is a `Z3` variable that specifies the column of the position of the knight before the move.

   (c) `rowX` is a `Z3` variable that specifies the row of the position of the knight after the move.

   (d) `colX` is a `Z3` variable that specifies the column of the position of the knight after the move.

   The function checks whether the move from position $\langle \texttt{R}_i, \texttt{C}_i \rangle$ to the position $\langle \texttt{R}_{i+1}, \texttt{C}_{i+1} \rangle$ is a legal move for a knight.

   In lines 13–14 we create the possible delta pairs. For example, the pair `[2, 1]` specifies that the knight moves 2 squares horizontally to the right and 1 square down. (The square with coordinates `[0, 0]` is the top left corner of the board.)

   In line 16 we specify that `rowX == row + deltaRow`, while line 17 specifies the equation

   ```
   colX == col + deltaCol.
   ```

   In line 18 we use the fact that the function `Z3.Or` can take any number of arguments. If `Formulas` is the set

```
import { init, Bool, BitVec } from 'z3-solver';
const { Context } = await init();
const Z3 = Context("main");

function row(i: number): string { return `R${i}`; }
function col(i: number): string { return `C${i}`; }

const toBitVec = (x: number): BitVec => Z3.BitVec.val  (x, 4);
const toBitVar = (x: string): BitVec => Z3.BitVec.const(x, 4);

function isKnightMove(row: BitVec, col: BitVec,
                      rowX: BitVec, colX: BitVec): Bool {
    const deltaSet = [ [ 2, 1], [ 2, -1], [-2, 1], [-2, -1],
                       [ 1, 2], [ 1, -2], [-1, 2], [-1, -2] ];
    const formulas = deltaSet.map(([deltaRow, deltaCol]) =>
        Z3.And(rowX.eq(row.add(toBitVec(deltaRow))),     // rowX == row + deltaRow,
               colX.eq(col.add(toBitVec(deltaCol))) ));  // colX == col + deltaCol
    return Z3.Or(...formulas);
}

function allDifferent(Rows: BitVec[], Cols: BitVec[]): Bool[] {
    return range(0, 62).flatMap(i =>
        range(i + 1, 63).map(j => Z3.Or(Rows[i].neq(Rows[j]), Cols[i].neq(Cols[j])))
    );
}

function allConstraints(Rows: BitVec[], Cols: BitVec[]): Bool[] {
    const zero  = toBitVec(0);
    const eight = toBitVec(8);
    return
        [ ...allDifferent(Rows, Cols),
          Rows[0].eq(zero), Cols[0].eq(zero),
          ...range(0, 62).map(
              i => isKnightMove(Rows[i], Cols[i], Rows[i+1], Cols[i+1])),
          ...range(0, 63).flatMap(
              i => [Rows[i].ult(eight), Cols[i].ult(eight)])
        ];
}
```

Figure 5.13: The Knight's Tour: Computing the constraints.

$$\texttt{Formulas} = \{f_1, \cdots, f_n\},$$

then the notation `z3.Or(...Formulas)` is expanded into the call

$$\texttt{z3.Or}(f_1, \cdots, f_n),$$

which computes the logical disjunction

$$f_1 \vee \cdots \vee f_n.$$

7. The function `allDifferent` takes two parameters:

   (a) `Rows` is a list of `Z3` variables. The `Z3` variable `Rows[i]` specifies the row of the position of the knight after the ith move.

   (b) `Cols` is a list of `Z3` variables. The `Z3` variable `Cols[i]` specifies the column of the position of the knight after the ith move.

   The function computes a set of formulas that state that the positions $\langle \texttt{R}_i, \texttt{C}_i \rangle$ for $i = 0, 1 \cdots, 63$ are all different from each other. Note that the position $\langle \texttt{R}_i, \texttt{C}_i \rangle$ is different from the position $\langle \texttt{R}_j, \texttt{C}_j \rangle$ iff R_i is different from R_j or C_i is different from C_j.

8. The function `allConstraints` computes the set of all constraints. The parameters for this function are the same as those for the function `allDifferent`. In addition to the constraints already discussed this function specifies that the knight starts its tour at the upper left corner of the board.

   Furthermore, there are constraints of the form

   $$\texttt{R}_i < 8 \quad \text{and} \quad \texttt{C}_i < 8.$$

   non-negative. These constraints are needed as we will model the variables with bit vectors of length 4. These constraints ensure that the knight does not jump off the board. Remember that the function `isKnightMove` performs additions like

   $$\texttt{R}_i + 2 \quad \text{and} \quad \texttt{R}_\texttt{i} - 2$$

   If $\texttt{R}_i$ is 7, then $\texttt{R}_i + 1$ would be 9 and hence the knight would no longer be on the board.

   Similarly, if $\texttt{R}_i$ is 0, then $\texttt{R}_i - 1$ would be interpreted as 15, since we are working with unsigned bit vectors of 4 bits.

Finally, the function `solve` that is shown in Figure 5.14 on page 150 can be used to solve the puzzle. The auxiliary function `parseVal` takes a bit vector and extracts the value as an integer.

The purpose of the function `solve` is to construct a CSP encoding the puzzle and to find a solution of this CSP using `Z3`. If successful, it returns a list of the form

$$\texttt{[[R}_0\texttt{, C}_0\texttt{], } \cdots \texttt{, [R}_{63}\texttt{, C}_{63}\texttt{]]}$$

that contains the positions of the knight after each of its 63 moves.

1. In lines 7 and 8 we create the `Z3` variables that specify the positions of the knight after its ith move. $\texttt{Rows}[i]$ specifies the row of the knight after its ith move, while $\texttt{Cols}[i]$ specifies the column.

2. We create a solver object in line 9 and add the constraints to this solver in the following line.

3. The function `check` tries to build a model satisfying the constraints, while the function `model` extracts this model if it exists.

```

function parseVal(expr: BitVec): number {
    return parseInt(expr.toString().substring(2), 16);
}

async function solve(): Promise<[number, number][]> {
    const Rows: BitVec[] = range(63).map(i => toBitVar(row(i)));
    const Cols: BitVec[] = range(63).map(i => toBitVar(col(i)));;
    const solver = new Z3.Solver();
    solver.add(...allConstraints(Rows, Cols));
    const check = await solver.check();
    if (check == 'sat') {
        const model = solver.model();
        const solution: [number, number][] = range(63).map(i =>
            [ parseVal(model.eval(Rows[i])),
              parseVal(model.eval(Cols[i])) ]);
        return solution;
    } else if (check == 'unsat') {
        console.log('The problem is not solvable.');
    } else {
        console.log('Z3 cannot determine whether the problem is solvable.');
    }
```

Figure 5.14: The function `solve`.

0	25	10	35	12	27	8	37
47	34	1	26	9	36	13	28
24	31	46	11	2	29	38	7
33	48	53	30	39	44	3	14
54	23	32	45	52	15	6	43
49	20	17	40	57	4	59	62
18	55	22	51	16	61	42	5
21	50	19	56	41	58	63	60

Figure 5.15: A solution of the knight's problem.

4. Finally, in lines 14–16 we create a list that contains the values of all our variables.

Figure 5.15 on page 150 shows a solution that has been computed by the program discussed above.

	3	9						7
			7			4	9	2
				6	5		8	3
			6		3	2	7	
				4		8		
5	6							
		5	2		9			1
	2	1					4	
7						5		

Table 5.1: A super hard sudoku from the magazine "Zeit Online".

Exercise 23: Table 5.1 on page 151 shows a sudoku that I have taken from the Zeit Online magazine. Solve this sudoku using Z3. You should start with the following file:

Sudoku-Z3-Frame.ipynb.

Hint: The function `Z3.Distinct`$(v_1, \cdots, v_n)$ takes an arbitrary number of variables $v_1, \cdots, v_n$ and creates a constraint that specifies that these variables are mutually distinct. ◇

5.6.3 Solving Search Problems with Z3

In this subsection we show how to solve a search problem using Z3. The idea is to formulate the search problem as a symbolic transition system, convert the symbolic transition system into a CSP, and then solve the CSP using Z3. We will explain our method by solving the missionaries and cannibals problem that has already been discussed earlier.

Figure 5.16 on page 152 shows how we can define the associated symbolic transition system using the constraint solver Z3.

1. In lines 1–3 we define the variables `M`, `C`, and `B`. These represent the number of missionaries, cannibals, and boats on the western shore. We have defined these variables as integers, although the set of values really is the set $\{0, 1, 2, 3\}$. We will later add appropriate inequalities to restrict these variables to this set.

   Furthermore, we define the variables `Mx`, `Cx`, and `Bx`. These variables are used to specify the transition formula. They represent the number of missionaries, cannibals, and boats in a transition after the boat has crossed the river.

2. The function `start` in line 5 creates the formula

   $$M = 3 \wedge C = 3 \wedge B = 1$$

   specifying that there are 3 missionaries, 3 cannibals, and 1 boat on the western shore.

   Note that we have to use the Z3 function `And` to form the conjunction of the conditions. The function takes an arbitrary number of arguments. The constraint solver Z3 also provides the

```
const M = Z3.Int.const('M'); const MX = Z3.Int.const('MX');
const C = Z3.Int.const('C'); const CX = Z3.Int.const('CX');
const B = Z3.Int.const('B'); const BX = Z3.Int.const('BX');

function start(M: Arith, C: Arith, B: Arith): Bool {
    return Z3.And(M.eq(3), C.eq(3), B.eq(1));
}

function goal(M: Arith, C: Arith, B: Arith): Bool {
    return Z3.And(M.eq(0), C.eq(0), B.eq(0));
}

function invariant(M: Arith, C: Arith, B: Arith): Bool[] {
    return [ Z3.Or(M.eq(0), M.eq(3), M.eq(C)),  // M = 0 ∨ M = 3 ∨ M = C
             M.ge(0), M.le(3),                  // 0 ≤ M ≤ 3
             C.ge(0), C.le(3),                  // 0 ≤ C ≤ 3
             B.ge(0), B.le(1)                   // 0 ≤ B ≤ 1
           ];
}

function boatOK(Ma: Arith, Ca: Arith, Ba: Arith,
                Mb: Arith, Cb: Arith, Bb: Arith): Bool
{
    const numMis = Ma.sub(Mb);           // Ma - Mb
    const numCan = Ca.sub(Cb);           // Ca - Cb
    const all    = numMis.add(numCan);   // Ma - Mb + Ca - Cb
    // B = 1 → 1 ≤ Ma - Mb + Ca - Cb ≤ 2 ∧ Mb ≤ Ma ∧ Cb ≤ Ca
    return Z3.Implies(Ba.eq(1),
                      Z3.And(all.ge(1), all.le(2), Mb.le(Ma), Cb.le(Ca)));
}

function transition(Ma: Arith, Ca: Arith, Ba: Arith,
                    Mb: Arith, Cb: Arith, Bb: Arith): Bool[] {
    const formulas = [ Bb.eq(Ba.mul(-1).add(1)) ]; // Bb = 1 - Ba
    formulas.push(boatOK(Ma, Ca, Ba, Mb, Cb, Bb));
    formulas.push(boatOK(Mb, Cb, Bb, Ma, Ca, Ba));
    return formulas;
}
```

Figure 5.16: Solving The Missionaries-and-Cannibals-Problem with `Z3`, part `I`.

functions `Or` and `Implies`. The function `Or` computes the disjunction of its arguments, while the function `Implies`(A, B) represents the formula $A \rightarrow B$.

3. The function `goal` in line 9 creates the formula

   $$M = 0 \wedge C = 0 \wedge B = 0$$

   specifying that there are 0 missionaries, 0 cannibals, and 0 boats on the western shore.

4. The definition of the invariant in lines 13–19 has become more complicated because in addition to the formula

   $$M = 0 \vee M = 3 \vee M = C$$

   we also have to add constraints that restrict the values of M and C to the set $\{0, 1, 2, 3\}$, while B has to be an element of the set $\{0, 1\}$.

5. The function `boatOK` takes 6 arguments:

   (a) `Ma` is the number of missionaries on the western shore before the crossing.

   (b) `Ca` is the number of infidels on the western shore before the crossing.

   (c) `Ba` is the number of boats on the western shore before the crossing.

   (d) `Mb` is the number of missionaries on the western shore after the crossing.

   (e) `Cb` is the number of infidels on the western shore after the crossing.

   (f) `Bb` is the number of boats on the western shore after the crossing.

   The function returns a formula that checks that the following condition holds when the boat crosses the river from the western shore to the eastern shore:

   $$\texttt{Ba} = 1 \rightarrow 1 \leq \texttt{Ma} - \texttt{Mb} + \texttt{Ca} - \texttt{Cb} \leq 2$$

   Here `Ma - Mb` is the number of missionaries on the boat and `Ca - Cb` is the number of infidels on the boat. The formula expresses that there is at least one but not more than two passengers on the boat. The condition $\texttt{Ba} = 1$ ensures that the boat is initially on the western shore, so we are crossing from the west to the east.

6. The function `transition` again takes six arguments, which are interpreted in the same way as in the function `boatOK`.

   It returns a formula that expresses the following:

   (a) The boat changes sides in every transition:

   $$\texttt{Bb} = 1 - \texttt{Ba}$$

   (b) If the boat crosses the river from the western shore to the eastern shore, then there is at least one and at most two passengers.

   (c) If the boat crosses the river from the eastern shore to the western shore, then there is at least one and at most two passengers.

Figure 5.17 on page 154 shows the implementation of the function `missionariesCSP` that takes a natural number n as its input and then creates a CSP that is solvable if and only if the symbolic transition system shown in Abbildung 5.16 auf Seite 152 has a solution involving n transitions.

1. Line 4 creates the solver object.

```
import { RecursiveMap as Map } from "recursive-set";

async function missionariesCSP(n: number): Promise<Map<string, number>> {
    const solver = new Z3.Solver();
    const Ms = [], Cs = [], Bs = [];
    for (let i = 0; i <= n; i++) {
        Ms.push(Z3.Int.const(`M${i}`));
        Cs.push(Z3.Int.const(`C${i}`));
        Bs.push(Z3.Int.const(`B${i}`));
    }
    solver.add(start(Ms[0], Cs[0], Bs[0]));
    solver.add( goal(Ms[n], Cs[n], Bs[n]));
    for (let i = 0; i < n; ++i) {
        solver.add(...invariant( Ms[i], Cs[i], Bs[i]));
        solver.add(...transition(Ms[i], Cs[i], Bs[i], Ms[i+1], Cs[i+1], Bs[i+1]));
    }
    const result = await solver.check();
    if (result == 'sat') {
        const model = solver.model();
        const solution = new Map<string, number>();
        for (let i = 0; i <= n; ++i) {
            solution.set(`M${i}`, parseInt(model.eval(Ms[i]).toString()));
            solution.set(`C${i}`, parseInt(model.eval(Cs[i]).toString()));
            solution.set(`B${i}`, parseInt(model.eval(Bs[i]).toString()));
        }
        return solution;
    }
}

async function findSolution(): Promise<Map<string, number>> {
    let n = 1;
    while (true) {
        console.log(`Trying n=${n}...`);
        const solution = await missionariesCSP(n);
        if (solution) {
            return solution;
        }
        n += 2;
    }
}
```

Figure 5.17: Solving The Missionaries-and-Cannibals-Problem with Z3, part II.

2. Lines 5–10 create the associated variables $\texttt{M}_i$, $\texttt{C}_i$, and $\texttt{B}_i$ for $i = 0, \cdots, n$.
3. Line 11 creates the constraint specifying the start state.
4. Line 12 creates the constraint corresponding to the goal state.
5. The `for`-loop in lines 13–16 ensures two things:
   (a) Every intermediate state has to satisfy the invariant.
   (b) Every transition from a state to the successor state has to satisfy the transition formula.
6. Line 17 checks whether the resulting CSP is solvable.
7. If it is solvable, we return a `Map` that associates every variable $\texttt{M}_i$, $\texttt{C}_i$, $\texttt{B}_i$ with the corresponding value of the solution.
8. The function `findSolution` searches for a value of $n \in \mathbb{N}$ such that the problem is solvable in n steps.

At this point you might well ask whether there is any benefit if we solve this problem with `Z3`. The answer is that it is about efficiency. While our simple backtracking solver took 1.5 seconds to solve this problem, `Z3` takes only 0.113 seconds to compute the solution. In the following exercise you will solve a more complicated search problem which would take much too long to solve if we only used backtracking.

Exercise 24: The following puzzle is part of a recruitment test in Japan.

A policeman, a convict, a father and his two sons Anton and Bruno, and a mother with her two daughters Cindy and Doris have to cross a river. On the boat there is only room for two passengers. During the crossing, the following conditions have to be observed:

(a) The father is not allowed to be on a shore with one of the daughters if the mother is on the other shore.

(b) The mother is not allowed to be on a shore with one of the sons if the father is on the other shore.

(c) If the convict is not alone, then the policeman must watch him.

(d) However, the convict can be alone on a shore, as his shackles prevent him from running away.

(e) Only the father, the mother, and the policeman are able to steer the boat.

You can try your luck online here.

Your task is to solve this puzzle by coding it as a symbolic transition system. You should then solve the transition system using the constraint solver `Z3`. Start with the following file:

Chapter-5/Japanese-Z3-Frame.ipynb. ◇

Exercise 25: The following puzzle offers a new twist to the river crossing puzzles encountered before, because we have to take the time used for a crossing into account.

In the darkness of the night, a group of four individuals encounters a river. A slender bridge stretches before them, capable of accommodating just two people simultaneously. Equipped with a single torch, they must rely on its flickering light to navigate the bridge. Each person possesses a distinct crossing time: Ariela takes 1 minute, Brian takes 2 minutes, Charly takes 5 minutes, and Dumpy takes 8 minutes. It is crucial to note that when two people cross together, they must synchronize their steps with the slower individual's pace. Given the torch's limited lifespan of 15 minutes, the pressing question arises: can all four individuals successfully traverse the bridge?

Your task is to solve this puzzle by coding it as a symbolic transition system. You should then solve the transition system using the constraint solver Z3. Start with the following file:

Bridge-Torch-Z3-Frame.ipynb. ◇

5.7 Check Your Comprehension

1. What is a signature?
2. How did we define the set $\mathcal{T}_\Sigma$ of Σ-terms?
3. Define atomic formulas!
4. How did we define the set $\mathbb{F}_\Sigma$ of Σ-formulas?
5. What is a Σ-structure?
6. Let $\mathcal{S}$ be a Σ-structure. How did we define the concept of an $\mathcal{S}$-variable assignment?
7. How did we define the semantics of Σ-formulas?
8. When is a first-order logic formula universally valid?
9. What does the notation $\mathcal{S} \models F$ mean for a Σ-structure $\mathcal{S}$ and a Σ-formula F?
10. When is a set of first-order logic formulas unsatisfiable?
11. What is a constraint satisfaction problem?
12. How does backtracking work?
13. Why does the order in which the different variables are instantiated matter in backtracking?
14. Are you able to solve a constraint satisfaction problem with the help of Z3?

Chapter 6

Automatic Theorem Proving

This chapter is structured as follows:

(a) First, we consider normal forms of first-order logic formulas and show how formulas can be transformed into first-order logic clauses.

(b) Next, we discuss the unification algorithm of Martelli and Montanari.

(c) Then we also discuss a first-order logic calculus, which is the basis of automatic reasoning in first-order logic.

(d) To conclude the chapter, we discuss the automatic theorem prover *Vampire*, as well as the programs *Prover9* and *Mace4*.

6.1 FOL Conjunctive Normal Form

In order to perform automatic theorem proving for first order logic (abbreviated as FOL in the following), it is necessary to transform FOL formulas into conjunctive normal form. FOL conjunctive normal form (abbreviated as FOL-CNF) provides a standardized representation for logical formulas. Its structure is defined as follows:

- A literal is an atomic formula (a predicate applied to terms) or the negation of an atomic formula. For example, $p(x, f(a))$ and $\neg q(y)$ are literals.
- A clause is a disjunction of one or more literals. For example, $p(x) \vee \neg q(y) \vee r(a, z)$ is a clause.

  As we have done in propositional logic, we will use set notation to represent clauses.
- A formula in CNF is a conjunction of clauses.

  Again, we will use set notation and represent the formula as a set of its clauses where the clauses themselves are represented as sets of literals.

A critical characteristic of clausal form in FOL is that **all variables appearing in the clauses are implicitly universally quantified**. The scope of these quantifiers is the individual clause in which the variable appears. Explicit universal quantifiers are typically dropped for conciseness.

6.1.1 Significance of Clausal Form in Automated Reasoning

The transformation of FOL formulas into clausal form is not merely a syntactic exercise; it is a cornerstone of automated reasoning and computational logic. The primary significance lies in its utility for automated theorem proving (ATP) systems. Clausal form offers a standardized and simplified representation of FOL formulas, which is highly amenable to computational manipulation. The uniformity of this structure—a conjunction of disjunctions of literals—simplifies the design and implementation of inference algorithms. Many ATP systems, particularly those based on the resolution principle, require their input to be in clausal form. The resolution principle is a powerful inference rule specifically designed to operate on clauses. It is a generalization of the cut rule from propositional logic.

The purpose of converting FOL formulas to clausal form is to make them suitable for processing by automated deduction systems. These systems often aim to determine the satisfiability or unsatisfiability of a set of formulas. Specifically, for theorem proving, the goal is frequently to show that a conclusion logically follows from a set of premises. This is often achieved through a **proof by refutation**: to prove a theorem T from some axioms $A_1, \cdots, A_n$ we show that the formula $A_1 \wedge \cdots \wedge A_n \wedge \neg T$ is unsatisfiable.

The conversion process yields a set of clauses that is **equisatisfiable** with the original formula (or the set of formulas including the negated theorem). This means the original formula is satisfiable if and only if the resulting set of clauses is satisfiable. If the clausal form is found to be unsatisfiable (e.g., by deriving an empty clause via resolution), then the original formula (or set of formulas) must also be unsatisfiable. This property is crucial for the soundness of refutation-based theorem proving.

6.2 The Transformation Process: From FOL to Clauses

The conversion of an arbitrary First-Order Logic formula into clausal form involves a sequence of steps. Each step systematically simplifies the formula or brings it closer to the desired structure of a conjunction of disjunctions of literals, with all variables implicitly universally quantified.

6.2.1 Step 1: Elimination of Implications and Biconditionals

The initial step aims to reduce the variety of logical connectives in the formula, retaining only negation ($\neg$), conjunction ($\wedge$), and disjunction ($\vee$), which are the fundamental connectives for CNF.

- **Rule for Biconditional Elimination:**

$$A \leftrightarrow B \quad \Leftrightarrow \quad (A \rightarrow B) \wedge (B \rightarrow A).$$

- **Rule for Implication Elimination:**

$$A \rightarrow B \quad \Leftrightarrow \quad \neg A \vee B.$$

6.2.2 Step 2: Conversion to Negation Normal Form - NNF

The objective of this step is to ensure that negation operators ($\neg$) apply only to atomic formulas (predicates). This form, where negations are pushed as deeply as possible into the formula, is known as Negation Normal Form (NNF). This is achieved by repeatedly applying the following equivalences:

(a) Double Negation Elimination:

$$\neg\neg A \quad \Leftrightarrow \quad A$$

(b) De Morgan's Laws for Connectives:

- $\neg(A \wedge B) \quad \Leftrightarrow \quad \neg A \vee \neg B$
- $\neg(A \vee B) \quad \Leftrightarrow \quad \neg A \wedge \neg B$

(c) De Morgan's Laws for Quantifiers:

- $\neg\forall x : p(x) \quad \Leftrightarrow \quad \exists x : \neg p(x)$
- $\neg\exists x : p(x) \quad \Leftrightarrow \quad \forall x : \neg p(x)$

After these transformations, negation symbols will only appear directly in front of predicate symbols, forming literals.

6.2.3 Step 3: Standardization of Variables (Renaming Variables)

The next step is crucial for ensuring that each quantifier in the formula binds a unique variable name. If a variable name is used by multiple quantifiers in different scopes, ambiguities and errors can arise during later stages, such as quantifier movement or quantifier elimination.

- If a variable name (e.g., x) is bound by more than one quantifier, all occurrences of that variable within the scope of one of these quantifiers (including the quantifier itself) must be renamed to a new, unique variable name (e.g., y) that does not appear elsewhere in the formula, or at least not in conflicting scopes. For example, the formula

  $$(\forall x : p(x)) \wedge (\exists x : q(x))$$

  should be transformed into

  $$(\forall x : p(x)) \wedge (\exists y : q(y)).$$

6.2.4 Step 4: Conversion to Prenex Normal Form

The goal of this step is to restructure the formula so that all quantifiers appear at the beginning of the formula, forming a "prefix." The remainder of the formula, which is quantifier-free, is called the matrix. Every FOL formula in classical logic is logically equivalent to a formula in prenex normal form (abbreviated as PNF). The conversion involves applying rules to move quantifiers outwards, past logical connectives. Assuming x is not a free variable in the subformula ψ (a condition ensured by prior variable standardization if necessary):

- $(\forall x : \phi) \wedge \psi \quad \Leftrightarrow \quad \forall x : (\phi \wedge \psi)$
- $(\forall x : \phi) \vee \psi \quad \Leftrightarrow \quad \forall x : (\phi \vee \psi)$
- $(\exists x : \phi) \wedge \psi \quad \Leftrightarrow \quad \exists x : (\phi \wedge \psi)$
- $(\exists x : \phi) \vee \psi \quad \Leftrightarrow \quad \exists x : (\phi \vee \psi)$

Similar rules apply when the quantified subformula is on the right (e.g.,

$$\psi \wedge (\forall x : \phi) \quad \Leftrightarrow \quad \forall x : (\psi \wedge \phi)).$$

These rules are applied repeatedly until all quantifiers are in the prefix. The resulting formula has the structure $Q_1 x_1 : Q_2 x_2 : \ldots Q_n x_n : M$, where each Q_i is a quantifier and M is the quantifier-free matrix.

6.2.5 Step 5: Skolemization: Eliminating Existential Quantifiers

Skolemization is a pivotal step that eliminates all existential quantifiers from the formula. This transformation is unique because it does not preserve logical equivalence; instead, it preserves equisatisfiability . The original formula and the Skolemized formula are either both satisfiable or both unsatisfiable. The process involves replacing existentially quantified variables with Skolem terms (constants or functions):

(a) **Skolem Constant:** If an existential quantifier $\exists y$ is not within the scope of any universal quantifiers in the PNF prefix (i.e., all quantifiers preceding $\exists y$, if any, are also existential), then every occurrence of the variable y in the matrix is replaced by a **new** Skolem constant c_{sk}. This constant symbol must not appear anywhere else in the formula or the knowledge base. The quantifier $\exists y$ is then removed. Example

- $\exists y : p(y)$ becomes $P(c_{\text{sk}})$.

(b) **Skolem Function:** If an existential quantifier $\exists y$ is within the scope of one or more universal quantifiers $\forall x_1, \forall x_2, \ldots, \forall x_k$ that precede it in the PNF prefix, then every occurrence of y in the matrix is replaced by a **new** Skolem function $f_{\text{sk}}(x_1, x_2, \ldots, x_k)$. The arguments of this function are precisely those universally quantified variables $x_1, \ldots, x_k$ whose quantifiers precede $\exists y$. The function symbol f_{sk} must be new. The quantifier $\exists y$ is then removed.

- $\forall x : \exists y : p(x, y)$ becomes $\forall x : p(x, f_{\text{sk}}(x))$.
- $\forall x_1 : \forall x_2 : \exists y : \forall x_3 : \exists z : q(x_1, x_2, y, x_3, z)$ would first transform $\exists y$ to $f_{\text{sk1}}(x_1, x_2)$, resulting in

  $$\forall x_1 : \forall x_2 : \forall x_3 : \exists z : q\big(x_1, x_2, f_{\text{sk1}}(x_1, x_2), x_3, z\big).$$

  Then, $\exists z$ (which is in the scope of $\forall x_1, \forall x_2, \forall x_3$) would be replaced by $g_{\text{sk2}}(x_1, x_2, x_3)$, yielding

  $$\forall x_1 : \forall x_2 : \forall x_3 : q\big(x_1, x_2, f_{\text{sk1}}(x_1, x_2), x_3, g_{\text{sk2}}(x_1, x_2, x_3)\big).$$

The "newness" of Skolem symbols is critical to avoid unintended relationships or assertions. Skolemization fundamentally alters the language of the formula by introducing these new symbols. The original formula asserts the *existence* of an entity, while the Skolemized formula refers to a *specific named entity* (via a Skolem constant) or a *specific function* that produces such an entity (via a Skolem function) that witnesses this existence. This change in language is why logical equivalence is lost. However, the transformation is designed such that if the original formula had a model, that model can be *extended* to provide an interpretation for these new Skolem symbols in a way that satisfies the Skolemized formula. Conversely, any model of the Skolemized formula directly provides the witnesses for the existential quantifiers in the original formula, thus satisfying it as well. This relationship ensures equisatisfiability.

6.2.6 Step 6: Dropping Universal Quantifiers

After Skolemization, the PNF prefix contains only universal quantifiers (e.g., $\forall x_1 : \forall x_2 : \ldots \forall x_k : M$). Since variables in clausal form are, by convention, implicitly universally quantified, these explicit universal quantifiers can be dropped.

- **Rule:** Remove all universal quantifiers $\forall x_i$ from the prefix. All variables remaining in the matrix M' are now considered free but are implicitly understood to be universally quantified over the scope of their respective clauses.

The formula is now just the quantifier-free matrix, M.

6.2.7 Step 7: Conversion of the Matrix to Conjunctive Normal Form (CNF)

The quantifier-free matrix M obtained from the previous step must now be converted into conjunctive normal form (CNF). A formula is in CNF if it is a conjunction of one or more clauses, where each clause is a disjunction of literals. This primarily involves applying the distributive laws to move disjunctions inwards over conjunctions:

- $A \vee (B \wedge C) \quad \Leftrightarrow \quad (A \vee B) \wedge (A \vee C)$,
- $(A \wedge B) \vee C \quad \Leftrightarrow \quad (A \vee C) \wedge (B \vee C)$.

These rules are applied repeatedly until the matrix is a conjunction of disjunctions. It is important to note that this step can, in the worst case, lead to an exponential increase in the size of the formula. For example, converting $(A_1 \wedge B_1) \vee (A_2 \wedge B_2) \vee \cdots \vee (A_n \wedge B_n)$ to CNF results in 2^n clauses. However, for many practical applications, the worst case does not occur.

6.2.8 Step 8: Using Set Notation

Once the matrix is in CNF, it has the form $C_1 \wedge C_2 \wedge \cdots \wedge C_m$, where each C_i is a disjunction of literals.

- **Rule:** Each conjunct C_i forms an individual clause. The entire formula can then be represented as a set of these clauses: $\{C_1, C_2, \ldots, C_m\}$. The conjunction symbol between the clauses is now implicit in the set notation.
- The clauses themselves are disjunctions of literals and are also written as sets.

6.2.9 Comprehensive Example Illustrating All Steps

Consider the First-Order Logic formula:

$$\phi := \forall x : \big(r(x) \rightarrow (\exists y : (p(x,y) \wedge \neg q(y)) \wedge \forall z : s(x,z))\big).$$

The transformation proceeds as follows:

1. **Step 1: Elimination of Implications and Biconditionals**
   Replace $A \rightarrow B$ with $\neg A \vee B$.

$$\phi \quad \Leftrightarrow \quad \forall x : \Big(\neg r(x) \vee \big(\exists y : (p(x,y) \wedge \neg q(y)\big) \wedge \forall z : s(x,z)\big)\Big)$$

2. **Step 2: Moving Negations Inwards (NNF)**
   The previous formula is already in NNF as all negations ($\neg R(x)$, $\neg Q(y)$) apply directly to atomic formulas.

3. **Step 3: Standardization of Variables**
   The variables x, y, z are bound by distinct quantifiers and their scopes are nested or separate as required. No renaming is needed at this stage for these variables.

4. **Step 4: Conversion to Prenex Normal Form (PNF)**
   Move all quantifiers to the front. We get

$$\phi \quad \leftrightarrow \quad \forall x : \exists y : \forall z : \Big(\neg r(x) \vee \big(p(x,y) \wedge \neg q(y) \wedge s(x,z)\big)\Big)$$

5. **Step 5: Skolemization**

   - The existential quantifier $\exists y$ is in the scope of the universal quantifier $\forall x$.
   - Therefore, we replace y with a new Skolem function $f(x)$ and remove the existential quantifier $\exists y$. This yields

$$\phi \approx_e \forall x : \forall z : \Big(\neg r(x) \vee \big(p(x,f(x)) \wedge \neg q(f(x)) \wedge s(x,z)\big)\Big)$$

6. **Step 6: Drop the Universal Quantifiers**
   Remove $\forall x$ and $\forall z$. Variables x and z are now implicitly universally quantified. Therefore

$$\phi \approx_e \neg r(x) \vee \big(p(x,f(x)) \wedge \neg q(f(x)) \wedge s(x,z)\big)$$

7. **Step 7: Conversion of the Matrix to Conjunctive Normal Form (CNF)** Applying the distributive law yields

$$\phi \approx_e \big(\neg r(x) \vee p(x,f(x))\big) \wedge \big(\neg r(x) \vee \neg q(f(x))\big) \wedge \big(\neg r(x) \vee s(x,z)\big)$$

8. **Step 8: Set Notation**
   The last formula is a conjunction of three clauses. The set of clauses is:

$$\Big\{\big\{\neg r(x), p(x,f(x))\big\}, \big\{\neg r(x), \neg q(f(x))\big\}, \big\{\neg r(x), s(x,z)\big\}\Big\}$$

Exercise 26: Transform the following formulas from first-order logic into sets of first-order clauses:

(a) $\forall x, y : \Big(\texttt{uncle}(x,y) \leftrightarrow \exists z : \big(\texttt{brother}(x,z) \wedge \texttt{parent}(z,y)\big)\Big)$,

(b) $\forall x, z : \Big(x < z \rightarrow \exists y : \big(x < y \wedge y < z\big)\Big)$,

(c) $\forall x : \big(p(x) \leftrightarrow \exists y : q(x,y)\big)$,

(d) $\forall x : \exists y : \big(\forall z : q(y,z) \rightarrow p(x,y)\big)$. ◇

The Jupyter notebook linked below contains a *TypeScript* program that we can use to transform predicate logic formulas into a satisfiability-equivalent set of first-order clauses.

Chapter-6/01-FOL-CNF.ipynb.

6.2.10 Refutational Theorem Proving

Next, our goal is to develop a method that can be used to verify that a first-order formula f that contains no free variables is universally valid, i.e. we want to be able to prove that

$$\models f$$

holds. We know that

$$\models f \quad \text{if and only if} \quad \{\neg f\} \models \bot,$$

because the closed formula f is universally valid iff there is no Σ-structure $\mathcal{S}$ such that

$$\mathcal{S} \models \neg f,$$

i.e. f is universally valid iff $\neg f$ is unsatisfiable. Therefore, in order to prove that f is universally valid, we transform the formula $\neg f$ into first-order normal form. By doing this we obtain clauses $k_1, \cdots, k_n$ such that

$$\neg f \approx_e k_1 \wedge \cdots \wedge k_n$$

holds. Next, we try to derive a contradiction from the clauses $k_1, \cdots, k_n$:

$$\{k_1, \cdots, k_n\} \vdash \bot$$

If this succeeds, then we know that the set $\{k_1, \cdots, k_n\}$ is unsatisfiable. This means that $\neg f$ is also unsatisfiable and hence f must be universally valid. In order to derive a contradiction from the clauses $k_1, \cdots, k_n$ we need a calculus $\vdash$ that works with first-order clauses. We will present this calculus in Section 6.4.

To explain the procedure in more detail, we will demonstrate it using an example. We want to investigate whether

$$\models \big(\exists x\colon \forall y\colon p(x,y)\big) \rightarrow \big(\forall y\colon \exists x\colon p(x,y)\big)$$

holds. We know that this is equivalent to

$$\Big\{\neg\Big(\big(\exists x\colon \forall y\colon p(x,y)\big) \rightarrow \big(\forall y\colon \exists x\colon p(x,y)\big)\Big)\Big\} \models \bot.$$

Therefore we transform the negated formula into prenex normal form.

$$\begin{aligned}
& \neg\Big(\big(\exists x\colon \forall y\colon p(x,y)\big) \rightarrow \big(\forall y\colon \exists x\colon p(x,y)\big)\Big) \\
\Leftrightarrow\ & \neg\Big(\neg\big(\exists x\colon \forall y\colon p(x,y)\big) \vee \big(\forall y\colon \exists x\colon p(x,y)\big)\Big) \\
\Leftrightarrow\ & \big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \neg\big(\forall y\colon \exists x\colon p(x,y)\big) \\
\Leftrightarrow\ & \big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \big(\exists y\colon \neg\exists x\colon p(x,y)\big) \\
\Leftrightarrow\ & \big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \big(\exists y\colon \forall x\colon \neg p(x,y)\big)
\end{aligned}$$

In order to proceed we have to rename the variables in the second part of the formula. We replace x

with u and y with v and are left with the following:

$$\begin{aligned}
& \big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \big(\exists y\colon \forall x\colon \neg p(x,y)\big) \\
\Leftrightarrow\ & \big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \big(\exists v\colon \forall u\colon \neg p(u,v)\big) \\
\Leftrightarrow\ & \exists v\colon \Big(\big(\exists x\colon \forall y\colon p(x,y)\big) \wedge \big(\forall u\colon \neg p(u,v)\big)\Big) \\
\Leftrightarrow\ & \exists v\colon \exists x\colon \Big(\big(\forall y\colon p(x,y)\big) \wedge \big(\forall u\colon \neg p(u,v)\big)\Big) \\
\Leftrightarrow\ & \exists v\colon \exists x\colon \forall y\colon \Big(p(x,y) \wedge \big(\forall u\colon \neg p(u,v)\big)\Big) \\
\Leftrightarrow\ & \exists v\colon \exists x\colon \forall y\colon \forall u\colon \Big(p(x,y) \wedge \neg p(u,v)\Big)
\end{aligned}$$

Now we have to skolemize in order to get rid of the existential quantifiers. In order to do this, we introduce two new function symbols s_1 and s_2. We have both $\texttt{arity}(s_1) = 0$ and $\texttt{arity}(s_2) = 0$, because the existential quantifiers are not preceded by universal quantifiers.

$$\begin{aligned}
& \exists v\colon \exists x\colon \forall y\colon \forall u\colon \Big(p(x,y) \wedge \neg p(u,v)\Big) \\
\approx_e\ & \exists x\colon \forall y\colon \forall u\colon \Big(p(x,y) \wedge \neg p(u,s_1)\Big) \\
\approx_e\ & \forall y\colon \forall u\colon \Big(p(s_2,y) \wedge \neg p(u,s_1)\Big)
\end{aligned}$$

At this point we are left with universal quantifiers. These can be omitted, as we have already agreed that all free variables are implicitly universally quantified. We proceed to present the first order CNF of our formula in set notation:

$$M := \Big\{\big\{p(s_2,y)\big\}, \big\{\neg p(u,s_1)\big\}\Big\}.$$

Next, we show that the set M is contradictory. To do this, we first consider the clause $\big\{p(s_2,y)\big\}$ and insert the constant s_1 in this clause for y. This gives us the clause

$$\big\{p(s_2,s_1)\big\}. \tag{1}$$

We justify the replacement of y by s_1 by the fact that the above clause is implicitly universally quantified and if something is true for all y, then it is certainly also true for $y = s_1$.

Next, we take the clause $\big\{\neg p(u,s_1)\big\}$ and insert the constant s_2 for the variable u. We obtain the clause

$$\big\{\neg p(s_2,s_1)\big\} \tag{2}$$

Now we apply the cut rule to the clauses (1) and (2) and have

$$\big\{p(s_2,s_1)\big\}, \quad \big\{\neg p(s_2,s_1)\big\} \quad \vdash \quad \{\}.$$

We have thus derived a contradiction and shown that the set M is unsatisfiable. Therefore, we also know that

$$\Big\{\neg\Big(\big(\exists x\colon \forall y\colon p(x,y)\big) \to \big(\forall y\colon \exists x\colon p(x,y)\big)\Big)\Big\}$$

is unsatisfiable and hence we have shown

$$\models \big(\exists x\colon \forall y\colon p(x,y)\big) \to \big(\forall y\colon \exists x\colon p(x,y)\big).$$

6.3 Unification

6.3.1 Substitutions

This section introduces the notion of a most general unifier of two terms. First, we define the notion of a Σ-substitution.

Definition 42 (Σ-Substitution) *Assume that a signature $\Sigma = \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, arity \rangle$ is given. A Σ-substitution σ is a map of the form*

$$\sigma : \mathcal{V} \to \mathcal{T}_\Sigma$$

such that the set $dom(\sigma) := \{x \in \mathcal{V} \mid \sigma(x) \neq x\}$ is finite. If we have $dom(\sigma) = \{x_1, \cdots, x_n\}$ and $t_i = \sigma(x_i)$ for all $i = 1, \cdots, n$, then we use the following notation:

$$\sigma = \{x_1 \mapsto t_1, \cdots, x_n \mapsto t_n\}.$$

The set of all Σ-substitutions is denoted as $\texttt{Subst}(\Sigma)$. ◇

A substitution $\sigma = \{x_1 \mapsto t_1, \cdots, x_n \mapsto t_n\}$ can be applied to a term t by replacing the variables x_i with the terms t_i. We will use the postfix notation $t\sigma$ to denote the application of the substitution σ to the term t. Formally, the notation $t\sigma$ is defined by induction on t:

1. $x\sigma := \sigma(x)$ for all $x \in \mathcal{V}$.
2. $c\sigma = c$ for every constant $c \in \mathcal{F}$.
3. $f(t_1, \cdots, t_n)\sigma := f\big(t_1\sigma, \cdots, t_n\sigma\big)$.

Next, we define the composition of two Σ-substitutions.

Definition 43 (Composition of Substitutions) *Assume that*

$$\sigma = \{x_1 \mapsto s_1, \cdots, x_m \mapsto s_m\} \quad \text{and} \quad \tau = \{y_1 \mapsto t_1, \cdots, y_n \mapsto t_n\}$$

are two substitutions such that $dom(\sigma) \cap dom(\tau) = \{\}$. We define the composition $\sigma\tau$ of σ and τ as

$$\sigma\tau := \{x_1 \mapsto s_1\tau, \cdots, x_m \mapsto s_m\tau,\ y_1 \mapsto t_1, \cdots, y_n \mapsto t_n\}$$ ◇

Example: If we define

$$\sigma := \{x_1 \mapsto c,\ x_2 \mapsto f(x_3)\} \quad \text{and} \quad \tau := \{x_3 \mapsto h(c, c),\ x_4 \mapsto d\},$$

then we have

$$\sigma\tau = \{x_1 \mapsto c,\ x_2 \mapsto f(h(c, c)),\ x_3 \mapsto h(c, c),\ x_4 \mapsto d\}.$$ ◇

Proposition 44 *If t is a term and σ and τ are substitutions such that $dom(\sigma) \cap dom(\tau) = \{\}$ holds, then we have*

$$(t\sigma)\tau = t(\sigma\tau).$$ ◇

This proposition may be proven by induction on t.

Definition 45 (Syntactical Equation) *A syntactical equation is a pair $\langle s, t\rangle$ of terms. It is written as $s \doteq t$. A system of syntactical equations is a set of syntactical equations.* ◇

Definition 46 (Unifier) *A substitution σ solves a syntactical equation $s \doteq t$ iff we have $s\sigma = t\sigma$. If E is a system of syntactical equations and σ is a substitution that solves every syntactical equation in E, then σ is a unifier of E.* ◇

If $E = \{s_1 \doteq t_1,\ \cdots,\ s_n \doteq t_n\}$ is a system of syntactical equations and σ is a substitution, then we define

$$E\sigma := \{s_1\sigma \doteq t_1\sigma,\ \cdots,\ s_n\sigma \doteq t_n\sigma\}.$$

Example: Let us consider the syntactical equation

$$p(x_1, f(x_4)) \doteq p(x_2, x_3)$$

and define the substitution

$$\sigma := \{x_1 \mapsto x_2,\ x_3 \mapsto f(x_4)\}.$$

Then σ solves the given syntactical equation because we have

$$p(x_1, f(x_4))\sigma = p(x_2, f(x_4)) \quad \text{and}$$
$$p(x_2, x_3)\sigma \quad = p(x_2, f(x_4)).$$ ◇

6.3.2 The Algorithm of Martelli and Montanari

Next we develop an algorithm for solving a system of syntactical equations. The algorithm we present was published by Martelli and Montanari [MM82]. To begin, we first consider the cases where a syntactical equation $s \doteq t$ is unsolvable. There are two cases: A syntactical equation of the form

$$f(s_1, \cdots, s_m) \doteq g(t_1, \cdots, t_n)$$

is certainly unsolvable if f and g are different function symbols. The reason is that for any substitution σ we have that

$$f(s_1, \cdots, s_m)\sigma = f(s_1\sigma, \cdots, s_m\sigma) \quad \text{and} \quad g(t_1, \cdots, t_n)\sigma = g(t_1\sigma, \cdots, t_n\sigma).$$

If $f \neq g$, then the terms $f(s_1, \cdots, s_m)\sigma$ and $g(t_1, \cdots, t_n)\sigma$ start with different function symbols and hence they can't be identical.

The other case where a syntactical equation is unsolvable is a syntactical equation of the following form:

$$x \doteq f(t_1, \cdots, t_n) \quad \text{where } x \in \texttt{var}\big(f(t_1, \cdots, t_n)\big).$$

This syntactical equation is unsolvable because the term $f(t_1, \cdots, t_n)\sigma$ will always contain at least one more occurrence of the function symbol f than the term $x\sigma$.

Now we are able to present an algorithm for solving a system of syntactical equations, provided the system is solvable. The algorithm will also discover if a system of syntactical equations is unsolvable. The algorithm works on pairs of the form $\langle F, \tau\rangle$ where F is a system of syntactical equations and τ is a substitution. The algorithm starts with the pair $\langle E, \{\}\rangle$. Here E is the system of syntactical

equations that is to be solved and $\{\}$ represents the empty substitution. The system works by simplifying the pairs $\langle F, \tau \rangle$ using certain reduction rules that are presented below. These reduction rules are applied until we either discover that the system of syntactical equations is unsolvable or else we reduce the pairs until we finally arrive at a pair of the form $\langle \{\}, \mu \rangle$. In this case μ is a unifier of the system of syntactical equations E. The reduction rules are as follows:

1. If $y \in \mathcal{V}$ is a variable that does not occur in the term t, then we can perform the following reduction:

   $$\Big\langle E \cup \{y \doteq t\}, \sigma \Big\rangle \quad \rightsquigarrow \quad \Big\langle E\{y \mapsto t\},\ \sigma\{y \mapsto t\} \Big\rangle \qquad \text{if } y \in \mathcal{V} \text{ and } y \notin \mathtt{var}(t)$$

   This reduction rule can be understood as follows: If the system of syntactical equations that is to be solved contains a syntactical equation of the form $y \doteq t$, where the variable y does not occur in the term t, then the syntactical equation $y \doteq t$ can be removed if we apply the substitution $\{y \mapsto t\}$ to both components of the pair

   $$\Big\langle E \cup \{y \doteq t\},\ \sigma \Big\rangle.$$

2. If the variable y occurs in the term t, i.e. if $y \in \mathit{Var}(t)$ and, furthermore, $t \neq y$, then the system of syntactical equations $E \cup \{y \doteq t\}$ has no solution. We write this as

   $$\Big\langle E \cup \{y \doteq t\},\ \sigma \Big\rangle \rightsquigarrow \Omega \quad \text{if } y \in \mathtt{var}(t) \text{ and } y \neq t.$$

3. If $y \in \mathcal{V}$ and $t \notin \mathcal{V}$, then we have:

   $$\Big\langle E \cup \{t \doteq y\},\ \sigma \Big\rangle \quad \rightsquigarrow \quad \Big\langle E \cup \{y \doteq t\},\ \sigma \Big\rangle \qquad \text{if } y \in \mathcal{V} \text{ and } t \notin \mathcal{V}.$$

   After we apply this rule, we can apply either the first or the second reduction rule thereafter.

4. Trivial syntactical equations can be deleted:

   $$\Big\langle E \cup \{x \doteq x\},\ \sigma \Big\rangle \quad \rightsquigarrow \quad \Big\langle E,\ \sigma \Big\rangle \qquad \text{if } x \in \mathcal{V}.$$

5. If f is an n-ary function symbol we have

   $$\Big\langle E \cup \big\{f(s_1, \cdots, s_n) \doteq f(t_1, \cdots, t_n)\big\},\ \sigma \Big\rangle \rightsquigarrow \Big\langle E \cup \{s_1 \doteq t_1, \cdots, s_n \doteq t_n\},\ \sigma \Big\rangle.$$

   This rule is the reason that we have to work with a system of syntactical equations, because even if we start with a single syntactical equation the rule given above can increase the number of syntactical equations.

   A special case of this rule is the following:

   $$\Big\langle E \cup \{c \doteq c\},\ \sigma \Big\rangle \rightsquigarrow \Big\langle E,\ \sigma \Big\rangle.$$

   Here c is a nullary function symbol.

6. The system of syntactical equations $E \cup \big\{f(s_1, \cdots, s_m) \doteq g(t_1, \cdots, t_n)\big\}$ has no solution if the function symbols f and g are different. Hence we have

   $$\Big\langle E \cup \big\{f(s_1, \cdots, s_m) \doteq g(t_1, \cdots, t_n)\big\},\ \sigma \Big\rangle \rightsquigarrow \Omega \qquad \text{provided } f \neq g.$$

If a system of syntactical equations E is given and we start with the pair $\langle E, \{\} \rangle$, then we can apply the rules given above until one of the following two cases happens:

1. We use the second or the sixth of the reduction rules given above. In this case the system of syntactical equations E is unsolvable.
2. The pair $\langle E, \{\}\rangle$ is reduced into a pair of the form $\langle \{\}, \mu\rangle$. Then μ is a unifier of E. In this case we write $\mu = \texttt{mgu}(E)$. If $E = \{s \doteq t\}$, we write $\mu = \texttt{mgu}(s, t)$. The abbreviation mgu is short for "most general unifier".

Example: We show how to solve the syntactical equation

$$p(x_1, f(x_4)) \doteq p(x_2, x_3).$$

We have the following reductions:

$$\begin{array}{rl} & \big\langle \big\{p(x_1, f(x_4)) \doteq p(x_2, x_3)\big\},\ \{\}\big\rangle \\ \leadsto & \big\langle \big\{x_1 \doteq x_2, f(x_4) \doteq x_3\big\},\ \{\}\big\rangle \\ \leadsto & \big\langle \big\{f(x_4) \doteq x_3\big\},\ \{x_1 \mapsto x_2\}\big\rangle \\ \leadsto & \big\langle \big\{x_3 \doteq f(x_4)\big\},\ \{x_1 \mapsto x_2\}\big\rangle \\ \leadsto & \big\langle \{\}, \{x_1 \mapsto x_2,\ x_3 \mapsto f(x_4)\}\big\rangle \end{array}$$

Hence the method is successful and the substitution

$$\{x_1 \mapsto x_2,\ x_3 \mapsto f(x_4)\}$$

is a solution of the syntactical equation given above. ◇

Example: Next we try to solve the following system of syntactical equations:

$$E = \big\{p(h(x_1, c)) \doteq p(x_2),\ q(x_2, d) \doteq q(h(d, c), x_4)\big\}$$

We have the following reductions:

$$\begin{array}{rl} & \big\langle \big\{p(h(x_1, c)) \doteq p(x_2),\ q(x_2, d) \doteq q(h(d, c), x_4)\big\},\ \{\}\big\rangle \\ \leadsto & \big\langle \big\{p(h(x_1, c)) \doteq p(x_2),\ x_2 \doteq h(d, c),\ d \doteq x_4\big\},\ \{\}\big\rangle \\ \leadsto & \big\langle \big\{p(h(x_1, c)) \doteq p(x_2),\ x_2 \doteq h(d, c),\ x_4 \doteq d\big\},\ \{\}\big\rangle \\ \leadsto & \big\langle \big\{p(h(x_1, c)) \doteq p(x_2),\ x_2 \doteq h(d, c)\big\},\ \{x_4 \mapsto d\}\big\rangle \\ \leadsto & \big\langle \big\{p(h(x_1, c)) \doteq p(h(d, c))\big\},\ \{x_4 \mapsto d,\ x_2 \mapsto h(d, c)\}\big\rangle \\ \leadsto & \big\langle \big\{h(x_1, c) \doteq h(d, c)\big\},\ \{x_4 \mapsto d,\ x_2 \mapsto h(d, c)\}\big\rangle \\ \leadsto & \big\langle \big\{x_1 \doteq d,\ c \doteq c\big\},\ \{x_4 \mapsto d,\ x_2 \mapsto h(d, c)\}\big\rangle \\ \leadsto & \big\langle \big\{x_1 \doteq d\big\},\ \{x_4 \mapsto d,\ x_2 \mapsto h(d, c)\}\big\rangle \\ \leadsto & \big\langle \{\},\ \{x_4 \mapsto d,\ x_2 \mapsto h(d, c),\ x_1 \mapsto d\}\big\rangle \end{array}$$

Hence the substitution $\{x_4 \mapsto d,\ x_2 \mapsto h(d, c),\ x_1 \mapsto d\}$ is a solution of the system of syntactical equations given above. ◇

The Jupyter notebook

Chapter-6/02-Unification.ipynb.

implements the algorithm that has been described above.

6.4 A Deductive System for First-Order Logic without Equality

In this section, we assume that our signature Σ does not use the equality sign, because this restriction makes it significantly easier to introduce a complete deductive system for first-order logic. Although there is also a complete deductive system for the case where the signature Σ contains the equality sign, it is considerably more complex than the system we are about to introduce and is therefore out of the scope of these lecture notes.

Definition 47 (Resolution) *Assume that*

1. *k_1 and k_2 are first-order logic clauses,*
2. *$p(s_1, \cdots, s_n)$ and $p(t_1, \cdots, t_n)$ are atomic formulas, and*
3. *the syntactic equation $p(s_1, \cdots, s_n) \doteq p(t_1, \cdots, t_n)$ is solvable with most general unifier*
$$\mu = \mathtt{mgu}\big(p(s_1, \cdots, s_n),\ p(t_1, \cdots, t_n)\big).$$

Then the following deduction rule

$$\frac{k_1 \cup \{p(s_1, \cdots, s_n)\} \qquad \{\neg p(t_1, \cdots, t_n)\} \cup k_2}{k_1\mu \cup k_2\mu}$$

is an application of the resolution rule. ◇

The resolution rule is a combination of the substitution rule and the cut rule. The substitution rule has the form

$$\frac{k}{k\sigma}.$$

Here, k is a first-order logic clause and σ is a substitution. In some cases, we may need to rename the variables in one of the two clauses before we can apply the resolution rule. Let us consider an example. The set of clauses

$$M = \Big\{\{p(x)\},\ \{\neg p(f(x))\}\Big\}$$

is contradictory. However, we cannot immediately apply the resolution rule because the syntactic equation

$$p(x) \doteq p(f(x))$$

is unsolvable. This is because the same variable happens to be used in both clauses. If we rename the variable x in the second clause to y, we get the set of clauses

$$\Big\{\{p(x)\},\ \{\neg p(f(y))\}\Big\}.$$

Here, we can apply the resolution rule because the syntactic equation

$$p(x) \doteq p(f(y))$$

has the solution $\{x \mapsto f(y)\}$. Then we obtain

$$\{p(x)\}, \quad \{\neg p(f(y))\} \quad \vdash \quad \{\}.$$

and thus we have proven the inconsistency of the clause set M.

The resolution rule by itself is not sufficient to derive the empty clause from a clause set M that is inconsistent in every case: we need a second rule. To see this, consider the following set of clauses:

$$M = \Big\{ \{p(f(x), y), p(u, g(v))\},\ \{\neg p(f(x), y), \neg p(u, g(v))\} \Big\}$$

We will soon show that the set M is contradictory. It can be shown that the resolution rule is not sufficient to prove that M is contradictory. However, we will soon see that M is contradictory.

Definition 48 (Factorization) *Assume that the following holds:*

1. *k is a first-order logic clause,*
2. *$p(s_1, \cdots, s_n)$ and $p(t_1, \cdots, t_n)$ are atomic formulas,*
3. *the syntactic equation $p(s_1, \cdots, s_n) \doteq p(t_1, \cdots, t_n)$ is solvable,*
4. *$\mu = \texttt{mgu}\big(p(s_1, \cdots, s_n),\ p(t_1, \cdots, t_n)\big)$.*

Then the following rules

$$\frac{k \cup \{p(s_1, \cdots, s_n),\ p(t_1, \cdots, t_n)\}}{k\mu \cup \{p(s_1, \cdots, s_n)\mu\}} \quad \textit{and} \quad \frac{k \cup \{\neg p(s_1, \cdots, s_n),\ \neg p(t_1, \cdots, t_n)\}}{k\mu \cup \{\neg p(s_1, \cdots, s_n)\mu\}}$$

are instances of the factorization rule. ◇

We will demonstrate how the inconsistency of the set M can be proven using the resolution and factorization rules.

1. First, we apply the factorization rule to the first clause. For this, we compute the unifier
   $$\mu = \texttt{mgu}\big(p(f(x), y), p(u, g(v))\big) = \{y \mapsto g(v), u \mapsto f(x)\}.$$
   Thus, we can apply the factorization rule:
   $$\{p(f(x), y), p(u, g(v))\} \quad \vdash \quad \{p(f(x), g(v))\}.$$
2. Next, we apply the factorization rule to the second clause. For this, we compute the unifier
   $$\mu = \texttt{mgu}\big(\neg p(f(x), y), \neg p(u, g(v))\big) = \{y \mapsto g(v), u \mapsto f(x)\}.$$
   Thus, we can apply the factorization rule:
   $$\{\neg p(f(x), y), \neg p(u, g(v))\} \quad \vdash \quad \{\neg p(f(x), g(v))\}.$$
3. We conclude the proof with an application of the resolution rule. The unifier used here is the empty substitution, thus $\mu = \{\}$:
   $$\frac{\{p(f(x), g(v))\} \quad \{\neg p(f(x), g(v))\}}{\{\}}.$$

If M is a set of first-order logic clauses and k is a first-order logic clause that can be derived from M by applying the resolution rule and the factorization rule, we write

$$M \vdash k.$$

This is read as M derives k .

Definition 49 (Universal closure) *If k is a first-order logic clause and $\{x_1, \cdots, x_n\}$ is the set of all variables that occur in k, we define the universal closure $\forall(k)$ of the clause k as*

$$\forall(k) := \forall x_1 \colon \cdots \forall x_n \colon k. \qquad \diamond$$

The essential properties of the notion $M \vdash k$ are summarized in the following two theorems.

Satz 50 (Correctness Theorem)
If $M = \{k_1, \cdots, k_n\}$ is a set of clauses and $M \vdash k$, then

$$\models \forall(k_1) \wedge \cdots \wedge \forall(k_n) \rightarrow \forall(k).$$

Thus, if a clause k can be derived from a set M, then k is indeed a consequence of M. □

The converse of the above correctness theorem holds only for the empty clause. It was proven in 1965 by John A. Robinson [Rob65].

Satz 51 (Refutational Completeness (Robinson, 1965))
If $M = \{k_1, \cdots, k_n\}$ is a set of clauses and M is unsatisfiable, i.e. we have

$$\models \forall(k_1) \wedge \cdots \wedge \forall(k_n) \rightarrow \bot,$$

then the empty clause can be derived from M, i.e. we have

$$M \vdash \{\}. \qquad \square$$

We now have a method to investigate whether $\models f$ holds for a given first-order logic formula f.

1. First, we compute the Skolem normal form of $\neg f$ and obtain something like

   $$\neg f \approx_e \forall x_1, \cdots, x_m \colon g.$$

2. Next, we convert the matrix g into conjunctive normal form:

   $$g \Leftrightarrow k_1 \wedge \cdots \wedge k_n.$$

   Hence, we now have

   $$\neg f \approx_e k_1 \wedge \cdots \wedge k_n$$

   and it holds that:

   $$\models f \quad \text{if and only if} \quad \{\neg f\} \models \bot \quad \text{if and only if} \quad \{k_1, \cdots, k_n\} \models \bot.$$

3. According to the correctness theorem and the theorem on refutational completeness, it holds that

   $$\{k_1, \cdots, k_n\} \models \bot \quad \text{if and only if} \quad \{k_1, \cdots, k_n\} \vdash \bot.$$

   Therefore, we now try to show the inconsistency of the set $M = \{k_1, \cdots, k_n\}$ by deriving the empty clause from M. If this succeeds, we have demonstrated the validity of the formula f.

Example: To conclude, we demonstrate the outlined method with an example. We turn to the zoology of dragonkind and start with the following axioms [Sch08]:

1. Every dragon is happy if all its children can fly.
2. Red dragons can fly.
3. The children of a red dragon are always red.

We will show that these axioms imply that

4. all red dragons are happy.

First, we formalize the axioms and the claim in first-order logic. We choose the signature

$$\Sigma_{Drache} := \langle \mathcal{V}, \mathcal{F}, \mathcal{P}, \mathit{arity} \rangle$$

where the sets $\mathcal{V}$, $\mathcal{F}$, $\mathcal{P}$, and arity are defined as follows:

1. $\mathcal{V} := \{x, y, z\}$.
2. $\mathcal{F} = \{\}$.
3. $\mathcal{P} := \{\mathit{red}, \mathit{flies}, \mathit{happy}, \mathit{child}\}$.
4. $\mathit{arity} := \{\mathit{red} \mapsto 1, \mathit{flies} \mapsto 1, \mathit{happy} \mapsto 1, \mathit{child} \mapsto 2\}$

The predicate $\mathit{child}(y, x)$ is true if and only if y is a child of x. Formalizing the axioms and the claim, we obtain the following formulas $f_1, \cdots, f_4$:

1. $f_1 := \forall x : \Big(\forall y : \big(\mathit{child}(y, x) \rightarrow \mathit{flies}(y)\big) \rightarrow \mathit{happy}(x)\Big)$
2. $f_2 := \forall x : \big(\mathit{red}(x) \rightarrow \mathit{flies}(x)\big)$
3. $f_3 := \forall x : \big(\mathit{red}(x) \rightarrow \forall y : \big(\mathit{child}(y, x) \rightarrow \mathit{red}(y)\big)\big)$
4. $f_4 := \forall x : \big(\mathit{red}(x) \rightarrow \mathit{happy}(x)\big)$

We want to show that the formula

$$f := f_1 \wedge f_2 \wedge f_3 \rightarrow f_4$$

is valid. We thus consider the formula $\neg f$ and note that

$$\begin{aligned}
\neg f &= \neg\big(f_1 \wedge f_2 \wedge f_3 \rightarrow f_4\big) \\
&\Leftrightarrow \neg\big(\neg(f_1 \wedge f_2 \wedge f_3) \vee f_4\big) \\
&\Leftrightarrow f_1 \wedge f_2 \wedge f_3 \wedge \neg f_4
\end{aligned}$$

Next, we need to convert the formula on the right side of this equivalence into a set of clauses. Since this is a conjunction of several formulas, we can convert the individual formulas f_1, f_2, f_3, and $\neg f_4$ into clauses separately.

1. The formula f_1 can be transformed as follows:

$$\begin{aligned}
f_1 &= \forall x : \Big(\forall y : \big(\mathit{child}(y,x) \to \mathit{flies}(y)\big) \to \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \Big(\neg\forall y : \big(\mathit{child}(y,x) \to \mathit{flies}(y)\big) \vee \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \Big(\neg\forall y : \big(\neg\mathit{child}(y,x) \vee \mathit{flies}(y)\big) \vee \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \Big(\exists y : \neg\big(\neg\mathit{child}(y,x) \vee \mathit{flies}(y)\big) \vee \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \Big(\exists y : \big(\mathit{child}(y,x) \wedge \neg\mathit{flies}(y)\big) \vee \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \exists y : \Big(\big(\mathit{child}(y,x) \wedge \neg\mathit{flies}(y)\big) \vee \mathit{happy}(x)\Big) \\
&\approx_e \forall x : \Big(\big(\mathit{child}(s(x),x) \wedge \neg\mathit{flies}(s(x))\big) \vee \mathit{happy}(x)\Big) \\
&\Leftrightarrow \forall x : \Big(\big(\mathit{child}(s(x),x) \vee \mathit{happy}(x)\big) \wedge \neg\big(\mathit{flies}(s(x))\big) \vee \mathit{happy}(x)\big)\Big)
\end{aligned}$$

   In the penultimate step, we have introduced the Skolem function s with $\mathit{arity}(s) = 1$. Intuitively, this function computes for each dragon x, who is not happy, a child $s(x)$, that cannot fly. In the last step, we have distributed the "$\vee$" in the matrix of this formula. In set notation, we obtain the following two clauses:

$$k_1 := \big\{\mathit{child}(s(x),x), \mathit{happy}(x)\big\},$$
$$k_2 := \big\{\neg\mathit{flies}(s(x)), \mathit{happy}(x)\big\}.$$

2. Similarly, for f_2 we find:

$$\begin{aligned}
f_2 &= \forall x : \big(\mathit{red}(x) \to \mathit{flies}(x)\big) \\
&\Leftrightarrow \forall x : \big(\neg\mathit{red}(x) \vee \mathit{flies}(x)\big)
\end{aligned}$$

   Thus, f_2 is equivalent to the following clause:

$$k_3 := \big\{\neg\mathit{red}(x), \mathit{flies}(x)\big\}.$$

3. For f_3, we see:

$$\begin{aligned}
f_3 &= \forall x : \Big(\mathit{red}(x) \to \forall y : \big(\mathit{child}(y,x) \to \mathit{red}(y)\big)\Big) \\
&\Leftrightarrow \forall x : \Big(\neg\mathit{red}(x) \vee \forall y : \big(\neg\mathit{child}(y,x) \vee \mathit{red}(y)\big)\Big) \\
&\Leftrightarrow \forall x : \forall y : \big(\neg\mathit{red}(x) \vee \neg\mathit{child}(y,x) \vee \mathit{red}(y)\big)
\end{aligned}$$

   This yields the following clause:

$$k_4 := \big\{\neg\mathit{red}(x), \neg\mathit{child}(y,x), \mathit{red}(y)\big\}.$$

4. Transforming the negation of f_4 gives:

$$\begin{aligned}
\neg f_4 &= \neg\forall x : \big(\mathit{red}(x) \to \mathit{happy}(x)\big) \\
&\Leftrightarrow \neg\forall x : \big(\neg\mathit{red}(x) \vee \mathit{happy}(x)\big) \\
&\Leftrightarrow \exists x : \neg\big(\neg\mathit{red}(x) \vee \mathit{happy}(x)\big) \\
&\Leftrightarrow \exists x : \big(\mathit{red}(x) \wedge \neg\mathit{happy}(x)\big) \\
&\approx_e \mathit{red}(d) \wedge \neg\mathit{happy}(d)\big)
\end{aligned}$$

The Skolem constant d introduced here stands for an unhappy red dragon. This leads to the following two clauses:

$$k_5 = \{red(d)\},$$
$$k_6 = \{\neg happy(d)\}.$$

Therefore, we have to investigate whether the set M, which consists of the following clauses, is contradictory:

1. $k_1 = \{child(s(x), x),\ happy(x)\}$
2. $k_2 = \{\neg flies(s(x)),\ happy(x)\}$
3. $k_3 = \{\neg red(x),\ flies(x)\}$
4. $k_4 = \{\neg red(x),\ \neg child(y, x),\ red(y)\}$
5. $k_5 = \{red(d)\}$
6. $k_6 = \{\neg happy(d)\}$

In order to do this, define $M := \{k_1, k_2, k_3, k_4, k_5, k_6\}$. We will show that $M \vdash \bot$ holds:

1. It holds that
   $$\texttt{mgu}\big(red(d), red(x)\big) = \{x \mapsto d\}.$$
   Therefore, we can apply the resolution rule to the clauses k_5 and k_4 as follows:
   $$\{red(d)\},\ \{\neg red(x), \neg child(y, x), red(y)\} \vdash \{\neg child(y, d), red(y)\}.$$
2. We now apply the resolution rule to the resulting clause and the clause k_1. For this, we first compute
   $$\texttt{mgu}\big(child(y, d), child(s(x), x)\big) = \{y \mapsto s(d), x \mapsto d\}.$$
   Then we have
   $$\{\neg child(y, d), red(y)\},\ \{child(s(x), x), happy(x)\} \vdash \{red(s(d)), happy(d)\}.$$
3. Now we apply the resolution rule to the derived clause and the clause k_6. We have:
   $$\texttt{mgu}\big(happy(d), happy(d)\big) = \{\}.$$
   Thus, we obtain
   $$\{red(s(d)), happy(d)\},\ \{\neg happy(d)\} \vdash \{red(s(d))\}.$$
4. We apply the resolution rule to the clause $\{red(s(d))\}$ and the clause k_3. First, we have
   $$\texttt{mgu}\big(red(s(d)), red(x)\big) = \{x \mapsto s(d)\}.$$
   Thus, the application of the resolution rule yields:
   $$\{red(s(d))\},\ \{\neg red(x), flies(x)\} \vdash \{flies(s(d))\}.$$

5. To resolve the clause $\{\mathit{flies}(s(d))\}$ with the clause k_2, we compute

$$\texttt{mgu}\big(\mathit{flies}(s(d)), \mathit{flies}(s(x))\big) = \{x \mapsto d\}.$$

   Then, the resolution rule gives

$$\{\mathit{flies}(s(d))\},\ \{\neg\mathit{flies}(s(x)), \mathit{happy}(x)\} \vdash \{\mathit{happy}(d)\}.$$

6. We now apply the resolution rule to the result $\{\mathit{happy}(d)\}$ and the clause k_6:

$$\{\mathit{happy}(d)\},\ \{\neg\mathit{happy}(d)\} \vdash \{\}.$$

Since we obtained the empty clause in the last step, we have proven that $M \vdash \bot$, and thus we have shown that all communist dragons are happy. ◇

Exercise 27: The *Russell set* R defined by Bertrand Russell is the set of all sets that do not contain themselves. Therefore, it holds that

$$\forall x : \big(x \in R \leftrightarrow \neg x \in x\big).$$

Using the calculus defined in this section, show that this formula is contradictory. ◇

Exercise 28: Given the following axioms:

1. Every barber shaves all persons who do not shave themselves.
2. No barber shaves anyone who shaves themselves.

These two axioms have an unsurprising consequence: Using only these axioms, show that all barbers are gay. ◇

6.5 Vampire*

The logical calculus described in the last section can be automated and forms the basis of modern automatic provers. This section presents the theorem prover Vampire [KV13]. We introduce this theorem prover via a small example from group theory.

6.5.1 Proving Theorems in Group Theory

A group is a triple $\mathcal{G} = \langle G, \mathrm{e}, \circ \rangle$ such that

1. G is a set.
2. e is an element of G.
3. $\circ$ is a binary operation on G, i.e. we have

$$\circ : G \times G \to G.$$

4. Furthermore, the following axioms hold:

   (a) $\forall x : \mathrm{e} \circ x = x$, (e is a left identity)

(b) $\forall x : \exists y : y \circ x = \mathrm{e}$, (every element has a left inverse)

(c) $\forall x : \forall y : \forall z : (x \circ y) \circ z = x \circ (y \circ z)$. ($\circ$ is associative)

It is a well-known fact that these axioms imply the following:

1. The element e is also a right identity, i.e. we have
$$\forall x : x \circ \mathrm{e} = x.$$
2. Every element has a right inverse, i.e. we have
$$\forall x : \exists y : x \circ y = e.$$

We will show both these claims with the help of *Vampire*. Figure 6.1 on page 176 shows the input file for *Vampire* that is used to prove that the left identity element e is also a right identity. We discuss this file line by line.

```
1  fof(identity, axiom, ! [X] : mult(e,X) = X).
2  fof(inverse,  axiom, ! [X] : ? [Y] : mult(Y, X) = e).
3  fof(assoc,    axiom, ! [X,Y,Z] : mult(mult(X, Y), Z) = mult(X, mult(Y, Z))).
4
5  fof(right, conjecture, ! [X] : mult(X, e) = X).
```

Figure 6.1: Prove that the left identity is also a right identity.

1. Line 1 states the axiom $\forall x : \mathrm{e} \circ x = x$. Every formula is written in the form
`fof`(*name*, *type*, *formula*) .
   - `fof` is an abbreviation for *first-order formula*.
   - *name* is a string giving the name of the formula. This name can be freely chosen, but should contain only letters, digits, and underscores. Furthermore, it should start with a letter.
   - *type* is either the string "`axiom`" or the string "`conjecture`". Every file must hold exactly one conjecture. The conjecture is the formula that has to be proven from the axioms.
   - *formula* is a first-order formula. The precise syntax of formulas will be described below.
2. Line 2 states the axiom $\forall x : \exists y : y \circ x = \mathrm{e}$.
3. Line 3 states the axiom $\forall x : \forall y : \forall z : (x \circ y) \circ z = x \circ (y \circ z)$.
4. Line 5 states the conjecture $\forall x : x \circ \mathrm{e} = x$. The keyword `conjecture` signifies that we want to prove this formula.

In order to understand the syntax of *Vampire* formulas we first have to note that all variables start with a capital letter, while function symbols and predicate symbols start with a lower case letter. As *Vampire* does not support binary operators, we had to introduce the function symbol `mult` to represent the operator $\circ$. Therefore, the term `mult`(x, y) is interpreted as $x \circ y$. Instead of `mult` we could have chosen any other name. Furthermore, *Vampire* uses the following operators:

(a) `! [X]`: F is interpreted as $\forall x : F$.

(b) `? [X]`: F is interpreted as $\exists x : F$.

(c) `$true` is interpreted as $\top$.

(d) `$false` is interpreted as $\bot$.

(e) `~`F is interpreted as $\neg F$.

(f) F `&` G is interpreted as $F \wedge G$.

(g) F `|` G is interpreted as $F \vee G$.

(h) F `=>` G is interpreted as $F \rightarrow G$.

(i) F `<=>` G is interpreted as $F \leftrightarrow G$.

When the text shown in Figure 6.1 is stored in a file with the name `group-right-identity.tptp`, then we can invoke *Vampire* with the following command

```
vampire group-right-identity.tptp
```

This will produce the output shown in Figure 6.2. This output shows that *Vampire* is trying to perform an indirect proof, i.e. *Vampire* negates the conjecture and then tries to derive a contradiction from the negated conjecture and the axioms.

If we want to prove that the left inverse is also a right inverse we can simply change the last line in Figure 6.1 to

```
fof(right, conjecture, ![X]: ?[Y]: mult(X, Y) = e).
```

Exercise 29: Use *Vampire* to show that in every group the left inverse is unique. ◇

6.5.2 Who killed Agatha?

Next, we solve the following puzzle.

> Someone who lives in Dreadbury Mansion killed Aunt Agatha. Agatha, the butler, and Charles live in Dreadbury Mansion, and are the only people who live therein. A killer always hates his victim, and is never richer than his victim. Charles hates no one that Aunt Agatha hates. Agatha hates everyone except the butler. The butler hates everyone not richer than Aunt Agatha. The butler hates everyone Aunt Agatha hates. No one hates everyone. Agatha is not the butler.

The question then is: Who killed Agatha? Let us first solve the puzzle by hand. As there are only three suspects who could have killed Agatha, we proceed with a case distinction.

1. Charles killed Agatha.
   (a) As a killer always hates its victim, Charles must then have hated Agatha.
   (b) As Charles hates no one that Agatha hates, Agatha cannot have hated herself.

```
vampire group-right-identity.tptp
% Running in auto input_syntax mode. Trying TPTP
% Refutation found. Thanks to Tanya!
% SZS status Theorem for group-right-identity
% SZS output start Proof for group-right-identity
1. ! [X0] : mult(e,X0) = X0 [input]
2. ! [X0] : ? [X1] : e = mult(X1,X0) [input]
3. ! [X0,X1,X2] : mult(mult(X0,X1),X2) = mult(X0,mult(X1,X2)) [input]
4. ! [X0] : mult(X0,e) = X0 [input]
5. ~! [X0] : mult(X0,e) = X0 [negated conjecture 4]
6. ? [X0] : mult(X0,e) != X0 [ennf transformation 5]
7. ! [X0] : (? [X1] : e = mult(X1,X0) => e = mult(sK0(X0),X0)) [choice axiom]
8. ! [X0] : e = mult(sK0(X0),X0) [skolemisation 2,7]
9. ? [X0] : mult(X0,e) != X0 => sK1 != mult(sK1,e) [choice axiom]
10. sK1 != mult(sK1,e) [skolemisation 6,9]
11. mult(e,X0) = X0 [cnf transformation 1]
12. e = mult(sK0(X0),X0) [cnf transformation 8]
13. mult(mult(X0,X1),X2) = mult(X0,mult(X1,X2)) [cnf transformation 3]
14. sK1 != mult(sK1,e) [cnf transformation 10]
16. mult(sK0(X2),mult(X2,X3)) = mult(e,X3) [superposition 13,12]
18. mult(sK0(X2),mult(X2,X3)) = X3 [forward demodulation 16,11]
22. mult(sK0(sK0(X1)),e) = X1 [superposition 18,12]
24. mult(X5,X6) = mult(sK0(sK0(X5)),X6) [superposition 18,18]
35. mult(X3,e) = X3 [superposition 24,22]
55. sK1 != sK1 [superposition 14,35]
56. $false [trivial inequality removal 55]
% SZS output end Proof for group-right-identity
% ------------------------------
% Version: Vampire 4.7 (commit )
% Termination reason: Refutation
```

Figure 6.2: Vampire proof that the left identity is a right identity.

(c) But Agatha hates everybody with the exception of the butler and since Agatha is not the butler, she must have hated herself.

This contradiction shows that Charles has not killed Agatha.

2. The butler killed Agatha.

(a) As a killer is never richer than his victim, the butler can then not be richer than Agatha.

(b) But as the butler hates everyone not richer than Agatha, he would then hate himself.

(c) As the butler also hates everyone that Agatha hates and Agatha hates everyone except the butler, the butler would then hate everyone.

(d) However, we know that no one hates everyone.

This contradiction shows that the butler has not killed Agatha.

3. Hence we must conclude that Agatha has killed herself.

Next, we show how *Vampire* can solve the puzzle. As there are only three possibilities, we try to prove the following conjectures one by one:

1. Charles killed Agatha.
2. The butler killed Agatha.
3. Agatha killed herself.

Figure 6.3 on page 179 shows the axioms and the conjecture of the last proof attempt. The first two proof attempts fail, but the last one is successful. Hence we have shown again that Agatha committed suicide.

```
% Someone who lives in Dreadbury Mansion killed Aunt Agatha.
fof(a1, axiom, ?[X] : (lives_at_dreadbury(X) & killed(X, agatha))).
% Agatha, the butler, and Charles live in Dreadbury Mansion, and are
% the only people who live therein.
fof(a2, axiom, ![X] : (lives_at_dreadbury(X) <=>
                        (X = agatha | X = butler | X = charles))).
% A killer always hates his victim.
fof(a3, axiom, ![X, Y]: (killed(X, Y) => hates(X, Y))).
% A killer is never richer than his victim.
fof(a4, axiom, ![X, Y]:  (killed(X, Y) => ~richer(X, Y))).
% Charles hates no one that Aunt Agatha hates.
fof(a5, axiom, ![X]: (hates(agatha, X) => ~hates(charles, X))).
% Agatha hates everyone except the butler.
fof(a6, axiom, ![X]: (hates(agatha, X) <=> X != butler)).
% The butler hates everyone not richer than Aunt Agatha.
fof(a7, axiom, ![X]: (~richer(X, agatha) => hates(butler, X))).
% The butler hates everyone Aunt Agatha hates.
fof(a8, axiom, ![X]: (hates(agatha, X) => hates(butler, X))).
%  No one hates everyone.
fof(a9, axiom, ![X]: ?[Y]: ~hates(X, Y)).
% Agatha is not the butler.
fof(a0, axiom, agatha != butler).

fof(c, conjecture, killed(agatha, agatha)).
```

Figure 6.3: Who killed Agatha?

6.6 *Prover9* and *Mace4**

The deductive system described in the last section can be automated and forms the basis of modern automatic provers. At the same time, the search for counterexamples can also be automated. In this section, we introduce two systems that serve these purposes.

1. *Prover9* is used to automatically prove first-order logic formulas.
2. *Mace4* examines whether a given set of first-order logic formulas is satisfiable in a finite structure. If so, this structure is computed.

The two programs, *Prover9* and *Mace4*, were developed by William McCune [McC10], are available under the GPL (*GNU General Public License*), and can be downloaded in source code form from

`http://www.cs.unm.edu/~mccune/prover9/download/`

First, we will discuss *Prover9* and then look at *Mace4*.

6.6.1 The Automatic Prover *Prover9*

Prover9 is a program that takes two sets of formulas as input. The first set of formulas is interpreted as a set of *axioms*, and the second set of formulas are the *theorems* to be proven from the axioms. For example, if we want to show that in group theory, the existence of a left-inverse element implies the existence of a right-inverse element, and that the left-neutral element is also right-neutral, we can axiomatize group theory as follows:

1. $\forall x : e \cdot x = x$,
2. $\forall x : \exists y : y \cdot x = e$,
3. $\forall x : \forall y : \forall z : (x \cdot y) \cdot z = x \cdot (y \cdot z)$.

We must now show that these axioms logically imply the two formulas

$$\forall x : x \cdot e = x \quad \text{and} \quad \forall x : \exists y : x \cdot y = e.$$

We can represent these formulas for *Prover9* as shown in Figure 6.4 on page 180.

```
1  formulas(sos).
2  all x (e * x = x).                              % left neutral
3  all x exists y (y * x = e).                     % left inverse
4  all x all y all z ((x * y) * z = x * (y * z)).  % associativity
5  end_of_list.
6
7  formulas(goals).
8  all x (x * e = x).                              % right neutral
9  all x exists y (x * y = e).                     % right inverse
10 end_of_list.
```

Figure 6.4: The axioms of group theory given in the syntax of Prover9.

The beginning of the axioms in this file is indicated by "`formulas(sos)`" and ends with the keyword "`end_of_list`." Note that both the keywords and each formula are terminated by a period "`.`". The axioms in lines 2, 3, and 4 express that:

1. `e` is a left-neutral element,

2. for every element x, there exists a left-inverse element y, and
3. the associative law holds.

From these axioms, it follows that `e` is also a right-neutral element and that for every element x, there exists a right-inverse element y. These two formulas are the goals to be proven and are marked in the file by "`formulas(goal)`." If the file shown in Figure 6.4 is named "`group2.in`," we can run the *Prover9* program with the command

```
prover9 -f group2.in
```

and obtain the information that the two formulas shown in lines 8 and 9 do indeed follow from the previously given axioms. If a formula cannot be proven, there are two possibilities: In certain cases, *Prover9* can actually recognize that a proof is impossible. In this case, the program stops the search for a proof with a corresponding message. If the situation is unfavorable, due to the undecidability of first-order logic, it is not possible to recognize that the search for a proof must fail. In such a case, the program runs until no more memory is available and then terminates with an error message.

Prover9 attempts to conduct an indirect proof. First, the axioms are converted into first-order logic clauses. Then, each theorem to be proven is negated, and the negated formula is also converted into clauses. Subsequently, *Prover9* attempts to derive the empty clause from the set of all axioms along with the clauses resulting from the negation of one of the theorems to be proven. If this succeeds, it is proven that the respective theorem indeed follows from the axioms. Figure 6.5 shows an input file for *Prover9*, in which an attempt is made to deduce the commutative law from the axioms of group theory. However, the proof attempt with *Prover9* fails. In this case, the proof search is not continued indefinitely. This is because *Prover9* manages to derive all formulas that follow from the given premises in finite time. Unfortunately, such a case is the exception. Most of the time *Prover9* will abort the attempt because it runs out of memory.

```
1  formulas(sos).
2  all x (e * x = x).                              % left neutral
3  all x exists y (y * x = e).                     % left inverse
4  all x all y all z ((x * y) * z = x * (y * z)).  % associativity
5  end_of_list.
6
7  formulas(goals).
8  all x all y (x * y = y * x).                    % * is commutative
9  end_of_list.
```

Figure 6.5: Does the commutative law apply to all groups?

6.6.2 *Mace4*

If a proof attempt with *Prover9* takes forever, it is unclear whether the theorem to be proven holds. To be sure that a formula does not follow from a given set of axioms, it is sufficient to construct a Σ-structure in which all the axioms are satisfied, but the theorem to be proven is false. The program *Mace4* is specifically designed to find such structures. This, of course, only works as long as the structures are finite. Figure 6.6 shows an input file that we can use to answer the question of whether

finite non-commutative groups exist using *Mace4*. Lines 2, 3, and 4 contain the axioms of group theory. The formula in line 5 postulates that for the two elements a and b, the commutative law does not hold, that is, $a \cdot b \neq b \cdot a$. If the text shown in Figure 6.6 is saved in a file named "*group.in*", we can start *Mace4* with the command

```
mace4 -f group.in
```

Mace4 searches for all positive natural numbers $n = 1, 2, 3, \cdots$, to see if there is a Σ-structure $\mathcal{S} = \langle \mathcal{U}, \mathcal{J} \rangle$ with $card(U) = n$ in which the specified formulas hold. For $n = 6$, *Mace4* succeeds and indeed computes a group with 6 elements in which the commutative law is violated.

```
1  formulas(theory).
2  all x (e * x = x).                                % left neutral
3  all x exists y (y * x = e).                       % left inverse
4  all x all y all z ((x * y) * z = x * (y * z)).    % associativity
5  a * b != b * a.                                   % a and b do not commute
6  end_of_list.
```

Figure 6.6: Is there a non-commutative group?

Figure 6.7 shows a part of the output produced by *Mace4*. The elements of the group are the numbers $0, \cdots, 5$, the constant a is the element 0, b is the element 1, and e is the element 2. Furthermore, we see that the inverse of 0 is 0, the inverse of 1 is 1, the inverse of 2 is 2, the inverse of 3 is 4, the inverse of 4 is 3, and the inverse of 5 is 5. The multiplication is realized by the following group table:

$\circ$	0	1	2	3	4	5
0	2	3	0	1	5	4
1	4	2	1	5	0	3
2	0	1	2	3	4	5
3	5	0	3	4	2	1
4	1	5	4	2	3	0
5	3	4	5	0	1	2

This group table shows that

$$a \circ b = 0 \circ 1 = 3, \quad \text{but} \quad b \circ a = 1 \circ 0 = 4.$$

Therefore, the commutative law is indeed violated.

Remark: The theorem prover *Prover9* is a successor of the theorem prover *Otter*. With the help of *Otter*, William McCune succeeded in proving the Robbins conjecture in 1996 [McC97]. This proof was even featured in the New York Times headline, which can be read at

`http://www.nytimes.com/library/cyber/week/1210math.html`.

This shows that automated theorem provers can indeed be useful tools. Nevertheless, first-order logic is undecidable, and so far, only a few open mathematical problems have been solved with the help of automated provers. It remains to be seen whether this will change in the future. ◇

```
1  ============================== DOMAIN SIZE 6 ========================
2
3  === Mace4 starting on domain size 6. ===
4
5  ============================== MODEL ================================
6
7  interpretation( 6, [number=1, seconds=0], [
8
9          function(a, [ 0 ]),
10
11         function(b, [ 1 ]),
12
13         function(e, [ 2 ]),
14
15         function(f1(_), [ 0, 1, 2, 4, 3, 5 ]),
16
17         function(*(_,_), [
18                             2, 3, 0, 1, 5, 4,
19                             4, 2, 1, 5, 0, 3,
20                             0, 1, 2, 3, 4, 5,
21                             5, 0, 3, 4, 2, 1,
22                             1, 5, 4, 2, 3, 0,
23                             3, 4, 5, 0, 1, 2 ])
24 ]).
25
26 ============================== end of model =========================
```

Figure 6.7: Output of *Mace4*.

6.7 Check Your Comprehension

1. What are first-order logic clauses and what steps have to be taken in order to convert a given first-order logic formula into a set of first-order clauses that is satisfiability-equivalent?
2. Define the notion of a substitution.
3. What is a unifier?
4. State the rules of the Martelli-Montanari algorithm for unification!
5. How is the resolution rule defined and why might it be necessary to rename variables before the resolution rule can be applied?
6. What is the factorization rule?
7. How do we proceed when we want to prove the validity of a first-order logic formula f?

Bibliography

[CPR11] Byron Cook, Andreas Podelski, and Andrey Rybalchenko. Proving program termination. *Communications of the ACM*, 54(5):88–98, 2011.

[DLL62] Martin Davis, George Logemann, and Donald Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962. ISSN 0001-0782.

[DP60] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, July 1960.

[HMU06] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.

[KV13] Laura Kovács and Andrei Voronkov. First-order theorem proving and vampire. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification*, pages 1–35. Springer, 2013.

[McC97] William McCune. Solution of the robbins problem. *Journal of Automated Reasoning*, 19: 263–276, December 1997. ISSN 0168-7433.

[McC10] William McCune. Prover9 and Mace4. `http://www.cs.unm.edu/~mccune/prover9/`, 2005–2010.

[MGS96] Martin Müller, Thomas Glaß, and Karl Stroetmann. Automated modular termination proofs for real prolog programs. In Radhia Cousot and David A. Schmidt, editors, *SAS*, volume 1145 of *Lecture Notes in Computer Science*, pages 220–237. Springer, 1996.

[MM82] Alberto Martelli and Ugo Montanari. An efficient unification algorithm. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 4(2):258–282, 1982.

[MMZ+01] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an Efficient SAT Solver. In *Proceedings of the 38th Design Automation Conference (DAC'01)*. ACM, 2001. URL `http://www.princeton.edu/~chaff/publication/DAC2001v56.pdf`.

[Ric53] Henry G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 83, 1953.

[Rob65] John A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[Sch08] Uwe Schöning. *Logic for Computer Scientists*. Birkhäuser, 2008.

[Sip96] Michael Sipser. *Introduction to the Theory of Computation*. PWS Publishing Company, 1996.

[Smu82] Raymond M. Smullyan. *The Lady or the Tiger? And Other Logic Puzzles*. Times Books, 1982.

[Sum06] George J. Summers. *Logical Deduction Puzzles*. Sterling Publishing, 2006.

[Tur37] Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1937.

[vR68] Thomas von Randow. *99 Logeleien von Zweistein: mit ihren Antworten*. Christian Wegner Verlag, 1968.

Index

$M \models \bot$, 67, 124
$M \vdash k$, 170
M derives k, 170
Σ-Formula, 117
Σ-structure, 120
⊖, 40
⊖, 40
⊖, 40
⊘, 40
⊘, 40
⊥, falsum, 38
↔ if and only iff, 38
$\mathbb{F}_\Sigma$, 117
$\mathcal{A}_\Sigma$, 116
$\mathcal{B} = \{\texttt{true}, \texttt{false}\}$, 40
$\mathcal{F}$: set of propositional formulas, 38
$\mathcal{I}$, propositional valuation, 40
$\mathcal{K}$ set of all clauses, 52
$\mathcal{L}$, set of all literals, 51
$\mathcal{P}$, set of propositional variables, 38
$\mathcal{S} \models F$, 123
$\mathcal{T}_\Sigma$, 115
$\models f$, 48
$\neg a$, 36
$\neg f$, 38
→, if $\cdots$, then, 38
$\mathcal{S}(\mathcal{I}, t)$, 122
$BV(F)$, 117
$FV(F)$, 117
$Var(t)$, 117
∨, or, 38
⊤, verum, 38
∧, and, 38
$\widehat{\mathcal{I}}(f)$, 40
$a \leftrightarrow b$, 36
$a \rightarrow b$, 36
$a \vee b$, 36
$a \wedge b$, 36
$s \doteq t$, 166
`cnf`, 61
`findValuation`, 73
`reduce`, 83
`saturate`, 82
`neg`, 58
`nnf`, 58
`reduce`, 78
`solve`, 79
`subsume`, 77
`unitCut`, 76

absorption, 49
algorithm of Martelli and Montanari, 166
Arity, 114
associative, 49
Atomic Formulas, 116
atomic formulas, 113
atomic proposition, 35

backtracking, 131
biconditional, 38
bound variable, 116
bound variables, 117

case distinction, 77
clause, 52
closed formula, 123
CNF, 53
commutative, 49
complement, 51
complementary literals, 52
composite propositions, 35
composition $\sigma\tau$, 165
Composition of Substitutions, 165
computational induction, 21
conjunction, 38
conjunctive normal form, 53
constant, 113, 114
constraint satisfaction problem, 126
constraints, 126

contradictory, 124
Correctness Theorem of First-Order Logic, 171
countable, 17
CSP, 126
Cut Rule, 64

Davis-Putnam Algorithm, 78
decidable, 14
declarative programming, 126
DeMorgan rules, 49
derivation, 63
disjunction, 38
distributive, 49
domain, 127

eight queens puzzle, 129
equisatisfiability, 160
equivalence problem, 18
equivalent, 48, 123

Factorization, 170
fallacy, 64
falsum, 38
first-order logic, 113
free variables, 117
free variable, 116
function symbol, 113
Function symbols, 114

halting problem, 11, 14

idempotency, 49
implication, 38
inference rule, 63
interpretation, 120

literal, 51

map colouring, 127
model, 123

negation, 38
negation-normal-form, 54
negative literal, 51
nested list, 42

Object variable, 113
Object variables, 114

partial variable assignment, 127
partially equivalent, 17
positive literal, 51
Predicate symbols, 113, 114
prenex normal form, 159
proof, 63
propositional formulas, 38
propositional valuation $\mathcal{I}$, 40
propositional variable, 114
propositional variables, 36, 38
propositions, 35

quantifiers, 113

Refutational Completeness of the Resolution Calculus, 171
Resolution, 169

satisfiable, 36, 67, 124
saturated, 68
semantics, 38, 40
set notation, 52
set notation for formulas in CNF, 53
signature, 114
solution, 67, 74, 75
solution of a CSP, 127
Substitution Rule, 169
subsumed, 76
sudoku, 151
surjective, 17
symbolic execution, 28
syntactical equation, 166
syntax, 38
system of syntactical equations, 166

tautology, 36, 48
terms, 115
test function, 12
trivial, 52
trivial set of clauses, 75
truth table, 40
truth values, 40
turing machine, 16
Turing, Alan, 15
twin prime, 16

unifier, 166
unit clause, 75
unit cut, 76

Universal closure, 171
Universally valid, 123
universally valid, 48
universe, 120
unsatisfiable, 36, 67

Vampire, 175
variable assignment, 121
verum, 38